LOGO

Advanced Python Programming for Datascience

BCT III/I

IOE TU | Khwopa College of Engineering

Chapter-1
Advanced Python Concepts and Best Practices

Review of Python essentials

Data Types

Data types define the type of data a variable can hold. Common Python data types include:

- **Integer (int)** – Whole numbers, e.g., 5, -10
- **Float (float)** – Decimal numbers, e.g., 3.14, -0.5
- **String (str)** – Sequence of characters, e.g., "Hello"
- **Boolean (bool)** – Logical values, True or False

Data Structures

Data structures are ways to organize and store multiple values efficiently. Common Python structures include:

- **List (list)** – Ordered, mutable collection, e.g., [1, 2, 3]
- **Tuple (tuple)** – Ordered, immutable collection, e.g., (1, 2, 3)
- **Set (set)** – Unordered, unique elements, e.g., {1, 2, 3}
- **Dictionary (dict)** – Key-value pairs, e.g., {"name": "Alice", "age": 25}

Mutable Data Types

Mutable data types are those whose values can be changed after they are created. **Examples:**

- **List (list)** – e.g., `mylist = [1, 2, 3]` → `mylist[0] = 10`
- **Dictionary (dict)** – e.g., `mydict = {"name": "Alice"}` → `mydict["name"] = "Bob"`
- **Set (set)** – e.g., `myset = {1, 2, 3}` → `myset.add(4)`

Immutable Data Types

Immutable data types are those whose values cannot be changed once they are created. Any modification creates a new object. **Examples:**

- **Integer (int)** – e.g., `x = 5` → `x + 1` creates a new integer object
- **Float (float)** – e.g., `y = 3.14`
- **String (str)** – e.g., `s = "Hello"` → `s.upper()` creates `"HELLO"`
- **Tuple (tuple)** – e.g., `t = (1, 2, 3)`

Sequence Types

Lists []

Ordered, mutable, heterogeneous collections.

```
fruits = ['apple', 'banana', 'orange']
fruits.append('grape')            # Add to end
fruits.insert(1, 'kiwi')          # Insert at index
fruits.remove('banana')           # Remove by value
popped = fruits.pop()             # Remove and return last
sliced = fruits[1:3]              # Slicing [start:end:step]
```

Tuples ()

Ordered, immutable, heterogeneous collections.

```
coordinates = (10, 20)
x, y = coordinates                # Tuple unpacking
single_item = (1,)                # Single item needs trailing comma
```

Strings '' or ""

Ordered, immutable sequences of characters.

```
text = "Python Programming"
text.upper()                       # Returns new string
text.split()                       # Split into list
" ".join(['Python', 'Data'])       # Join list into string
```

Ranges

Immutable sequences of numbers.

```
range(5)           # 0, 1, 2, 3, 4
range(2, 10, 2)    # 2, 4, 6, 8
```

Mapping Types

Dictionaries {}

Key-value pairs (Python 3.7+ preserves insertion order).

```python
person = {'name': 'Alice', 'age': 30}
person.get('city', 'Unknown')       # Safe access with default
person.keys()                       # View of keys
person.values()                     # View of values
person.items()                      # View of key-value pairs
person.setdefault('country', 'USA') # Set if key doesn't exist
```

defaultdict (from `collections`)

Provides default values for missing keys.

```python
from collections import defaultdict
word_count = defaultdict(int)         # Default 0 for missing keys
word_count['apple'] += 1              # No KeyError

group_by_first = defaultdict(list)    # Default empty list
group_by_first['a'].append('apple')   # Auto-creates list
```

OrderedDict (from `collections`)

Remembers insertion order (less needed in Python 3.7+).

```python
from collections import OrderedDict
od = OrderedDict()
od.move_to_end('key')                 # Move to beginning/end
```

Set Types

set

Unordered collection of unique, hashable objects.

```python
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}
set1.add(5)                           # Add element
set1.remove(3)                        # Remove (raises KeyError if missing)
set1.discard(10)                       # Remove (no error if missing)

# Set operations
set1.union(set2)                       # {1, 2, 3, 4, 5, 6}
set1.intersection(set2)                # {3, 4}
set1.difference(set2)                  # {1, 2}
set1.symmetric_difference(set2)        # {1, 2, 5, 6}
set1.issubset(set2)                    # Check if subset
```

frozenset

Immutable set (hashable, can be dict key).

```python
immutable_set = frozenset([1, 2, 3])
```

NoneType

Represents absence of value.

- Single value `None`.
- Falsy in boolean contexts.

- Default return of functions without a return statement.
- Use `is None` for comparison (not `==`).

Control Flow

Conditional Statements

Conditional statements allow decision-making in code based on logical conditions.

```
# Basic if-elif-else
if condition1:
    # code block
elif condition2:
    # code block
else:
    # code block

# Ternary expression
value = x if condition else y

# Pattern matching (Python 3.10+)
match point:
    case (0, 0):
        print("Origin")
    case (x, 0):
        print(f"X-axis at {x}")
    case (0, y):
        print(f"Y-axis at {y}")
    case (x, y):
        print(f"Point at {x}, {y}")
```

Loops

Loops allow repeated execution of code blocks.

For loops: Iterate over sequences.

```
for item in iterable:
    # code block

# Enumerate for index and value
for i, value in enumerate(iterable):
    print(f"Index {i}: {value}")

# Zip for parallel iteration
for a, b in zip(list1, list2):
    print(a, b)

# Iterate over dictionary
for key, value in dictionary.items():
    print(key, value)
```

While loops: Repeat while a condition is true.

```
while condition:
    # code block
    if exit_condition:
        break
    if skip_condition:
        continue
```

Loop else clause: Executes only if loop completes without `break`.

```
for item in items:
    if condition(item):
        break
else:
    print("No item matched condition")
```

Comprehensions

Concise way to create collections from iterables.

```
# List comprehension
squares = [x**2 for x in range(10)]
evens = [x for x in range(20) if x % 2 == 0]
matrix = [[i*j for j in range(3)] for i in range(3)]

# Dictionary comprehension
square_dict = {x: x**2 for x in range(5)}
filtered_dict = {k: v for k, v in original.items() if v > 10}

# Set comprehension
unique_lengths = {len(word) for word in words}

# Generator expressions
sum_of_squares = sum(x**2 for x in range(1000000))
gen = (x**2 for x in range(5))
print(list(gen))  # [0, 1, 4, 9, 16]
print(list(gen))  # []  (exhausted)
```

Functions

First-Class Functions

Functions can be assigned, passed, and returned.

```
def greet(name):
    return f"Hello, {name}"

say_hello = greet
print(say_hello("Alice"))

def apply_twice(func, arg):
    return func(func(arg))

def add_one(x):
    return x + 1

result = apply_twice(add_one, 5)  # 7
```

Lambda Functions

Anonymous, single-expression functions.

```
square = lambda x: x**2
add = lambda x, y: x + y

pairs = [(1, 'one'), (3, 'three'), (2, 'two')]
pairs.sort(key=lambda pair: pair[0])
evens = list(filter(lambda x: x % 2 == 0, numbers))
squared = list(map(lambda x: x**2, numbers))
```

*args and **kwargs

Handle variable positional and keyword arguments.

```
def sum_all(*args):
    return sum(args)

def print_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

def complex_function(a, b, *args, **kwargs):
    print(f"a: {a}, b: {b}")
    print(f"args: {args}")
    print(f"kwargs: {kwargs}")

# Unpacking arguments
def add(a, b, c):
    return a + b + c

numbers = [1, 2, 3]
print(add(*numbers))  # Unpack list

kwargs = {'a': 1, 'b': 2, 'c': 3}
print(add(**kwargs))  # Unpack dict
```

Decorators

Functions that modify behavior of other functions.

```
def timer(func):
    import time
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end-start:.4f} seconds")
        return result
    return wrapper

@timer
def slow_function():
    import time
    time.sleep(1)
    return "Done"

# Decorators with arguments
def repeat(n_times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(n_times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def greet(name):
    print(f"Hello {name}")
```

Closures

Functions capturing variables from outer scope.

```python
def outer_function(message):
    def inner_function():
        print(message)
    return inner_function

closure = outer_function("Hello World")
closure()  # Still remembers "Hello World"

def counter(start=0):
    count = start
    def increment():
        nonlocal count
        count += 1
        return count
    return increment

c1 = counter(10)
print(c1())  # 11
print(c1())  # 12
```

Error Handling

try, except, else, finally

Handle exceptions gracefully.

```python
try:
    risky_operation()
except SomeSpecificError as e:
    print(f"Error occurred: {e}")
except Exception as e:
    print(f"Unexpected error: {e}")
else:
    print("Operation successful!")
finally:
    cleanup_resources()
```

Custom Exceptions

Define specific errors for clarity.

```python
class DataScienceError(Exception): pass
class DataValidationError(DataScienceError): pass

class ModelTrainingError(DataScienceError):
    def __init__(self, message, model_name=None):
        self.model_name = model_name
        super().__init__(f"{message} (Model: {model_name})")

# Usage
def train_model(data, model_name):
    if not data:
        raise DataValidationError("Empty dataset provided")
    try:
        # training code
        pass
    except Exception as e:
        raise ModelTrainingError(f"Training failed: {e}", model_name)

# Handling custom exceptions
try:
```

```
    train_model([], "RandomForest")
except DataValidationError as e:
    print(f"Validation issue: {e}")
except ModelTrainingError as e:
    print(f"Training issue with {e.model_name}: {e}")
except DataScienceError as e:
    print(f"General data science error: {e}")
```

Context Managers

Ensure resource management using `with`.

```
with open('file.txt', 'r') as file:
    content = file.read()

# Custom context manager using contextlib
from contextlib import contextmanager

@contextmanager
def managed_file(filename, mode='r'):
    file = open(filename, mode)
    try:
        yield file
    finally:
        file.close()

# Multiple context managers
with open('input.txt') as infile, open('output.txt', 'w') as outfile:
    outfile.write(infile.read())

# Timing context manager
import time

@contextmanager
def timer():
    start = time.time()
    try:
        yield
    finally:
        end = time.time()
        print(f"Operation took {end - start:.4f} seconds")
```

Naming conventions

Naming conventions are a set of standardized rules and guidelines for choosing names for program identifiers, such as variables, functions, classes, and constants. They ensure:

- **Consistency** – Uniform style across the codebase
- **Readability** – Code is easier to understand at a glance
- **Maintainability** – Easier to update and debug

By following naming conventions, identifiers convey meaning about their purpose and scope.

Common Python Conventions / PEP Guidelines

- **PEP 8** – Style Guide for Python Code

- **PEP 257** – Docstring Conventions
- **PEP 484** – Type Hints
- **PEP 20** – The Zen of Python

PEP 8 Style Guide

Naming Conventions

- **Variables and Functions:** `snake_case`
- **Classes:** `CapWords` / `CamelCase`
- **Constants:** `UPPER_CASE`
- **Internal use:** `_single_leading_underscore`
- **Private attributes:** `__double_leading_underscore`
- **Magic / special methods:** `dunder` (e.g., `__init__`)

Code Layout

- 4 spaces per indentation level
- No tabs
- Maximum line length: 79 characters
- Maximum docstring length: 72 characters
- 2 blank lines between top-level functions/classes
- 1 blank line between class methods
- UTF-8 encoding for files

Imports

- One import per line
- Imports grouped in order: standard library → third-party → local
- Absolute imports preferred
- No wildcard imports (avoid `from module import *`)

Whitespace

- Space around binary operators: `a + b`, not `a+b`
- No spaces inside parentheses, brackets, or braces: `func(a, b)`
- Space after commas: `[1, 2, 3]`
- No space before commas

- Space after colon in dictionaries: {'key': value}
- No space before colon

Comments

- Inline comments start with #: # comment
- Block comments explain complex logic
- Comments should explain **why**, not **what**

String Quotes

- Consistency within a module
- Triple quotes for docstrings: """Docstring"""

Line Breaks

- Break before binary operators
- Align with opening delimiter or use hanging indent

PEP 8 Examples

Naming Conventions Programs

```
# Variables and Functions: snake_case
customer_name = "John Smith"
total_amount = 150.75
is_active_member = True

def calculate_discount(price, discount_rate):
    """Calculate discounted price."""
    return price * (1 - discount_rate)

def fetch_user_data(user_id):
    """Retrieve user information from database."""
    # Database query logic
    return {"id": user_id, "name": "Alice"}

# Classes: CapWords/CamelCase
class CustomerAccount:
    """Represents a customer account."""
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

class DataProcessor:
    """Process and transform data."""
```

```
    def process_csv(self, filepath):
        """Process CSV file."""
        pass

# Constants: UPPER_CASE
MAX_LOGIN_ATTEMPTS = 5
DEFAULT_TIMEOUT_SECONDS = 30
API_BASE_URL = "https://api.example.com/v1"
PI = 3.14159265359
INTEREST_RATE = 0.05
```

Code Layout Programs

```
# 4 spaces per indentation level
def process_orders(orders):
    """Process multiple orders."""
    results = []
    for order in orders:
        customer_id = order.get('customer_id')
        if customer_id:
            items = order.get('items', [])
            for item in items:
                price = item.get('price', 0)
                quantity = item.get('quantity', 1)
                total = price * quantity
                results.append(total)
    return results

# No tabs - using spaces consistently
class OrderManager:
    """Manage customer orders."""
    def __init__(self, orders):
        self.orders = orders
        self.total = 0

    def calculate_total(self):
        for order in self.orders:
            for item in order['items']:
                self.total += item['price'] * item['quantity']
        return self.total
```

Imports Programs

```
# Standard Library
import os
import sys
import json
from datetime import datetime
from collections import defaultdict, Counter

# Third-party
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split

# Local imports
from myproject.config import settings
from myproject.database import get_connection
from myproject.models import Customer
```

Whitespace, Comments, Strings, Line Breaks

```
# Whitespace examples
x = 5 + 3
function_call(arg1, arg2, arg3)
user = {'name': 'Alice', 'age': 30}

# Comments
def calculate_tax(price, tax_rate):
    """Calculate tax amount."""
    # Adjust for tax-exempt items
    if tax_rate < 0:
        tax_rate = 0
    return price * tax_rate

# Strings
name = "Alice"
message = f"{name}, welcome!"
def multiply(a, b):
    """Multiply two numbers."""
    return a * b

# Line breaks
total = (first_variable
         + second_variable
         - third_variable
         * fourth_variable)
```

Complete Example: Data Processing Pipeline

```
"""
data_pipeline.py
Complete data processing pipeline demonstrating PEP 8 conventions.
"""
import csv
import json
import logging
from pathlib import Path
from typing import List
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from .config import settings
from .validators import validate_data

MAX_FILE_SIZE_MB = 100
DEFAULT_ENCODING = 'utf-8'
```

```

class DataPipeline:
    def __init__(self, source_path: Path, destination_path: Path):
        self.source_path = source_path
        self.destination_path = destination_path
        self.data = None
        self._processed = False

    def load_data(self) -> pd.DataFrame:
        if not self.source_path.exists():
            raise FileNotFoundError(f"{self.source_path} not found")
        self.data = pd.read_csv(self.source_path, encoding=
    DEFAULT_ENCODING)
        return self.data

    def process(self, normalize_cols: List[str] = None) -> pd.DataFrame:
        self._clean_data()
        if normalize_cols:
            self._normalize_features(normalize_cols)
        self._processed = True
        return self.data

    def _clean_data(self):
        self.data = self.data.drop_duplicates()
        for col in self.data.columns:
            if self.data[col].dtype in ['int64', 'float64']:
                self.data[col].fillna(self.data[col].median(), inplace=
    True)
            else:
                self.data[col].fillna('Unknown', inplace=True)

    def _normalize_features(self, columns: List[str]):
        scaler = StandardScaler()
        self.data[columns] = scaler.fit_transform(self.data[columns])

    def export_data(self, format='csv'):
        if not self._processed:
            raise ValueError("No processed data to export")
        if format == 'csv':
            self.data.to_csv(self.destination_path, encoding=
    DEFAULT_ENCODING, index=False)
```

Types of Advanced Data Structures in Python

- **Collections** – Specialized container datatypes like `namedtuple`, `defaultdict`, `Counter`, `OrderedDict`, `deque`, `ChainMap`.
- **Iterators** – Objects that implement the iterator protocol with `__iter__()` and `__next__()` methods.
- **Generators** – Functions using `yield` or generator expressions that produce values lazily.
- **Decorators** – Functions or classes that modify behavior of other functions or classes using `@decorator` syntax.

Collections Module

namedtuple

```
from collections import namedtuple

# Creating a namedtuple type
Point = namedtuple('Point', ['x', 'y'])
p = Point(10, 20)
print(p.x, p.y)  # Access by name
print(p[0], p[1])  # Access by index also works
```

defaultdict

```
from collections import defaultdict

# int provides default value 0
word_counts = defaultdict(int)
# list provides default empty list
grouped_data = defaultdict(list)
# set provides default empty set
unique_sets = defaultdict(set)
```

Counter

```
from collections import Counter

# Count elements in an iterable
freq = Counter(['a', 'b', 'a', 'c', 'a', 'b'])
# Most common elements
top_two = freq.most_common(2)
```

OrderedDict

```
from collections import OrderedDict

od = OrderedDict()
od['first'] = 1
od['second'] = 2
od.move_to_end('first')  # Moves to end
```

deque

```
from collections import deque

dq = deque(maxlen=3)  # Fixed-size queue
dq.append(1)          # Add to right
dq.appendleft(2)      # Add to left
dq.pop()              # Remove from right
dq.popleft()          # Remove from left
```

ChainMap

```python
from collections import ChainMap

defaults = {'theme': 'light', 'language': 'en'}
user_prefs = {'theme': 'dark'}
settings = ChainMap(user_prefs, defaults)
print(settings['theme'])    # 'dark'
print(settings['language']) # 'en'
```

Iterators

Iterator Protocol

```python
class CountDown:
    """Iterator that counts down from n to 1."""
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        self.current -= 1
        return self.current + 1
```

Iterable vs Iterator

```python
# List is iterable but not an iterator
my_list = [1, 2, 3]
my_iter = iter(my_list)
print(next(my_iter))  # 1
```

Built-in Iterator Functions

```python
# enumerate adds indices
for idx, val in enumerate(['a', 'b', 'c']):
    print(idx, val)

# zip combines iterables
for a, b in zip([1, 2], ['x', 'y']):
    print(a, b)
```

Infinite Iterators

```python
from itertools import count, cycle, repeat

# count(start=0, step=1) - infinite counter
for i in count(5, 2):
    if i > 15: break

# cycle(iterable) - infinitely repeats
for item in cycle(['red', 'green']):
    pass
```

Generators

Generator Functions

```python
def countdown(n):
    """Generator that counts down from n."""
    while n > 0:
        yield n
        n -= 1

gen = countdown(5)
print(next(gen))  # 5
print(next(gen))  # 4
```

Generator Expressions

```python
# List comprehension - creates entire list
squares_list = [x**2 for x in range(1000)]

# Generator expression - produces values on demand
squares_gen = (x**2 for x in range(1000))
total = sum(x**2 for x in range(1000000))
```

Memory Efficiency

```python
def read_large_file(file_path):
    """Read file line by line without loading entire file."""
    with open(file_path) as file:
        for line in file:
            yield line.strip()

for line in read_large_file('huge_dataset.txt'):
    process_line(line)
```

Generator Pipeline

```python
def read_data(source):
    for item in source:
        yield item
```

```python

def filter_data(stream, condition):
    for item in stream:
        if condition(item):
            yield item

def transform_data(stream, func):
    for item in stream:
        yield func(item)

pipeline = transform_data(
    filter_data(
        read_data(data_source),
        lambda x: x > 0
    ),
    lambda x: x ** 2
)
```

yield from

Theory: The `yield from` expression allows a generator to delegate part of its operations to another generator or iterable. It yields all values from the sub-iterator, making it easier to compose generators and flatten nested generator structures.

```python
def chain(*iterables):
    """Chain multiple iterables together."""
    for iterable in iterables:
        yield from iterable

# Equivalent to:
def chain_alt(*iterables):
    for iterable in iterables:
        for item in iterable:
            yield item
```

Generator State and Closing

Theory: Generators maintain their state between yields, including local variables and instruction pointer. They can be closed using `.close()` which raises `GeneratorExit` inside the generator. The yield expression can also receive values using `.send()`.

```python
def echo():
    """Generator that echoes received values."""
    while True:
        received = yield
        print(f"Received: {received}")

gen = echo()
next(gen)  # Advance to first yield
gen.send('hello')  # Send value to generator
gen.close()  # Close generator
```

Function Decorators

Theory: Decorators are functions that modify the behavior of other functions without changing their source code. They use the `@decorator` syntax and are applied at function definition time.

```python
def timer(func):
    """Decorator that measures execution time."""
    import time
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.time()-start:.4f}s")
        return result
    return wrapper

@timer
def slow_function():
    import time
    time.sleep(1)
```

Decorator Syntax

Theory: The `@decorator` syntax is syntactic sugar for `function = decorator(function)`.

```python
# These are equivalent
@decorator
def func():
    pass

# Same as
def func():
    pass
func = decorator(func)
```

Decorators with Arguments

Theory: Decorators can accept arguments by creating nested functions.

```python
def repeat(n_times):
    """Decorator factory that repeats function n times."""
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(n_times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def greet(name):
    print(f"Hello {name}")
```

Preserving Function Metadata

Theory: Use `functools.wraps` to preserve function metadata.

```python
from functools import wraps

def logged(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper
```

```

@logged
def add(x, y):
    """Add two numbers."""
    return x + y

print(add.__name__)  # 'add'
print(add.__doc__)   # 'Add two numbers.'
```

Class Decorators

Theory: Decorators can also be applied to classes to modify or enhance behavior.

```
def singleton(cls):
    """Decorator that ensures only one instance of class."""
    instances = {}
    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return get_instance

@singleton
class DatabaseConnection:
    def __init__(self):
        self.connected = False
```

Property Decorator

Theory: The `@property` decorator transforms methods into managed attributes.

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    @property
    def celsius(self):
        return self._celsius

    @property
    def fahrenheit(self):
        return (self._celsius * 9/5) + 32

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Temperature below absolute zero")
        self._celsius = value
```

Common Decorator Patterns

Theory: Patterns include logging, timing, memoization, access control, retry, and validation.

```
def memoize(func):
    """Caching decorator for expensive functions."""
    cache = {}
    @wraps(func)
    def wrapper(*args):
        if args in cache:
```

```
            return cache[args]
        result = func(*args)
        cache[args] = result
        return result
    return wrapper

def retry(max_attempts=3):
    """Retry decorator for unreliable operations."""
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(max_attempts):
                try:
                    return func(*args, **kwargs)
                except Exception:
                    if attempt == max_attempts - 1:
                        raise
            return None
        return wrapper
    return decorator
```

Stacked Decorators

Theory: Multiple decorators can be stacked; execution order is bottom to top.

```
@timer
@logged
@memoize
def fibonacci(n):
    """Calculate nth Fibonacci number."""
    if n < 2:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
```

Decorator Factories

Theory: Factories parameterize decorators.

```
def rate_limit(max_calls, period):
    """Decorator factory for rate limiting."""
    def decorator(func):
        calls = []
        @wraps(func)
        def wrapper(*args, **kwargs):
            from time import time
            now = time()
            calls[:] = [c for c in calls if now - c < period]
            if len(calls) >= max_calls:
                raise Exception("Rate limit exceeded")
            calls.append(now)
            return func(*args, **kwargs)
        return wrapper
    return decorator

@rate_limit(max_calls=5, period=60)
def api_call():
    """Limited to 5 calls per minute."""
    return "API response"
```

Integration Examples

```
from collections import defaultdict, Counter, deque
from functools import wraps
import time

def timing(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__}: {time.time()-start:.3f}s")
        return result
    return wrapper

def read_chunks(file_path, chunk_size=1024):
    """Generator to read file in chunks."""
    with open(file_path, 'rb') as f:
        while True:
            chunk = f.read(chunk_size)
            if not chunk:
                break
            yield chunk

class RollingWindow:
    """Iterator that yields rolling windows over data."""
    def __init__(self, data, window_size):
        self.data = data
        self.window_size = window_size
        self.index = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.index + self.window_size > len(self.data):
            raise StopIteration
        result = self.data[self.index:self.index + self.window_size]
        self.index += 1
        return result

@timing
def process_data(data_stream):
    """Process data using collections and generators."""
    recent = deque(maxlen=10)
    word_freq = Counter()
    word_groups = defaultdict(list)

    for chunk in data_stream:
        words = chunk.decode().split()
        word_freq.update(words)

        for word in words:
            word_groups[word[0]].append(word)
            recent.append(word)

        yield {
            'unique_words': len(word_freq),
            'most_common': word_freq.most_common(3),
            'recent_words': list(recent),
            'group_counts': {k: len(v) for k, v in word_groups.items()}
        }
```

Functions

Function Definition and Structure

Functions are reusable blocks of code that perform specific tasks. In Python, functions are first-class objects, meaning they can be assigned to variables, passed as arguments, and returned from other functions. The `def` keyword defines a function, followed by a name, parameters in parentheses, and a colon. The function body must be indented.

```
def fibonacci(N):
    """Generate first N Fibonacci numbers."""
    L = []
    a, b = 0, 1
    while len(L) < N:
        a, b = b, a + b
        L.append(a)
    return L

# Function call
result = fibonacci(10)
print(result)  # [1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
```

Key Characteristics:

- Functions are defined with `def` statement
- Can take any number of arguments
- Return any Python object (simple or compound)
- No type declarations required

Return Values

Python functions can return multiple values by packing them into a tuple. Functions without a return statement return `None`.

```
def real_imag_conj(val):
    """Return real part, imaginary part, and conjugate."""
    return val.real, val.imag, val.conjugate()

r, i, c = real_imag_conj(3 + 4j)
print(r, i, c)  # 3.0 4.0 (3-4j)

def print_message(msg):
    print(msg)

result = print_message("Hello")
print(result)  # None
```

Default Argument Values

Default arguments allow functions to be called with fewer arguments than defined. Mutable default arguments can persist across calls—use `None` as default to avoid this.

```
def fibonacci(N, a=0, b=1):
    L = []
    while len(L) < N:
        a, b = b, a + b
        L.append(a)
```

```
    return L

print(fibonacci(10))         # Defaults
print(fibonacci(10, 0, 2))   # Positional override
print(fibonacci(10, b=3, a=1))   # Keyword override

def add_item(item, L=[]):   # Dangerous!
    L.append(item)
    return L

def add_item_safe(item, L=None):
    if L is None:
        L = []
    L.append(item)
    return L
```

Positional and Keyword Arguments

Arguments can be passed positionally or by keyword. Keyword arguments must come after positional arguments.

```
def greet(name, greeting="Hello", punctuation="!"):
    return f"{greeting}, {name}{punctuation}"

print(greet("Alice", "Hi", "?"))
print(greet(name="Bob", punctuation="!!!"))
print(greet("Charlie", greeting="Hey"))
```

Variable-Length Arguments

*args - Variable Positional Arguments

Theory: Collects extra positional arguments into a tuple.

```
def catch_all(*args):
    print("args =", args)

catch_all(1, 2, 3)
catch_all('a', 'b', 'c', 'd')

def sum_all(*args):
    return sum(args)

print(sum_all(1, 2, 3, 4, 5))
```

**kwargs - Variable Keyword Arguments

Collects extra keyword arguments into a dictionary.

```
def catch_all_kwargs(**kwargs):
    print("kwargs =", kwargs)

catch_all_kwargs(a=4, b=5)
catch_all_kwargs(name="Alice", age=30)

def configure(**kwargs):
    for key, value in kwargs.items():
        print(f"Setting {key} = {value}")

```

```
configure(host="localhost", port=8080, debug=True)
```

Combining *args and **kwargs

Functions can accept both variable positional and keyword arguments.

```
def catch_both(*args, **kwargs):
    print("args =", args)
    print("kwargs =", kwargs)

catch_both(1, 2, 3, a=4, b=5)

def add(a, b, c):
    return a + b + c

numbers = [1, 2, 3]
print(add(*numbers))

kwargs = {'a': 10, 'b': 20, 'c': 30}
print(add(**kwargs))
```

Positional-Only and Keyword-Only Arguments

Python 3.8+ supports positional-only (before /) and keyword-only (after *).

```
def precise_function(a, b, /, c, d, *, e, f):
    return a + b + c + d + e + f

print(precise_function(1, 2, 3, 4, e=5, f=6))
print(precise_function(1, 2, c=3, d=4, e=5, f=6))
```

Lambda Expressions

Lambda Syntax and Theory

Lambda expressions create anonymous, single-expression functions.

```
square = lambda x: x ** 2
print(square(5))

add = lambda x, y: x + y
constant = lambda: 42
```

Use Cases: map, filter, reduce, sorting

Lambdas are useful for sorting, transformation, filtering, and reducing iterables.

```
# Sorting
data = [{'first': 'Guido'}, {'first': 'Grace'}, {'first': 'Alan'}]
sorted_by_first = sorted(data, key=lambda item: item['first'])

# map
numbers = [1, 2, 3]
squared = list(map(lambda x: x**2, numbers))

# filter
evens = list(filter(lambda x: x % 2 == 0, range(1,11)))
```

```

# reduce
from functools import reduce
total = reduce(lambda x, y: x+y, [1,2,3,4,5])
```

Nested Lambda Functions

Lambdas can return other lambdas for higher-order functionality.

```
adder = lambda x: (lambda y: x + y)
add5 = adder(5)
print(add5(10))
```

Higher-Order Functions

Functions as First-Class Objects

Functions can be passed, returned, and assigned.

```
def greet(name): return f"Hello, {name}"
say_hello = greet
print(say_hello("Alice"))

def apply_twice(func, arg): return func(func(arg))
```

Function Composition

Combine functions to form new functions.

```
def compose(f, g):
    return lambda x: f(g(x))

add_then_square = compose(lambda x: x**2, lambda x: x+1)
print(add_then_square(3))
```

Function Decorators

Basic Decorator Pattern

Decorators modify function behavior without changing its code.

```
def timer(func):
    import time
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end-start:.4f} seconds")
        return result
    return wrapper

@timer
def slow_function():
    import time
    time.sleep(1)
    return "Done"
```

Scoping Rules

LEGB Rule

Python resolves names using Local, Enclosing, Global, Built-in scopes.

```
x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print("inner:", x)
    inner()
    print("outer:", x)

outer()
print("global:", x)

# nonlocal example
def counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment

c = counter()
print(c())
print(c())
```

Summary

Concept	Purpose	Syntax	Best Used For
def functions	Named, reusable code	def name(params):	Complex logic, reuse
Lambda	Anonymous, single-expression	lambda params: expr	Simple callbacks, key functions
*args	Variable positional args	def f(*args):	Wrappers, variable inputs
**kwargs	Variable keyword args	def f(**kwargs):	Configuration, wrappers
Default args	Optional parameters	def f(x=default):	Common-case values
Higher-order	Functions as objects	def f(): return g	Composition, decorators

Why OOP in Data Science?

Object-oriented programming transforms data science work from exploratory scripts into maintainable, scalable code. It enables:

- **Encapsulation:** Bundling data and operations into cohesive units
- **State Management:** Tracking transformations and preserving fitted parameters
- **Code Organization:** Moving from scattered code to structured components
- **Reusability:** Building components that can be shared across projects

Core Class Structure

```python
class DataProcessor:
    """Template for processing data with configurable operations."""

    def __init__(self, name: str, scale_factor: float = 1.0):
        """
        Initialize processor with configuration.

        Args:
            name: Identifier for the processor instance
            scale_factor: Multiplier to apply during processing
        """
        self.name = name                    # Public attribute
        self._scale_factor = scale_factor    # Protected by convention
        self.__processed_data = None         # Private (name mangling)
        self.status = "initialized"
        self.raw_data = None

    def load_data(self, data):
        """Load data into the processor."""
        print(f"{self.name}: Loading data...")
        self.raw_data = data
        self.status = "loaded"

    def process(self):
        """Apply configured transformations."""
        if self.raw_data is not None:
            print(f"Processing with scale factor: {self._scale_factor}")
            self.__processed_data = [x * self._scale_factor for x in self.raw_data]
            self.status = "processed"
            return self.__processed_data
        else:
            raise ValueError("No data loaded. Call load_data() first.")

    def get_results(self):
        """Safely access processed data."""
        return self.__processed_data if self.__processed_data is not None else None

    def reset(self):
        """Reset processor state."""
        self.__processed_data = None
        self.raw_data = None
        self.status = "reset"

    def __repr__(self):
        """Provide readable representation."""
        return f"DataProcessor(name='{self.name}', status='{self.status}')"

    def __len__(self):
        """Return length of raw data if loaded."""
        return len(self.raw_data) if self.raw_data else 0

# Usage Example
processor = DataProcessor("test_processor", scale_factor=2.5)
print(processor)

data = [1, 2, 3, 4, 5]
processor.load_data(data)
result = processor.process()
print(f"Processed result: {result}")
print(f"Data length: {len(processor)}")
```

Encapsulation and Data Hiding

Encapsulation protects internal state and controls data access. This is crucial for maintaining data integrity in pipelines.

```
class FeatureScaler:
    """Scale features while protecting fitted parameters."""

    def __init__(self, method='standard'):
        self.method = method
        self._fitted = False
        self._parameters = {}  # Protected - internal use only
        self.__cache = {}       # Private - for internal caching

    def fit(self, data):
        """Learn scaling parameters from data."""
        if not data:
            raise ValueError("Empty data provided")

        if self.method == 'standard':
            self._parameters['mean'] = sum(data) / len(data)
            self._parameters['std'] = (
                sum((x - self._parameters['mean']) ** 2 for x in data) / len(
    data)
            ) ** 0.5

        elif self.method == 'minmax':
            self._parameters['min'] = min(data)
            self._parameters['max'] = max(data)

        self._fitted = True
        self.__cache = {}  # Clear cache on new fit
        return self

    def transform(self, data):
        """Apply scaling using fitted parameters."""
        if not self._fitted:
            raise RuntimeError("Scaler must be fitted before transform")

        # Check cache first
        data_tuple = tuple(data)
        if data_tuple in self.__cache:
            return self.__cache[data_tuple].copy()

        # Apply transformation
        if self.method == 'standard':
            result = [
                (x - self._parameters['mean']) / self._parameters['std']
                for x in data
            ]

        elif self.method == 'minmax':
            range_val = self._parameters['max'] - self._parameters['min']
            if range_val == 0:
                result = [0.5] * len(data)
            else:
                result = [
                    (x - self._parameters['min']) / range_val
                    for x in data
                ]

        # Cache result
        self.__cache[data_tuple] = result.copy()
```

```
        return result

    def fit_transform(self, data):
        """Convenience method combining fit and transform."""
        self.fit(data)
        return self.transform(data)

    @property
    def parameters(self):
        """Read-only access to parameters."""
        return self._parameters.copy()

    @property
    def is_fitted(self):
        """Check if scaler has been fitted."""
        return self._fitted

# Usage Example
scaler = FeatureScaler(method='standard')
data = [10, 20, 30, 40, 50]

# Fit and transform
scaled_data = scaler.fit_transform(data)
print(f"Scaled data: {scaled_data}")
print(f"Parameters: {scaler.parameters}")

# Transform new data using same parameters
new_data = [15, 25, 35]
transformed = scaler.transform(new_data)
print(f"Transformed new data: {transformed}")
```

Inheritance and Polymorphism

Inheritance enables code reuse and establishes clear hierarchies. Polymorphism allows different objects to respond to the same interface.

```
from abc import ABC, abstractmethod
import math

class BaseModel(ABC):
    """Abstract base class for all models."""

    def __init__(self, name):
        self.name = name
        self._fitted = False
        self._metrics = {}

    @abstractmethod
    def fit(self, X, y):
        """Train the model on data."""
        pass

    @abstractmethod
    def predict(self, X):
        """Make predictions for input data."""
        pass

    def score(self, X, y):
        """Calculate prediction accuracy."""
        predictions = self.predict(X)
        correct = sum(1 for p, t in zip(predictions, y) if p == t)
        accuracy = correct / len(y)
```

```
        self._metrics['accuracy'] = accuracy
        return accuracy

    def get_metrics(self):
        """Return model performance metrics."""
        return self._metrics.copy()

    def __repr__(self):
        return f"{self.__class__.__name__}(name='{self.name}', fitted={self._fitted})"


class LogisticRegression(BaseModel):
    """Logistic regression classifier."""

    def __init__(self, name, learning_rate=0.01, iterations=1000):
        super().__init__(name)
        self.learning_rate = learning_rate
        self.iterations = iterations
        self.weights = None
        self.bias = None

    def _sigmoid(self, z):
        """Sigmoid activation function."""
        return 1 / (1 + math.exp(-z))

    def fit(self, X, y):
        """Train using gradient descent."""
        n_samples = len(X)
        n_features = len(X[0])

        # Initialize parameters
        self.weights = [0.0] * n_features
        self.bias = 0.0

        # Gradient descent
        for _ in range(self.iterations):
            # Forward pass
            linear_predictions = [
                sum(w * x for w, x in zip(self.weights, xi)) + self.bias
                for xi in X
            ]
            predictions = [self._sigmoid(z) for z in linear_predictions]

            # Gradient calculation
            dw = [0.0] * n_features
            db = 0.0

            for i in range(n_samples):
                error = predictions[i] - y[i]
                for j in range(n_features):
                    dw[j] += (1/n_samples) * error * X[i][j]
                db += (1/n_samples) * error

            # Update parameters
            for j in range(n_features):
                self.weights[j] -= self.learning_rate * dw[j]
            self.bias -= self.learning_rate * db

        self._fitted = True
        return self

    def predict(self, X):
```

```
        """Make binary predictions."""
        if not self._fitted:
            raise RuntimeError("Model must be fitted before prediction")

        predictions = []
        for xi in X:
            linear = sum(w * x for w, x in zip(self.weights, xi)) + self.bias
            prob = self._sigmoid(linear)
            predictions.append(1 if prob >= 0.5 else 0)

        return predictions


class DecisionTree(BaseModel):
    """Simple decision tree classifier."""

    class Node:
        """Internal tree node."""
        def __init__(self, feature=None, threshold=None, left=None, right=None, value=None):
            self.feature = feature
            self.threshold = threshold
            self.left = left
            self.right = right
            self.value = value  # For leaf nodes

    def __init__(self, name, max_depth=5):
        super().__init__(name)
        self.max_depth = max_depth
        self.tree = None

    def _gini(self, groups, classes):
        """Calculate Gini impurity."""
        n_instances = sum(len(group) for group in groups)
        gini = 0.0

        for group in groups:
            size = len(group)
            if size == 0:
                continue

            score = 0.0
            for class_val in classes:
                proportion = [row[-1] for row in group].count(class_val) / size
                score += proportion ** 2

            gini += (1 - score) * (size / n_instances)

        return gini

    def _split(self, data, feature, threshold):
        """Split data based on feature and threshold."""
        left = [row for row in data if row[feature] <= threshold]
        right = [row for row in data if row[feature] > threshold]
        return left, right

    def _get_best_split(self, data):
        """Find best feature and threshold to split on."""
        class_values = list(set(row[-1] for row in data))
        best_feature, best_threshold, best_score, best_groups = None, None, float('inf'), None

        for feature in range(len(data[0]) - 1):
```

```
            thresholds = list(set(row[feature] for row in data))

            for threshold in thresholds:
                groups = self._split(data, feature, threshold)
                gini = self._gini(groups, class_values)

                if gini < best_score:
                    best_feature, best_threshold, best_score, best_groups = (
                        feature, threshold, gini, groups
                    )

        return {
            'feature': best_feature,
            'threshold': best_threshold,
            'groups': best_groups
        }

    def _build_tree(self, data, depth):
        """Recursively build decision tree."""
        classes = list(set(row[-1] for row in data))

        # Check stopping conditions
        if len(classes) == 1 or depth >= self.max_depth:
            # Create leaf node with majority class
            class_counts = {c: [row[-1] for row in data].count(c) for c in
    classes}
            leaf_value = max(class_counts, key=class_counts.get)
            return self.Node(value=leaf_value)

        # Find best split
        split = self._get_best_split(data)

        if split['groups'] is None:
            class_counts = {c: [row[-1] for row in data].count(c) for c in
    classes}
            leaf_value = max(class_counts, key=class_counts.get)
            return self.Node(value=leaf_value)

        # Recursively build children
        left = self._build_tree(split['groups'][0], depth + 1)
        right = self._build_tree(split['groups'][1], depth + 1)

        return self.Node(
            feature=split['feature'],
            threshold=split['threshold'],
            left=left,
            right=right
        )

    def fit(self, X, y):
        """Build decision tree from training data."""
        # Combine X and y for tree building
        data = [list(x) + [y[i]] for i, x in enumerate(X)]
        self.tree = self._build_tree(data, 0)
        self._fitted = True
        return self

    def _predict_row(self, node, row):
        """Predict class for a single row."""
        if node.value is not None:
            return node.value

        if row[node.feature] <= node.threshold:
```

```
            return self._predict_row(node.left, row)
        else:
            return self._predict_row(node.right, row)

    def predict(self, X):
        """Make predictions for input data."""
        if not self._fitted:
            raise RuntimeError("Model must be fitted before prediction")

        return [self._predict_row(self.tree, row) for row in X]


# Usage Example - Polymorphism in action
def evaluate_model(model, X_train, y_train, X_test, y_test):
    """Function that works with any model implementing fit/predict."""
    print(f"\nEvaluating {model.name}:")

    # Fit model
    model.fit(X_train, y_train)
    print(f"Model fitted: {model._fitted}")

    # Make predictions
    predictions = model.predict(X_test)
    print(f"Predictions: {predictions}")

    # Calculate accuracy
    accuracy = model.score(X_test, y_test)
    print(f"Accuracy: {accuracy:.2f}")
    print(f"Metrics: {model.get_metrics()}")

    return accuracy

# Sample data
X_train = [[1, 2], [2, 3], [3, 4], [4, 5]]
y_train = [0, 0, 1, 1]
X_test = [[1.5, 2.5], [3.5, 4.5]]
y_test = [0, 1]

# Both models can be used interchangeably
log_reg = LogisticRegression("LR_Model", learning_rate=0.1)
tree = DecisionTree("DT_Model", max_depth=3)

# Same function works for both
evaluate_model(log_reg, X_train, y_train, X_test, y_test)
evaluate_model(tree, X_train, y_train, X_test, y_test)
```

Composition over Inheritance

Composition builds complex objects by combining simpler ones, often providing more flexibility than deep inheritance.

```
class DataLoader:
    """Handle data loading from various sources."""

    def __init__(self):
        self.source = None
        self.loaded_data = None

    def from_csv(self, filepath):
        """Load data from CSV file."""
        print(f"Loading CSV from {filepath}")
        # Simulated CSV loading
```

```
        self.loaded_data = [
            [1, 2, 3],
            [4, 5, 6],
            [7, 8, 9]
        ]
        self.source = filepath
        return self.loaded_data

    def from_json(self, filepath):
        """Load data from JSON file."""
        print(f"Loading JSON from {filepath}")
        # Simulated JSON loading
        self.loaded_data = [
            {'feature1': 1, 'feature2': 2},
            {'feature1': 3, 'feature2': 4},
            {'feature1': 5, 'feature2': 6}
        ]
        self.source = filepath
        return self.loaded_data


class DataCleaner:
    """Handle data cleaning operations."""

    def __init__(self):
        self.cleaning_log = []

    def remove_nulls(self, data):
        """Remove rows with null values."""
        if not data:
            return data

        # Handle both list of lists and list of dicts
        if isinstance(data[0], dict):
            cleaned = [row for row in data if all(v is not None for v in row.
    values())]
        else:
            cleaned = [row for row in data if all(v is not None for v in row)]

        self.cleaning_log.append(f"Removed nulls: {len(data) - len(cleaned)}
    rows")
        return cleaned

    def normalize_types(self, data):
        """Ensure consistent data types."""
        if not data or isinstance(data[0], dict):
            return data

        # Convert all to float
        cleaned = []
        for row in data:
            cleaned.append([float(x) if x is not None else 0.0 for x in row])

        self.cleaning_log.append("Normalized types to float")
        return cleaned

    def get_log(self):
        """Return cleaning operation log."""
        return self.cleaning_log.copy()


class FeatureEngineer:
    """Create and transform features."""
```

```

    def __init__(self):
        self.features_created = []

    def add_interaction(self, data, col1, col2):
        """Add interaction feature."""
        if isinstance(data[0], dict):
            for row in data:
                row[f'interaction_{col1}_{col2}'] = row[col1] * row[col2]
        else:
            for row in data:
                row.append(row[col1] * row[col2])

        self.features_created.append(f'interaction_{col1}_{col2}')
        return data

    def add_polynomial(self, data, col, degree=2):
        """Add polynomial features."""
        if isinstance(data[0], dict):
            for d in range(2, degree + 1):
                for row in data:
                    row[f'{col}^{d}'] = row[col] ** d
        else:
            for d in range(2, degree + 1):
                for i, row in enumerate(data):
                    data[i] = row + [row[col] ** d]

        self.features_created.append(f'poly_{col}_deg{degree}')
        return data


class ModelTrainer:
    """Train and evaluate models."""

    def __init__(self, model):
        self.model = model
        self.training_history = []

    def train(self, X, y, validation_split=0.2):
        """Train model with validation."""
        split_idx = int(len(X) * (1 - validation_split))

        X_train, X_val = X[:split_idx], X[split_idx:]
        y_train, y_val = y[:split_idx], y[split_idx:]

        # Train model
        self.model.fit(X_train, y_train)

        # Validate
        train_score = self.model.score(X_train, y_train)
        val_score = self.model.score(X_val, y_val)

        self.training_history.append({
            'train_score': train_score,
            'val_score': val_score,
            'samples': len(X)
        })

        return self.model

    def get_history(self):
        """Return training history."""
        return self.training_history.copy()
```

```
class DataSciencePipeline:
    """Complete pipeline using composition."""

    def __init__(self, name):
        self.name = name
        self.loader = DataLoader()
        self.cleaner = DataCleaner()
        self.engineer = FeatureEngineer()
        self.trainer = None
        self.data = None
        self.labels = None

    def set_model(self, model):
        """Set the model to use."""
        self.trainer = ModelTrainer(model)
        return self

    def load_data(self, source_type, filepath):
        """Load data from source."""
        if source_type == 'csv':
            self.data = self.loader.from_csv(filepath)
        elif source_type == 'json':
            self.data = self.loader.from_json(filepath)
        else:
            raise ValueError(f"Unknown source type: {source_type}")

        # Assume last column is target for simplicity
        if self.data and not isinstance(self.data[0], dict):
            self.labels = [row[-1] for row in self.data]
            self.data = [row[:-1] for row in self.data]

        return self

    def clean_data(self):
        """Apply cleaning operations."""
        if self.data:
            self.data = self.cleaner.remove_nulls(self.data)
            self.data = self.cleaner.normalize_types(self.data)
        return self

    def engineer_features(self):
        """Create new features."""
        if self.data and len(self.data[0]) >= 2:
            self.data = self.engineer.add_interaction(self.data, 0, 1)
            self.data = self.engineer.add_polynomial(self.data, 0, degree=2)
        return self

    def run(self):
        """Execute the complete pipeline."""
        if not self.trainer:
            raise RuntimeError("No model set. Call set_model() first.")

        print(f"\nRunning pipeline: {self.name}")
        print("=" * 40)

        # Train model
        self.trainer.train(self.data, self.labels)

        # Report results
        print(f"Cleaning log: {self.cleaner.get_log()}")
        print(f"Features created: {self.engineer.features_created}")
```

```
        print(f"Training history: {self.trainer.get_history()}")

        return self.trainer.model

# Usage Example
pipeline = DataSciencePipeline("Experiment_1")
pipeline.set_model(LogisticRegression("LR", learning_rate=0.01))
pipeline.load_data('csv', 'data.csv')
pipeline.clean_data()
pipeline.engineer_features()
model = pipeline.run()
```

Magic Methods for Data Science

Special methods allow custom objects to integrate seamlessly with Python's syntax and built-in functions.

```
class DataFrame:
    """Custom DataFrame implementation with magic methods."""

    def __init__(self, data, columns=None):
        """
        Initialize DataFrame.

        Args:
            data: List of lists or list of dicts
            columns: Column names (for list of lists)
        """
        self._data = []
        self.columns = columns
        self._shape = None
        self._dtypes = {}

        if data:
            self._initialize_data(data)

    def _initialize_data(self, data):
        """Initialize internal data structure."""
        if isinstance(data[0], dict):
            # List of dictionaries
            self.columns = list(data[0].keys())
            self._data = [[row[col] for col in self.columns] for row in data]
        else:
            # List of lists
            self._data = [list(row) for row in data]
            if not self.columns:
                self.columns = [f"col_{i}" for i in range(len(data[0]))]

        self._infer_dtypes()
        self._update_shape()

    def _infer_dtypes(self):
        """Infer column data types."""
        if not self._data:
            return

        for i, col in enumerate(self.columns):
            col_data = [row[i] for row in self._data if row[i] is not None]
            if all(isinstance(x, int) for x in col_data):
                self._dtypes[col] = 'int'
            elif all(isinstance(x, float) for x in col_data):
                self._dtypes[col] = 'float'
```

```
            elif all(isinstance(x, str) for x in col_data):
                self._dtypes[col] = 'str'
            else:
                self._dtypes[col] = 'object'

    def _update_shape(self):
        """Update shape information."""
        self._shape = (len(self._data), len(self.columns) if self._data else 0)

    # Container Magic Methods
    def __len__(self):
        """Return number of rows."""
        return len(self._data)

    def __getitem__(self, key):
        """
        Support indexing and slicing.
        - Integer: returns row
        - String: returns column
        - Slice: returns DataFrame subset
        """
        if isinstance(key, int):
            # Row indexing
            return dict(zip(self.columns, self._data[key]))

        elif isinstance(key, str):
            # Column indexing
            if key not in self.columns:
                raise KeyError(f"Column '{key}' not found")

            col_idx = self.columns.index(key)
            return [row[col_idx] for row in self._data]

        elif isinstance(key, slice):
            # Row slicing
            new_data = self._data[key]
            return DataFrame(new_data, self.columns.copy())

        elif isinstance(key, list):
            # Multiple columns
            if all(isinstance(k, str) for k in key):
                col_indices = [self.columns.index(k) for k in key]
                new_data = [[row[i] for i in col_indices] for row in self._data]
                return DataFrame(new_data, key.copy())

        raise TypeError(f"Invalid key type: {type(key)}")

    def __setitem__(self, key, value):
        """Support column assignment."""
        if isinstance(key, str):
            if key in self.columns:
                # Update existing column
                col_idx = self.columns.index(key)
                for i, row in enumerate(self._data):
                    row[col_idx] = value[i] if hasattr(value, '__getitem__')
else value
            else:
                # Add new column
                self.columns.append(key)
                for i, row in enumerate(self._data):
                    val = value[i] if hasattr(value, '__getitem__') else value
                    row.append(val)

```

```
        self._infer_dtypes()
        self._update_shape()

    def __delitem__(self, key):
        """Support column deletion."""
        if key in self.columns:
            col_idx = self.columns.index(key)
            self.columns.pop(col_idx)
            for row in self._data:
                row.pop(col_idx)
            self._update_shape()

    # Numeric Magic Methods
    def __add__(self, other):
        """Element-wise addition."""
        result = DataFrame([], self.columns.copy())

        if isinstance(other, (int, float)):
            # Scalar addition
            result._data = [[x + other for x in row] for row in self._data]

        elif isinstance(other, DataFrame):
            # DataFrame addition
            if self._shape != other._shape:
                raise ValueError("DataFrames must have same shape")

            result._data = [
                [self._data[i][j] + other._data[i][j] for j in range(len(self.
columns))]
                for i in range(len(self._data))
            ]

        return result

    def __sub__(self, other):
        """Element-wise subtraction."""
        result = DataFrame([], self.columns.copy())

        if isinstance(other, (int, float)):
            result._data = [[x - other for x in row] for row in self._data]

        elif isinstance(other, DataFrame):
            if self._shape != other._shape:
                raise ValueError("DataFrames must have same shape")

            result._data = [
                [self._data[i][j] - other._data[i][j] for j in range(len(self.
columns))]
                for i in range(len(self._data))
            ]

        return result

    def __mul__(self, other):
        """Element-wise multiplication."""
        result = DataFrame([], self.columns.copy())

        if isinstance(other, (int, float)):
            result._data = [[x * other for x in row] for row in self._data]

        elif isinstance(other, DataFrame):
            if self._shape != other._shape:
                raise ValueError("DataFrames must have same shape")
```

```
169
170             result._data = [
171                 [self._data[i][j] * other._data[i][j] for j in range(len(self.
    columns))]
172                 for i in range(len(self._data))
173             ]
174
175         return result
176
177     # Comparison Magic Methods
178     def __eq__(self, other):
179         """Element-wise equality comparison."""
180         result = DataFrame([], self.columns.copy())
181
182         if isinstance(other, (int, float, str)):
183             result._data = [[x == other for x in row] for row in self._data]
184
185         return result
186
187     def __gt__(self, other):
188         """Element-wise greater than comparison."""
189         result = DataFrame([], self.columns.copy())
190
191         if isinstance(other, (int, float)):
192             result._data = [[x > other for x in row] for row in self._data]
193
194         return result
195
196     def __lt__(self, other):
197         """Element-wise less than comparison."""
198         result = DataFrame([], self.columns.copy())
199
200         if isinstance(other, (int, float)):
201             result._data = [[x < other for x in row] for row in self._data]
202
203         return result
204
205     # String Representation
206     def __repr__(self):
207         """Detailed string representation."""
208         if not self._data:
209             return "Empty DataFrame"
210
211         lines = []
212         lines.append(f"DataFrame: {self._shape[0]} rows    {self._shape[1]}
    columns")
213         lines.append(f"Columns: {', '.join(self.columns)}")
214         lines.append(f"Types: {self._dtypes}")
215
216         # Show first few rows
217         lines.append("\nFirst 5 rows:")
218         header = "  ".join(f"{col:<10}" for col in self.columns[:5])
219         lines.append(header)
220         lines.append("-" * len(header))
221
222         for i, row in enumerate(self._data[:5]):
223             row_str = "  ".join(f"{str(x):<10}"[:10] for x in row[:5])
224             lines.append(row_str)
225
226         if len(self._data) > 5:
227             lines.append("...")
228
229         return "\n".join(lines)
```

```
230
231     def __str__(self):
232         """Simple string representation."""
233         return f"DataFrame({self._shape[0]}x{self._shape[1]})"
234
235     # Callable Magic Method
236     def __call__(self, *args, **kwargs):
237         """Make DataFrame callable for common operations."""
238         if args and callable(args[0]):
239             # Apply function to all elements
240             func = args[0]
241             result = DataFrame([], self.columns.copy())
242             result._data = [[func(x) for x in row] for row in self._data]
243             return result
244         return self
245
246     # Context Manager Magic Methods
247     def __enter__(self):
248         """Enter context manager."""
249         print(f"Entering DataFrame context: {self._shape}")
250         self._backup = [row.copy() for row in self._data]
251         return self
252
253     def __exit__(self, exc_type, exc_val, exc_tb):
254         """Exit context manager."""
255         if exc_type is not None:
256             print(f"Error occurred: {exc_val}")
257             print("Restoring backup...")
258             self._data = self._backup
259         else:
260             print("Exiting DataFrame context normally")
261         del self._backup
262         return False  # Don't suppress exceptions
263
264     # Iterator Magic Methods
265     def __iter__(self):
266         """Iterate over rows."""
267         self._iter_index = 0
268         return self
269
270     def __next__(self):
271         """Get next row."""
272         if self._iter_index >= len(self._data):
273             raise StopIteration
274
275         row = dict(zip(self.columns, self._data[self._iter_index]))
276         self._iter_index += 1
277         return row
278
279     # Utility methods
280     @property
281     def shape(self):
282         """Return DataFrame shape."""
283         return self._shape
284
285     @property
286     def dtypes(self):
287         """Return column data types."""
288         return self._dtypes.copy()
289
290     def head(self, n=5):
291         """Return first n rows."""
292         return DataFrame(self._data[:n], self.columns.copy())
```

```
293
294     def tail(self, n=5):
295         """Return last n rows."""
296         return DataFrame(self._data[-n:], self.columns.copy())
297
298     def describe(self):
299         """Generate descriptive statistics."""
300         stats = {}
301         for i, col in enumerate(self.columns):
302             col_data = [row[i] for row in self._data if isinstance(row[i], (int,
     float))]
303             if col_data:
304                 stats[col] = {
305                     'count': len(col_data),
306                     'mean': sum(col_data) / len(col_data),
307                     'min': min(col_data),
308                     'max': max(col_data),
309                     'std': (sum((x - sum(col_data)/len(col_data))**2 for x in
    col_data) / len(col_data))**0.5
310                 }
311         return stats
312
313 # Usage Example
314 print("=" * 50)
315 print("DEMONSTRATING MAGIC METHODS")
316 print("=" * 50)
317
318 # Create DataFrame
319 data = [
320     [1, 2, 3],
321     [4, 5, 6],
322     [7, 8, 9],
323     [10, 11, 12]
324 ]
325 df = DataFrame(data, columns=['A', 'B', 'C'])
326 print(df)
327 print()
328
329 # Container operations
330 print("Row 0:", df[0])
331 print("Column A:", df['A'])
332 print("First 2 rows:\n", df[:2])
333 print()
334
335 # Assignment
336 df['D'] = [10, 20, 30, 40]
337 print("After adding column D:\n", df.head(2))
338 print()
339
340 # Arithmetic operations
341 df2 = df + 10
342 print("df + 10:\n", df2.head(2))
343 print()
344
345 # Comparisons
346 print("df > 5:\n", (df > 5).head(2))
347 print()
348
349 # Iteration
350 print("Iterating over rows:")
351 for row in df.head(2):
352     print(row)
353 print()
```

```

# Context manager
with df as d:
    d['E'] = [100, 200, 300, 400]
    print("Inside context:\n", d.head(2))
print()

# Callable
squared = df(lambda x: x**2)
print("Squared values:\n", squared.head(2))
print()

# Statistics
print("Descriptive statistics:")
stats = df.describe()
for col, stat in stats.items():
    print(f"{col}: {stat}")
```

Properties for Controlled Access

Properties provide getter/setter logic while maintaining simple attribute syntax.

```
class TimeSeriesData:
    """Time series data with validation and automatic alignment."""

    def __init__(self, dates, values, name="Series"):
        self._dates = list(dates)
        self._values = list(values)
        self._name = name
        self._freq = None
        self._cached_stats = {}

        # Validate on initialization
        self._validate_data()

    def _validate_data(self):
        """Validate time series data."""
        if len(self._dates) != len(self._values):
            raise ValueError("Dates and values must have same length")

        if len(self._dates) == 0:
            raise ValueError("Empty time series")

        # Check date order
        for i in range(1, len(self._dates)):
            if self._dates[i] <= self._dates[i-1]:
                raise ValueError("Dates must be strictly increasing")

    @property
    def name(self):
        """Get series name."""
        return self._name

    @name.setter
    def name(self, value):
        """Set series name with validation."""
        if not isinstance(value, str):
            raise TypeError("Name must be a string")
        if not value:
            raise ValueError("Name cannot be empty")
        self._name = value
        self._cached_stats.clear()  # Invalidate cache
```

```

    @property
    def values(self):
        """Get values (read-only copy)."""
        return self._values.copy()

    @property
    def dates(self):
        """Get dates (read-only copy)."""
        return self._dates.copy()

    @property
    def length(self):
        """Get series length."""
        return len(self._values)

    @property
    def start_date(self):
        """Get first date."""
        return self._dates[0]

    @property
    def end_date(self):
        """Get last date."""
        return self._dates[-1]

    @property
    def date_range(self):
        """Get date range as tuple."""
        return (self.start_date, self.end_date)

    @property
    def freq(self):
        """Infer frequency from dates."""
        if self._freq is None and len(self._dates) > 1:
            # Simple frequency inference
            diffs = [self._dates[i] - self._dates[i-1] for i in range(1, len(
self._dates))]
            if all(d == diffs[0] for d in diffs):
                self._freq = diffs[0]
            else:
                self._freq = None
        return self._freq
```

Exception Handling

Theory

Exceptions are errors detected during program execution. Python provides a robust mechanism to handle these errors gracefully so that programs do not crash unexpectedly. Exception handling allows developers to detect errors and respond to them in a controlled manner.

try block: Encloses code that might raise an exception.

except block: Catches and handles specific exceptions.

else block: Executes if no exception occurs in the `try` block.

finally block: Always executes regardless of whether an exception occurred. It is ideal for cleanup tasks such as closing files or releasing resources.

raise: Used to manually trigger an exception.

Common Built-in Exceptions

- **ZeroDivisionError**: Raised when division by zero occurs.
- **ValueError**: Raised when a function receives an invalid value.
- **TypeError**: Raised when an operation is performed on an inappropriate data type.
- **FileNotFoundError**: Raised when a specified file does not exist.
- **KeyError**: Raised when a dictionary key is not found.
- **IndexError**: Raised when an invalid index is used in a list.

Example Programs

Basic try/except with else and finally

```
def divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("Error: Cannot divide by zero.")
        result = None
    else:
        print("Division successful.")
    finally:
        print("Cleanup actions (if any) go here.")
    return result

print(divide(10, 2))   # Output: 5.0
print(divide(10, 0))   # Error: Cannot divide by zero. None
```

Handling Multiple Exception Types

```
def read_file(filename):
    try:
        with open(filename, 'r') as f:
            return f.read()
    except FileNotFoundError:
        print(f"File '{filename}' not found.")
    except PermissionError:
        print(f"Permission denied for '{filename}'.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
```

Raising Exceptions Manually

Sometimes programmers need to deliberately trigger an exception when invalid conditions occur. Python provides the `raise` keyword to manually generate exceptions. This is useful for enforcing constraints and validating input data.

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    print(f"Age set to {age}")
```

Using `else` and `finally` Together

The `else` block executes only when no exception occurs in the `try` block. The `finally` block always executes regardless of whether an exception occurred or not. This combination is useful for performing operations after successful execution while still ensuring cleanup activities are performed.

```
def process_data(data):
    try:
        value = int(data)
    except ValueError:
        print("Invalid data: not an integer")
    else:
        print(f"Processing integer: {value}")
    finally:
        print("Data processing attempt finished.")
```

Debugging

Debugging is the process of identifying, analyzing, and fixing errors (bugs) in a program. It is an essential skill in software development because even well-written programs may contain logical, syntax, or runtime errors. Python provides several tools and techniques to assist in debugging.

Print Debugging: One of the simplest debugging techniques is inserting `print()` statements in the code to track variable values and program execution flow.

Assertions: Assertions are used to check assumptions in the program. The syntax `assert condition, message` verifies that the condition is true. If the condition evaluates to false, Python raises an `AssertionError`.

Using `pdb` (Python Debugger): Python includes a built-in interactive debugger called `pdb`. It allows developers to pause execution and inspect variables and program flow.

- `pdb.set_trace()`: Sets a breakpoint in the program.
- `n`: Execute the next line.
- `c`: Continue execution until the next breakpoint.
- `p variable`: Print the value of a variable.
- `q`: Quit the debugger.

Post-mortem Debugging: This technique starts the debugger automatically after an exception occurs. For example, in IPython the command `%pdb` enables automatic debugging after errors.

IDE Debuggers: Modern development environments such as VS Code and PyCharm provide graphical debugging tools including breakpoints, step execution, variable inspection, and watch expressions.

Example Programs

Using Assertions

```
def average(numbers):
    assert len(numbers) > 0, "List must not be empty"
    return sum(numbers) / len(numbers)

# average([])  # Would raise AssertionError
```

Inserting a Breakpoint with pdb

```
import pdb

def buggy_function(x):
    y = x + 1
    pdb.set_trace()          # Execution pauses here
    z = y * 2
    return z

result = buggy_function(5)

# At the (Pdb) prompt, you can inspect variables:
# p y
# p x
# then continue with:
# c
```

Post-mortem Debugging in IPython

```
# Enable automatic debugger on exception
%pdb on

def faulty():
    return 1 / 0

faulty()   # After ZeroDivisionError, the debugger opens automatically
```

Using breakpoint() (Python 3.7+)

```
def compute(a, b):
    breakpoint()   # Same as pdb.set_trace() but more convenient
    return a / b
```

Logging

Logging is the preferred way to record runtime information in software applications. It is more flexible and configurable than using simple `print()` statements. Logging helps developers monitor program behavior, diagnose problems, and maintain a record of application events.

Logging Levels (in increasing severity):

- **DEBUG**: Detailed diagnostic information useful for debugging.
- **INFO**: Confirmation that the program is functioning as expected.
- **WARNING**: Indicates a potential problem or unusual situation.
- **ERROR**: A serious issue that prevents part of the program from working.
- **CRITICAL**: A severe error indicating the program may not continue running.

Basic Configuration: The function `logging.basicConfig()` is used to configure the logging system. It allows the developer to specify the logging level, output format, and destination (such as console or file).

Components of Logging:

- **Logger**: The object responsible for generating log messages.
- **Handler**: Determines where the log messages are sent (e.g., console or file).
- **Formatter**: Defines the structure and appearance of log messages.

Best Practices:

- Use loggers named after modules using `__name__`.
- Avoid using `print()` statements in production code; use logging instead.
- Configure logging once at the start of the application.

Example Programs

Basic Logging to Console

```
import logging

logging.basicConfig(level=logging.INFO,
format='%(asctime)s - %(levelname)s - %(message)s')

def process(data):
    logging.info("Starting processing")
    try:
        result = data / 2
    except Exception as e:
        logging.error(f"Error: {e}")
    else:
        logging.debug(f"Result: {result}")   # Won't appear (level=INFO)
        return result

process(10)
```

Logging to a File with Custom Format

```
import logging

logging.basicConfig(
    filename='app.log',
    level=logging.DEBUG,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)

def read_data(filepath):
    logging.info(f"Attempting to read {filepath}")
    try:
        with open(filepath) as f:
            data = f.read()
        logging.debug(f"Read {len(data)} characters")
        return data
    except FileNotFoundError:
        logging.error(f"File {filepath} not found")
```

Using Multiple Handlers (Console + File)

```
import logging

# Create logger
logger = logging.getLogger('my_app')
logger.setLevel(logging.DEBUG)

# Console handler (only warnings and above)
c_handler = logging.StreamHandler()
c_handler.setLevel(logging.WARNING)
c_format = logging.Formatter('%(name)s - %(levelname)s - %(message)s')
c_handler.setFormatter(c_format)

# File handler (all levels)
f_handler = logging.FileHandler('app.log')
f_handler.setLevel(logging.DEBUG)
f_format = logging.Formatter('%(asctime)s - %(name)s - %(levelname)s - %(message
    )s')
f_handler.setFormatter(f_format)

# Add handlers
logger.addHandler(c_handler)
logger.addHandler(f_handler)

# Test
logger.debug('Debug message (file only)')
logger.warning('Warning message (both)')
```

Logging in a Data Processing Context

```
import logging
import pandas as pd

logging.basicConfig(
level=logging.INFO,
filename='pipeline.log',
format='%(asctime)s - %(message)s'
)

def load_and_clean(csv_file):
    logging.info(f"Loading {csv_file}")
    df = pd.read_csv(csv_file)
    initial = len(df)
    df.dropna(inplace=True)
    dropped = initial - len(df)
    logging.info(f"Dropped {dropped} rows with missing values")
    return df
```

Summary for Quick Revision

Exception Handling

Key Elements:

- `try`, `except`, `else`, and `finally` blocks
- `raise` keyword for manually triggering exceptions
- Built-in exceptions such as `ZeroDivisionError`, `ValueError`, `TypeError`, and `FileNotFoundError`

Common Pitfalls:

- Catching exceptions too broadly (e.g., using a generic `except`)
- Forgetting to use `finally` for cleanup operations such as closing files

Debugging

Key Elements:

- `print()` statements for tracing program execution
- `assert` statements for validating assumptions
- `pdb` debugger and `breakpoint()` for interactive debugging
- IDE debugging tools such as breakpoints, step execution, and variable inspection

Common Pitfalls:

- Excessive reliance on `print()` debugging
- Leaving debugging statements in production code

Logging

Key Elements:

- Python `logging` module
- Logging levels: `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`
- Handlers for directing logs to files or console
- Formatters for customizing log message structure
- Proper configuration using `logging.basicConfig()`

Common Pitfalls:

- Using `print()` instead of logging in real applications
- Not setting appropriate logging levels
- Misconfiguring handlers or logging formats

Working with Modules and Packages in Python

Modules and packages are fundamental for organizing and reusing code in Python. They allow developers to divide large programs into smaller, manageable components that can be reused across different projects.

1.1 What is a Module?

A module is a single `.py` file that contains Python definitions and statements. It can define functions, classes, variables, and may also include executable code.

Modules provide the following advantages:

- Help break large programs into smaller, manageable components.
- Promote code reuse across multiple programs.
- Improve maintainability and readability of software projects.

1.2 What is a Package?

A package is a collection of modules organized in directories. A directory must contain a special file named `__init__.py` to be recognized as a package.

- The `__init__.py` file can be empty or contain initialization code.
- Packages can include subpackages (nested directories).
- Packages help organize large codebases into logical groups.

1.3 Importing Modules

Python provides multiple ways to import modules.

- `import module_name` – imports the whole module. Access items using `module_name.item`.
- `from module_name import item1, item2` – imports specific items into the current namespace.
- `from module_name import *` – imports all public items (defined by `__all__`). This approach should generally be avoided in production code.
- `import module_name as alias` – imports a module with an alias.
- `from package import module` – imports a module from a package.
- `from package.module import item` – imports a specific item from a module inside a package.

1.4 Module Search Path (`sys.path`)

When Python imports a module, it searches in the following locations:

- The directory containing the script or the current working directory.
- Directories listed in the `PYTHONPATH` environment variable.
- Installation-dependent default paths such as the standard library and `site-packages`.

Although it is possible to modify `sys.path` at runtime, this practice is generally discouraged.

1.5 `__name__` and `__main__`

Every Python module contains a special variable named `__name__`.

- When a module is executed directly using `python module.py`, `__name__` is set to `"__main__"`.
- When the module is imported, `__name__` is set to the module's actual name.

A common pattern is:

```
if __name__ == "__main__":
    # Code that runs only when the file is executed directly
```

1.6 `__init__.py` and Package Initialization

The file `__init__.py` is executed when a package is imported.

- It may be empty or contain initialization logic.
- It can define the variable `__all__` to control exported names.
- It may import submodules to make them accessible directly from the package.

1.7 `__all__`

`__all__` is a list of strings that defines which names are exported when using:

```
from module import *
```

If `__all__` is not defined, Python imports all names that do not start with an underscore.

1.8 Relative Imports

Within a package, relative imports can be used.

- `from . import module` – import from the same package.
- `from .. import module` – import from the parent package.
- `from .subpackage import module`

Relative imports work only inside packages and not in top-level scripts.

2. Example Programs

2.1 Creating and Using a Simple Module

File: mymath.py

```
# mymath.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

PI = 3.14159
```

Using the module

```
import mymath

print(mymath.add(5, 3))
print(mymath.PI)

from mymath import subtract
print(subtract(10, 4))
```

2.2 Creating a Package

Directory Structure

```
mycalculator/
    __init__.py
    basic.py
    advanced/
        __init__.py
        stats.py
        trig.py
```

File: mycalculator/__init__.py

```
from .basic import add, subtract
```

File: mycalculator/basic.py

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

File: mycalculator/advanced/stats.py

```
def mean(data):
    return sum(data) / len(data)
```

Using the package

```
import mycalculator
from mycalculator.basic import add
from mycalculator.advanced.stats import mean

print(mycalculator.add(2, 3))
print(add(10, 5))
print(mean([1, 2, 3, 4]))
```

2.3 Relative Imports Inside a Package

```
# mycalculator/advanced/trig.py
from .stats import mean

def some_function():
    pass
```

2.4 Using __all__ to Control Import

```
__all__ = ['add', 'PI']

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

PI = 3.14159
SECRET = "hidden"
```

2.5 Running a Module as a Script

```python
def say_hello(name):
    return f"Hello, {name}!"

if __name__ == "__main__":
    print(say_hello("Exam Student"))
```

3. Summary for Quick Revision

Modules

Key Elements

- `.py` files
- `import`, `from ... import`, `as`
- `__name__` and `if __name__ == "__main__"`

Common Pitfalls

- Circular imports between modules
- Modifying `sys.path` unnecessarily
- Forgetting the `__name__` guard causing code to run during import

Packages

Key Elements

- Directory containing `__init__.py`
- Subpackages
- `__all__`
- Relative imports using `.` and `..`

Common Pitfalls

- Missing `__init__.py` (in older Python versions)
- Incorrect relative import syntax
- Not exposing submodules in `__init__.py` when needed

Import System

Key Elements

- `sys.path`
- Search order
- `PYTHONPATH`
- Import caching using compiled `.pyc` files

Common Pitfalls

- Naming conflicts with standard library modules
- Importing from unexpected paths
- Relying on the current working directory

Chapter-2
Data Sources and APIs

Data sources refer to the origins, locations, or systems from which data is obtained for processing, analysis, and interpretation. They encompass a wide spectrum of storage mechanisms, formats, and delivery methods, each with distinct characteristics that influence how data must be accessed, extracted, and prepared for analysis.

Classification of Data Sources

Based on reference literature, data sources can be classified along several dimensions.

By Structure

- **Structured Data Sources:** Data with a predefined schema, typically organized in tables with rows and columns. Examples include relational databases and CSV files.
- **Semi-structured Data Sources:** Data with a flexible structure that uses tags, markers, or key-value pairs to organize information. Examples include JSON, XML, and HTML.
- **Unstructured Data Sources:** Data without a predefined structure. Examples include text files, images, audio, and video.

By Access Method

- **File-based Sources:** Data stored in files that can be accessed locally or over a network, such as CSV, Excel, or text files.
- **Database Sources:** Data stored in databases and accessed using query languages such as SQL or through NoSQL systems.
- **API Sources:** Data obtained through web service endpoints using protocols such as REST or SOAP.
- **Stream Sources:** Continuous data flows generated in real time, such as sensor data or system logs.

By Storage Location

- **Local Sources:** Data stored on a local machine or device.
- **Network Sources:** Data accessed through network resources such as shared servers or distributed systems.
- **Cloud Sources:** Data stored and managed through cloud-based services such as Amazon S3, Google Cloud Storage, or Microsoft Azure.

By Data Origin

- **Primary Sources:** Data collected firsthand for a specific purpose, such as surveys, experiments, or sensor measurements.
- **Secondary Sources:** Data obtained from existing repositories or previously collected datasets, such as public datasets or institutional databases.
- **Third-party Sources:** Data provided by external organizations, such as commercial data providers or public APIs.

Reading and Writing Structured and Unstructured Data

CSV Files (Comma-Separated Values)

Definition and Characteristics

CSV files represent tabular data where each line corresponds to a row, and fields within each row are separated by commas. This format originated in early computing as a simple way to exchange data between different systems and applications.

Historical Context

CSV emerged as a portable format when transferring data between incompatible database systems. Its simplicity became its greatest strength—any system that can read and write text can process CSV files.

Structural Components

- **Records:** Each line represents one record or row of data.
- **Fields:** Individual data elements separated by delimiters.
- **Header Row (optional):** First row containing column names.
- **Footer Row (optional):** Last row containing summaries or metadata.

Important Theoretical Considerations

Delimiters

Although commas are standard delimiters, other separators may be used:

- Tabs (TSV – Tab-Separated Values)
- Pipes (`|`)
- Semicolons (`;`)
- Spaces in fixed-width formatted exports

Headers

The first row often contains column names, although this is not mandatory. When headers are absent, columns are referenced using index positions (0, 1, 2, ...).

Quoting Mechanisms

- **QUOTE_ALL**
- **QUOTE_MINIMAL**
- **QUOTE_NONNUMERIC**
- **QUOTE_NONE**

Best Practices for Working with CSV

- Inspect the first few rows before loading the full dataset.
- Check delimiter consistency.
- Specify encoding such as UTF-8.
- Handle missing values explicitly.
- Use proper error handling.

JSON (JavaScript Object Notation)

Theoretical Background

JSON is a widely used data interchange format, especially for APIs and web services. Although derived from JavaScript syntax, it is now language-independent.

Structural Components

Objects

Objects contain key-value pairs enclosed in curly braces.

```
{"name": "Alice", "age": 30}
```

Arrays

Arrays are ordered collections enclosed in square brackets.

```
["apple", "banana", 42, true]
```

Advantages of JSON

- Human-readable format
- Lightweight structure
- Flexible schema
- Language independent

Excel Files

File Format Evolution

- XLS – legacy binary format
- XLSX – XML-based modern format
- XLSM – macro-enabled format

Unique Challenges

- Multiple sheets
- Mixed data types
- Hidden rows and columns
- Formatting interference

Text Files

Conceptual Framework

Text files represent sequences of characters encoded in a specific encoding format.

Classification of Text Files

Structured Text

- CSV
- TSV
- Fixed-width files
- XML or HTML

Semi-structured Text

- Log files
- Configuration files
- JSON

Unstructured Text

- Emails
- Articles
- Social media posts
- Transcripts

Common Text File Formats

- LOG
- INI/CFG
- Markdown
- reStructuredText
- LaTeX

1.1 Reading CSV Files using CSV module

A. Reading as Lists (csv.reader)

```
import csv

# Basic CSV reading - returns rows as lists
with open('employees.csv', 'r', encoding='utf-8') as file:
    csv_reader = csv.reader(file)

    # Get the header row (first row)
    header = next(csv_reader)
    print(f"Header: {header}")

    # Read and process data rows
    print("\nEmployee Data:")
    for row_num, row in enumerate(csv_reader, start=2):
        print(f"Row {row_num}: {row}")
        # Access by index: row[0] = name, row[1] = age, etc.
```

B. Reading as Dictionaries (csv.DictReader)

```
import csv

# Reading CSV with DictReader - rows as dictionaries
with open('employees.csv', 'r', encoding='utf-8') as file:
    csv_reader = csv.DictReader(file)

    # Access field names
    print(f"Columns: {csv_reader.fieldnames}")

    # Read and process data rows
    for row in csv_reader:
        print(f"Name: {row['name']}, Age: {row['age']}, "
              f"Department: {row['department']}")
```

1.2 Writing CSV Files

A. Writing with csv.writer

```
import csv

# Data to write: header row + data rows
data = [
    ['name', 'age', 'department', 'salary'],  # Header
```

```
    ['Alice Brown', 32, 'Engineering', 82000],
    ['Charlie Wilson', 41, 'Sales', 91000],
    ['Diana Prince', 29, 'Marketing', 67000]
]

# Write to CSV file
with open('output.csv', 'w', newline='', encoding='utf-8') as file:
    writer = csv.writer(file)

    # Write header
    writer.writerow(data[0])

    # Write all data rows at once
    writer.writerows(data[1:])

print("File written successfully")
```

B. Writing with csv.DictWriter

```
import csv

# Data as list of dictionaries
employees = [
    {'name': 'Eve Adams', 'age': 27, 'department': 'Engineering', 'salary': 72000},
    {'name': 'Frank Miller', 'age': 38, 'department': 'Sales', 'salary': 85000},
    {'name': 'Grace Lee', 'age': 31, 'department': 'Marketing', 'salary': 68000}
]

# Write using DictWriter
with open('employees_dict.csv', 'w', newline='', encoding='utf-8') as file:
    fieldnames = ['name', 'age', 'department', 'salary']
    writer = csv.DictWriter(file, fieldnames=fieldnames)

    # Write header
    writer.writeheader()

    # Write all rows
    writer.writerows(employees)
```

Using pandas

2.1 Reading CSV Files with pandas

A. Basic Reading

```
import pandas as pd

# Simple CSV read
df = pd.read_csv('employees.csv')

print("DataFrame from CSV:")
print(df.head())  # First 5 rows

print(f"\nShape: {df.shape}")
print(f"\nData Types:\n{df.dtypes}")
```

B. Reading with Common Parameters

```
import pandas as pd

# Reading with various options
df = pd.read_csv('employees.csv',
                 index_col=0,
                 header=0,
                 names=['name', 'age', 'dept', 'salary'],
                 skiprows=[1,2],
                 nrows=100,
                 usecols=['name', 'salary'],
                 dtype={'age': int, 'salary': float},
                 parse_dates=['hire_date'],
                 na_values=['NA', 'Missing'],
                 encoding='utf-8')

print(df.head())
```

C. Handling Missing Values

```
import pandas as pd
import numpy as np

df = pd.read_csv('data_with_missing.csv',
                 na_values=['', 'NA', 'NULL', '-999'],
                 keep_default_na=True)

print("Missing values per column:")
print(df.isnull().sum())
```

2.2 Writing CSV Files with pandas

A. Basic Writing

```
import pandas as pd

data = {
    'name': ['Alice', 'Bob', 'Charlie', 'Diana'],
    'age': [25, 30, 35, 28],
    'department': ['HR', 'IT', 'IT', 'HR'],
    'salary': [50000, 60000, 65000, 52000]
}

df = pd.DataFrame(data)

df.to_csv('output_pandas.csv',
          index=False,
          encoding='utf-8')
```

B. Writing with Common Parameters

```
df.to_csv('output_formatted.csv',
          index=False,
          header=True,
          columns=['name', 'salary'],
          sep=';',
          na_rep='NULL',
```

```
            float_format='%.2f',
            encoding='utf-8-sig',
            date_format='%Y-%m-%d')
```

Operation	csv Module	pandas
Read	`csv.reader()` `csv.DictReader()`	`pd.read_csv()`
Write	`csv.writer()` `csv.DictWriter()`	`df.to_csv()`
Parameters	`delimiter, quotechar`	`sep, encoding, dtype`
Output Control	`writerow(), writerows()`	`index, header, columns`

Table 1: Comparison between Python csv module and pandas for CSV operations

Database Access with Relational and Non-Relational Databases

Introduction to Database Access

Database access in Python follows a standard convention where data is represented as sequences of tuples. All database interactions occur through SQL queries, with each input or output row represented as a tuple.

Key Components:

- **Connection Objects:** Represent the communication channel between Python and the database, handling authentication, network communication, and session management.
- **Cursor Objects:** Execute SQL statements and fetch results, providing query execution and result set iteration.
- **Parameterized Queries:** Using placeholders (?, %s, or named parameters) instead of string formatting to prevent SQL injection attacks.
- **Transactions:** Groups of operations that execute as a single unit with commit or rollback capabilities.

Relational Databases

Definition:

A relational database is a collection of tables containing data organized by relations between data items. Relationships can be established between rows in different tables or between columns inside a table.

Core Concepts:

Concept	Description
Table	Collection of related data organized in rows and columns
Row (Record)	Single entry in a table representing one instance
Column (Field)	Specific attribute of the data
Primary Key	Uniquely identifies each row in a table
Foreign Key	Establishes relationships between tables
Index	Improves query performance
View	Virtual table based on SELECT query
Schema	Defines table structure and relationships

ACID Properties:

- **Atomicity:** Transactions are all-or-nothing; if any part fails, entire transaction rolls back.
- **Consistency:** Transactions maintain database rules and constraints.
- **Isolation:** Concurrent transactions do not interfere with each other.
- **Durability:** Committed transactions persist even after system failures.

SQL Categories:

Category	Purpose	Examples
DDL	Define structure	CREATE, ALTER, DROP
DML	Manipulate data	SELECT, INSERT, UPDATE, DELETE
DCL	Control access	GRANT, REVOKE
TCL	Manage transactions	COMMIT, ROLLBACK

Common Relational Databases:

- SQLite (embedded, serverless)
- MySQL / MariaDB
- PostgreSQL
- Oracle
- Microsoft SQL Server

Non-Relational Databases (NoSQL)

Definition:
Not Only SQL (NoSQL) databases are designed for big data and web applications. They allow data storage without fixed schemas and are optimized for specific data models and access patterns.

Types of NoSQL Databases:

- **Document Stores**
  - Description: Data stored as JSON-like documents
  - Examples: MongoDB, CouchDB
  - Use Cases: Content management, catalogs
- **Key-Value Stores**
  - Description: Simple key-value pairs
  - Examples: Redis, DynamoDB
  - Use Cases: Caching, session storage
- **Column-Family Stores**
  - Description: Data stored in columns rather than rows
  - Examples: Cassandra, HBase
  - Use Cases: Time-series data, analytics

- **Graph Databases**
  - Description: Data stored as nodes and edges representing relationships
  - Examples: Neo4j, ArangoDB
  - Use Cases: Social networks, recommendation systems

Key Characteristics:

- **Schema-less Design:** Documents in the same collection can have different fields.
- **Horizontal Scaling:** Designed to distribute across many servers.
- **BASE Properties:** Basically Available, Soft state, Eventually consistent.
- **Denormalization:** Data duplication accepted for query performance.
- **Embedded Documents:** Related data can be nested rather than split across tables.

BASE vs ACID:

ACID	BASE
Strong consistency	Weak consistency (eventual)
Isolation	Availability first
Focus on commit	Focus on continue
Pessimistic	Optimistic
Complex	Simple

RELATIONAL DATABASE PROGRAMS

SQLite Programs

A. Basic SQLite Operations

```
import sqlite3

# Connect to database
conn = sqlite3.connect('company.db')
print("Connected to database successfully")

# Create a cursor
cursor = conn.cursor()

# Create tables
cursor.execute('''
    CREATE TABLE IF NOT EXISTS departments (
        dept_id INTEGER PRIMARY KEY,
        dept_name TEXT NOT NULL,
        location TEXT
    )
''')

cursor.execute('''
    CREATE TABLE IF NOT EXISTS employees (
        emp_id INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        age INTEGER,
        salary REAL,
        dept_id INTEGER,
```

```
        hire_date TEXT,
        FOREIGN KEY (dept_id) REFERENCES departments (dept_id)
    )
''')
conn.commit()
print("Tables created successfully")

# Insert data
departments = [
    (1, 'Engineering', 'Building A'),
    (2, 'Marketing', 'Building B'),
    (3, 'Sales', 'Building C')
]
cursor.executemany('INSERT INTO departments VALUES (?, ?, ?)', departments)

employees = [
    (101, 'Alice Brown', 32, 82000, 1, '2022-03-15'),
    (102, 'Bob Smith', 45, 95000, 1, '2021-06-01'),
    (103, 'Charlie Wilson', 38, 72000, 2, '2023-01-10'),
    (104, 'Diana Prince', 29, 68000, 3, '2023-08-22')
]
cursor.executemany('INSERT INTO employees VALUES (?, ?, ?, ?, ?, ?)', employees
    )
conn.commit()
print(f"Inserted {cursor.rowcount} rows")
```

B. Querying Data

```
import sqlite3

conn = sqlite3.connect('company.db')
cursor = conn.cursor()

# SELECT all employees
cursor.execute('SELECT * FROM employees')
all_employees = cursor.fetchall()
print("All Employees:")
for emp in all_employees:
    print(f"ID: {emp[0]}, Name: {emp[1]}, Salary: ${emp[4]}")

# SELECT with condition
cursor.execute('SELECT name, salary FROM employees WHERE dept_id = ?', (1,))
engineers = cursor.fetchall()
print("\nEngineering Department Employees:")
for name, salary in engineers:
    print(f"{name}: ${salary}")

# SELECT with JOIN
cursor.execute('''
    SELECT e.name, e.salary, d.dept_name, d.location
    FROM employees e
    JOIN departments d ON e.dept_id = d.dept_id
    WHERE salary > ?
''', (70000,))
high_earners = cursor.fetchall()
print("\nEmployees earning > $70,000:")
for name, salary, dept, loc in high_earners:
    print(f"{name} ({dept}): ${salary}")

# Aggregation queries
cursor.execute('''
    SELECT d.dept_name, COUNT(*) as emp_count, AVG(e.salary) as avg_salary
```

```
    FROM employees e
    JOIN departments d ON e.dept_id = d.dept_id
    GROUP BY d.dept_name
''')
stats = cursor.fetchall()
print("\nDepartment Statistics:")
for dept, count, avg in stats:
    print(f"{dept}: {count} employees, Avg Salary: ${avg:.2f}")
```

C. Update and Delete Operations

```
import sqlite3

conn = sqlite3.connect('company.db')
cursor = conn.cursor()

# UPDATE operation
cursor.execute('''
    UPDATE employees
    SET salary = salary * 1.10
    WHERE dept_id = ?
''', (2,))
conn.commit()
print(f"Updated {cursor.rowcount} rows in Marketing department")

# DELETE operation
cursor.execute('DELETE FROM employees WHERE name = ?', ('Bob Smith',))
conn.commit()
print(f"Deleted {cursor.rowcount} rows")

# Parameterized queries
def update_employee_salary(emp_id, new_salary):
    cursor.execute('''
        UPDATE employees
        SET salary = ?
        WHERE emp_id = ?
    ''', (new_salary, emp_id))
    conn.commit()
    return cursor.rowcount

update_employee_salary(101, 85000)
print("Employee salary updated")

conn.close()
```

D. Transaction Management

```
import sqlite3

def transfer_employee(emp_id, new_dept_id):
    conn = sqlite3.connect('company.db')
    cursor = conn.cursor()

    try:
        # Check if employee exists
        cursor.execute('SELECT name FROM employees WHERE emp_id = ?', (emp_id,)
    )
        if not cursor.fetchone():
            raise Exception("Employee not found")

```

```
        # Check if department exists
        cursor.execute('SELECT dept_name FROM departments WHERE dept_id = ?', (new_dept_id,))
        if not cursor.fetchone():
            raise Exception("Department not found")

        # Update employee department
        cursor.execute('''
            UPDATE employees
            SET dept_id = ?
            WHERE emp_id = ?
        ''', (new_dept_id, emp_id))

        conn.commit()
        print(f"Employee {emp_id} transferred successfully")

    except Exception as e:
        conn.rollback()
        print(f"Transaction failed: {e}")

    finally:
        conn.close()

# Test the transaction
transfer_employee(101, 3)
```

E. Using pandas with SQLite

```
import sqlite3
import pandas as pd

conn = sqlite3.connect('company.db')

# Read SQL query into DataFrame
df_employees = pd.read_sql_query("SELECT * FROM employees", conn)
print("Employees DataFrame:")
print(df_employees.head())

# Read with conditions
df_high_salary = pd.read_sql_query('''
    SELECT e.name, e.salary, d.dept_name
    FROM employees e
    JOIN departments d ON e.dept_id = d.dept_id
    WHERE salary > 70000
''', conn)
print("\nHigh Salary Employees:")
print(df_high_salary)

# Write DataFrame to SQL table
new_employees = pd.DataFrame({
    'name': ['Eve Adams', 'Frank Miller'],
    'age': [27, 41],
    'salary': [72000, 89000],
    'dept_id': [1, 3],
    'hire_date': ['2024-01-15', '2024-02-01']
})
new_employees.to_sql('employees', conn, if_exists='append', index=False)
print("\nNew employees added to database")

conn.close()
```

MySQL/PostgreSQL Programs

A. MySQL Connection and Operations

```python
import mysql.connector
from mysql.connector import Error

try:
    # Establish connection
    conn = mysql.connector.connect(
        host='localhost',
        database='company_db',
        user='root',
        password='password',
        port=3306
    )

    if conn.is_connected():
        print("Connected to MySQL database")
        cursor = conn.cursor()

        # Create table
        cursor.execute('''
            CREATE TABLE IF NOT EXISTS products (
                product_id INT AUTO_INCREMENT PRIMARY KEY,
                product_name VARCHAR(100) NOT NULL,
                price DECIMAL(10,2),
                quantity INT,
                category VARCHAR(50)
            )
        ''')

        # Insert data
        insert_query = '''
            INSERT INTO products (product_name, price, quantity, category)
            VALUES (%s, %s, %s, %s)
        '''
        products = [
            ('Laptop', 999.99, 10, 'Electronics'),
            ('Mouse', 24.99, 50, 'Electronics'),
            ('Desk Chair', 199.99, 15, 'Furniture')
        ]
        cursor.executemany(insert_query, products)
        conn.commit()
        print(f"Inserted {cursor.rowcount} rows")

        # Query with conditions
        cursor.execute('''
            SELECT product_name, price, quantity
            FROM products
            WHERE category = %s AND price > %s
        ''', ('Electronics', 500))

        results = cursor.fetchall()
        for row in results:
            print(f"Product: {row[0]}, Price: ${row[1]}")

except Error as e:
    print(f"Error: {e}")
finally:
    if conn.is_connected():
        cursor.close()
        conn.close()
```

```
        print("MySQL connection closed")
```

B. PostgreSQL Connection

```
import psycopg2
from psycopg2 import sql

# Connect to PostgreSQL
conn = psycopg2.connect(
    host="localhost",
    database="company_db",
    user="postgres",
    password="password",
    port="5432"
)

cursor = conn.cursor()

# Create table with SERIAL for auto-increment
cursor.execute('''
    CREATE TABLE IF NOT EXISTS customers (
        customer_id SERIAL PRIMARY KEY,
        name VARCHAR(100) NOT NULL,
        email VARCHAR(100) UNIQUE,
        joined_date DATE DEFAULT CURRENT_DATE
    )
''')

# Insert using parameterized queries
cursor.execute('''
    INSERT INTO customers (name, email)
    VALUES (%s, %s) RETURNING customer_id
''', ('John Doe', 'john@email.com'))
customer_id = cursor.fetchone()[0]
conn.commit()
print(f"Inserted customer with ID: {customer_id}")

cursor.close()
conn.close()
```

MongoDB (Document Database)

A. Basic MongoDB Operations

```
from pymongo import MongoClient
from pymongo.errors import ConnectionFailure

# Connect to MongoDB
try:
    client = MongoClient('mongodb://localhost:27017/', serverSelectionTimeoutMS
    =5000)
    client.admin.command('ping')  # Test connection
    print("Connected to MongoDB successfully")

    db = client['company_db']
    employees_collection = db['employees']

    # Insert single document
    employee = {
        'emp_id': 101,
```

```
        'name': 'Alice Brown',
        'age': 32,
        'skills': ['Python', 'SQL', 'MongoDB'],
        'address': {'street': '123 Main St', 'city': 'New York', 'zip': '10001'},
        'department': 'Engineering',
        'salary': 82000,
        'is_active': True,
        'join_date': '2022-03-15'
    }

    result = employees_collection.insert_one(employee)
    print(f"Inserted document with ID: {result.inserted_id}")

    # Insert multiple documents
    new_employees = [
        {'emp_id': 102, 'name': 'Bob Smith', 'age': 45, 'skills': ['Java', 'AWS'],
         'department': 'Engineering', 'salary': 95000},
        {'emp_id': 103, 'name': 'Charlie Wilson', 'age': 38, 'skills': ['Marketing', 'SEO'],
         'department': 'Marketing', 'salary': 72000}
    ]
    result = employees_collection.insert_many(new_employees)
    print(f"Inserted {len(result.inserted_ids)} documents")

except ConnectionFailure as e:
    print(f"Could not connect to MongoDB: {e}")
```

B. Querying MongoDB

```
from pymongo import MongoClient

client = MongoClient('mongodb://localhost:27017/')
db = client['company_db']
employees = db['employees']

# Find one document
alice = employees.find_one({'name': 'Alice Brown'})
print("Found employee:")
print(alice)

# Find all documents
all_employees = employees.find()
print("\nAll employees:")
for emp in all_employees:
    print(f"{emp['name']} - {emp['department']}")

# Find with conditions
high_salary = employees.find({'salary': {'$gt': 80000}})
print("\nEmployees with salary > $80,000:")
for emp in high_salary:
    print(f"{emp['name']}: ${emp['salary']}")

# Multiple conditions
engineers_young = employees.find({
    '$and': [{'department': 'Engineering'}, {'age': {'$lt': 40}}]
})
print("\nEngineers under 40:")
for emp in engineers_young:
    print(f"{emp['name']} - Age: {emp['age']}")

```

```
# Projection (specific fields)
names_only = employees.find({}, {'name': 1, 'department': 1, '_id': 0})
print("\nNames and departments only:")
for emp in names_only:
    print(f"{emp['name']} works in {emp['department']}")

# Count documents
count = employees.count_documents({'department': 'Engineering'})
print(f"\nNumber of engineers: {count}")

# Sort results
sorted_emps = employees.find().sort('salary', -1).limit(3)
print("\nTop 3 highest paid:")
for emp in sorted_emps:
    print(f"{emp['name']}: ${emp['salary']}")
```

C. Update and Delete in MongoDB

```
from pymongo import MongoClient

client = MongoClient('mongodb://localhost:27017/')
db = client['company_db']
employees = db['employees']

# Update one document
result = employees.update_one(
    {'name': 'Alice Brown'},
    {'$set': {'salary': 90000, 'skills': ['Python', 'SQL', 'MongoDB', 'Data
    Science']}}
)
print(f"Modified {result.modified_count} document")

# Update multiple documents
result = employees.update_many(
    {'department': 'Engineering'},
    {'$inc': {'salary': 5000}}
)
print(f"Modified {result.modified_count} documents")

# Add new field to all documents
employees.update_many({}, {'$set': {'active_status': 'Active'}})

# Delete one document
result = employees.delete_one({'name': 'Bob Smith'})
print(f"Deleted {result.deleted_count} document")

# Delete many documents
result = employees.delete_many({'department': 'Marketing'})
print(f"Deleted {result.deleted_count} documents")

# Drop collection
employees.drop()
print("Collection dropped")
```

D. Advanced MongoDB Operations

```
from pymongo import MongoClient
from datetime import datetime

client = MongoClient('mongodb://localhost:27017/')
```

```
db = client['company_db']
orders = db['orders']

# Insert orders with nested documents
orders_data = [
    {
        'order_id': 1001,
        'customer': {'name': 'John Smith', 'email': 'john@email.com',
                     'address': {'city': 'New York', 'state': 'NY'}},
        'items': [{'product': 'Laptop', 'price': 999.99, 'quantity': 1},
                  {'product': 'Mouse', 'price': 24.99, 'quantity': 2}],
        'total': 1049.97,
        'date': datetime.now()
    },
    {
        'order_id': 1002,
        'customer': {'name': 'Jane Doe', 'email': 'jane@email.com',
                     'address': {'city': 'Boston', 'state': 'MA'}},
        'items': [{'product': 'Desk Chair', 'price': 199.99, 'quantity': 1}],
        'total': 199.99,
        'date': datetime.now()
    }
]
orders.insert_many(orders_data)

# Query nested documents
ny_orders = orders.find({'customer.address.city': 'New York'})
print("Orders from New York:")
for order in ny_orders:
    print(f"Order {order['order_id']} - Customer: {order['customer']['name']}")

# Query array elements
large_orders = orders.find({'items': {'$elemMatch': {'price': {'$gt': 500}}}})
print("\nOrders with items > $500:")
for order in large_orders:
    print(f"Order {order['order_id']} - Total: ${order['total']}")

# Aggregation pipeline
pipeline = [
    {'$unwind': '$items'},
    {'$group': {
        '_id': '$items.product',
        'total_quantity': {'$sum': '$items.quantity'},
        'total_revenue': {'$sum': {'$multiply': ['$items.price', '$items.
    quantity']}}
    }},
    {'$sort': {'total_revenue': -1}}
]

results = orders.aggregate(pipeline)
print("\nProduct sales summary:")
for result in results:
    print(f"Product: {result['_id']}")
    print(f"  Quantity: {result['total_quantity']}")
    print(f"  Revenue: ${result['total_revenue']:.2f}")

client.close()
```

Redis (Key-Value Store)

```
import redis

```

```
# Connect to Redis
r = redis.Redis(host='localhost', port=6379, db=0, decode_responses=True)

# Test connection
try:
    r.ping()
    print("Connected to Redis")
except redis.ConnectionError:
    print("Could not connect to Redis")

# Set key-value pairs
r.set('user:1001:name', 'Alice Brown')
r.set('user:1001:email', 'alice@email.com')
r.set('user:1001:age', 32)

# Get values
name = r.get('user:1001:name')
email = r.get('user:1001:email')
print(f"User: {name}, Email: {email}")

# Set multiple keys at once
r.mset({
    'user:1002:name': 'Bob Smith',
    'user:1002:email': 'bob@email.com',
    'user:1002:age': 45
})

# Check if key exists
if r.exists('user:1001:name'):
    print("User 1001 exists")

# Set with expiration (10 seconds)
r.setex('temp:session', 10, 'session_data')

# Working with lists
r.lpush('recent_searches:1001', 'laptop', 'mouse', 'keyboard')
searches = r.lrange('recent_searches:1001', 0, -1)
print(f"Recent searches: {searches}")

# Working with hashes
r.hset('user:1003', mapping={
    'name': 'Charlie Wilson',
    'email': 'charlie@email.com',
    'age': 38,
    'department': 'Engineering'
})

# Get all fields from hash
user_data = r.hgetall('user:1003')
print(f"User 1003 data: {user_data}")

# Get specific fields
user_name = r.hget('user:1003', 'name')
print(f"Name: {user_name}")

# Delete keys
r.delete('temp:session')
print("Deleted temporary key")

# Get all keys matching pattern
keys = r.keys('user:*')
print(f"All user keys: {keys}")

```

```
# Close connection
r.close()
```

Cassandra (Column-Family Store)

```
from cassandra.cluster import Cluster
from cassandra.query import SimpleStatement
from datetime import datetime
from uuid import uuid4

# Connect to Cassandra
cluster = Cluster(['127.0.0.1'])
session = cluster.connect()

# Create keyspace
session.execute("""
    CREATE KEYSPACE IF NOT EXISTS company_keyspace
    WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1}
""")

# Use keyspace
session.set_keyspace('company_keyspace')

# Create table
session.execute("""
    CREATE TABLE IF NOT EXISTS employee_timeline (
        emp_id UUID,
        event_time timestamp,
        event_type text,
        description text,
        PRIMARY KEY (emp_id, event_time)
    ) WITH CLUSTERING ORDER BY (event_time DESC)
""")

# Insert data
emp_id = uuid4()
session.execute("""
    INSERT INTO employee_timeline (emp_id, event_time, event_type, description)
    VALUES (%s, %s, %s, %s)
""", (emp_id, datetime.now(), 'LOGIN', 'User logged into system'))

# Batch insert
batch = []
for i in range(5):
    batch.append((emp_id, datetime.now(), 'ACTIVITY', f'Performed action {i}'))

prepared = session.prepare("""
    INSERT INTO employee_timeline (emp_id, event_time, event_type, description)
    VALUES (?, ?, ?, ?)
""")
for item in batch:
    session.execute(prepared, item)

# Query data
rows = session.execute("""
    SELECT * FROM employee_timeline
    WHERE emp_id = %s
    LIMIT 10
""", (emp_id,))

print("Employee timeline:")
for row in rows:
```

```
    print(f"{row.event_time}: {row.event_type} - {row.description}")

# Close connection
cluster.shutdown()
```

COMPARISON AND BEST PRACTICES

When to Use Each Database Type

- **Structured data with relationships:** Recommended Database: Relational (SQLite, PostgreSQL) Reason: ACID compliance, joins, constraints
- **Unstructured/semi-structured data:** Recommended Database: MongoDB Reason: Flexible schema, nested documents
- **Caching, session management:** Recommended Database: Redis Reason: Ultra-fast key-value access
- **Time-series data, analytics:** Recommended Database: Cassandra Reason: High write throughput, wide columns
- **Complex relationships:** Recommended Database: Neo4j Reason: Graph traversal optimization

SUMMARY OF DATABASE OPERATIONS

- **Connect:**
  - SQLite: `sqlite3.connect()`
  - MySQL/PostgreSQL: `mysql.connector.connect()` / `psycopg2.connect()`
  - MongoDB: `MongoClient()`
  - Redis: `redis.Redis()`
- **Insert:**
  - SQLite: `cursor.execute('INSERT ...')`
  - MySQL/PostgreSQL: Same as SQLite (parameterized query)
  - MongoDB: `collection.insert_one()`
  - Redis: `r.set()`
- **Query:**
  - SQLite: `cursor.execute('SELECT ...')`
  - MySQL/PostgreSQL: Same as SQLite
  - MongoDB: `collection.find()`
  - Redis: `r.get()`
- **Update:**
  - SQLite: `cursor.execute('UPDATE ...')`
  - MySQL/PostgreSQL: Same as SQLite
  - MongoDB: `collection.update_one()`

  - Redis: `r.set()`

- **Delete:**
  - SQLite: `cursor.execute('DELETE ...')`
  - MySQL/PostgreSQL: Same as SQLite
  - MongoDB: `collection.delete_one()`
  - Redis: `r.delete()`

- **With pandas:**
  - SQLite: `pd.read_sql_query()`
  - MySQL/PostgreSQL: Same as SQLite
  - MongoDB: `pd.DataFrame(list(collection.find()))`
  - Redis: Not applicable

Accessing and Processing Data from APIs (REST, SOAP)

Introduction to APIs

Definition: An API (Application Programming Interface) is a set of protocols, routines, and tools that allows different software applications to communicate with each other. In the context of web services, APIs enable programs to request data or services from remote servers over HTTP, providing a standardized way to access functionality without needing to understand the internal implementation of the service.

Purpose: APIs have become a primary method for acquiring data from online sources. They allow developers to leverage existing services, access remote data, and build integrated systems without understanding the internal workings of the service provider.

REST APIs

Definition: REST (Representational State Transfer) is an architectural style for designing web services. A RESTful API uses HTTP requests to `GET`, `PUT`, `POST`, and `DELETE` data. It is built on the concept of resources, where each resource is identified by a URL.

Core Principles of REST:

- **Stateless:** Each request from client to server must contain all information needed to understand the request. The server does not store any client context between requests. This improves scalability because the server doesn't need to maintain session state.
- **Client-Server:** Separation of concerns between the user interface (client) and data storage (server), allowing them to evolve independently. Clients don't need to know about data storage, and servers don't need to know about the user interface.
- **Cacheable:** Responses must implicitly or explicitly define themselves as cacheable or non-cacheable. This improves performance by allowing clients to reuse responses for equivalent requests.
- **Uniform Interface:** Resources are identified in requests, manipulated through representations, and include self-descriptive messages. This simplifies the overall system architecture.

- **Layered System:** Intermediary servers (proxies, gateways, load balancers) can exist between client and server without either knowing. This improves scalability and enables security policies.
- **Code on Demand (optional):** Servers can temporarily extend client functionality by transferring executable code (like JavaScript).

HTTP Methods in REST

- **GET** Purpose: Retrieve resource
  Safe: Yes
  Idempotent: Yes
  Description: Request data from server; never changes server state
- **POST** Purpose: Create resource
  Safe: No
  Idempotent: No
  Description: Submit data to server to create a new resource
- **PUT** Purpose: Update/replace resource
  Safe: No
  Idempotent: Yes
  Description: Update entire resource; multiple identical requests have same effect
- **PATCH** Purpose: Partial update
  Safe: No
  Idempotent: No
  Description: Partially modify resource
- **DELETE** Purpose: Remove resource
  Safe: No
  Idempotent: Yes
  Description: Delete resource; multiple identical requests have same effect
- **HEAD** Purpose: GET without body
  Safe: Yes
  Idempotent: Yes
  Description: Retrieve headers only, useful for checking resource existence
- **OPTIONS** Purpose: Available methods
  Safe: Yes
  Idempotent: Yes
  Description: Discover allowed operations on a resource

HTTP Status Codes

- **Success (2xx)**
  - **200 OK** – Request succeeded; response body contains requested data
  - **201 Created** – Resource created successfully; typically returned after POST
  - **204 No Content** – Success but no response body; common for DELETE operations
- **Redirection (3xx)**
  - **301 Moved Permanently** – Resource has new URL; client should update bookmarks

  - **304 Not Modified** – Resource hasn't changed; client can use cached version

- **Client Error (4xx)**
  - **400 Bad Request** – Malformed request syntax; server cannot understand
  - **401 Unauthorized** – Authentication required; client must authenticate
  - **403 Forbidden** – Authenticated but not authorized to access resource
  - **404 Not Found** – Resource doesn't exist at the specified URL
  - **429 Too Many Requests** – Rate limit exceeded; client should slow down
- **Server Error (5xx)**
  - **500 Internal Server Error** – Generic server error; unexpected condition
  - **502 Bad Gateway** – Invalid response from upstream server
  - **503 Service Unavailable** – Server overloaded or down for maintenance

Request Components in API Calls

- **URL/Endpoint:** The address where the API resource is located (e.g., `https://api.example.com/users`)
- **HTTP Method:** Indicates the desired operation: `GET`, `POST`, `PUT`, `DELETE`, etc.
- **Headers:** Metadata about the request, including:
  - **Content-Type:** Format of data being sent (e.g., `application/json`, `application/xml`)
  - **Accept:** Expected format of response
  - **Authorization:** Authentication credentials
  - **User-Agent:** Client identification
- **Parameters:** Can be included in different locations:
  - **Query parameters:** Appended to URL after `?` (e.g., `?page=1&limit=10`)
  - **Path parameters:** Part of URL path (e.g., `/users/123` where 123 is a parameter)
- **Request Body:** Data sent with `POST` or `PUT` requests, usually in JSON or XML format

Response Components

- **Status Code:** Three-digit number indicating success or failure of the request
- **Headers:** Metadata about the response, such as `Content-Type`, `Content-Length`, `Cache-Control`
- **Body:** The actual data returned, usually in JSON format

Authentication Methods

- **API Keys:** Simple token passed in headers, URL, or body. Easy to implement but less secure. Commonly used for public APIs with basic access control. **Example:** `Headers: {"X-API-Key": "your_api_key_here"}`
- **Basic Authentication:** Username and password sent in the `Authorization` header, Base64 encoded. Requires HTTPS for security as encoding is not encryption. **Example:** `Headers: {"Authorization": "Basic dXNlcm5hbWU6cGFzc3dvcmQ="}`

- **Bearer Tokens (OAuth 2.0):** Token-based authentication where a token is issued after authentication and included in subsequent requests. Tokens have limited lifetime and can be scoped to specific permissions. **Example:**
  `Headers: {"Authorization": "Bearer eyJhbGciOiJIUzI1NiIs..."}`
- **JWT (JSON Web Tokens):** Self-contained tokens that include claims (user information, permissions) and a digital signature. The server can verify token authenticity without storing session state.

SOAP APIs

Definition: SOAP (Simple Object Access Protocol) is a protocol specification for exchanging structured information in web services. It uses XML for message format and typically relies on HTTP or SMTP for transmission. SOAP has strict standards and formal contracts between services.

Key Characteristics:

- **XML-Based:** All messages must be well-formed XML following strict schemas
- **Protocol Agnostic:** Can run over HTTP, SMTP, TCP, JMS, not limited to web protocols
- **Language and Platform Independent:** Any language on any platform can implement SOAP clients and servers
- **Strict Standards:** Formal contracts and validation through WSDL ensure compatibility
- **Extensible:** WS-* specifications add capabilities like security, transactions, and reliable messaging
- **Built-in Error Handling:** SOAP Faults provide structured error information

SOAP Message Structure:

```
<?xml version="1.0"?>
<soap:Envelope
    xmlns:soap="http://www.w3.org/2003/05/soap-envelope">

    <soap:Header>
        <!-- Optional header elements for metadata -->
        <auth:Authentication xmlns:auth="http://example.com/auth">
            <auth:Username>user</auth:Username>
            <auth:Password>pass</auth:Password>
        </auth:Authentication>
    </soap:Header>

    <soap:Body>
        <!-- Required body with request or response data -->
        <m:GetPrice xmlns:m="http://example.com/stock">
            <m:Symbol>IBM</m:Symbol>
        </m:GetPrice>
    </soap:Body>

    <soap:Fault>
        <!-- Optional fault element for errors (only in responses) -->
```

```
        <faultcode>soap:Server</faultcode>
        <faultstring>Processing error</faultstring>
        <detail>Error details here</detail>
    </soap:Fault>
</soap:Envelope>
```

SOAP Message Components

- **Envelope (Required)** Root element that wraps the entire message; identifies the XML as a SOAP message
- **Header (Optional)** Contains metadata such as authentication, routing instructions, or transaction context
- **Body (Required)** Contains the actual request or response data, including method name and parameters
- **Fault (Optional)** Provides error information returned when a request fails; only appears in responses

WSDL (Web Services Description Language)

WSDL is an XML document that describes a web service interface, providing a complete contract for clients. Key components include:

- **Types:** Data type definitions using XML Schema
- **Messages:** Abstract definitions of the data being exchanged between client and service
- **PortTypes:** Operations that can be performed and the messages involved
- **Bindings:** Protocol and data format details for each port type
- **Services:** Collection of ports with network locations (endpoints)

SOAP vs. REST Comparison

- **Protocol:** SOAP – Protocol with strict rules
  REST – Architectural style
- **Message Format:** SOAP – XML only
  REST – JSON, XML, HTML, plain text
- **Transport:** SOAP – HTTP, SMTP, JMS, TCP, etc.
  REST – HTTP/HTTPS only
- **State:** SOAP – Can be stateless or stateful
  REST – Stateless
- **Security:** SOAP – WS-Security (built-in to protocol)
  REST – HTTPS, OAuth (external)
- **Caching:** SOAP – Not supported natively
  REST – HTTP caching mechanisms
- **Performance:** SOAP – Slower (XML parsing overhead)
  REST – Faster (lightweight formats)

- **Error Handling:** SOAP – Built-in Fault elements
  REST – HTTP status codes
- **Tooling:** SOAP – Extensive enterprise tooling
  REST – Simple HTTP tools
- **Use Cases:** SOAP – Enterprise, banking, telecom, healthcare
  REST – Web, mobile, public APIs

API Best Practices

Rate Limiting: Rate limiting restricts how frequently a client can make requests within a given time frame. This prevents abuse, ensures fair usage among clients, and protects server resources.

- **Common Rate Limit Headers:**
  - `X-RateLimit-Limit:` Maximum requests allowed per time window
  - `X-RateLimit-Remaining:` Number of requests remaining in current window
  - `X-RateLimit-Reset:` Time when the limit resets (Unix timestamp)
  - `Retry-After:` Seconds to wait when rate limited (sent with 429 status)

Error Handling: Proper error handling distinguishes between different types of failures:

- **HTTP Errors (4xx, 5xx):** Server returned an error response
- **Connection Errors:** Network issues, DNS failures, refused connections
- **Timeout Errors:** Server took too long to respond
- **SSL Errors:** Certificate validation failures

Exception Patterns:

- **Use Specific Exceptions:** Different error types should raise different exceptions
- **Include Context:** Exceptions should include URL, status code, and error details
- **Retry Logic:** Transient errors (timeouts, 5xx) should be retried with backoff
- **Fail Fast:** Validation errors should fail immediately without making request

REST API PROGRAMS

Basic GET Requests

Installing and Importing Requests

```
# Install requests library
# pip install requests

import requests
import json
from pprint import pprint
```

Simple GET Request

```python
import requests

# Send a GET request to a public API
response = requests.get("https://jsonplaceholder.typicode.com/posts/1")

# Check status code
print(f"Status Code: {response.status_code}")

# Check if request was successful
if response.status_code == 200:
    print("Request successful!")

    # Response content as string
    print(f"Raw response (first 100 chars): {response.text[:100]}...")

    # Parse JSON response
    data = response.json()
    print(f"\nParsed data type: {type(data)}")
    print(f"Title: {data['title']}")
    print(f"Body: {data['body'][:50]}...")
else:
    print(f"Error: {response.status_code}")
```

Examining Response Headers

```python
import requests

# Send GET request to API
response = requests.get("https://jsonplaceholder.typicode.com/posts")

# Basic response info
print(f"Status Code: {response.status_code}")
print(f"Response URL: {response.url}")

# View all response headers
print("\n--- Response Headers ---")
for key, value in response.headers.items():
    print(f"{key}: {value}")

# Check specific headers
content_type = response.headers.get('Content-Type')
print(f"\nContent Type: {content_type}")
print(f"Server: {response.headers.get('Server', 'Not specified')}")
```

Parsing Different Response Formats

```python
import requests

# Send GET request to API
response = requests.get("https://jsonplaceholder.typicode.com/posts/1")

# Method 1: Get as text (raw string)
text_response = response.text
print(f"Text response (type: {type(text_response)})")
print(f"First 100 chars: {text_response[:100]}...")

# Method 2: Get as bytes (for binary data)
bytes_response = response.content
```

```
print(f"\nBytes response (type: {type(bytes_response)})")
print(f"First 50 bytes: {bytes_response[:50]}")

# Method 3: Parse as JSON (for JSON APIs)
json_response = response.json()
print(f"\nJSON response (type: {type(json_response)})")
print(f"Title: {json_response['title']}")
```

GET Request with Query Parameters

```
import requests

# Method 1: Pass parameters as dictionary
params = {
    'userId': 1,
    '_limit': 3
}

response = requests.get(
    "https://jsonplaceholder.typicode.com/posts",
    params=params
)

print(f"URL called: {response.url}")  # Shows full URL with parameters
posts = response.json()

print(f"\nRetrieved {len(posts)} posts for user 1:")
for i, post in enumerate(posts, 1):
    print(f"{i}. {post['title']}")

# Method 2: Different endpoint with parameters
response2 = requests.get(
    "https://jsonplaceholder.typicode.com/comments",
    params={'postId': 1, '_limit': 2}
)
comments = response2.json()
print(f"\nComments for post 1: {len(comments)}")
for comment in comments:
    print(f"- {comment['name']}")
```

Accessing Nested JSON Data

```
import requests

# Get a todo item
response = requests.get("https://jsonplaceholder.typicode.com/todos/1")
todo = response.json()

# Access nested data (though this API is flat, pattern applies)
print("Todo Item:")
print(f"  ID: {todo.get('id')}")
print(f"  Title: {todo.get('title')}")
print(f"  Completed: {todo.get('completed')}")

# Get user details
user_response = requests.get(f"https://jsonplaceholder.typicode.com/users/{todo
    .get('userId')}")
user = user_response.json()

print(f"\nAssigned User:")
```

```
print(f"  Name: {user.get('name')}")
print(f"  Email: {user.get('email')}")
print(f"  City: {user.get('address', {}).get('city')}")  # Nested access
```

POST Requests

Sending JSON Data

```
import requests

# Data to send (as Python dictionary)
new_post = {
    'title': 'My New Post',
    'body': 'This is the content of my new post created via API.',
    'userId': 1
}

# Send POST request with JSON data
response = requests.post(
    "https://jsonplaceholder.typicode.com/posts",
    json=new_post  # 'json' parameter automatically sets Content-Type header
)

print(f"Status Code: {response.status_code}")

if response.status_code == 201:  # 201 = Created
    created_post = response.json()
    print(f"Created post with ID: {created_post.get('id')}")
    print(f"Title: {created_post.get('title')}")
    print(f"Body: {created_post.get('body')}")
```

Sending Form Data

```
import requests

# Form data (like HTML form submission)
form_data = {
    'username': 'john_doe',
    'email': 'john@example.com',
    'age': 30
}

# Send as form-encoded data using 'data' parameter
response = requests.post(
    "https://httpbin.org/post",  # Test endpoint that echoes request
    data=form_data
)

result = response.json()
print("Form data received by server:")
for key, value in result['form'].items():
    print(f"  {key}: {value}")

# Note the Content-Type header
print(f"\nContent-Type sent: {result['headers'].get('Content-Type')}")
# Should be: application/x-www-form-urlencoded
```

Sending Custom Headers

```python
import requests

# Custom headers
headers = {
    'User-Agent': 'MyApplication/1.0',
    'Accept': 'application/json',
    'X-Custom-Header': 'custom-value-123',
    'X-API-Version': '2'
}

data = {
    'name': 'Test Product',
    'price': 29.99,
    'category': 'electronics'
}

response = requests.post(
    "https://httpbin.org/post",
    json=data,
    headers=headers
)

result = response.json()
print("Headers sent to server:")
for key, value in result['headers'].items():
    print(f"  {key}: {value}")

print("\nData sent:")
print(f"  {result['json']}")
```

PUT and PATCH Requests

PUT Request (Full Update)

```python
import requests

# PUT replaces the entire resource
updated_post = {
    'id': 1,
    'title': 'Updated Title',
    'body': 'This post has been completely updated.',
    'userId': 1
}

response = requests.put(
    "https://jsonplaceholder.typicode.com/posts/1",
    json=updated_post
)

print(f"PUT Status: {response.status_code}")
if response.status_code == 200:
    result = response.json()
    print(f"Updated title: {result['title']}")
```

PATCH Request (Partial Update)

```python
import requests

# PATCH updates only specified fields
partial_update = {
```

```
    'title': 'Partially Updated Title'
}

response = requests.patch(
    "https://jsonplaceholder.typicode.com/posts/1",
    json=partial_update
)

print(f"PATCH Status: {response.status_code}")
if response.status_code == 200:
    result = response.json()
    print(f"Updated title: {result['title']}")
```

PATCH Request (Partial Update)

```
import requests

# PATCH only updates specified fields
partial_update = {
    'title': 'Partially Updated Title'  # Only change the title
}

response = requests.patch(
    "https://jsonplaceholder.typicode.com/posts/1",
    json=partial_update
)

print(f"PATCH Status: {response.status_code}")
if response.status_code == 200:
    result = response.json()
    print(f"New title: {result['title']}")
    print(f"Body unchanged: {result['body'][:30]}...")
```

DELETE Requests

```
import requests

# DELETE request
response = requests.delete("https://jsonplaceholder.typicode.com/posts/1")

print(f"DELETE Status: {response.status_code}")

# 204 No Content indicates successful deletion with no response body
if response.status_code == 204:
    print("Resource deleted successfully")
elif response.status_code == 200:
    print("Resource deleted, response body:", response.json())
elif response.status_code == 404:
    print("Resource not found")
```

Authentication Examples

API Key Authentication

```
import requests

# API key in headers (common pattern)
headers = {
    'X-API-Key': 'your_api_key_here',
```

```
    'Content-Type': 'application/json'
}

response = requests.get(
    "https://api.example.com/v1/data",
    headers=headers
)

if response.status_code == 200:
    data = response.json()
    print("Authenticated successfully")
else:
    print(f"Authentication failed: {response.status_code}")
```

Basic Authentication

```
import requests
from requests.auth import HTTPBasicAuth

# Method 1: Using HTTPBasicAuth helper
response = requests.get(
    "https://api.example.com/protected",
    auth=HTTPBasicAuth('username', 'password')
)

# Method 2: Shortcut syntax (recommended)
response = requests.get(
    "https://api.example.com/protected",
    auth=('username', 'password')
)

print(f"Status: {response.status_code}")
if response.status_code == 200:
    print("Authentication successful")
```

Bearer Token Authentication

```
import requests

class APIClient:
    """Simple API client with bearer token authentication"""

    def __init__(self, base_url, access_token):
        self.base_url = base_url.rstrip('/')
        self.access_token = access_token
        self.session = requests.Session()

        # Set default headers for all requests
        self.session.headers.update({
            'Authorization': f'Bearer {access_token}',
            'Content-Type': 'application/json',
            'Accept': 'application/json'
        })

    def get(self, endpoint, params=None):
        """Make authenticated GET request"""
        url = f"{self.base_url}/{endpoint.lstrip('/')}"
        response = self.session.get(url, params=params)
        return response

```

```
    def post(self, endpoint, data=None):
        """Make authenticated POST request"""
        url = f"{self.base_url}/{endpoint.lstrip('/')}"
        response = self.session.post(url, json=data)
        return response

    def close(self):
        """Close the session"""
        self.session.close()

# Usage
client = APIClient(
    base_url="https://api.example.com/v1",
    access_token="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
)

try:
    response = client.get("users/me")
    if response.status_code == 200:
        user_data = response.json()
        print(f"Authenticated as: {user_data.get('name', 'Unknown')}")
finally:
    client.close()
```

Session Management

```
import requests

# Using Session for multiple requests
# Sessions maintain cookies and connection pooling
session = requests.Session()

# Set default headers for all requests in this session
session.headers.update({
    'User-Agent': 'MyApp/1.0',
    'Accept': 'application/json'
})

# Login once (if authentication sets cookies)
login_data = {'username': 'user', 'password': 'pass'}
login_response = session.post(
    "https://api.example.com/login",
    json=login_data
)

# Session automatically maintains cookies
if login_response.status_code == 200:
    print("Logged in successfully")

    # Subsequent requests use the same session
    # No need to send credentials again
    profile_response = session.get("https://api.example.com/profile")
    dashboard_response = session.get("https://api.example.com/dashboard")

    print(f"Profile status: {profile_response.status_code}")
    print(f"Dashboard status: {dashboard_response.status_code}")

    # Access protected resources
    data_response = session.get(
        "https://api.example.com/data",
        params={'limit': 10}
    )
```

```

    if data_response.status_code == 200:
        data = data_response.json()
        print(f"Retrieved {len(data)} items")

# Close session when done
session.close()
```

Error Handling

```
import requests
from requests.exceptions import RequestException, ConnectionError, HTTPError,
    Timeout

def safe_api_request(url, timeout=10):
    """Make API request with comprehensive error handling"""

    try:
        response = requests.get(url, timeout=timeout)

        # Raise exception for HTTP errors (4xx, 5xx)
        response.raise_for_status()

        # Success - return parsed JSON
        return response.json()

    except ConnectionError as e:
        print(f"Connection Error: Could not connect to server - {e}")
        print("Check your internet connection or if the server is available")
        return None

    except Timeout as e:
        print(f"Timeout Error: Request timed out after {timeout} seconds")
        print("The server is taking too long to respond")
        return None

    except HTTPError as e:
        print(f"HTTP Error: {response.status_code}")

        # Handle specific status codes
        if response.status_code == 400:
            print("Bad Request - Check your parameters")
        elif response.status_code == 401:
            print("Unauthorized - Check your authentication")
        elif response.status_code == 403:
            print("Forbidden - You don't have permission")
        elif response.status_code == 404:
            print("Not Found - The resource doesn't exist")
        elif response.status_code == 429:
            print("Rate Limited - Too many requests")
            retry_after = response.headers.get('Retry-After', 'unknown')
            print(f"Retry after {retry_after} seconds")
        elif response.status_code >= 500:
            print("Server Error - The server had a problem")
        return None

    except RequestException as e:
        print(f"General Request Error: {e}")
        return None

# Usage
data = safe_api_request("https://jsonplaceholder.typicode.com/posts/1")
```

```
if data:
    print(f"Success! Retrieved: {data['title']}")
```

Complete REST API Client Example

```
import requests
import time
import logging
from requests.exceptions import RequestException

# Set up logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

class RESTAPIClient:
    """
    Complete REST API Client with authentication, error handling,
    retry logic, and session management
    """

    def __init__(self, base_url, auth_token=None, timeout=30, max_retries=3):
        self.base_url = base_url.rstrip('/')
        self.timeout = timeout
        self.max_retries = max_retries
        self.session = requests.Session()
        self.session.headers.update({
            'User-Agent': 'RESTAPIClient/1.0',
            'Accept': 'application/json',
            'Content-Type': 'application/json'
        })
        if auth_token:
            self.session.headers.update({'Authorization': f'Bearer {auth_token}
    '})
        logger.info(f"API Client initialized for {base_url}")

    def _request(self, method, endpoint, params=None, data=None, retry_count=0)
    :
        url = f"{self.base_url}/{endpoint.lstrip('/')}"
        try:
            logger.debug(f"Making {method} request to {url}")
            response = self.session.request(
                method=method,
                url=url,
                params=params,
                json=data,
                timeout=self.timeout
            )
            if response.status_code == 429:
                retry_after = int(response.headers.get('Retry-After', 5))
                logger.warning(f"Rate limited. Waiting {retry_after} seconds")
                if retry_count < self.max_retries:
                    time.sleep(retry_after)
                    return self._request(method, endpoint, params, data,
    retry_count + 1)
                else:
                    logger.error("Max retries exceeded for rate limit")
                    response.raise_for_status()
            response.raise_for_status()
            return response
        except requests.exceptions.ConnectionError as e:
            logger.error(f"Connection error: {e}")
            if retry_count < self.max_retries:
```

```
                wait = 2 ** retry_count
                logger.info(f"Retrying in {wait} seconds...")
                time.sleep(wait)
                return self._request(method, endpoint, params, data,
    retry_count + 1)
            raise
        except requests.exceptions.Timeout as e:
            logger.error(f"Timeout error: {e}")
            if retry_count < self.max_retries:
                wait = 2 ** retry_count
                logger.info(f"Retrying in {wait} seconds...")
                time.sleep(wait)
                return self._request(method, endpoint, params, data,
    retry_count + 1)
            raise
        except requests.exceptions.HTTPError as e:
            logger.error(f"HTTP error {response.status_code}: {e}")
            raise
        except Exception as e:
            logger.error(f"Unexpected error: {e}")
            raise

    def get(self, endpoint, params=None):
        response = self._request('GET', endpoint, params=params)
        return response.json() if response.content else None

    def post(self, endpoint, data=None):
        response = self._request('POST', endpoint, data=data)
        return response.json() if response.content else None

    def put(self, endpoint, data=None):
        response = self._request('PUT', endpoint, data=data)
        return response.json() if response.content else None

    def patch(self, endpoint, data=None):
        response = self._request('PATCH', endpoint, data=data)
        return response.json() if response.content else None

    def delete(self, endpoint):
        response = self._request('DELETE', endpoint)
        return response.status_code == 204

    def close(self):
        self.session.close()
        logger.info("API Client closed")

# Usage example
if __name__ == "__main__":
    client = RESTAPIClient(
        base_url="https://jsonplaceholder.typicode.com",
        timeout=10,
        max_retries=2
    )
    try:
        posts = client.get("posts", params={"_limit": 3})
        print(f"Retrieved {len(posts)} posts")
        for post in posts:
            print(f"  - {post['title']}")
        new_post = client.post("posts", data={
            "title": "API Test Post",
            "body": "Created via REST client",
            "userId": 1
        })
```

```
        print(f"\nCreated post with ID: {new_post.get('id')}")
        single = client.get(f"posts/{new_post.get('id')}")
        print(f"Retrieved: {single['title']}")
        updated = client.put(f"posts/{new_post.get('id')}", data={
            "id": new_post.get('id'),
            "title": "Updated Title",
            "body": "Updated content",
            "userId": 1
        })
        print(f"Updated: {updated['title']}")
        success = client.delete(f"posts/{new_post.get('id')}")
        print(f"Deleted: {success}")
    finally:
        client.close()
```

SOAP API PROGRAMS

Introduction to Zeep

Zeep is a Python library for SOAP web services. It inspects WSDL documents and generates the corresponding code to interact with the SOAP web service.

```
# Install zeep
# pip install zeep

from zeep import Client
from zeep.exceptions import Fault, TransportError
import logging

# Enable logging for debugging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)
```

Basic SOAP Client

Creating a SOAP Client from WSDL

```
from zeep import Client

# Create a client from WSDL URL
wsdl_url = "http://www.dneonline.com/calculator.asmx?wsdl"  # Public calculator service
client = Client(wsdl_url)

print("SOAP Client created successfully")
print(f"Service: {client.wsdl.services[0].name}")
```

Viewing Available Operations

```
from zeep import Client
from pprint import pprint

client = Client("http://www.dneonline.com/calculator.asmx?wsdl")

# Get service
service = client.wsdl.services[0]
print(f"Service: {service.name}")

# Get ports
```

```
for port in service.ports.values():
    print(f"\nPort: {port.name}")
    print("Operations:")
    for operation_name, operation in port.binding._operations.items():
        print(f"  - {operation_name}")
        print(f"    Input: {operation.input.signature}")
```

Calling SOAP Methods

```
from zeep import Client

# Calculator service
client = Client("http://www.dneonline.com/calculator.asmx?wsdl")

# Call Add method
result = client.service.Add(intA=10, intB=20)
print(f"10 + 20 = {result}")

# Call Subtract method
result = client.service.Subtract(intA=50, intB=15)
print(f"50 - 15 = {result}")

# Call Multiply method
result = client.service.Multiply(intA=6, intB=7)
print(f"6 * 7 = {result}")

# Call Divide method
result = client.service.Divide(intA=100, intB=4)
print(f"100 / 4 = {result}")
```

Working with Complex Types

```
from zeep import Client
from zeep.helpers import serialize_object

# Example with a more complex SOAP service
# Note: This is a conceptual example using a public SOAP service
wsdl_url = "http://www.dneonline.com/calculator.asmx?wsdl"
client = Client(wsdl_url)

# For complex types, you may need to create objects
# The exact structure depends on the WSDL

# Example of inspecting types
for name, type_obj in client.wsdl.types.types.items():
    print(f"Type: {name}")
```

SOAP with Authentication

```
from zeep import Client
from zeep.transports import Transport
from requests import Session
from requests.auth import HTTPBasicAuth

# Create session with authentication
session = Session()
session.auth = HTTPBasicAuth('username', 'password')

# Create transport with the authenticated session
```

```python
transport = Transport(session=session)

# Create client with transport
client = Client(
    "https://example.com/secure/service?wsdl",
    transport=transport
)

# Now all SOAP calls will include authentication
try:
    result = client.service.SecureOperation(param1="value")
    print(f"Result: {result}")
except Exception as e:
    print(f"SOAP call failed: {e}")
```

Web Scraping using Requests and BeautifulSoup

Introduction to Web Scraping

Definition:

Web scraping is the practice of using a computer program to sift through a web page and gather the data that you need in a format most useful for you.

Purpose and Applications:

Web scraping is used to collect data from websites for various purposes including data analysis, research, and building datasets when APIs are not available.

1.2 Web Scraping Workflow

The process of web scraping can be broken down into two parts: actually fetching the web page (downloading it) and then extracting information from it.

Step 1: Fetching the Web Page

This involves sending an HTTP request to the server and receiving the HTML content of the page.

Step 2: Extracting Information

Once you have the HTML, you need to parse it and extract the specific pieces of data you're interested in.

1.3 The Requests Library

The `requests` library is the standard tool for making HTTP requests from Python. It handles the fetching part of web scraping.

Key Capabilities:

- Sending GET and POST requests
- Handling headers and parameters
- Managing cookies and sessions
- Processing response status codes
- Following redirects

1.4 The BeautifulSoup Library

BeautifulSoup is a library for parsing HTML documents and extracting data from them. It creates a parse tree from the HTML that makes it easy to navigate and search.

Key Capabilities:

- Parsing HTML regardless of how messy it is
- Navigating the document tree
- Searching for elements by tags, classes, IDs, or attributes
- Extracting text and attribute values

1.5 The HTML Document Object Model

HTML documents have a hierarchical structure. A simple HTML document looks like:

```
<html>
    <head>
        <title>Page Title</title>
    </head>
    <body>
        <h1>Main Heading</h1>
        <p>First paragraph</p>
        <p>Second paragraph</p>
    </body>
</html>
```

In this structure, the `<html>` tag contains all other tags, `<head>` contains metadata, and `<body>` contains the visible content.

1.6 Ethical and Legal Considerations

Robots.txt

Before scraping a website, you should check its `robots.txt` file (e.g., `https://example.com/robots.txt`) which specifies which parts of the site can be accessed by automated tools.

Rate Limiting

You should implement delays between requests to avoid overloading the server. This is both polite and helps avoid being blocked.

User-Agent Identification

You should identify your bot with a proper User-Agent string rather than pretending to be something you're not.

Terms of Service

Always review the website's terms of service to ensure scraping is permitted.

INSTALLATION AND BASIC SETUP

2.1 Installing Required Libraries

```
# Install using pip
# pip install requests
# pip install beautifulsoup4

# For better performance, also install lxml parser
# pip install lxml
```

2.2 Importing Libraries

```
import requests
from bs4 import BeautifulSoup
```

FETCHING WEB PAGES WITH REQUESTS

3.1 Basic GET Request

```
import requests

# Make a request to a web page
response = requests.get('https://example.com')

# Check the status code
print(f"Status code: {response.status_code}")

# A status code of 200 means the request was successful
if response.status_code == 200:
    # Get the HTML content
    html_content = response.text
    print(f"Retrieved {len(html_content)} bytes")
else:
    print(f"Failed to retrieve page: {response.status_code}")
```

3.2 Adding Headers

```
import requests

# Some websites require a proper User-Agent header
headers = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
    '
}

response = requests.get('https://example.com', headers=headers)
print(f"Status code: {response.status_code}")
```

3.3 Handling Errors

```
import requests
from requests.exceptions import RequestException

def fetch_page(url):
    """Safely fetch a web page"""
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()  # Raise exception for 4xx/5xx status codes
        return response.text
    except RequestException as e:
        print(f"Error fetching page: {e}")
        return None

html = fetch_page('https://example.com')
if html:
    print(f"Retrieved {len(html)} bytes")
```

PARSING HTML WITH BEAUTIFULSOUP

4.1 Creating a BeautifulSoup Object

```
from bs4 import BeautifulSoup

# Sample HTML document
html_doc = """
<html>
<head>
    <title>Test Page</title>
</head>
<body>
    <h1>Main Heading</h1>
    <p class="content">First paragraph</p>
    <p class="content">Second paragraph</p>
    <a href="https://example.com/link" id="link">Click here</a>
</body>
</html>
"""

# Create a BeautifulSoup object
soup = BeautifulSoup(html_doc, 'html.parser')

# The prettify method formats the HTML for display
print(soup.prettify())
```

4.2 Accessing Elements by Tag Name

```
from bs4 import BeautifulSoup

html_doc = "<html><body><h1>Title</h1><p>Paragraph</p></body></html>"
soup = BeautifulSoup(html_doc, 'html.parser')

# Access the title tag
title_tag = soup.title
print(f"Title tag: {title_tag}")

# Access the body tag
body_tag = soup.body
print(f"Body tag: {body_tag.name}")

# Access the first paragraph
first_p = soup.p
print(f"First paragraph: {first_p.string}")

# Access all paragraphs
all_p = soup.find_all('p')
print(f"Number of paragraphs: {len(all_p)}")
```

NAVIGATING THE PARSE TREE

5.1Getting Tag Contents

```
from bs4 import BeautifulSoup

html_doc = """
<html>
```

```
<body>
    <p>This is a <strong>simple</strong> paragraph.</p>
</body>
</html>
"""

soup = BeautifulSoup(html_doc, 'html.parser')
p_tag = soup.p

# Getting the text content (all text inside the tag)
print(f"Text: {p_tag.get_text()}")

# Getting the string (only if tag has one child)
print(f"String: {p_tag.string}")

# Getting the raw HTML
print(f"Raw HTML: {p_tag}")
```

5.2 Navigating with Tag Names

```
from bs4 import BeautifulSoup

html_doc = """
<html>
<body>
    <div>
        <h1>Title</h1>
        <p>First paragraph</p>
        <p>Second paragraph</p>
    </div>
</body>
</html>
"""

soup = BeautifulSoup(html_doc, 'html.parser')

# Access children of body
body = soup.body
for child in body.children:
    print(f"Child: {child.name}")

# Access all descendants
for descendant in body.descendants:
    if descendant.name:  # Only print tag names
        print(f"Descendant: {descendant.name}")
```

5.3 Navigating with Parent and Sibling

```
from bs4 import BeautifulSoup

html_doc = """
<html>
<body>
    <h1>Title</h1>
    <p>First paragraph</p>
    <p>Second paragraph</p>
</body>
</html>
"""

```

```
soup = BeautifulSoup(html_doc, 'html.parser')
first_p = soup.p

# Parent
print(f"Parent: {first_p.parent.name}")

# Next sibling
next_sib = first_p.next_sibling
while next_sib and next_sib.name is None:  # Skip NavigableString objects
    next_sib = next_sib.next_sibling
print(f"Next sibling: {next_sib.name if next_sib else None}")

# Previous sibling
prev_sib = first_p.previous_sibling
while prev_sib and prev_sib.name is None:
    prev_sib = prev_sib.previous_sibling
print(f"Previous sibling: {prev_sib.name if prev_sib else None}")
```

SEARCHING THE PARSE TREE

6.1 Searching with `find` and `find_all`

```
from bs4 import BeautifulSoup

html_doc = """
<html>
<body>
    <div class="container">
        <h1 class="title">Main Title</h1>
        <p class="content">First paragraph</p>
        <p class="content">Second paragraph</p>
        <p class="footer">Footer paragraph</p>
    </div>
</body>
</html>
"""

soup = BeautifulSoup(html_doc, 'html.parser')

# find_all returns a list of all matching tags
all_p = soup.find_all('p')
print(f"All p tags: {len(all_p)}")

# find returns the first matching tag
first_p = soup.find('p')
print(f"First p tag: {first_p.text}")

# Search by class
content_p = soup.find_all('p', class_='content')
print(f"Content paragraphs: {len(content_p)}")

# Search by multiple attributes
specific = soup.find_all('p', {'class': 'footer'})
print(f"Footer paragraph: {specific[0].text if specific else 'None'}")
```

6.2 Searching with CSS Selectors

```
from bs4 import BeautifulSoup

```

```
html_doc = """
<html>
<body>
    <div id="main">
        <p class="content">First</p>
        <p class="content special">Second</p>
        <div class="nested">
            <p class="content">Nested</p>
        </div>
    </div>
</body>
</html>
"""

soup = BeautifulSoup(html_doc, 'html.parser')

# Select by tag
content_p = soup.select('p')
print(f"All p tags: {len(content_p)}")

# Select by class
content = soup.select('.content')
print(f"Class 'content': {len(content)}")

# Select by id
main = soup.select('#main')
print(f"Element with id 'main': {main[0].name if main else 'None'}")

# Select nested elements
nested = soup.select('#main .nested .content')
print(f"Nested content: {nested[0].text if nested else 'None'}")
```

6.3 Getting Attribute Values

```
from bs4 import BeautifulSoup

html_doc = """
<html>
<body>
    <a href="https://example.com" class="link" id="link1">Example Link</a>
    <img src="image.jpg" alt="Sample Image" width="300">
</body>
</html>
"""

soup = BeautifulSoup(html_doc, 'html.parser')
link = soup.a
img = soup.img

# Getting attributes using dictionary syntax
print(f"Link href: {link['href']}")
print(f"Link class: {link['class']}")

# Using get method (safe if attribute might be missing)
print(f"Link id: {link.get('id', 'No id')}")
print(f"Link target: {link.get('target', 'No target')}")

# Getting multiple attributes from an image
print(f"Image src: {img['src']}")
print(f"Image alt: {img['alt']}")
print(f"Image width: {img['width']}")
```

COMPLETE WEB SCRAPING EXAMPLES

7.1 Basic Example: Scraping a Simple Page

```python
import requests
from bs4 import BeautifulSoup

# Step 1: Fetch the page
url = "http://quotes.toscrape.com"
response = requests.get(url)

if response.status_code == 200:
    # Step 2: Parse the HTML
    soup = BeautifulSoup(response.text, 'html.parser')

    # Step 3: Find all quotes
    quotes = soup.find_all('div', class_='quote')

    # Step 4: Extract data from each quote
    for quote in quotes:
        text = quote.find('span', class_='text').text
        author = quote.find('small', class_='author').text

        tags = quote.find_all('a', class_='tag')
        tag_list = [tag.text for tag in tags]

        print(f"Quote: {text}")
        print(f"Author: {author}")
        print(f"Tags: {', '.join(tag_list)}")
        print("-" * 50)
else:
    print(f"Failed to retrieve page: {response.status_code}")
```

7.2 Saving Scraped Data to CSV

```python
import requests
from bs4 import BeautifulSoup
import csv

# Fetch and parse the page
url = "http://quotes.toscrape.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, 'html.parser')

# Find all quotes
quotes = soup.find_all('div', class_='quote')

# Prepare data for CSV
data = []
for quote in quotes:
    text = quote.find('span', class_='text').text
    author = quote.find('small', class_='author').text

    tags = quote.find_all('a', class_='tag')
    tags_str = ', '.join([tag.text for tag in tags])

    data.append([text, author, tags_str])

# Write to CSV
with open('quotes.csv', 'w', newline='', encoding='utf-8') as file:
    writer = csv.writer(file)
```

```
    writer.writerow(['Quote', 'Author', 'Tags'])  # Header
    writer.writerows(data)

print(f"Saved {len(data)} quotes to quotes.csv")
```

7.3 Scraping Multiple Pages

```
import requests
from bs4 import BeautifulSoup
import time

base_url = "http://quotes.toscrape.com/page/{}/"
all_quotes = []
page = 1

while True:
    print(f"Scraping page {page}...")
    url = base_url.format(page)
    response = requests.get(url)

    if response.status_code != 200:
        print(f"Failed to retrieve page {page}")
        break

    soup = BeautifulSoup(response.text, 'html.parser')
    quotes = soup.find_all('div', class_='quote')

    if not quotes:
        print(f"No quotes found on page {page}, stopping.")
        break

    for quote in quotes:
        text = quote.find('span', class_='text').text
        author = quote.find('small', class_='author').text
        all_quotes.append({'text': text, 'author': author})

    print(f"Found {len(quotes)} quotes on page {page}")

    # Be polite - don't hammer the server
    time.sleep(2)
    page += 1

print(f"Total quotes scraped: {len(all_quotes)}")
```

7.4 Scraping with Proper Headers

```
import requests
from bs4 import BeautifulSoup

# Some websites block requests without proper headers
headers = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36',
    'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8',
    'Accept-Language': 'en-US,en;q=0.5',
    'Connection': 'keep-alive',
}

url = "http://quotes.toscrape.com"
```

```
response = requests.get(url, headers=headers)

if response.status_code == 200:
    soup = BeautifulSoup(response.text, 'html.parser')
    print(f"Page title: {soup.title.text}")
else:
    print(f"Failed: {response.status_code}")
```

7.5 Handling Different Parsers

```
from bs4 import BeautifulSoup

# Sample HTML with some issues
messy_html = """
<html>
<body>
    <p>This is a paragraph with <b>bold text
    <p>This paragraph is missing a closing tag
"""

# Using the built-in parser
soup1 = BeautifulSoup(messy_html, 'html.parser')
print("html.parser result:")
print(soup1.prettify()[:200])

# Using lxml (faster, better at handling broken HTML)
try:
    soup2 = BeautifulSoup(messy_html, 'lxml')
    print("\nlxml result:")
    print(soup2.prettify()[:200])
except:
    print("lxml not installed")
```

SUMMARY

- **requests.get():** Fetch web pages **Key Methods:** get(), post(), status_code, text
- **BeautifulSoup():** Parse HTML **Key Methods:** find(), find_all(), select()
- **Navigating:** Move through the parse tree **Key Methods:** parent, children, next_sibling
- **Extracting:** Get data from tags **Key Methods:** get_text(), get(), `['attribute']`
- **Searching:** Find elements in HTML **Key Methods:** By tag, class, id, CSS selector

Handling Large Datasets with Chunking and Lazy Evaluation

1.1 The Challenge of Large Datasets

Definition: Working with large datasets in Python often leads to a common problem: when you attempt to load everything into memory simultaneously, your program slows down or crashes entirely with out-of-memory (OOM) errors.

The Memory Constraint: Traditional tools like Pandas load the entire dataset into RAM before processing. For datasets larger than available memory, this leads to:

- Slow performance due to excessive swapping
- System crashes with memory errors

- Limited interactivity and exploration capability

1.2 Chunking: Divide and Conquer

Definition: Chunking divides a large dataset into smaller, manageable pieces (chunks) that are processed sequentially. Each chunk fits comfortably in memory, and results from all chunks are combined at the end.

Theoretical Principle: Instead of loading all 10 million rows at once, you load 100,000 rows at a time. Your RAM only ever holds 100,000 rows, no matter how big the file is.

When to Use Chunking:

- Performing aggregations (sum, count, average)
- Applying filtering operations
- Processing data that cannot fit entirely in RAM

1.3 Lazy Evaluation

Definition: Lazy evaluation is a strategy where expressions are not evaluated until their values are actually needed. This contrasts with eager evaluation, where all data is loaded and processed immediately.

Core Characteristics:

- **Delayed Computation:** Operations are only performed when results are requested
- **Memory Efficiency:** Values are generated one at a time as needed
- **Infinite Sequences:** Can represent potentially infinite data structures
- **Composition:** Operations can be chained without intermediate storage

Lazy vs. Eager Evaluation

- **Memory Usage:**
  - Lazy Evaluation: Low (current item only)
  - Eager Evaluation: High (entire dataset)
- **Startup Time:**
  - Lazy Evaluation: Fast (immediate)
  - Eager Evaluation: Slow (loading time)
- **Processing Model:**
  - Lazy Evaluation: Streaming
  - Eager Evaluation: Batch
- **Random Access:**
  - Lazy Evaluation: Not supported
  - Eager Evaluation: Full support
- **Use Case:**
  - Lazy Evaluation: Large datasets, pipelines
  - Eager Evaluation: Small to medium datasets

1.4 Generators: The Heart of Lazy Evaluation

- **Definition:**
  - Generator functions use the `yield` statement to produce values.
  - Execution can be paused and resumed, continuing from the last `yield` when the next value is requested.
- **Key Characteristics:**
  - When called, a generator function does not execute immediately.
  - Returns a generator object.
  - Values are generated one at a time as needed.
  - The function's state is preserved between calls.

1.5 Memory Complexity

- **Traditional Approach:** O(n) memory, where n is the dataset size.
- **Lazy/Chunking Approach:** O(1) memory, regardless of dataset size.

IMPORTANT PROGRAMS

2.1 Basic Generator Function

```
def read_large_file(file_path):
    """Read a large file line by line using generator"""
    with open(file_path, 'r') as file:
        for line in file:
            yield line.strip()

# Usage - process large log file line by line
for line in read_large_file("huge_log.txt"):
    if "ERROR" in line:
        print(f"Found error: {line[:100]}")
```

2.2 Generator Expression vs List Comprehension

```
import sys

# List comprehension (eager) - loads all into memory
list_squares = [x*x for x in range(1000000)]
print(f"List memory: {sys.getsizeof(list_squares)} bytes")

# Generator expression (lazy) - produces on demand
gen_squares = (x*x for x in range(1000000))
print(f"Generator memory: {sys.getsizeof(gen_squares)} bytes")

# Using the generator
for i, square in enumerate(gen_squares):
    if i < 5:
        print(square, end=" ")
    else:
        break
```

2.3 Generator Pipeline

```
def read_lines(filename):
    """Generator: read lines from file"""
    with open(filename) as f:
        for line in f:
            yield line.strip()

def filter_pattern(lines, pattern):
    """Generator: filter lines containing pattern"""
    for line in lines:
        if pattern in line:
            yield line

def extract_field(lines, field_num, delimiter=','):
    """Generator: extract specific field"""
    for line in lines:
        fields = line.split(delimiter)
        if len(fields) > field_num:
            yield fields[field_num]

# Build pipeline
lines = read_lines("large_data.txt")
filtered = filter_pattern(lines, "ERROR")
timestamps = extract_field(filtered, 0, ' ')

# Execute pipeline lazily
for ts in timestamps:
    print(f"Timestamp: {ts}")
    break  # Only process first one
```

2.4 Basic Chunking with Pandas

```
import pandas as pd

# Read CSV file in chunks
chunk_size = 100000  # Number of rows per chunk
total_rows = 0
total_sum = 0

print(f"Processing file in chunks of {chunk_size} rows...")

for chunk in pd.read_csv('large_file.csv', chunksize=chunk_size):
    # Process each chunk
    rows_in_chunk = len(chunk)
    total_rows += rows_in_chunk
    total_sum += chunk['value_column'].sum()

    print(f"Processed chunk with {rows_in_chunk} rows. Total so far: {
    total_rows}")

average = total_sum / total_rows if total_rows > 0 else 0
print(f"\nFinal result: Average = {average:.2f}")
```

2.5 Chunking with Multiple Operations

```
import pandas as pd

def process_large_file(filename, chunk_size=50000):
    """
```

```
    Process large file with multiple aggregations
    """
    results = {
        'total_rows': 0,
        'sum_amount': 0,
        'max_amount': float('-inf'),
        'min_amount': float('inf'),
        'unique_categories': set()
    }

    for chunk in pd.read_csv(filename, chunksize=chunk_size):
        # Update aggregations
        results['total_rows'] += len(chunk)
        results['sum_amount'] += chunk['amount'].sum()
        results['max_amount'] = max(results['max_amount'], chunk['amount'].max
    ())
        results['min_amount'] = min(results['min_amount'], chunk['amount'].min
    ())

        # Add unique values
        results['unique_categories'].update(chunk['category'].unique())

        print(f"Processed chunk. Total rows: {results['total_rows']}")

    # Calculate final average
    results['avg_amount'] = results['sum_amount'] / results['total_rows']

    return results

# Usage
stats = process_large_file('sales_data.csv')
print(f"Total rows: {stats['total_rows']}")
print(f"Total amount: ${stats['sum_amount']:,.2f}")
print(f"Average amount: ${stats['avg_amount']:.2f}")
print(f"Unique categories: {len(stats['unique_categories'])}")
```

2.6 Filtering While Reading in Chunks

```
import pandas as pd

# Read and filter in chunks
chunk_size = 100000
filtered_chunks = []

for chunk in pd.read_csv('transactions.csv', chunksize=chunk_size):
    # Filter each chunk
    filtered = chunk[chunk['amount'] > 1000]
    if len(filtered) > 0:
        filtered_chunks.append(filtered)
        print(f"Found {len(filtered)} rows with amount > 1000 in this chunk")

# Combine only the filtered results
if filtered_chunks:
    result = pd.concat(filtered_chunks, ignore_index=True)
    print(f"\nTotal filtered rows: {len(result)}")
```

2.7 Column Selection with Chunking

```
import pandas as pd

```

```python
# Only load specific columns to save memory
columns_needed = ['id', 'name', 'amount', 'date']

chunk_size = 100000
total_amount = 0

for chunk in pd.read_csv('transactions.csv',
                         chunksize=chunk_size,
                         usecols=columns_needed):
    # Work with only the columns we need
    total_amount += chunk['amount'].sum()
    print(f"Running total: ${total_amount:,.2f}")

print(f"\nFinal total amount: ${total_amount:,.2f}")
```

2.8 Custom Iterator Class

```python
class FileChunkIterator:
    """
    Custom iterator that reads files in chunks
    """
    def __init__(self, filename, chunk_size=1024):
        self.filename = filename
        self.chunk_size = chunk_size
        self.file = None
        self.eof = False

    def __iter__(self):
        return self

    def __enter__(self):
        self.file = open(self.filename, 'rb')
        return self

    def __exit__(self, *args):
        if self.file:
            self.file.close()

    def __next__(self):
        if self.eof:
            raise StopIteration

        chunk = self.file.read(self.chunk_size)
        if not chunk:
            self.eof = True
            raise StopIteration

        return chunk

# Usage
with FileChunkIterator('large_file.bin', chunk_size=4096) as it:
    for i, chunk in enumerate(it):
        print(f"Chunk {i+1}: {len(chunk)} bytes")
        if i >= 4:  # Stop after 5 chunks
            break
```

2.9 CSV Chunk Reader with Iterator Protocol

```python
import csv

```

```python
class CSVChunkReader:
    """
    Read CSV file in chunks using iterator protocol
    """
    def __init__(self, filename, chunk_size=1000):
        self.filename = filename
        self.chunk_size = chunk_size
        self.file = None
        self.reader = None
        self.header = None

    def __enter__(self):
        self.file = open(self.filename, 'r')
        self.reader = csv.reader(self.file)
        self.header = next(self.reader)  # Read header
        return self

    def __exit__(self, *args):
        if self.file:
            self.file.close()

    def __iter__(self):
        return self

    def __next__(self):
        """Return next chunk of rows"""
        chunk = []
        for i in range(self.chunk_size):
            try:
                row = next(self.reader)
                chunk.append(row)
            except StopIteration:
                break
        if not chunk:
            raise StopIteration
        return chunk

# Usage
with CSVChunkReader('large_data.csv', chunk_size=1000) as reader:
    for chunk_num, chunk in enumerate(reader, 1):
        print(f"Chunk {chunk_num}: {len(chunk)} rows")
        if chunk_num >= 5:
            break
```

2.10 Infinite Sequence Generator

```python
def fibonacci_generator():
    """Generate Fibonacci numbers infinitely"""
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

# Get first 10 Fibonacci numbers
fib = fibonacci_generator()
first_ten = [next(fib) for _ in range(10)]
print(f"First 10 Fibonacci: {first_ten}")

# Get next 5
next_five = [next(fib) for _ in range(5)]
print(f"Next 5: {next_five}")
```

2.11 Using itertools for Lazy Processing

```
from itertools import islice, count

# count: infinite counter
counter = count(start=1, step=1)
first_10 = list(islice(counter, 10))
print(f"First 10 counts: {first_10}")

# islice: slice any iterator
numbers = range(1000000)
first_5 = list(islice(numbers, 5))
last_5 = list(islice(numbers, 999995, 1000000))
print(f"First 5: {first_5}")
```

2.12 Complete Large File Processor Class

```
import pandas as pd

class LargeCSVProcessor:
    """
    Complete solution for processing large CSV files
    """
    def __init__(self, filename, chunk_size=50000):
        self.filename = filename
        self.chunk_size = chunk_size
        self.total_rows = 0
        self.chunk_count = 0

    def process(self, operations):
        """
        Apply operations to each chunk and combine results
        """
        results = {}
        for chunk in pd.read_csv(self.filename, chunksize=self.chunk_size):
            self.chunk_count += 1
            self.total_rows += len(chunk)
            print(f"Processing chunk {self.chunk_count} ({len(chunk)} rows)")

            # Apply each operation to this chunk
            for op_name, op_func in operations.items():
                chunk_result = op_func(chunk)

                # Combine results
                if op_name not in results:
                    results[op_name] = chunk_result
                else:
                    if isinstance(chunk_result, (int, float)):
                        results[op_name] += chunk_result
                    elif isinstance(chunk_result, set):
                        results[op_name].update(chunk_result)

        results['total_rows'] = self.total_rows
        results['chunks'] = self.chunk_count
        return results

# Define operations
def sum_amount(chunk):
    return chunk['amount'].sum()

def avg_amount(chunk):
    return chunk['amount'].mean()
```

```

def unique_categories(chunk):
    return set(chunk['category'].unique())

# Usage
processor = LargeCSVProcessor('sales_data.csv', chunk_size=100000)
operations = {
    'total_amount': sum_amount,
    'unique_categories': unique_categories
}
results = processor.process(operations)

print(f"\nProcessed {results['total_rows']:,} rows in {results['chunks']}
    chunks")
print(f"Total amount: ${results['total_amount']:,.2f}")
print(f"Unique categories: {len(results['unique_categories'])}")
```

2.13 Memory-Efficient Log File Analysis

```
def analyze_log_file(filename):
    """
    Analyze large log file line by line
    """
    error_count = 0
    warning_count = 0
    unique_ips = set()

    with open(filename, 'r') as f:
        for line_num, line in enumerate(f, 1):
            if "ERROR" in line:
                error_count += 1
            elif "WARNING" in line:
                warning_count += 1

            # Extract IP if present
            parts = line.split()
            if len(parts) > 0 and '.' in parts[0]:
                unique_ips.add(parts[0])

            # Progress indicator
            if line_num % 100000 == 0:
                print(f"Processed {line_num:,} lines...")

    print(f"\nAnalysis complete:")
    print(f"Total lines: {line_num:,}")
    print(f"Errors: {error_count:,}")
    print(f"Warnings: {warning_count:,}")
    print(f"Unique IPs: {len(unique_ips):,}")

# analyze_log_file("webserver.log")
```

2.14 JSON Streaming

```
import json

def stream_json(filename):
    """
    Stream JSON objects from a file containing
    one JSON object per line
    """
```

```
    with open(filename, 'r') as f:
        for line in f:
            yield json.loads(line)

# Usage
for record in stream_json('large_data.jsonl'):
    if record.get('status') == 'active':
        print(f"Active record: {record['id']}")
        break  # Process only first one
```

Summary of Key Programs

- **Basic Generator** — Line-by-line file reading; Use for log files and text processing.
- **Pandas Chunking** — CSV aggregation; Use for large CSV files.
- **Filtering Chunks** — Selective data extraction; Use when filtering large datasets.
- **Custom Iterator** — Binary file chunking; Use for non-text files.
- **Generator Pipeline** — Multi-step processing; Use for complex data transformations.
- **CSVChunkReader** — Custom CSV handling; Use when pandas is not available.
- **LargeCSVProcessor** — Complete solution; Use in production applications.

Chapter-3
Advanced Data Wrangling and Transformation

Data Wrangling and Transformation

This section covers essential theory of data wrangling and transformation for exam preparation and practical data analysis.

1. Definition and Importance of Data Wrangling

Data wrangling is the process of cleaning, transforming, and mapping raw data from one form into another format suitable for downstream purposes such as analytics, machine learning, or visualization.

The majority of real-world data is not ready for immediate analysis. Before applying algorithms or statistical methods, data must be cleaned and transformed. This preparatory work often consumes more time and effort than analysis itself—sometimes up to 80% of project time.

Data cleaning and transformation are integral parts of exploratory data analysis (EDA). Understanding a dataset inevitably reveals inconsistencies, missing values, and structural issues that must be addressed before meaningful analysis.

Data analysis is rarely linear; it involves an iterative cycle of cleaning, transforming, visualizing, and modeling. Proficiency in data wrangling tools enables smooth navigation through this cycle.

Robust data processing requires attention to data formats, encoding, and error handling. Production pipelines must handle malformed records gracefully.

2. Tidy Data Concept

Tidy data is a standardized way of structuring datasets:

- Each variable forms a column
- Each observation forms a row
- Each type of observational unit forms a table

This framework ensures consistent organization, making data easier to manipulate, visualize, and model. Most Pandas operations are designed to work seamlessly with tidy data.

3. Data Types and Structures

3.1 Basic Data Types

- Numeric: Integers and floating-point numbers

- Text/String: Character data
- Boolean: True/False values
- DateTime: Temporal data with special operations
- Categorical: Limited number of distinct values

3.2 Common Data Structures in Python

- **Series**: One-dimensional labeled array holding any data type
- **DataFrame**: Two-dimensional labeled data structure with columns of potentially different types
- **Panel**: Three-dimensional data structure (less commonly used)

4. Data Cleaning

Data cleaning involves identifying and correcting errors, inconsistencies, and inaccuracies.

4.1 Common Data Quality Issues

- Missing data
- Duplicate data
- Inconsistent formatting
- Outliers
- Invalid values
- Inconsistent categorizations

4.2 Approaches to Missing Data

- Removal: Drop rows/columns with missing values
- Imputation: Fill missing values with mean, median, mode, or forward/backward fill
- Interpolation: Estimate missing values from surrounding data
- Flagging: Create indicator columns for missingness

4.3 Data Type Conversion

- String to numeric (e.g., "1,234" $\rightarrow$ 1234)
- String to datetime
- Categorical encoding

4.4 String Cleaning

- Stripping whitespace
- Changing case
- Removing/replacing characters
- Extracting patterns with regular expressions
- Splitting and combining columns

5. Data Transformation

5.1 Filtering and Selection

- Select subsets of rows based on conditions
- Select subsets of columns
- Random sampling

5.2 Creating New Columns

- Derived from existing columns (ratios, differences)
- Applying functions
- Conditional assignments

5.3 Handling Duplicates

- Identify duplicate rows
- Remove duplicates
- Keep first or last occurrence

5.4 Discretization and Binning

- Equal-width binning
- Equal-frequency binning
- Custom bin edges

5.5 Encoding Categorical Variables

- Label encoding: integers for categories
- One-hot encoding: binary columns per category
- Dummy encoding: k-1 columns to avoid multicollinearity

5.6 Normalization and Standardization

- Normalization: scale data to [0,1] or [-1,1]
- Standardization: center data (mean=0, variance=1)

5.7 Applying Functions

- Element-wise operations
- Row-wise or column-wise operations
- Window/rolling operations

6. Combining Datasets

6.1 Concatenation

Stack datasets vertically (add rows) or horizontally (add columns). Attention to indices is important.

6.2 Merging and Joining

Combine datasets based on keys or indices (like SQL joins):

- Inner join: Keep matching keys only
- Outer join: Keep all keys, fill missing with NaN
- Left join: Keep all keys from left
- Right join: Keep all keys from right

6.3 Key Considerations for Joins

- Key columns may differ in name
- One-to-one, one-to-many, many-to-many relationships
- Overlapping column names handled with suffixes
- Index-based vs. column-based joining

7. Reshaping Data

7.1 Wide vs. Long Format

- Wide: One row per subject, multiple columns
- Long: Multiple rows per subject, key column indicates measurement type

7.2 Pivoting (Long to Wide)

Convert long format to wide format. Useful for summary tables or modeling.

7.3 Melting (Wide to Long)

Convert wide format to long format (key-value pairs), often needed for visualization.

7.4 Stacking and Unstacking

- Stack: Columns $\rightarrow$ rows (creates MultiIndex)
- Unstack: Rows $\rightarrow$ columns

8. Group Operations

8.1 Split-Apply-Combine Strategy

- Split: Break data into groups
- Apply: Perform operation per group
- Combine: Assemble results into new structure

8.2 Common Group Operations

- Aggregation: summary statistics per group
- Transformation: group-specific computations aligned with original data
- Filtration: discard groups based on properties
- Apply arbitrary functions

9. Handling Time Series Data

9.1 Date and Time Fundamentals

- Parsing from strings
- Time zone handling
- Date ranges and frequencies

9.2 Time Series Operations

- Resampling (changing frequency)
- Time shifts and lags
- Rolling windows
- Date-based indexing and selection

10. Common Challenges in Data Wrangling

10.1 Scale and Performance

- Large datasets that don't fit in memory
- Computational efficiency
- Vectorization vs. iteration

10.2 Data Inconsistencies

- Naming convention differences
- Unit mismatches (metric vs. imperial)
- Different levels of aggregation

10.3 Reproducibility

- Document transformation steps
- Reusable pipelines
- Version control for data and transformations

10.4 Data Provenance

- Track data origin
- Record transformations applied
- Maintain audit trails

Summary for Quick Revision

- Data wrangling: Cleaning, transforming, mapping raw data to usable formats; consumes majority of analysis time
- Tidy data framework: Variables as columns, observations as rows, each unit as a separate table
- Data cleaning: Handle missing values, duplicates, inconsistent formatting, outliers, invalid values
- Data transformation: Filtering, creating columns, binning, encoding, normalization
- Combining datasets: Concatenation and merging/joining based on keys
- Reshaping: Pivoting, melting, stacking/unstacking
- Group operations: Split-apply-combine paradigm
- Time series: Resampling, shifting, rolling windows
- Challenges: Scale, inconsistency, reproducibility, provenance

Merging and Joining DataFrames

1.1 Theory

Merging combines DataFrames based on common columns or indices using `pd.merge()`. It is similar to SQL joins.

Types of joins:

- **inner**: keep only rows with matching keys in both DataFrames
- **outer**: keep all rows, fill missing with NaN
- **left**: keep all rows from left DataFrame, match from right
- **right**: keep all rows from right DataFrame, match from left

Key parameters:

- `on`: column(s) to join on (must exist in both)
- `left_on, right_on`: specify different key columns

- `left_index, right_index`: use index as join key
- `suffixes`: tuple of strings to append to overlapping column names
- `how`: type of join (`'inner'`, `'outer'`, `'left'`, `'right'`)

`DataFrame.join()` joins on index by default and can also join on a column using `on`.

1.2 Example Programs

a) Inner join

```
import pandas as pd

df1 = pd.DataFrame({'key': ['A', 'B', 'C', 'D'],
                    'value1': [1, 2, 3, 4]})
df2 = pd.DataFrame({'key': ['B', 'D', 'E', 'F'],
                    'value2': [5, 6, 7, 8]})

merged_inner = pd.merge(df1, df2, on='key', how='inner')
print(merged_inner)
```

Output:

key	value1	value2
B	2	5
D	4	6

b) Left, right, and outer joins

```
# Left join
merged_left = pd.merge(df1, df2, on='key', how='left')
print(merged_left)

# Right join
merged_right = pd.merge(df1, df2, on='key', how='right')
print(merged_right)

# Outer join
merged_outer = pd.merge(df1, df2, on='key', how='outer')
print(merged_outer)
```

Left join output:

key	value1	value2
A	1	NaN
B	2	5
C	3	NaN
D	4	6

Right join output:

key	value1	value2
B	2	5
D	4	6
E	NaN	7
F	NaN	8

Outer join output:

key	value1	value2
A	1	NaN
B	2	5
C	3	NaN
D	4	6
E	NaN	7
F	NaN	8

c) Merging on different column names

```
df1 = pd.DataFrame({'key1': ['A', 'B', 'C'],
                    'value': [1, 2, 3]})
df2 = pd.DataFrame({'key2': ['B', 'C', 'D'],
                    'score': [4, 5, 6]})

merged_diff = pd.merge(df1, df2, left_on='key1', right_on='key2')
print(merged_diff)
```

Output:

key1	value	key2	score
B	2	B	4
C	3	C	5

d) Using join method (index-based)

```
df1 = pd.DataFrame({'A': [1, 2, 3]}, index=['x', 'y', 'z'])
df2 = pd.DataFrame({'B': [4, 5, 6]}, index=['x', 'y', 'w'])

joined_left = df1.join(df2, how='left')
print(joined_left)
```

Output:

Index	A	B
x	1	4
y	2	5
z	3	NaN

Reshaping DataFrames

Reshaping changes the layout of a DataFrame without altering the data itself.

- `stack()`: Compresses a level from the column index to the row index, creating a MultiIndex Series.
- `unstack()`: Inverse of stack; pivots a level of the row index to the column index.
- `melt()`: Unpivots a DataFrame from wide format to long format; identifier columns remain.
- `pivot()`: Creates a reshaped DataFrame from long to wide format; requires unique index/column combinations. For duplicates, use `pivot_table()`.

2.2 Example Programs

a) stack() and unstack()

```
import pandas as pd

df = pd.DataFrame({'A': [1, 2], 'B': [3, 4]}, index=['X', 'Y'])
print(df)

stacked = df.stack()
print(stacked)

unstacked = stacked.unstack()
print(unstacked)
```

Original DataFrame:

Index	A	B
X	1	3
Y	2	4

Stacked output:

Index	Column	Value
X	A	1
X	B	3
Y	A	2
Y	B	4

Unstacked output (back to original):

Index	A	B
X	1	3
Y	2	4

b) melt() – wide to long

```
df_wide = pd.DataFrame({
    'id': [1, 2, 3],
    'name': ['Alice', 'Bob', 'Charlie'],
    'math': [85, 90, 78],
    'science': [92, 88, 81]
})

melted = pd.melt(df_wide, id_vars=['id','name'],
                 value_vars=['math','science'],
                 var_name='subject', value_name='score')
print(melted)
```

Melted output:

id	name	subject	score
1	Alice	math	85
2	Bob	math	90
3	Charlie	math	78
1	Alice	science	92
2	Bob	science	88
3	Charlie	science	81

c) pivot() – long to wide (unique combinations)

```
pivoted = melted.pivot(index='id', columns='subject', values='score')
pivoted.reset_index(inplace=True)
pivoted.columns.name = None
print(pivoted)
```

Pivoted output:

id	math	science
1	85	92
2	90	88
3	78	81

d) pivot_table() – aggregation when duplicates exist

```
df_sales2 = pd.DataFrame({
    'date': ['2023-01-01','2023-01-01','2023-01-01','2023-01-02'],
    'product': ['A','A','B','B'],
    'sales': [100,50,150,175]
})

pivot_sum = df_sales2.pivot_table(index='date',
                                  columns='product',
                                  values='sales',
                                  aggfunc='sum')
print(pivot_sum)
```

Pivot table with sum aggregation:

date	A	B
2023-01-01	150	150
2023-01-02	NaN	175

Advanced: MultiIndex and Reshaping

Theory

MultiIndex (hierarchical index) allows working with higher-dimensional data in a 2D structure.

Common operations:

- `set_index()` / `reset_index()`: create or flatten MultiIndex
- `swaplevel()`: swap levels of a MultiIndex
- `sort_index()`: sort by index levels
- Stacking/Unstacking with MultiIndex

3.2 Example: Creating and Unstacking MultiIndex

```
import pandas as pd

# Create MultiIndex DataFrame
arrays = [['A', 'A', 'B', 'B'], [1, 2, 1, 2]]
index = pd.MultiIndex.from_arrays(arrays, names=['letter','number'])
df = pd.DataFrame({'value': [10, 20, 30, 40]}, index=index)
print(df)

# Unstack the 'number' level
unstacked = df.unstack(level='number')
print(unstacked)
```

Original MultiIndex DataFrame:

letter	number	value
A	1	10
A	2	20
B	1	30
B	2	40

Unstacked DataFrame (number level → columns):

letter	1	2
A	10	20
B	30	40

4. Summary for Quick Revision

Merging (`pd.merge`):

- Combine DataFrames on columns.
- `how`: inner, outer, left, right.
- `left_on`/`right_on`: for different key names.
- `suffixes`: handle overlapping column names.

Joining (`df.join`):

- Convenient index-based join.

Reshaping:

- `stack()`: columns → rows.
- `unstack()`: rows → columns.
- `melt()`: wide → long (unpivot).
- `pivot()`: long → wide (when no duplicates).
- `pivot_table()`: long → wide with aggregation (handles duplicates).

Common Pitfalls:

- Forgetting to specify `how` in merge may lead to default inner join (data loss).
- Using `pivot()` with duplicates raises error; use `pivot_table()`.
- Not resetting index after pivoting leads to MultiIndex columns.
- Confusing `id_vars` and `value_vars` in `melt()`.

Handling Missing Data

Missing data is a common issue in real-world datasets. Proper handling is crucial because most analytical methods cannot process missing values directly.

Types of Missing Data:

- **Missing Completely at Random (MCAR):** The probability of missing is the same for all observations.
- **Missing at Random (MAR):** The probability of missing depends on observed data.
- **Missing Not at Random (MNAR):** The probability of missing depends on unobserved data.

Common Representations of Missing Data:

- `NaN` (Not a Number) in pandas
- `None` in Python
- Placeholder values like -999, 0, or empty strings

Detection Methods:

- `isnull()` / `isna()`: Detect missing values
- `notnull()` / `notna()`: Detect non-missing values
- `info()`: Summary of non-null counts per column
- `isnull().sum()`: Count missing values per column

Handling Strategies:

- **Removal / Dropping:** Use `dropna()` to remove rows or columns with missing values. Suitable when missing data is minimal and random.
- **Imputation / Filling:** Use `fillna()` to replace missing values.
  - **Constant imputation:** Fill with a specific value (e.g., 0, "Unknown").
  - **Statistical imputation:** Use mean, median, or mode.
  - **Forward/backward fill:** Propagate previous or next value.
  - **Interpolation:** Estimate missing values based on surrounding data.
- **Flagging:** Create indicator columns to mark missing values. This preserves information about the missingness pattern.
- **Advanced Methods:**
  - Model-based imputation using regression or machine learning.
  - Multiple imputation for statistical inference.

1.2 Example Programs

a) Detecting Missing Values

```
import pandas as pd
import numpy as np

# Create DataFrame with missing values
df = pd.DataFrame({
    'A': [1, 2, np.nan, 4, 5],
    'B': [np.nan, 2, 3, np.nan, 5],
    'C': ['a', 'b', np.nan, 'd', 'e'],
    'D': [10, 20, 30, 40, 50]
})

print("Original DataFrame:")
print(df)

# Check for missing values
print(df.isnull())

# Count missing per column
print(df.isnull().sum())

# Total missing values
print(f"Total missing values: {df.isnull().sum().sum()}")

# Rows with any missing values
print(df[df.isnull().any(axis=1)])

# Rows with no missing values
print(df[df.notnull().all(axis=1)])
```

Output Table: Original DataFrame

A	B	C	D
1	NaN	a	10
2	2	b	20
NaN	3	NaN	30
4	NaN	d	40
5	5	e	50

b) Dropping Missing Values

```
# Drop rows with any missing value
df_drop_rows = df.dropna()
print(df_drop_rows)

# Drop rows where all values are missing
df_all_missing = pd.DataFrame({
    'X': [np.nan, 2, np.nan],
    'Y': [np.nan, 3, np.nan],
    'Z': [np.nan, 4, np.nan]
})
```

```
print(df_all_missing.dropna(how='all'))

# Drop columns with any missing
df_drop_cols = df.dropna(axis=1)
print(df_drop_cols)

# Drop columns with fewer than a threshold of non-null values
df_thresh = df.dropna(axis=1, thresh=3)
print(df_thresh)
```

Output Table: Drop Rows with Any Missing Values

A	B	C
2	2	b
5	5	e

Output Table: Drop Rows Where All Values are Missing

X	Y	Z
2	3	4

Output Table: Drop Columns with Any Missing Values

C
a
b
NaN
d
e

Output Table: Keep Columns with At Least 3 Non-Null Values

A	C
1	a
2	b
NaN	NaN
4	d
5	e

Handling Missing Data

1. Detecting Missing Values

```
import pandas as pd
import numpy as np

# Create DataFrame with missing values
df = pd.DataFrame({
    'A': [1, 2, np.nan, 4, 5],
    'B': [np.nan, 2, 3, np.nan, 5],
    'C': ['a', 'b', np.nan, 'd', 'e'],
    'D': [10, 20, 30, 40, 50]
})

print("Original DataFrame:")
print(df)

# Check for missing values
print("\nIs null?")
```

```
print(df.isnull())

# Count missing per column
print("\nMissing count per column:")
print(df.isnull().sum())

# Total missing values
print(f"\nTotal missing values: {df.isnull().sum().sum()}")

# Rows with any missing
print("\nRows with any missing:")
print(df[df.isnull().any(axis=1)])

# Rows with all values present
print("\nRows with no missing:")
print(df[df.notnull().all(axis=1)])
```

Output:

```
Original DataFrame:
     A    B    C   D
0  1.0  NaN    a  10
1  2.0  2.0    b  20
2  NaN  3.0  NaN  30
3  4.0  NaN    d  40
4  5.0  5.0    e  50

Is null?
       A      B      C      D
0   False   True  False  False
1   False  False  False  False
2    True  False   True  False
3   False   True  False  False
4   False  False  False  False

Missing count per column:
A    1
B    2
C    1
D    0
dtype: int64

Total missing values: 4

Rows with any missing:
     A    B    C   D
0  1.0  NaN    a  10
2  NaN  3.0  NaN  30
3  4.0  NaN    d  40

Rows with no missing:
     A    B  C   D
```

```
1  2.0  2.0  b  20
4  5.0  5.0  e  50
```

2. Dropping Missing Values

```
# Drop rows with any missing value
df_drop_rows = df.dropna()
print("\nDrop rows with any missing:")
print(df_drop_rows)

# Drop rows where all values are missing
df_all_missing = pd.DataFrame({
    'X': [np.nan, 2, np.nan],
    'Y': [np.nan, 3, np.nan],
    'Z': [np.nan, 4, np.nan]
})
print("\nDrop rows where all values are missing:")
print(df_all_missing.dropna(how='all'))

# Drop columns with any missing
df_drop_cols = df.dropna(axis=1)
print("\nDrop columns with any missing:")
print(df_drop_cols)

# Drop columns with fewer than a threshold of non-null values
df_thresh = df.dropna(axis=1, thresh=3)
print("\nKeep columns with at least 3 non-null:")
print(df_thresh)
```

Output:

```
Drop rows with any missing:
     A    B  C   D
1  2.0  2.0  b  20
4  5.0  5.0  e  50

Drop rows where all values are missing:
     X    Y    Z
0  NaN  NaN  NaN
1  2.0  3.0  4.0
2  NaN  NaN  NaN

Drop columns with any missing:
   D
0  10
1  20
2  30
3  40
4  50

Keep columns with at least 3 non-null:
     A    B   D
0  1.0  NaN  10
```

```
1  2.0  2.0  20
2  NaN  3.0  30
3  4.0  NaN  40
4  5.0  5.0  50
```

3. Filling Missing Values

```
df = pd.DataFrame({
    'A': [1, 2, np.nan, 4, 5, np.nan],
    'B': [10, np.nan, 30, np.nan, 50, 60],
    'C': [100, 200, 300, 400, 500, 600]
})

# Fill with constant
df_fill_constant = df.fillna(0)
print("\nFill with 0:")
print(df_fill_constant)

# Fill with column-specific values
df_fill_dict = df.fillna({'A':999, 'B':-1})
print("\nFill with column-specific values:")
print(df_fill_dict)

# Forward fill
df_ffill = df.fillna(method='ffill')
print("\nForward fill:")
print(df_ffill)

# Backward fill
df_bfill = df.fillna(method='bfill')
print("\nBackward fill:")
print(df_bfill)

# Fill with mean
df_fill_mean = df.copy()
df_fill_mean['A'] = df['A'].fillna(df['A'].mean())
df_fill_mean['B'] = df['B'].fillna(df['B'].mean())
print("\nFill with column means:")
print(df_fill_mean)

# Limit forward fill
df_limit = df.fillna(method='ffill', limit=1)
print("\nForward fill with limit=1:")
print(df_limit)
```

Output:

```
Fill with 0:
     A     B    C
0  1.0  10.0  100
1  2.0   0.0  200
2  0.0  30.0  300
3  4.0   0.0  400
```

```
4  5.0  50.0  500
5  0.0  60.0  600

Fill with column-specific values:
       A   B    C
0    1.0  10  100
1    2.0  -1  200
2  999.0  30  300
3    4.0  -1  400
4    5.0  50  500
5  999.0  60  600

Forward fill:
     A     B    C
0  1.0  10.0  100
1  2.0  10.0  200
2  2.0  30.0  300
3  4.0  30.0  400
4  5.0  50.0  500
5  5.0  60.0  600

Backward fill:
     A     B    C
0  1.0  10.0  100
1  2.0  30.0  200
2  4.0  30.0  300
3  4.0  50.0  400
4  5.0  50.0  500
5  NaN  60.0  600

Fill with column means:
          A     B    C
0  1.000000  10.0  100
1  2.000000  37.5  200
2  3.000000  30.0  300
3  4.000000  37.5  400
4  5.000000  50.0  500
5  3.000000  60.0  600

Forward fill with limit=1:
     A     B    C
0  1.0  10.0  100
1  2.0  10.0  200
2  2.0  30.0  300
3  4.0  30.0  400
4  5.0  50.0  500
5  5.0  60.0  600
```

4. Interpolation

```
df = pd.DataFrame({'value':[1,2,np.nan,np.nan,5,6,np.nan,8,9,10]})
```

```
# Linear
df_linear = df.interpolate(method='linear')
print("\nLinear interpolation:")
print(df_linear)

# Polynomial order 2
df_poly = df.interpolate(method='polynomial', order=2)
print("\nPolynomial interpolation:")
print(df_poly)

# Time-based
dates = pd.date_range('2023-01-01', periods=10)
df_time = pd.DataFrame({'value':[1,2,np.nan,np.nan,5,6,np.nan,8,9,10]}, index=dates)
df_time_interp = df_time.interpolate(method='time')
print("\nTime-based interpolation:")
print(df_time_interp)
```

Output:

```
Linear interpolation:
    value
0   1.0
1   2.0
2   3.0
3   4.0
4   5.0
5   6.0
6   7.0
7   8.0
8   9.0
9  10.0

Polynomial interpolation:
     value
0   1.0
1   2.0
2   3.0
3   4.0
4   5.0
5   6.0
6   7.0
7   8.0
8   9.0
9  10.0

Time-based interpolation:
            value
2023-01-01    1.0
2023-01-02    2.0
2023-01-03    3.0
2023-01-04    4.0
2023-01-05    5.0
```

```
2023-01-06    6.0
2023-01-07    7.0
2023-01-08    8.0
2023-01-09    9.0
2023-01-10   10.0
```

5. Flagging Missing Values

```
df = pd.DataFrame({
    'age':[25,30,np.nan,35,40,np.nan],
    'income':[50000,60000,75000,np.nan,82000,90000]
})

# Create flags
df['age_missing'] = df['age'].isnull()
df['income_missing'] = df['income'].isnull()

# Fill missing while preserving flags
df['age_filled'] = df['age'].fillna(df['age'].mean())
df['income_filled'] = df['income'].fillna(df['income'].median())
print(df)
```

Output:

```
    age   income  age_missing  income_missing  age_filled  income_filled
0  25.0  50000.0        False           False   25.000000        50000.0
1  30.0  60000.0        False           False   30.000000        60000.0
2   NaN  75000.0         True           False   32.5            75000.0
3  35.0      NaN        False            True   35.000000        71000.0
4  40.0  82000.0        False           False   40.000000        82000.0
5   NaN  90000.0         True           False   32.5            90000.0
```

6. Complete Missing Data Handling Pipeline

```
df_raw = pd.DataFrame({
    'id':[1,2,3,4,5,6],
    'name':['Alice','Bob',np.nan,'David','Eve','Frank'],
    'age':[25,np.nan,35,np.nan,45,50],
    'salary':[50000,60000,np.nan,80000,np.nan,100000],
    'department':['HR','IT','IT',np.nan,'HR','Finance']
})

# Analyze
missing_summary = df_raw.isnull().sum()
missing_percent = (df_raw.isnull().sum()/len(df_raw))*100

# Clean
df_clean = df_raw.dropna(subset=['name'])
df_clean['age'] = df_clean['age'].fillna(df_clean['age'].median())
df_clean['salary_missing'] = df_clean['salary'].isnull()
df_clean['salary'] = df_clean['salary'].fillna(df_clean['salary'].mean())
df_clean['department'] = df_clean['department'].fillna(df_clean['department'].mode()[0])
```

```
print(df_clean)
print(df_clean.isnull().sum())
```

Output:

```
   id   name   age   salary department  salary_missing
0   1  Alice  25.0  50000.0         HR           False
1   2    Bob  37.5  60000.0         IT           False
3   4  David  37.5  80000.0         HR           False
4   5    Eve  45.0  71250.0         HR            True
5   6  Frank  50.0 100000.0    Finance           False

id                0
name              0
age               0
salary            0
department        0
salary_missing    0
dtype: int64
```

2. Handling Categorical Data

Theory

Categorical data represents variables with a limited, fixed number of possible values. Proper handling is essential for statistical analysis and machine learning.

Types of Categorical Data

- **Nominal:** Categories with no inherent order. *Examples:* Colors, countries, gender, department names.
- **Ordinal:** Categories with a natural order. *Examples:* Education level (High School ¡ Bachelor's ¡ Master's ¡ PhD), ratings (poor, average, good).
- **Binary:** Special case with only two categories. *Examples:* Yes/No, True/False, Male/Female.

Representation in `pandas`

- `object dtype`: String values
- `category dtype`: Special categorical type (memory efficient)
- `bool dtype`: Boolean values

Common Operations

Frequency Analysis:

- `value_counts()`: Count occurrences of each category.
- `crosstab()`: Create contingency tables for two categorical variables.

Encoding Methods:

- **Label Encoding:** Convert categories to integers (preserves order for ordinal variables).
- **One-Hot Encoding:** Create binary columns for each category.
- **Dummy Encoding:** One-hot with $k-1$ columns to avoid multicollinearity.
- **Ordinal Encoding:** Map categories to ordered integers.

Creating Categorical Variables:

- `cut()`: Bin continuous data into categories.
- `qcut()`: Bin continuous data based on quantiles.
- `astype('category')`: Convert a column to categorical type.

Handling New Categories:

- Plan for unseen categories in test data.
- Use a special "other" category or implement appropriate handling.

Factorizing:

- `factorize()`: Encode categorical data as integer categories.

2.2 Example Programs

a) Creating and Exploring Categorical Data

```
import pandas as pd
import numpy as np

# Create DataFrame with categorical columns
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'department': ['HR', 'IT', 'IT', 'Finance', 'HR'],
    'rating': ['Good', 'Excellent', 'Average', 'Good', 'Poor'],
    'gender': ['F', 'M', 'M', 'M', 'F'],
    'salary': [50000, 80000, 75000, 90000, 55000]
})

print(df)
```

Output:

name	department	rating	gender	salary
Alice	HR	Good	F	50000
Bob	IT	Excellent	M	80000
Charlie	IT	Average	M	75000
David	Finance	Good	M	90000
Eve	HR	Poor	F	55000

Table 1: Original DataFrame

```
# Convert to categorical type
df['department'] = df['department'].astype('category')
df['rating'] = df['rating'].astype('category')
df['gender'] = df['gender'].astype('category')

print(df.dtypes)
```

Output:

Column	Data Type
name	object
department	category
rating	category
gender	category
salary	int64

Table 2: Data types after converting to categorical

```
print("Department categories:", df['department'].cat.categories)
print("Rating categories:", df['rating'].cat.categories)
print("Gender categories:", df['gender'].cat.categories)
```

Output:

Column	Categories
department	Finance, HR, IT
rating	Average, Excellent, Good, Poor
gender	F, M

Table 3: Categorical column categories

b) Value Counts and Frequency Analysis

```
df = pd.DataFrame({
    'department': ['HR', 'IT', 'IT', 'Finance', 'HR', 'IT', 'HR', 'Finance'],
    'rating': ['Good', 'Excellent', 'Average', 'Good', 'Poor', 'Excellent', '
    Good', 'Average']
})

# Department counts
print(df['department'].value_counts())

# Rating counts
print(df['rating'].value_counts())

# Rating counts normalized
print(df['rating'].value_counts(normalize=True))

# Cross-tabulation
print(pd.crosstab(df['department'], df['rating']))

# Cross-tab with margins
print(pd.crosstab(df['department'], df['rating'], margins=True))
```

Output:

Department	Count
HR	3
IT	3
Finance	2

Rating	Count
Good	3
Excellent	2
Average	2
Poor	1

Department	Average	Excellent	Good	Poor
Finance	1	0	1	0
HR	0	0	2	1
IT	1	2	0	0

Table 4: Cross-tabulation Department vs Rating

Department	Average	Excellent	Good	Poor	All
Finance	1	0	1	0	2
HR	0	0	2	1	3
IT	1	2	0	0	3
All	2	2	3	1	8

Table 5: Cross-tab with totals

c) Label Encoding (Ordinal Encoding)

```
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'rating': ['Good', 'Excellent', 'Average', 'Good', 'Poor']
})

# Manual mapping
rating_map = {'Poor':1, 'Average':2, 'Good':3, 'Excellent':4}
df['rating_encoded'] = df['rating'].map(rating_map)

# Factorize
df['rating_factorized'], unique = pd.factorize(df['rating'])
print(df)
print("Unique values:", unique.tolist())
```

Output:

name	rating	rating_encoded	rating_factorized
Alice	Good	3	0
Bob	Excellent	4	1
Charlie	Average	2	2
David	Good	3	0
Eve	Poor	1	3

Table 6: Label encoding outputs

d) One-Hot Encoding

```
df = pd.DataFrame({
    'id': [1,2,3,4,5],
    'department': ['HR','IT','IT','Finance','HR'],
    'gender': ['F','M','M','M','F']
```

```
})

df_encoded = pd.get_dummies(df, columns=['department','gender'])
df_dummy = pd.get_dummies(df, columns=['department','gender'], drop_first=True)
df_prefix = pd.get_dummies(df, columns=['department'], prefix='dept')
```

Output (one-hot encoded):

id	department_Finance	department_HR	department_IT	gender_F	gender_M
1	0	1	0	1	0
2	0	0	1	0	1
3	0	0	1	0	1
4	1	0	0	0	1
5	0	1	0	1	0

Table 7: One-hot encoding output

3. Handling Time-Series Data

3.1 Theory

Time-series data consists of observations indexed by time. Proper handling requires specialized techniques and understanding of temporal structures.

Key Concepts:

- **Timestamp:** Specific point in time, e.g., 2023-03-11 09:00:00.
- **Time Period:** Interval between two timestamps.
- **Time Delta:** Duration or difference between times.
- **Frequency:** Regularity of observations (daily, monthly, hourly, etc.).

Common Operations:

1. Date/Time Parsing

- `pd.to_datetime()`: Convert strings to datetime objects.
- `pd.date_range()`: Create sequences of dates with specified frequency.

2. Indexing and Selection

- `DatetimeIndex` as row index for efficient selection.
- Partial string indexing: e.g., `df['2023-01']` selects all January 2023 data.
- Slicing with dates: `df['2023-01-01':'2023-01-31']`.

3. Resampling

- Change frequency of time series: upsampling or downsampling.
- Aggregation functions (mean, sum, etc.) for downsampling.
- Interpolation methods for upsampling.

4. Shifting and Lags

- `shift()`: Move data forward or backward in time.
- Create lagged features for predictive modeling.

5. Rolling Windows

- `rolling()`: Apply functions over a sliding time window.
- Use for moving averages, rolling statistics, or feature engineering.

6. Time Zone Handling

- `tz_localize()`: Assign timezone to naive datetime.
- `tz_convert()`: Convert datetime between timezones.

7. Date Components Extraction

- Extract `year`, `month`, `day`, `weekday`, `hour`, `minute`, etc., from datetime for analysis or modeling.

3.2 Example Programs

a) Creating Datetime Objects and Indexes

```
import pandas as pd
import numpy as np
from datetime import datetime

# Create Timestamp from string
ts1 = pd.Timestamp('2023-01-15')
ts2 = pd.Timestamp('2023-12-31')

# Create from datetime object
dt = datetime(2023, 6, 15, 14, 30)
ts3 = pd.Timestamp(dt)

# Create date range
dates = pd.date_range(start='2023-01-01', end='2023-01-10', freq='D')

# Create with specific frequency
month_starts = pd.date_range(start='2023-01-01', periods=5, freq='MS')  # Month start
business_days = pd.date_range(start='2023-01-01', periods=10, freq='B')  # Business days

# Create DataFrame with datetime index
df = pd.DataFrame({'value': np.random.randn(len(dates))}, index=dates)
```

b) Parsing and Converting to Datetime

```
# Various date string formats
df = pd.DataFrame({
    'date_str1': ['2023-01-15', '2023-02-20', '2023-03-25'],
    'date_str2': ['15/01/2023', '20/02/2023', '25/03/2023'],
    'date_str3': ['Jan 15, 2023', 'Feb 20, 2023', 'Mar 25, 2023'],
    'value': [100, 200, 300]
```

```
})

# Parse different formats
df['date1'] = pd.to_datetime(df['date_str1'])
df['date2'] = pd.to_datetime(df['date_str2'], format='%d/%m/%Y')
df['date3'] = pd.to_datetime(df['date_str3'])

# Handle errors
df_bad = pd.DataFrame({
    'date': ['2023-01-15', 'invalid_date', '2023-03-25'],
    'value': [100, 200, 300]
})
df_bad['date_coerce'] = pd.to_datetime(df_bad['date'], errors='coerce')
```

c) Indexing and Selecting Time Series Data

```
# Create time series data
dates = pd.date_range('2023-01-01', periods=100, freq='D')
df = pd.DataFrame({
    'value': np.random.randn(100).cumsum(),
    'volume': np.random.randint(100, 1000, 100)
}, index=dates)

# Select by year
df['2023']

# Select by year-month
df['2023-01']

# Select by date range
df['2023-01-15':'2023-01-25']

# Select with loc on specific date
df.loc['2023-01-15']

# Extract date components
df['year'] = df.index.year
df['month'] = df.index.month
df['day'] = df.index.day
df['dayofweek'] = df.index.dayofweek  # Monday=0, Sunday=6
df['quarter'] = df.index.quarter
print("\nWith date components:")
print(df[['value', 'year', 'month', 'day', 'dayofweek', 'quarter']].head())
```

d) Resampling Time Series Data

```
import pandas as pd
import numpy as np

# Create high-frequency data (hourly)
dates = pd.date_range('2023-01-01 00:00', periods=24*7, freq='H')  # One week of
    hourly data
df = pd.DataFrame({'value': np.random.randn(len(dates)).cumsum()}, index=dates)

# Downsampling: Hourly to daily
daily_mean = df.resample('D').mean()
daily_sum = df.resample('D').sum()
daily_max = df.resample('D').max()

# Downsampling with multiple aggregations
daily_stats = df.resample('D').agg(['mean', 'std', 'min', 'max'])
```

```
15
16 # Create lower-frequency data (daily)
17 dates_daily = pd.date_range('2023-01-01', periods=30, freq='D')
18 df_daily = pd.DataFrame({'value': np.random.randn(30).cumsum()}, index=
       dates_daily)
19
20 # Upsampling: Daily to hourly (interpolation)
21 hourly_interpolated = df_daily.resample('H').interpolate(method='linear')
22
23 # Upsampling with forward fill
24 hourly_ffill = df_daily.resample('H').ffill()
25
26 # Custom resampling functions
27 def custom_range(x):
28     return x.max() - x.min()
29
30 daily_range = df.resample('D').apply(custom_range)
```

e) Shifting and Lagging

```
1 import pandas as pd
2 import numpy as np
3
4 # Create simple time series
5 dates = pd.date_range('2023-01-01', periods=10, freq='D')
6 df = pd.DataFrame({'value': [10, 12, 15, 14, 18, 20, 22, 21, 25, 28]}, index=
      dates)
7
8 # Shift forward (lag) and backward (lead)
9 df['lag_1'] = df['value'].shift(1)
10 df['lag_2'] = df['value'].shift(2)
11 df['lag_-1'] = df['value'].shift(-1)
12
13 # Create features for time series prediction
14 df['value_lag1'] = df['value'].shift(1)
15 df['value_lag2'] = df['value'].shift(2)
16 df['value_lag3'] = df['value'].shift(3)
17 df['value_lead1'] = df['value'].shift(-1)
18
19 # Calculate differences and percentage change
20 df['diff_1'] = df['value'].diff(1)
21 df['diff_2'] = df['value'].diff(2)
22 df['pct_change'] = df['value'].pct_change()
```

f) Rolling Window Operations

```
1 import pandas as pd
2 import numpy as np
3
4 # Create time series data
5 dates = pd.date_range('2023-01-01', periods=30, freq='D')
6 df = pd.DataFrame({'value': np.random.randn(30).cumsum()}, index=dates)
7
8 print("Original data:")
9 print(df.head(10))
10
11 # Rolling mean (moving average)
12 df['rolling_mean_3'] = df['value'].rolling(window=3).mean()
13 df['rolling_mean_7'] = df['value'].rolling(window=7).mean()
14
15 print("\nWith rolling means:")
```

```
print(df.head(10))

# Other rolling statistics
df['rolling_std_3'] = df['value'].rolling(window=3).std()
df['rolling_max_3'] = df['value'].rolling(window=3).max()
df['rolling_min_3'] = df['value'].rolling(window=3).min()
df['rolling_sum_3'] = df['value'].rolling(window=3).sum()

print("\nWith various rolling statistics:")
print(df.head(10))

# Rolling with custom function
df['rolling_range_3'] = df['value'].rolling(window=3).apply(lambda x: x.max() - x.min())
print("\nRolling range (max-min) with window 3:")
print(df[['value', 'rolling_range_3']].head(10))

# Centered rolling window
df['centered_ma_3'] = df['value'].rolling(window=3, center=True).mean()
print("\nCentered moving average (window 3):")
print(df[['value', 'centered_ma_3']].head(10))

# Rolling with min_periods
df['rolling_min5_3'] = df['value'].rolling(window=3, min_periods=2).mean()
print("\nRolling mean with min_periods=2:")
print(df[['value', 'rolling_min5_3']].head(10))
```

g) Time Zone Handling

```
import pandas as pd
import numpy as np

# Create time series without timezone
dates = pd.date_range('2023-01-01 09:00', periods=5, freq='H')
df = pd.DataFrame({'value': [100, 110, 120, 130, 140]}, index=dates)

print("Original (naive):")
print(df)

# Localize to a timezone (assign timezone)
df_tz = df.copy()
df_tz.index = df_tz.index.tz_localize('UTC')
print("\nLocalized to UTC:")
print(df_tz)

# Convert to another timezone
df_ny = df_tz.tz_convert('America/New_York')
print("\nConverted to New York time:")
print(df_ny)

df_london = df_tz.tz_convert('Europe/London')
print("\nConverted to London time:")
print(df_london)

# Create with timezone directly
dates_tz = pd.date_range('2023-01-01 09:00', periods=5, freq='H', tz='Asia/Tokyo')
df_tokyo = pd.DataFrame({'value': [200, 210, 220, 230, 240]}, index=dates_tz)
print("\nCreated with Tokyo timezone:")
print(df_tokyo)
```

h) Extracting Date Components

```python
import pandas as pd
import numpy as np

# Create time series
dates = pd.date_range('2023-01-01', periods=365, freq='D')
df = pd.DataFrame({'value': np.random.randn(365).cumsum()}, index=dates)

# Extract various components
df['year'] = df.index.year
df['month'] = df.index.month
df['day'] = df.index.day
df['dayofweek'] = df.index.dayofweek  # Monday=0, Sunday=6
df['day_name'] = df.index.day_name()
df['month_name'] = df.index.month_name()
df['quarter'] = df.index.quarter
df['week'] = df.index.isocalendar().week
df['dayofyear'] = df.index.dayofyear
df['is_month_end'] = df.index.is_month_end
df['is_month_start'] = df.index.is_month_start
df['is_quarter_end'] = df.index.is_quarter_end

print("Date components (first 10 rows):")
print(df.head(10))

# Group by date components
monthly_avg = df.groupby('month')['value'].mean()
print("\nMonthly averages:")
print(monthly_avg)

# Count by day of week
dow_counts = df.groupby('day_name').size()
print("\nCount by day of week:")
print(dow_counts)
```

i) Complete Time Series Pipeline

```python
import pandas as pd
import numpy as np
from datetime import datetime, timedelta

# Generate sample sales data
np.random.seed(42)
dates = pd.date_range('2023-01-01', '2023-12-31', freq='D')
df = pd.DataFrame({
    'date': dates,
    'sales': np.random.poisson(lam=100, size=len(dates)) +
             np.sin(np.arange(len(dates)) * 2 * np.pi / 365) * 20 +  # Seasonal
             np.random.randn(len(dates)) * 10  # Noise
})

print("Raw sales data:")
print(df.head())

# Step 1: Set datetime as index
df['date'] = pd.to_datetime(df['date'])
df.set_index('date', inplace=True)

# Step 2: Check for missing dates
date_range = pd.date_range(start=df.index.min(), end=df.index.max(), freq='D')
if len(df) < len(date_range):
```

```
    print(f"Missing {len(date_range) - len(df)} dates")
    df = df.reindex(date_range)

# Step 3: Handle any missing values
if df['sales'].isnull().any():
    print(f"Filling {df['sales'].isnull().sum()} missing values")
    df['sales'] = df['sales'].interpolate(method='time')

# Step 4: Extract time features
df['year'] = df.index.year
df['month'] = df.index.month
df['day'] = df.index.day
df['dayofweek'] = df.index.dayofweek
df['day_name'] = df.index.day_name()
df['quarter'] = df.index.quarter
df['weekend'] = (df.index.dayofweek >= 5).astype(int)

# Step 5: Create lag features for prediction
for lag in [1, 2, 3, 7, 14, 30]:
    df[f'sales_lag_{lag}'] = df['sales'].shift(lag)

# Step 6: Create rolling statistics
df['rolling_mean_7'] = df['sales'].rolling(window=7).mean()
df['rolling_std_7'] = df['sales'].rolling(window=7).std()
df['rolling_mean_30'] = df['sales'].rolling(window=30).mean()

# Step 7: Create differences
df['sales_diff_1'] = df['sales'].diff(1)
df['sales_diff_7'] = df['sales'].diff(7)
df['sales_pct_change'] = df['sales'].pct_change()

# Step 8: Resample to different frequencies
weekly_sales = df['sales'].resample('W').sum()
monthly_sales = df['sales'].resample('M').mean()
quarterly_sales = df['sales'].resample('Q').sum()

print("\nProcessed data (first 10 rows):")
print(df.head(10))
print(f"\nColumns in processed data: {df.columns.tolist()}")
print(f"Shape: {df.shape}")

print("\nWeekly sales (first 5 weeks):")
print(weekly_sales.head())

print("\nMonthly average sales:")
print(monthly_sales.head())
```

Quick Revision Summary

1. Missing Data

- **Detection:** `isnull()`, `notnull()`, `info()`, `isnull().sum()`
- **Removal:** `dropna()` with parameters `how`, `thresh`, `subset`
- **Imputation:** `fillna()` with constant, method, or statistics
- **Interpolation:** `interpolate()` with various methods
- **Flagging:** Create indicator columns for missingness

2. Categorical Data

- **Types:**
  - Nominal (no order)
  - Ordinal (has order)
  - Binary
- **Analysis:** `value_counts()`, `crosstab()`, `groupby()`
- **Encoding:**
  - Label encoding (`map`)
  - One-hot (`get_dummies`)
  - Dummy (`drop_first`)
- **Binning:** `cut()` (equal-width), `qcut()` (equal-frequency)
- **Categorical dtype:** `astype('category')` for memory efficiency

3. Time-Series Data

- **Creation:** `to_datetime()`, `date_range()`, `Timestamp`
- **Indexing:** DatetimeIndex, partial string indexing, slicing
- **Resampling:** `resample()` with aggregation (downsampling) or interpolation (upsampling)
- **Shifting:** `shift()` for lags/leads, `diff()` for differences, `pct_change()` for percentage change
- **Rolling:** `rolling()` for window statistics (mean, std, min, max, sum, custom)
- **Components:** Extract `year`, `month`, `day`, `dayofweek`, `quarter`, etc.
- **Time zones:** `tz_localize()`, `tz_convert()`

Feature Transformation, Scaling, and Encoding

1. Feature Transformation

1.1 Theory

Feature transformation involves applying mathematical functions to modify the distribution or relationships of features. It is often used to:

- Make data more normally distributed (many statistical models assume normality)
- Stabilize variance
- Linearize relationships
- Reduce skewness

1.2 Common Transformations

- **Log Transformation**: $\log(x)$ — Useful for right-skewed positive data.
- **Square Root Transformation**: $\sqrt{x}$ — Useful for count data or moderately skewed data.
- **Reciprocal Transformation**: $1/x$ — Useful when values represent rates or ratios.
- **Box-Cox Transformation**: $(x^\lambda - 1)/\lambda$ (if $\lambda \neq 0$) — General power transformation for positive data.
- **Yeo-Johnson Transformation**: Similar to Box-Cox but can handle negative values — Useful for data with zeros or negative values.
- **Power Transformation**: x^λ — General power transformation for adjusting skewness or variance.

1.3 Key Considerations

- Log transformation requires positive values (add a constant if zeros exist)
- Box-Cox automatically finds the optimal λ
- Transformations should be applied to training data, then the same parameters used on test/validation data

Common Transformations with Examples

1. Log Transformation

Formula:

$$y = \log(x) \quad \text{or} \quad y = \log(1 + x) \quad (x \geq 0)$$

Purpose: Compresses the right tail of a distribution; suitable for right-skewed positive data. Use $\log(1 + x)$ if zeros are present.

Example: $x = [1, 2, 5, 10, 100]$

$$\log(x) = [0.0000, 0.6931, 1.6094, 2.3026, 4.6052]$$

2. Square Root Transformation

Formula:

$$y = \sqrt{x}$$

Purpose: Reduces moderate right skewness; often used for count data.

Example: $x = [1, 4, 9, 16, 25]$

$$\sqrt{x} = [1.0000, 2.0000, 3.0000, 4.0000, 5.0000]$$

3. Reciprocal Transformation

Formula:

$$y = \frac{1}{x}$$

Purpose: Useful for rates or ratios; strongly reduces right skewness, sensitive to small values.

Example: $x = [1, 2, 5, 10, 20]$

$$\frac{1}{x} = [1.0000, 0.5000, 0.2000, 0.1000, 0.0500]$$

4. Box–Cox Transformation

Formula:

$$y(\lambda) = \begin{cases} \dfrac{x^\lambda - 1}{\lambda}, & \lambda \neq 0 \\ \log(x), & \lambda = 0 \end{cases}$$

Purpose: Power transformation to make data more normal; requires $x > 0$.

Example: $x = [1, 2, 3, 4, 5]$, $\lambda = 2$

$$y = \frac{x^2 - 1}{2} = [0.0, 1.5, 4.0, 7.5, 12.0]$$

5. Yeo–Johnson Transformation

Formula:

$$y(\lambda) = \begin{cases} \dfrac{(x+1)^\lambda - 1}{\lambda}, & x \geq 0, \lambda \neq 0 \\ \log(x+1), & x \geq 0, \lambda = 0 \\ -\dfrac{(-x+1)^{2-\lambda} - 1}{2 - \lambda}, & x < 0, \lambda \neq 2 \\ -\log(-x+1), & x < 0, \lambda = 2 \end{cases}$$

Purpose: Extension of Box–Cox; handles zero and negative values.
Example: $x = [-2, -1, 0, 1, 2]$, $\lambda = 0.5$

$$y \approx [-2.7975, -1.2189, 0, 0.8284, 1.4641]$$

6. Power Transformation

Formula:

$$y = x^{\lambda}$$

Purpose: General transformation to reduce skewness or stabilize variance.
Example: $x = [1, 2, 3, 4, 5]$, $\lambda = 2$

$$x^2 = [1, 4, 9, 16, 25]$$

Summary of Transformations

Transformation	Formula	Example Input	Example Output
Log	$\log(x)$	10	2.3026
Square Root	$\sqrt{x}$	16	4
Reciprocal	$1/x$	5	0.2
Box–Cox	$(x^{\lambda} - 1)/\lambda$ $(\lambda \neq 0)$	4, $\lambda = 2$	7.5
Yeo–Johnson	Piecewise	-1, $\lambda = 0.5$	-1.2189
Power	x^{λ}	3, $\lambda = 2$	9

Table 8: Common feature transformations with formulas and examples

a) Log Transformation

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Create skewed data
np.random.seed(42)
data = np.random.exponential(scale=2, size=1000)
df = pd.DataFrame({'original': data})

# Apply log transformation
df['log'] = np.log(df['original'])
df['log1p'] = np.log1p(df['original'])  # handles zeros

# Compare skewness
print("Original - Skewness:", df['original'].skew())
print("Log - Skewness:", df['log'].skew())
print("Log1p - Skewness:", df['log1p'].skew())

# Optional visualization
# df.hist(bins=50, figsize=(10,4))
# plt.show()
```

Listing 1: Log Transformation on Skewed Data

b) Square Root Transformation

```
import pandas as pd
import numpy as np

# Poisson count data
np.random.seed(42)
counts = np.random.poisson(lam=5, size=1000)
df = pd.DataFrame({'counts': counts})

# Square root transformation
df['sqrt'] = np.sqrt(df['counts'])

print("Original - Skewness:", df['counts'].skew())
print("Sqrt - Skewness:", df['sqrt'].skew())
```

Listing 2: Square Root Transformation for Count Data

c) Box-Cox Transformation

```
import pandas as pd
import numpy as np
from scipy import stats

# Generate right-skewed data
np.random.seed(42)
data = np.random.gamma(shape=2, scale=2, size=1000)
df = pd.DataFrame({'original': data})

# Box-Cox transformation (add 1 if zeros present)
df['boxcox'], fitted_lambda = stats.boxcox(df['original'] + 1)

print(f"Optimal lambda: {fitted_lambda:.4f}")
print("Original - Skewness:", df['original'].skew())
print("Box-Cox - Skewness:", df['boxcox'].skew())
```

Listing 3: Box-Cox Transformation

d) Yeo-Johnson Transformation (handles negative values)

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import PowerTransformer

# Data with negative values
np.random.seed(42)
data = np.random.randn(1000) * 2 + 1
df = pd.DataFrame({'original': data})

# Yeo-Johnson via PowerTransformer
pt = PowerTransformer(method='yeo-johnson')
df['yeojohnson'] = pt.fit_transform(df[['original']])

print("Original - Skewness:", df['original'].skew())
print("Yeo-Johnson - Skewness:", df['yeojohnson'].skew())
print("Lambda used:", pt.lambdas_[0])
```

Listing 4: Yeo-Johnson Transformation using scikit-learn

e) Applying Transformation in a Pipeline

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PowerTransformer
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import r2_score

# Generate synthetic data
np.random.seed(42)
X = np.random.exponential(size=(1000, 1))
y = 2 * np.log(X).ravel() + np.random.randn(1000) * 0.5

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Pipeline: transform then model
pipeline = make_pipeline(
    PowerTransformer(method='box-cox'),  # automatically finds lambda
    LinearRegression()
)
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
print(f"R  score: {r2_score(y_test, y_pred):.4f}")
```

Listing 5: Feature Transformation in ML Pipeline

Feature Scaling

Theory

Feature scaling adjusts the range or distribution of numerical features. It is essential for algorithms that rely on distance measures, such as k-NN, SVM, or PCA, and for methods that use gradient descent, like neural networks and linear models with regularization.

Common Scaling Techniques

- **Standardization (Z-score):** Transform data by subtracting the mean and dividing by the standard deviation:
$$z = \frac{x - \mu}{\sigma}$$
Resulting distribution has mean 0 and variance 1. Useful when data is roughly Gaussian and for many machine learning algorithms.

- **Min-Max Scaling:** Scale data to a fixed range, typically [0,1], using:
$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$
Useful when a bounded range is needed, e.g., for neural networks.

- **Robust Scaling:** Center data by the median and scale according to the interquartile range (IQR):
$$x' = \frac{x - \text{median}}{\text{IQR}}$$
Robust to outliers and effective when data contains extreme values.

- **MaxAbs Scaling:** Scale data by the maximum absolute value:

$$x' = \frac{x}{\max(|x|)}$$

  Produces a range [-1,1] and is suitable for sparse data.

Important Considerations

- Always fit the scaler on the training data only, then apply the transformation to validation or test sets.
- Standardization does not guarantee a fixed range, whereas Min-Max scaling produces an exact bounded range.

Example Programs

a) Standardization (Z-score)

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

# Sample data
df = pd.DataFrame({
    'age': [25, 30, 35, 40, 45],
    'income': [50000, 60000, 75000, 90000, 120000]
})

scaler = StandardScaler()
scaled = scaler.fit_transform(df)
df_scaled = pd.DataFrame(scaled, columns=df.columns)

print("Original mean:\n", df.mean())
print("Original std:\n", df.std())
print("\nAfter scaling mean:\n", df_scaled.mean())
print("After scaling std:\n", df_scaled.std())
```

b) Min-Max Scaling

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

df = pd.DataFrame({
    'age': [25, 30, 35, 40, 45],
    'income': [50000, 60000, 75000, 90000, 120000]
})

scaler = MinMaxScaler()
scaled = scaler.fit_transform(df)
df_scaled = pd.DataFrame(scaled, columns=df.columns)

print("Original min:\n", df.min())
print("Original max:\n", df.max())
print("\nAfter scaling min:\n", df_scaled.min())
print("After scaling max:\n", df_scaled.max())
```

c) Robust Scaling (Median and IQR)

```
import pandas as pd
from sklearn.preprocessing import RobustScaler

# Include an outlier
df = pd.DataFrame({
    'value': [10, 12, 11, 100, 13, 14, 12]  # 100 is outlier
})

scaler = RobustScaler()
scaled = scaler.fit_transform(df)
df['scaled'] = scaled

print("Original:\n", df)
print("\nMedian:", df['value'].median())
print("IQR:", df['value'].quantile(0.75) - df['value'].quantile(0.25))
print("Scaled median:", df['scaled'].median())
```

d) Applying Scaling in a Pipeline

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error

# Generate data with different scales
np.random.seed(42)
X = np.random.rand(100, 2) * [100, 0.1]  # one feature large scale, one small
y = X[:, 0] * 2 + X[:, 1] * 100 + np.random.randn(100) * 5

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# SVM without scaling
svm_raw = SVR()
svm_raw.fit(X_train, y_train)
y_pred_raw = svm_raw.predict(X_test)
mse_raw = mean_squared_error(y_test, y_pred_raw)

# SVM with scaling
pipeline = make_pipeline(StandardScaler(), SVR())
pipeline.fit(X_train, y_train)
y_pred_scaled = pipeline.predict(X_test)
mse_scaled = mean_squared_error(y_test, y_pred_scaled)

print(f"MSE without scaling: {mse_raw:.4f}")
print(f"MSE with scaling:    {mse_scaled:.4f}")
```

e) Comparison of Scaling Methods

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler,
    MaxAbsScaler

# Create data with outliers
np.random.seed(42)
```

```
data = np.concatenate([np.random.normal(0, 1, 50), np.random.normal(10, 1, 5)])  # outliers
df = pd.DataFrame({'original': data})

scalers = {
    'Standard': StandardScaler(),
    'MinMax': MinMaxScaler(),
    'Robust': RobustScaler(),
    'MaxAbs': MaxAbsScaler()
}

for name, scaler in scalers.items():
    scaled = scaler.fit_transform(df[['original']])
    df[name] = scaled

print(df.describe())
```

Example Programs

a) Standardization (Z-score)

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

# Sample data
df = pd.DataFrame({
    'age': [25, 30, 35, 40, 45],
    'income': [50000, 60000, 75000, 90000, 120000]
})

scaler = StandardScaler()
scaled = scaler.fit_transform(df)
df_scaled = pd.DataFrame(scaled, columns=df.columns)

print("Original mean:\n", df.mean())
print("Original std:\n", df.std())
print("\nAfter scaling mean:\n", df_scaled.mean())
print("After scaling std:\n", df_scaled.std())
```

b) Min-Max Scaling

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

df = pd.DataFrame({
    'age': [25, 30, 35, 40, 45],
    'income': [50000, 60000, 75000, 90000, 120000]
})

scaler = MinMaxScaler()
scaled = scaler.fit_transform(df)
df_scaled = pd.DataFrame(scaled, columns=df.columns)

print("Original min:\n", df.min())
print("Original max:\n", df.max())
print("\nAfter scaling min:\n", df_scaled.min())
print("After scaling max:\n", df_scaled.max())
```

c) Robust Scaling (Median and IQR)

```
import pandas as pd
from sklearn.preprocessing import RobustScaler

# Include an outlier
df = pd.DataFrame({
    'value': [10, 12, 11, 100, 13, 14, 12]  # 100 is outlier
})

scaler = RobustScaler()
scaled = scaler.fit_transform(df)
df['scaled'] = scaled

print("Original:\n", df)
print("\nMedian:", df['value'].median())
print("IQR:", df['value'].quantile(0.75) - df['value'].quantile(0.25))
print("Scaled median:", df['scaled'].median())
```

d) Applying Scaling in a Pipeline

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error

# Generate data with different scales
np.random.seed(42)
X = np.random.rand(100, 2) * [100, 0.1]  # one feature large scale, one small
y = X[:, 0] * 2 + X[:, 1] * 100 + np.random.randn(100) * 5

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# SVM without scaling
svm_raw = SVR()
svm_raw.fit(X_train, y_train)
y_pred_raw = svm_raw.predict(X_test)
mse_raw = mean_squared_error(y_test, y_pred_raw)

# SVM with scaling
pipeline = make_pipeline(StandardScaler(), SVR())
pipeline.fit(X_train, y_train)
y_pred_scaled = pipeline.predict(X_test)
mse_scaled = mean_squared_error(y_test, y_pred_scaled)

print(f"MSE without scaling: {mse_raw:.4f}")
print(f"MSE with scaling:    {mse_scaled:.4f}")
```

e) Comparison of Scaling Methods

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler,
    MaxAbsScaler

# Create data with outliers
np.random.seed(42)
```

```
data = np.concatenate([np.random.normal(0, 1, 50), np.random.normal(10, 1, 5)])
    # outliers
df = pd.DataFrame({'original': data})

scalers = {
    'Standard': StandardScaler(),
    'MinMax': MinMaxScaler(),
    'Robust': RobustScaler(),
    'MaxAbs': MaxAbsScaler()
}

for name, scaler in scalers.items():
    scaled = scaler.fit_transform(df[['original']])
    df[name] = scaled

print(df.describe())
```

Feature Encoding

Theory

Feature encoding converts categorical data into numerical form suitable for machine learning algorithms. Proper encoding ensures models can interpret categorical variables correctly.

Types of Categorical Encoding:

- **Label Encoding:** Assigns a unique integer to each category. Useful for ordinal data or tree-based models.
- **One-Hot Encoding:** Creates binary columns for each category. Suitable for nominal data and linear models.
- **Dummy Encoding:** Similar to one-hot but uses $k-1$ columns to avoid multicollinearity. Used when intercept is present.
- **Ordinal Encoding:** Maps categories to ordered integers. Useful for ordinal data with a known order.
- **Frequency Encoding:** Replaces each category with its frequency. Effective for high cardinality features and tree models.
- **Target Encoding:** Replaces each category with the mean of the target variable. Suitable for classification/regression tasks with high cardinality features. Must use cross-validation to avoid overfitting.
- **Binary Encoding:** Converts categories to binary digits and splits them into separate columns. Memory-efficient for high-cardinality features.

Key Considerations:

- One-hot encoding can create many columns for features with high cardinality.
- Target encoding can lead to overfitting; always use cross-validation.
- For tree-based models, simple label encoding may be sufficient.

Feature Encoding Examples with Input and Output

a) Label Encoding

```python
import pandas as pd
from sklearn.preprocessing import LabelEncoder

df = pd.DataFrame({'color': ['red', 'blue', 'green', 'blue', 'red']})

# Using sklearn
le = LabelEncoder()
df['color_encoded'] = le.fit_transform(df['color'])

# Using pandas factorize
df['color_factor'] = pd.factorize(df['color'])[0]

print(df)
print("Classes:", le.classes_)
# Output:
#    color  color_encoded  color_factor
# 0    red              2             2
# 1   blue              0             0
# 2  green              1             1
# 3   blue              0             0
# 4    red              2             2
# Classes: ['blue' 'green' 'red']
```

b) One-Hot Encoding

```python
df = pd.DataFrame({
    'id': [1,2,3,4],
    'color': ['red','blue','green','blue'],
    'size': ['S','M','L','M']
})

df_encoded = pd.get_dummies(df, columns=['color','size'])
df_dummy = pd.get_dummies(df, columns=['color','size'], drop_first=True)

print(df_encoded)
print(df_dummy)
# Output:
# df_encoded:
#    id  color_blue  color_green  color_red  size_L  size_M  size_S
# 0   1           0            0          1       0       0       1
# 1   2           1            0          0       0       1       0
# 2   3           0            1          0       1       0       0
# 3   4           1            0          0       0       1       0
# df_dummy:
#    id  color_green  color_red  color_blue  size_M  size_S
# 0   1            0          1           0       0       1
# 1   2            0          0           1       1       0
# 2   3            1          0           0       0       0
# 3   4            0          0           1       1       0
```

c) Ordinal Encoding

```python
df = pd.DataFrame({'education': ['Bachelor','Master','PhD','Bachelor','Master','PhD']})
education_order = {'Bachelor':1, 'Master':2, 'PhD':3}
df['education_ordinal'] = df['education'].map(education_order)
```

```
print(df)
# Output:
#    education  education_ordinal
# 0   Bachelor                  1
# 1     Master                  2
# 2        PhD                  3
# 3   Bachelor                  1
# 4     Master                  2
# 5        PhD                  3
```

d) Frequency Encoding

```
df = pd.DataFrame({'city': ['NYC','LA','NYC','Chicago','LA','NYC','Chicago','Boston']})
freq = df['city'].value_counts(normalize=True)
df['city_freq'] = df['city'].map(freq)
print(df)
# Output:
#      city  city_freq
# 0     NYC      0.375
# 1      LA      0.250
# 2     NYC      0.375
# 3  Chicago      0.250
# 4      LA      0.250
# 5     NYC      0.375
# 6  Chicago      0.250
# 7   Boston      0.125
```

e) Target Encoding (Mean Encoding)

```
df = pd.DataFrame({
    'category': ['A','B','A','C','B','A','C','B','A','C'],
    'target': [1,0,1,0,1,0,1,0,1,1]
})

# Simple target encoding
target_map = df.groupby('category')['target'].mean()
df['target_encoded_simple'] = df['category'].map(target_map)

# Cross-validated target encoding
from sklearn.model_selection import KFold
import numpy as np
df['target_encoded_cv'] = np.nan
kf = KFold(n_splits=3, shuffle=True, random_state=42)
for train_idx, val_idx in kf.split(df):
    train, val = df.iloc[train_idx], df.iloc[val_idx]
    means = train.groupby('category')['target'].mean()
    df.loc[val_idx, 'target_encoded_cv'] = val['category'].map(means)
df['target_encoded_cv'] = df['target_encoded_cv'].fillna(df['target'].mean())

print(df)
# Output: df with columns category, target, target_encoded_simple, target_encoded_cv
```

f) Binary Encoding (Manual)

```
df = pd.DataFrame({'color': ['red','blue','green','yellow']})
unique = df['color'].unique()
mapping = {cat:i for i,cat in enumerate(unique)}
```

```
df['code'] = df['color'].map(mapping)
max_bits = len(bin(len(unique)-1))-2
for i in range(max_bits):
    df[f'bit_{i}'] = (df['code'] >> i) & 1
print(df)
# Output:
#    color  code  bit_0  bit_1
# 0    red     0      0      0
# 1   blue     1      1      0
# 2  green     2      0      1
# 3 yellow     3      1      1
```

g) Combining Encodings in a Pipeline

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

# Assume X_train, X_test, y_train, y_test exist
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), ['height']),
        ('cat', OneHotEncoder(drop='first'), ['color','size'])
    ])

pipeline = Pipeline(steps=[('preprocessor',preprocessor),
                           ('classifier', RandomForestClassifier(random_state
    =42))])
pipeline.fit(X_train, y_train)
score = pipeline.score(X_test, y_test)
print(f"Accuracy: {score:.4f}")
# Output: Accuracy ~0.80-0.95 depending on random seed
```

Complete Preprocessing Pipeline Example

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error

# Create a mixed dataset
np.random.seed(42)
n_samples = 1000
df = pd.DataFrame({
    'age': np.random.randint(18, 70, n_samples),
    'income': np.random.exponential(50, n_samples) * 1000,
    'education': np.random.choice(['High School', 'Bachelor', 'Master', 'PhD'],
    n_samples),
    'city': np.random.choice(['NYC', 'LA', 'Chicago', 'Houston', 'Phoenix'],
    n_samples),
    'purchases': np.random.poisson(5, n_samples) * 10
})

# Target: function of features + noise
df['spending'] = (df['age']*2 +
                  df['income']*0.01 +
```

```
                    (df['education'].map({'High School':1,'Bachelor':2,'Master':3,'PhD':4})*50) +
                    np.random.randn(n_samples)*100)

# Separate features and target
X = df.drop('spending', axis=1)
y = df['spending']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Preprocessing
numeric_features = ['age','income','purchases']
categorical_features = ['education','city']

numeric_transformer = Pipeline(steps=[('scaler', StandardScaler())])

categorical_transformer = Pipeline(steps=[('onehot', OneHotEncoder(drop='first', sparse_output=False))])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ])

pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', RandomForestRegressor(n_estimators=100, random_state=42))
])

# Train
pipeline.fit(X_train, y_train)

# Predict and evaluate
y_pred = pipeline.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print(f"RMSE: {rmse:.2f}")
# Sample Output: RMSE: 267.43 (may vary slightly)

# Inspect transformed feature names
preprocessor.fit(X_train)
cat_feature_names = preprocessor.named_transformers_['cat'].named_steps['onehot'].get_feature_names_out(categorical_features)
all_feature_names = numeric_features + list(cat_feature_names)
print(f"Number of features: {len(all_feature_names)}")
# Sample Output: Number of features: 17 (depends on categories)
```

Memory Optimization and Efficient Data Processing

1. Core Theory and Concepts

1.1 Python Memory Management Fundamentals

Python manages memory through:

- **Reference counting**: Each object tracks references; deallocated when count reaches zero.
- **Cyclic garbage collector**: Handles reference cycles.

Memory Profiling Tools: `tracemalloc`, `memory_profiler`, `psutil`.

1.2 Views vs. Copies

```python
# View (pointer, minimal memory)
test_list = [1, 2, 3]
view_list = test_list
view_list[1] = 4
print(test_list)  # Output: [1, 4, 3]

# Copy (independent, full memory)
test_list = [1, 2, 3]
copy_list = test_list.copy()
copy_list[1] = 4
print(test_list)  # Output: [1, 2, 3]
```

1.3 Vectorization and Broadcasting

NumPy and Pandas achieve high performance through vectorization—applying operations to entire arrays without explicit Python loops. This leverages:

- **C implementation**: Core functions are written in compiled C, avoiding Python interpreter overhead
- **SIMD instructions**: Modern processors apply one instruction across multiple variables simultaneously
- **Broadcasting**: Operations are applied element-wise without looping

Performance comparison (sum of 1 million elements):

```python
# Python loop (slow)
total = 0
for i in range(1_000_000):
    total += i

# NumPy vectorized (fast)
import numpy as np
total = np.sum(np.arange(1_000_000))
```

1.4 Chunking and Out-of-Core Processing

When datasets exceed available RAM, chunking processes data in manageable pieces:

- **Pandas chunking**: Read CSV files in chunks using `chunksize` parameter
- **Out-of-core computing**: Streams data from disk without loading entirely into memory

2.1 Use Generators and Iterators for Streaming

Process one item at a time rather than loading entire datasets:

```python
# Anti-pattern: loads all into memory
def fetch_all_pages(urls):
    results = []
    for url in urls:
        results.append(fetch_page(url))
    return results

# Optimized: generator yields one at a time
def fetch_pages_stream(urls):
    for url in urls:
        yield fetch_page(url)  # yields without storing all
```

2.2 Choose Data Structures Wisely

- **For numeric data:**
  - Use `NumPy` arrays instead of Python lists for large numeric collections.
  - Arrays store data contiguously in memory, enabling vectorized operations.
- **For objects with many instances:**
  - Using `__slots__` can reduce memory overhead by 30-50% because it prevents each object from creating a `__dict__`.

```python
# Without __slots__ (each object has __dict__ overhead)
class Product:
    def __init__(self, id, name, price):
        self.id = id
        self.name = name
        self.price = price

# With __slots__ (reduces memory 30-50%)
class ProductOptimized:
    __slots__ = ('id', 'name', 'price')
    def __init__(self, id, name, price):
        self.id = id
        self.name = name
        self.price = price
```

3.1 Optimize Data Types

Pandas often uses unnecessarily large data types. Downcasting numeric columns can dramatically reduce memory usage.

```python
import pandas as pd
import numpy as np

def reduce_mem_usage(df):
    """Iterate through columns and downcast numeric types"""
    start_mem = df.memory_usage().sum() / 1024**2
    print(f"Initial memory: {start_mem:.2f} MB")

    for col in df.columns:
        col_type = df[col].dtype

        if col_type != object:
            c_min = df[col].min()
            c_max = df[col].max()

            # Test if column can be integer
            if str(col_type)[:3] == 'int':
                if c_min >= 0:
                    if c_max < 255:
                        df[col] = df[col].astype(np.uint8)
                    elif c_max < 65535:
                        df[col] = df[col].astype(np.uint16)
                    elif c_max < 4294967295:
                        df[col] = df[col].astype(np.uint32)
                    else:
                        df[col] = df[col].astype(np.uint64)
                else:
                    if c_min > np.iinfo(np.int8).min and c_max < np.iinfo(np.
    int8).max:
```

```
                        df[col] = df[col].astype(np.int8)
                    elif c_min > np.iinfo(np.int16).min and c_max < np.iinfo(np.
    int16).max:
                        df[col] = df[col].astype(np.int16)
                    elif c_min > np.iinfo(np.int32).min and c_max < np.iinfo(np.
    int32).max:
                        df[col] = df[col].astype(np.int32)
            else:
                df[col] = df[col].astype(np.float32)

    end_mem = df.memory_usage().sum() / 1024**2
    print(f"Final memory: {end_mem:.2f} MB")
    print(f"Reduced by: {(1 - end_mem/start_mem)*100:.1f}%")
    return df
```

3.2 Use Categorical Data for Low-Cardinality Columns

Converting string columns with few unique values to `category` type saves significant memory.

```
# Before: object dtype (expensive)
df['country'] = ['USA', 'Canada', 'USA', 'Mexico', ...]  # ~50MB

# After: categorical (much smaller)
df['country'] = df['country'].astype('category')  # ~5MB
```

3.3 Read Data in Chunks

For large CSV files, never load the entire file at once. Process in manageable chunks.

```
# Bad: loads entire file into memory
df = pd.read_csv('huge_file.csv')

# Good: read and process in chunks
chunk_size = 100000
for chunk in pd.read_csv('huge_file.csv', chunksize=chunk_size):
    # Process each chunk
    process_chunk(chunk)
    # Memory is freed after each iteration
```

3.4 Drop Unnecessary Columns Early

Drop unneeded columns as early as possible to save memory.

```
# Load only the needed columns
df = pd.read_csv('data.csv', usecols=['id', 'value', 'timestamp'])

# Or drop columns after loading
df.drop(['temporary_column', 'debug_info'], axis=1, inplace=True)
```

3.5 Use Method Chaining with inplace=True

Avoid creating multiple intermediate DataFrames; use method chaining and inplace operations.

```
# Anti-pattern: creates multiple intermediate DataFrames
df_temp = df[df['value'] > 0]
df_temp = df_temp.dropna()
df_temp = df_temp.rename(columns={'old': 'new'})

# Optimized: method chaining with inplace operations
df_filtered = df[df['value'] > 0].copy()
df_filtered.dropna(inplace=True)
df_filtered.rename(columns={'old': 'new'}, inplace=True)
```

4.1 Vectorization Over Loops

NumPy vectorization avoids Python loops and is significantly faster.

```
import numpy as np
import time

# Create large array
arr = np.random.randn(10_000_000)

# Slow: Python loop
start = time.time()
result = [np.tanh(x) for x in arr]
print(f"Loop time: {time.time() - start:.2f}s")

# Fast: vectorized
start = time.time()
result = np.tanh(arr)
print(f"Vectorized time: {time.time() - start:.2f}s")
# Typically 50-100x faster
```

4.2 Use Views Instead of Copies

Using views avoids unnecessary memory usage compared to creating copies.

```
# Creates a copy (memory intensive)
arr_copy = arr[arr > 0]

# Creates a view (memory efficient)
mask = arr > 0
arr_view = arr[mask]  # Still a copy in this case, but...

# Better: use boolean indexing without creating intermediate copy
result = arr[arr > 0].mean()  # No persistent copy
```

4.3 In-Place Operations

In-place operations modify existing arrays, avoiding new memory allocation.

```
# Creates new array (memory intensive)
arr = arr + 1

# Modifies in place (memory efficient)
arr += 1
```

5.1 Parallel Pandas with Multiprocessing

For CPU-bound operations, split work across multiple cores.

```
import pandas as pd
import numpy as np
import multiprocessing as mp

def parallel_apply(df, func, cores=None):
    """Apply function to DataFrame in parallel"""
    if cores is None:
        cores = mp.cpu_count()

    # Split DataFrame
    df_split = np.array_split(df, cores)

    # Create pool
```

```
    pool = mp.Pool(cores)

    # Process in parallel
    df = pd.concat(pool.map(func, df_split))

    pool.close()
    pool.join()
    return df

# Example usage
def process_chunk(chunk):
    chunk['new_col'] = chunk['value'] * 2
    return chunk

result = parallel_apply(large_df, process_chunk)
```

5.2 Dask for Out-of-Core Computing

For datasets too large for memory, Dask provides parallel, out-of-core DataFrames.

```
import dask.dataframe as dd

# Read large CSV with Dask (doesn't load into memory)
ddf = dd.read_csv('huge_file.csv')

# Operations are lazy
result = ddf.groupby('category').value.mean()

# Compute only when needed
final_result = result.compute()
```

6.1 Using memory_profiler

```
# Install: pip install memory_profiler
from memory_profiler import profile
import numpy as np

@profile
def memory_intensive_function():
    large_list = [i for i in range(1_000_000)]
    large_array = np.arange(1_000_000)
    del large_list
    return large_array.mean()

# Run the script:
# python -m memory_profiler script.py
```

6.2 Using tracemalloc for Detailed Tracking

```
import tracemalloc
import pandas as pd

tracemalloc.start()

# Your code here
df = pd.read_csv('large_file.csv')
result = df.groupby('category').sum()

current, peak = tracemalloc.get_traced_memory()
print(f"Current memory: {current / 1024**2:.1f} MB")
```

```
print(f"Peak memory: {peak / 1024**2:.1f} MB")

tracemalloc.stop()
```

6.3 Using psutil for Process Monitoring

```
import psutil
import os

def memory_usage():
    process = psutil.Process(os.getpid())
    mem = process.memory_info().rss / 1024**2  # MB
    return mem

print(f"Memory usage: {memory_usage():.2f} MB")
```

7. Complete Optimization Pipeline Example

```
import pandas as pd
import numpy as np
import psutil
import tracemalloc

def optimized_pipeline(filepath):
    """
    Complete optimized data processing pipeline
    """
    tracemalloc.start()
    start_mem = psutil.Process().memory_info().rss / 1024**2

    # 1. Read only necessary columns in chunks
    chunk_size = 50000
    chunks = []

    for chunk in pd.read_csv(filepath, chunksize=chunk_size,
                             usecols=['id', 'category', 'value', 'timestamp']):

        # 2. Optimize data types per chunk
        chunk['category'] = chunk['category'].astype('category')

        # 3. Downcast numeric columns
        if chunk['value'].dtype == 'float64':
            chunk['value'] = pd.to_numeric(chunk['value'], downcast='float')

        # 4. Process chunk (vectorized operations)
        chunk['value_squared'] = chunk['value'] ** 2
        chunk['value_log'] = np.log1p(chunk['value'])

        # 5. Filter early
        chunk = chunk[chunk['value'] > 0]

        chunks.append(chunk)

        # 6. Clear any temporary variables
        del chunk

    # 7. Combine results
    final_df = pd.concat(chunks, ignore_index=True)

    # 8. Final aggregations
    result = final_df.groupby('category').agg({
```

```
        'value': ['mean', 'std', 'count'],
        'value_squared': 'mean'
    })

    # Memory reporting
    current, peak = tracemalloc.get_traced_memory()
    end_mem = psutil.Process().memory_info().rss / 1024**2

    print(f"Start memory: {start_mem:.1f} MB")
    print(f"End memory: {end_mem:.1f} MB")
    print(f"Peak memory (tracemalloc): {peak / 1024**2:.1f} MB")
    print(f"Memory saved: {(1 - end_mem/start_mem)*100:.1f}%")

    tracemalloc.stop()
    return result
```

Summary for Quick Revision

- **Memory Optimization Principles**
  - Measure first: Use `memory_profiler`, `tracemalloc`, `psutil`
  - Understand views vs. copies: Views share memory, copies duplicate
  - Choose right data structures: Arrays vs. lists, `__slots__` for classes
- **Pandas Optimizations**
  - Downcast numeric types: `pd.to_numeric(..., downcast='integer/float')`
  - Use categorical: `astype('category')` for low-cardinality strings
  - Read in chunks: `chunksize` parameter
  - Drop columns early: `usecols` or `drop()`
  - Avoid `apply`: Use vectorized operations
- **NumPy Optimizations**
  - Vectorize everything: Replace loops with array operations
  - Use in-place operations: `+=`, `*=` to avoid copies
  - Fancy indexing carefully: Creates copies, not views
- **Parallel Processing**
  - Multiprocessing: Split DataFrames across cores
  - Dask: Out-of-core processing for datasets larger than RAM
  - GPU computing: For deep learning, use mixed precision (FP16)
- **Key Anti-Patterns to Avoid**
  - Loading entire datasets into memory at once
  - Using Python loops instead of vectorized operations
  - Creating unnecessary copies with slicing or methods
  - Keeping large intermediate DataFrames alive
  - Using `object` dtype for categorical data

Building a Reusable Data-Cleaning Pipeline

This guide covers the theory and practical implementation of reusable data-cleaning pipelines based on authoritative Python data analysis resources. A well-designed pipeline automates repetitive cleaning tasks, ensures consistency across projects, and makes data preparation more maintainable.

1.1 What is a Data Pipeline?

A data pipeline is a series of data processing steps where the output of one step becomes the input for the next. Think of it like an assembly line in manufacturing—each step performs a specific function, transforming raw materials (messy data) into finished products (clean, analysis-ready data).

The ETL framework provides the foundation:

- **Extract:** Pull messy or scattered data from various sources (CSV files, databases, APIs, JSON)
- **Transform:** Clean, reformat, validate, and enrich the data
- **Load:** Save the polished result to a destination (new file, database, data warehouse)

1.2 Why Build Reusable Pipelines?

Without a pipeline, data cleaning becomes a maze of ever-growing Jupyter notebooks and manual steps. Each new dataset means re-running cells in the right order, copying formulas, and hoping nothing breaks. This repetition is draining, and the chance of subtle mistakes is ever present.

A reusable pipeline offers several advantages:

- **Consistency:** Apply the same cleaning rules across multiple datasets
- **Reproducibility:** Run the exact same transformations on refreshed data
- **Maintainability:** Update cleaning logic in one place rather than scattered scripts
- **Testing:** Validate each component independently
- **Documentation:** The pipeline structure itself documents the cleaning process

1.3 Key Principles of Pipeline Design

- **Modularity:** Each cleaning operation should be a separate, focused function. This makes the pipeline easier to understand, test, and modify.
- **Configuration over hard-coding:** Define cleaning rules, thresholds, and parameters in configuration files rather than embedding them directly in code.
- **Idempotency:** Running the same pipeline multiple times on the same input should produce identical output. This is crucial for reproducibility.
- **Error handling:** The pipeline should handle problematic records gracefully—capturing errors without crashing, and continuing to process valid data.
- **Reporting:** Track what changes were made during processing (duplicates removed, missing values filled, validation errors) to provide transparency.

2. Core Components of a Data-Cleaning Pipeline

2.1 Data Ingestion

The first step is reading data from various sources. A reusable pipeline should handle multiple formats:

- CSV, TSV, and other delimited files
- Excel spreadsheets
- JSON (including nested structures)
- Databases via SQL
- APIs

Best practices:

- Always check file formats and encodings before processing
- Create a consistent organization system (e.g., dictionary of DataFrames keyed by source name)
- Verify data after reading—examine a few rows to ensure proper loading

2.2 Data Profiling

Before cleaning, understand what you're working with. Profiling examines:

- Data types and their appropriateness
- Missing value counts and patterns
- Unique values and cardinality
- Statistical summaries (mean, median, min, max, quartiles)
- Potential outliers

This information guides cleaning decisions.

2.3 Data Cleaning Operations

Common cleaning tasks that belong in a pipeline include:

Missing Data Handling

- Removal of rows or columns with excessive missing values
- Imputation using mean, median, mode, or forward/backward fill
- Creating indicator columns to flag missing values

Duplicate Removal

- Removing exact duplicates
- Identifying near-duplicates using similarity thresholds

Data Type Correction

- Converting strings to numeric values
- Parsing dates and timestamps
- Converting low-cardinality columns to categorical types

Standardization

- Converting text to consistent case (upper, lower, title)
- Removing extra whitespace
- Standardizing categorical labels (e.g., "NYC" $\rightarrow$ "New York City")

Outlier Handling

- Capping or flooring extreme values
- Removing values that are clearly erroneous

2.4 Data Validation

Validation ensures data meets business rules and quality standards. Each record should be checked against a schema that defines:

- Required fields
- Expected data types
- Value ranges or allowed categories
- Format patterns (e.g., email address or phone number)

When validation fails, the pipeline should:

- Capture detailed error information
- Separate valid rows from invalid ones
- Continue processing valid data

2.5 Feature Engineering

For machine learning pipelines, feature engineering transforms raw data into model-ready features:

- Creating derived columns (e.g., age from birth date)
- Binning continuous variables
- Encoding categorical variables (one-hot, ordinal, frequency encoding)
- Scaling numerical features

2.6 Output and Reporting

The final pipeline should produce:

- Cleaned and validated datasets
- Error reports containing invalid records
- Processing statistics such as counts of duplicates removed or values imputed
- Logs for auditing, debugging, and monitoring pipeline behavior

0.0.1 3. Implementation Patterns

3.1 Function-Based Pipeline

The simplest approach chains together functions, each performing one cleaning operation:

```
def remove_duplicates(df):
    initial_count = len(df)
    df = df.drop_duplicates()
    stats = {'duplicates_removed': initial_count - len(df)}
    return df, stats

def fill_missing_values(df):
    stats = {}

    # Handle numeric columns
    num_cols = df.select_dtypes(include=['number']).columns
    for col in num_cols:
        missing = df[col].isnull().sum()
        if missing > 0:
            df[col] = df[col].fillna(df[col].median())
            stats[f'{col}_filled'] = missing

    # Handle categorical columns
    cat_cols = df.select_dtypes(include=['object', 'category']).columns
    for col in cat_cols:
        missing = df[col].isnull().sum()
        if missing > 0:
            df[col] = df[col].fillna('Unknown')
            stats[f'{col}_filled'] = missing

    return df, stats

def standardize_text(df, columns):
    for col in columns:
        if col in df.columns:
            df[col] = df[col].str.strip().str.lower()
    return df, {'text_standardized': len(columns)}

# Pipeline orchestrator
def run_pipeline(df, steps):
    """Execute a sequence of cleaning steps"""
    all_stats = {}

    for step_name, step_func, *args in steps:
        if args:
            df, stats = step_func(df, *args[0])
        else:
            df, stats = step_func(df)
        all_stats[step_name] = stats

```

```
    return df, all_stats


# Usage
steps = [
    ('remove_duplicates', remove_duplicates),
    ('fill_missing', fill_missing_values),
    ('standardize', standardize_text, ['name', 'address'])
]

clean_data, stats = run_pipeline(raw_data, steps)
```

3. Implementation Patterns

3.1 Function-Based Pipeline

The simplest approach chains together functions, each performing one cleaning operation:

```
def remove_duplicates(df):
    initial_count = len(df)
    df = df.drop_duplicates()
    stats = {'duplicates_removed': initial_count - len(df)}
    return df, stats

def fill_missing_values(df):
    stats = {}

    # Handle numeric columns
    num_cols = df.select_dtypes(include=['number']).columns
    for col in num_cols:
        missing = df[col].isnull().sum()
        if missing > 0:
            df[col] = df[col].fillna(df[col].median())
            stats[f'{col}_filled'] = missing

    # Handle categorical columns
    cat_cols = df.select_dtypes(include=['object', 'category']).columns
    for col in cat_cols:
        missing = df[col].isnull().sum()
        if missing > 0:
            df[col] = df[col].fillna('Unknown')
            stats[f'{col}_filled'] = missing

    return df, stats

def standardize_text(df, columns):
    for col in columns:
        if col in df.columns:
            df[col] = df[col].str.strip().str.lower()
    return df, {'text_standardized': len(columns)}

# Pipeline orchestrator
def run_pipeline(df, steps):
    """Execute a sequence of cleaning steps"""
    all_stats = {}

    for step_name, step_func, *args in steps:
        if args:
            df, stats = step_func(df, *args[0])
        else:
            df, stats = step_func(df)
        all_stats[step_name] = stats

```

```
    return df, all_stats


# Usage
steps = [
    ('remove_duplicates', remove_duplicates),
    ('fill_missing', fill_missing_values),
    ('standardize', standardize_text, ['name', 'address'])
]

clean_data, stats = run_pipeline(raw_data, steps)
```

3.2 Class-Based Pipeline

A class encapsulates state (configuration, statistics) and methods:

```
class DataCleaningPipeline:
    def __init__(self, config=None):
        self.config = config or {}
        self.stats = {
            'duplicates_removed': 0,
            'missing_filled': 0,
            'validation_errors': 0,
            'outliers_capped': 0
        }
        self.errors = []

    def clean_data(self, df):
        """Main cleaning workflow"""
        df = self._remove_duplicates(df)
        df = self._handle_missing_values(df)
        df = self._standardize_columns(df)
        df = self._handle_outliers(df)
        return df

    def _remove_duplicates(self, df):
        initial = len(df)
        df = df.drop_duplicates(
            subset=self.config.get('dedup_columns'),
            keep=self.config.get('dedup_keep', 'first')
        )
        self.stats['duplicates_removed'] = initial - len(df)
        return df

    def _handle_missing_values(self, df):
        for col in df.columns:
            missing = df[col].isnull().sum()
            if missing == 0:
                continue

            if col in self.config.get('numeric_columns', []):
                strategy = self.config.get('numeric_impute', 'median')
                if strategy == 'median':
                    df[col] = df[col].fillna(df[col].median())
                elif strategy == 'mean':
                    df[col] = df[col].fillna(df[col].mean())
            elif col in self.config.get('categorical_columns', []):
                df[col] = df[col].fillna(
                    self.config.get('missing_category', 'Unknown')
                )

            self.stats['missing_filled'] += missing
        return df
```

```
    def _standardize_columns(self, df):
        text_cols = self.config.get('text_columns', [])
        for col in text_cols:
            if col in df.columns:
                df[col] = df[col].str.strip().str.lower()
        return df

    def _handle_outliers(self, df):
        for col in self.config.get('numeric_columns', []):
            if col not in df.columns:
                continue

            q1 = df[col].quantile(0.25)
            q3 = df[col].quantile(0.75)
            iqr = q3 - q1
            lower_bound = q1 - 1.5 * iqr
            upper_bound = q3 + 1.5 * iqr

            outliers = (
                (df[col] < lower_bound) | (df[col] > upper_bound)
            ).sum()

            if outliers > 0 and self.config.get('cap_outliers', True):
                df[col] = df[col].clip(lower_bound, upper_bound)
                self.stats['outliers_capped'] += outliers

        return df

    def validate_data(self, df, schema):
        """Validate against a schema"""
        valid_rows = []

        for idx, row in df.iterrows():
            try:
                # Apply validation rules from schema
                validated = self._validate_row(row, schema)
                valid_rows.append(validated)
            except Exception as e:
                self.errors.append({'row': idx, 'error': str(e)})
                self.stats['validation_errors'] += 1

        return pd.DataFrame(valid_rows)

    def run(self, df):
        """Execute complete pipeline"""
        cleaned = self.clean_data(df)

        if hasattr(self, 'validation_schema'):
            cleaned = self.validate_data(cleaned, self.validation_schema)

        return {
            'data': cleaned,
            'stats': self.stats,
            'errors': self.errors
        }


# Usage
config = {
    'numeric_columns': ['age', 'salary', 'experience'],
    'categorical_columns': ['department', 'gender'],
    'text_columns': ['name', 'address'],
```

```
    'numeric_impute': 'median',
    'cap_outliers': True
}

pipeline = DataCleaningPipeline(config)
result = pipeline.run(raw_data)

clean_df = result['data']
print(result['stats'])
```

3.3 Pipeline with Configuration Files

For maximum reusability, separate configuration from code.

Configuration file (YAML)

```yaml
# config.yaml
data_sources:
  main:
    path: "data/raw/sales.csv"
    format: csv
    encoding: utf-8
    dtype:
      customer_id: str
      amount: float64

cleaning:
  duplicates:
    subset: [customer_id, date]
    keep: first

  missing_values:
    numeric_strategy: median
    categorical_fill: "Unknown"
    max_missing_pct: 0.3  # drop columns exceeding this

  outliers:
    method: iqr
    multiplier: 1.5
    cap: true

  text_columns:
    - customer_name
    - product_description
    operations:
      - strip
      - lower

validation:
  schema:
    customer_id:
      type: str
      required: true
      pattern: "^CUST\\d{6}$"
    amount:
      type: float
      required: true
      min: 0
      max: 1000000
    date:
      type: datetime
      required: true
```

Loading and using the configuration

```
import yaml
import pandas as pd

class ConfigurablePipeline:
    def __init__(self, config_path):
        with open(config_path, 'r') as f:
            self.config = yaml.safe_load(f)
        self.stats = {}

    def run(self):
        # Extract
        df = self._extract()

        # Transform
        df = self._clean_data(df)

        # Validate
        df, errors = self._validate(df)

        # Load
        self._load(df)

        return {
            'data': df,
            'stats': self.stats,
            'errors': errors
        }

    def _extract(self):
        sources = self.config['data_sources']
        # Implementation for reading from various sources
        # ...

    def _clean_data(self, df):
        # Apply cleaning steps from config
        # ...
        return df
```

3.4 Pipeline with Error Handling and Reporting

A robust pipeline captures detailed information about what was fixed:

```
class ReportingPipeline:
    def __init__(self):
        self.report = {
            'input_rows': 0,
            'output_rows': 0,
            'steps': [],
            'errors': [],
            'warnings': []
        }

    def add_step_result(self, step_name, before_count, after_count, details=None
    ):
        self.report['steps'].append({
            'step': step_name,
            'rows_removed': before_count - after_count,
            'details': details or {}
        })

    def generate_report(self):
        """Create human-readable report"""
        lines = []
```

```
        lines.append("=" * 50)
        lines.append("DATA CLEANING PIPELINE REPORT")
        lines.append("=" * 50)
        lines.append(f"Input rows: {self.report['input_rows']}")
        lines.append(f"Output rows: {self.report['output_rows']}")
        lines.append(
            f"Rows removed: {self.report['input_rows'] - self.report['
    output_rows']}"
        )
        lines.append("\nStep Details:")

        for step in self.report['steps']:
            lines.append(
                f"  - {step['step']}: removed {step['rows_removed']} rows"
            )
            for key, value in step['details'].items():
                lines.append(f"      {key}: {value}")

        if self.report['errors']:
            lines.append("\nErrors:")
            for err in self.report['errors']:
                lines.append(f"  - {err}")

        return "\n".join(lines)


# Pipeline with reporting
def run_with_reporting(df, pipeline_funcs):
    reporter = ReportingPipeline()
    reporter.report['input_rows'] = len(df)

    current_df = df.copy()

    for name, func in pipeline_funcs:
        before = len(current_df)
        current_df, details = func(current_df)
        after = len(current_df)
        reporter.add_step_result(name, before, after, details)

    reporter.report['output_rows'] = len(current_df)

    return current_df, reporter
```

4. Testing and Validation

4.1 Unit Testing Pipeline Components

Each cleaning function should have tests:

```
import pytest
import pandas as pd
import numpy as np

def test_remove_duplicates():
    # Test data with duplicates
    df = pd.DataFrame({
        'id': [1, 1, 2, 3, 3],
        'value': ['a', 'a', 'b', 'c', 'c']
    })

    cleaned, stats = remove_duplicates(df)

    assert len(cleaned) == 3  # IDs 1, 2, 3
```

```
    assert stats['duplicates_removed'] == 2
    assert cleaned['id'].tolist() == [1, 2, 3]


def test_fill_missing_values():
    df = pd.DataFrame({
        'numeric': [1, np.nan, 3, np.nan, 5],
        'category': ['a', 'b', np.nan, 'd', 'e']
    })

    cleaned, stats = fill_missing_values(df)

    assert cleaned['numeric'].isnull().sum() == 0
    assert cleaned['numeric'].iloc[1] == 3  # median of [1,3,5]
    assert cleaned['category'].isnull().sum() == 0
    assert cleaned['category'].iloc[2] == 'Unknown'
```

4.2 Pipeline Integration Testing

Test the entire pipeline on a small sample:

```
def test_full_pipeline():
    # Create small messy dataset
    test_data = pd.DataFrame({
        'id': [1, 1, 2, 3, 4],
        'name': ['Alice', 'Alice', 'Bob', None, 'David'],
        'age': [25, 25, -5, 35, 150],
        'score': [85, 85, 92, None, 78]
    })

    pipeline = DataCleaningPipeline(config)
    result = pipeline.run(test_data)

    assert result['stats']['duplicates_removed'] == 1
    assert result['stats']['missing_filled'] >= 1
    assert result['stats']['validation_errors'] >= 1
    assert len(result['data']) < len(test_data)  # Some rows removed
```

5. Complete Production-Ready Pipeline Example

Here is a comprehensive example combining all the concepts discussed earlier. This pipeline includes configuration loading, logging, error handling, validation, transformation, and reporting.

```
import pandas as pd
import numpy as np
import logging
from datetime import datetime
from typing import Dict, List, Tuple
import yaml
import json
import re

class ProductionDataPipeline:
    """
    Production-ready data cleaning pipeline with logging,
    error handling, and comprehensive reporting.
    """

    def __init__(self, config_path: str, log_path: str = None):
        with open(config_path, 'r') as f:
            self.config = yaml.safe_load(f)

```

```
        # Logging setup
        self.logger = logging.getLogger(__name__)
        if log_path:
            handler = logging.FileHandler(log_path)
            formatter = logging.Formatter(
                '%(asctime)s - %(name)s - %(levelname)s - %(message)s'
            )
            handler.setFormatter(formatter)
            self.logger.addHandler(handler)
            self.logger.setLevel(logging.INFO)

        self.stats = {
            'start_time': datetime.now(),
            'end_time': None,
            'input_rows': 0,
            'output_rows': 0,
            'steps': [],
            'errors': []
        }

    def extract(self) -> pd.DataFrame:
        """Extract data from configured source"""
        source = self.config['data_source']

        if source['format'] == 'csv':
            df = pd.read_csv(source['path'])
        elif source['format'] == 'excel':
            df = pd.read_excel(source['path'])
        elif source['format'] == 'json':
            df = pd.read_json(source['path'])
        else:
            raise ValueError("Unsupported format")

        self.stats['input_rows'] = len(df)
        return df

    def clean(self, df: pd.DataFrame) -> pd.DataFrame:
        """Apply cleaning operations"""

        # Remove duplicates
        before = len(df)
        df = df.drop_duplicates()
        removed = before - len(df)

        self.stats['steps'].append({
            'step': 'remove_duplicates',
            'rows_removed': removed
        })

        # Handle missing values
        missing_filled = 0
        for col in df.columns:
            missing = df[col].isnull().sum()

            if missing == 0:
                continue

            if pd.api.types.is_numeric_dtype(df[col]):
                df[col] = df[col].fillna(df[col].median())
            else:
                df[col] = df[col].fillna("Unknown")

            missing_filled += missing
```

```
        self.stats['steps'].append({
            'step': 'handle_missing',
            'values_filled': missing_filled
        })

        return df

    def validate(self, df: pd.DataFrame) -> Tuple[pd.DataFrame, List[Dict]]:
        """Validate dataset using schema rules"""

        schema = self.config.get('validation', {}).get('schema', {})
        valid_rows = []
        errors = []

        for idx, row in df.iterrows():
            row_errors = []

            for field, rules in schema.items():
                value = row.get(field)

                if rules.get('required') and pd.isna(value):
                    row_errors.append(f"{field} missing")

                if 'min' in rules and value < rules['min']:
                    row_errors.append(f"{field} below minimum")

                if 'max' in rules and value > rules['max']:
                    row_errors.append(f"{field} above maximum")

            if row_errors:
                errors.append({
                    "row_index": idx,
                    "errors": row_errors
                })
            else:
                valid_rows.append(row)

        return pd.DataFrame(valid_rows), errors

    def transform(self, df: pd.DataFrame) -> pd.DataFrame:
        """Apply feature transformations"""

        transforms = self.config.get("transformations", {})

        for encode in transforms.get("encoding", []):
            col = encode["column"]

            if encode["method"] == "one_hot":
                dummies = pd.get_dummies(df[col], prefix=col)
                df = pd.concat([df, dummies], axis=1)
                df.drop(columns=[col], inplace=True)

        return df

    def load(self, df: pd.DataFrame, errors: List[Dict]):
        """Save cleaned data and reports"""

        output = self.config["output"]

        df.to_csv(output["clean_path"], index=False)

        if errors:
```

```
            pd.DataFrame(errors).to_csv(
                output["error_path"], index=False
            )

        self.stats["end_time"] = datetime.now()
        self.stats["output_rows"] = len(df)

        with open(output["stats_path"], "w") as f:
            json.dump(self.stats, f, indent=2, default=str)

    def run(self) -> Dict:
        """Run full pipeline"""

        try:
            df = self.extract()
            df = self.clean(df)
            df, errors = self.validate(df)

            if self.config.get("transformations"):
                df = self.transform(df)

            self.load(df, errors)

            return {
                "success": True,
                "stats": self.stats,
                "error_count": len(errors)
            }

        except Exception as e:
            return {
                "success": False,
                "error": str(e),
                "stats": self.stats
            }


# Usage
if __name__ == "__main__":

    pipeline = ProductionDataPipeline(
        config_path="pipeline_config.yaml",
        log_path="pipeline.log"
    )

    result = pipeline.run()

    if result["success"]:
        print("Pipeline completed successfully")
        print("Processed rows:", result["stats"]["output_rows"])
    else:
        print("Pipeline failed:", result["error"])
```

Summary for Quick Revision

Key Principles

- **Modularity:** Each cleaning operation should be implemented as a separate function.
- **Configuration:** Define pipeline rules in configuration files (YAML/JSON) instead of hardcoding them inside the program.

- **Idempotency:** Running the pipeline multiple times with the same input should produce the same output every time.
- **Error Handling:** Pipelines should fail gracefully and capture errors for debugging.
- **Reporting:** Maintain detailed reports of all transformations and operations.

Pipeline Components

- **Extract:** Read data from sources such as CSV, Excel, JSON, or databases.
- **Clean:** Remove duplicates, handle missing values, standardize text fields, and manage outliers.
- **Validate:** Check the dataset against schema rules including required fields, data types, ranges, and patterns.
- **Transform:** Perform feature engineering such as derived columns, encoding categorical variables, and scaling numerical features.
- **Load:** Save the cleaned dataset along with logs, statistics, and error reports.

Implementation Patterns

- **Function-based Pipeline:** Uses a simple sequence of functions for data cleaning.
- **Class-based Pipeline:** Encapsulates state, configuration, and logic inside classes.
- **Configuration-driven Pipeline:** Uses YAML or JSON configuration files to control pipeline behavior.
- **Production-ready Pipeline:** Includes logging, error handling, monitoring, and detailed reporting.

Testing Strategies

- **Unit Testing:** Test individual cleaning functions separately.
- **Integration Testing:** Test the complete pipeline using small sample datasets.
- **Validation Testing:** Compare outputs against expected or known correct results.

Best Practices

- Always work on copies of data to avoid modifying the original dataset.
- Track statistics for every transformation step.
- Log all operations for debugging and reproducibility.
- Handle edge cases such as empty DataFrames or columns with all missing values.
- Plan for schema evolution as datasets and requirements change over time.

Introduction to Data Pipeline Components

A data pipeline is a sequence of processes used to collect, process, and store data so that it can be analyzed efficiently. In data science and data engineering, pipelines automate the movement of data from its source to its final destination while ensuring data quality and consistency. The main components of a typical data pipeline include data ingestion, data transformation, and data storage.

1. Data Ingestion

Data ingestion is the process of collecting and importing data from various sources into a system for further processing. The sources may include databases, CSV files, APIs, web applications, sensors, or streaming platforms.

Data ingestion can be classified into two main types:

- **Batch Ingestion:** Data is collected and processed in large batches at scheduled intervals.
- **Real-Time (Streaming) Ingestion:** Data is continuously ingested as it is generated.

The goal of data ingestion is to ensure that raw data from different sources is reliably transferred into the pipeline.

2. Data Transformation

Data transformation involves cleaning, organizing, and converting raw data into a format that is suitable for analysis or machine learning tasks. This stage is crucial because raw data often contains missing values, inconsistencies, or errors.

Common transformation operations include:

- Data cleaning (handling missing values and duplicates)
- Standardizing text and formats
- Feature engineering and data enrichment
- Encoding categorical variables
- Scaling or normalizing numerical features

The objective of this step is to produce high-quality and structured data that can be easily used for analysis and modeling.

3. Data Storage

Data storage refers to storing the processed data in systems where it can be accessed for analysis, reporting, or machine learning. Proper storage ensures efficient retrieval, scalability, and security of data.

Common storage systems include:

- Relational databases (e.g., SQL databases)
- Data warehouses
- Data lakes
- Cloud storage systems

After storage, the data can be accessed by analytics tools, dashboards, or machine learning pipelines.

Summary

A well-designed data pipeline ensures smooth data flow from ingestion to transformation and finally to storage. By automating these stages, organizations can efficiently process large volumes of data and support reliable data-driven decision making.

Chapter-4
Applied Statistics and Exploratory Analysis

4.1 Statistical Measures

Understanding the shape, spread, and relationships within your data is fundamental to exploratory analysis. This section covers key statistical measures that quantify these aspects.

4.1.1 Covariance and Correlation

When analyzing two variables, it is crucial to understand if and how they are related.

Covariance is a measure of the joint variability of two random variables. It indicates the directional relationship between them.

- **Positive Covariance:** When a larger value of one variable corresponds to a larger value of the other variable.
- **Negative Covariance:** When a larger value of one variable corresponds to a smaller value of the other variable.

Formula: For two samples X_i and Y_i, the covariance is calculated as:

$$\text{Cov}(X, Y) = \frac{\sum_{i=1}^{n}(X_i - \bar{X})(Y_i - \bar{Y})}{n - 1} \tag{1}$$

where $\bar{X}$ and $\bar{Y}$ are the sample means.

A key limitation is that its value is not standardized and is difficult to interpret.

Correlation (Coefficient) addresses the limitation of covariance by standardizing the measure. It is a dimensionless quantity that always lies between -1 and $+1$.

- $+1$: Perfect positive linear relationship
- 0: No linear relationship
- -1: Perfect negative linear relationship

Formula (Pearson Correlation): The most common type, measuring the strength of a linear relationship.

$$r_{XY} = \frac{\text{Cov}(X, Y)}{s_X s_Y} \tag{2}$$

where s_X and s_Y are the sample standard deviations of X and Y.

The `cov2corr` function is a standard utility to convert a covariance matrix into a correlation matrix by dividing by the outer product of the standard deviations.

4.1.2 Skewness

Skewness is a measure of the asymmetry of the probability distribution of a real-valued random variable about its mean.

- **Symmetrical Distribution:** The distribution is bell-shaped and balanced (e.g., Normal distribution). Skewness is approximately zero.

Positive Skew (Right-skewed): The distribution has a long tail on the right. The mean is typically greater than the median.

The formula for sample skewness is often based on the third central moment:

$$\text{Skewness} = \frac{\frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^3}{s^3} \tag{3}$$

Negative Skew (Left-skewed): The distribution has a long tail on the left. The mean is typically less than the median.

4.1.3 Kurtosis

Kurtosis is a measure of the "tailedness" of the probability distribution. It describes the combined weight of the tails relative to the center of the distribution.

- **High Kurtosis (Leptokurtic):** The distribution has heavy tails and a sharp, narrow peak. This indicates more outliers.
- **Low Kurtosis (Platykurtic):** The distribution has light tails and a flat, spread-out peak. This indicates fewer or no outliers.

Formula: Kurtosis is based on the fourth central moment. It is often reported as *excess kurtosis*, which is the kurtosis value minus 3 (the kurtosis of a normal distribution).

$$\text{Kurtosis} = \frac{\frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^4}{s^4} \tag{4}$$

4.1 Statistical Measures: Python Programs

This section provides practical implementations of the concepts discussed above using Python's core data science libraries.

Program 1: Calculating Covariance and Correlation

This program demonstrates how to calculate covariance and the Pearson correlation coefficient between two variables using NumPy and Pandas.

```
import numpy as np
import pandas as pd

# Sample data: Height (in cm) and Weight (in kg)
height = [170, 175, 168, 180, 178, 182, 165, 172]
weight = [65, 70, 63, 80, 75, 85, 60, 68]

print("--- Covariance and Correlation Calculation ---")
print(f"Height data: {height}")
print(f"Weight data: {weight}\n")

# Using NumPy
print("Using NumPy:")
cov_numpy = np.cov(height, weight, ddof=1)[0, 1]
corr_numpy = np.corrcoef(height, weight)[0, 1]

print(f"  Covariance: {cov_numpy:.2f}")
print(f"  Correlation (Pearson r): {corr_numpy:.4f}\n")

# Using Pandas
print("Using Pandas:")
df = pd.DataFrame({'Height': height, 'Weight': weight})

cov_pandas = df['Height'].cov(df['Weight'])
corr_pandas = df['Height'].corr(df['Weight'])

print(f"  Covariance: {cov_pandas:.2f}")
print(f"  Correlation (Pearson r): {corr_pandas:.4f}")

# Interpretation
if abs(corr_pandas) > 0.7:
    print("\nInterpretation: Strong linear relationship.")
else:
    print("\nInterpretation: Moderate or weak linear relationship.")
```

Listing 1: Calculating Covariance and Correlation in Python

Explanation

- `np.cov()` and `np.corrcoef()` compute the covariance and correlation matrices. The element `[0,1]` represents the relationship between the two variables.
- The parameter `ddof=1` ensures that the sample covariance is calculated.
- Pandas provides convenient methods `.cov()` and `.corr()` to compute these statistics directly from DataFrame columns.

The positive value confirms that height and weight increase together, and the correlation value close to 1 indicates a strong positive linear relationship.

Program 2: Calculating Skewness and Kurtosis

This program demonstrates how to assess the shape of a distribution using SciPy.

```
import numpy as np
from scipy.stats import skew, kurtosis
import pandas as pd

np.random.seed(42)

# 1. Symmetrical distribution (Normal)
normal_data = np.random.normal(loc=100, scale=10, size=1000)

# 2. Positively skewed distribution
positive_skew = np.random.exponential(scale=2, size=1000)

# 3. Negatively skewed distribution
negative_skew = -np.random.exponential(scale=2, size=1000) + 5

print("--- Skewness and Kurtosis Calculation ---")

print("\n1. Symmetrical (Normal) Data:")
print(f"Skewness: {skew(normal_data):.4f}")
print(f"Kurtosis: {kurtosis(normal_data, fisher=True):.4f}")
print(f"Kurtosis (Pearson): {kurtosis(normal_data, fisher=False):.4f}")

print("\n2. Positively Skewed Data:")
print(f"Skewness: {skew(positive_skew):.4f}")
print(f"Excess Kurtosis: {kurtosis(positive_skew, fisher=True):.4f}")

print("\n3. Negatively Skewed Data:")
print(f"Skewness: {skew(negative_skew):.4f}")
print(f"Excess Kurtosis: {kurtosis(negative_skew, fisher=True):.4f}")

print("\n--- Pandas Summary Statistics ---")

```

```python
df = pd.DataFrame({
    'Normal': normal_data,
    'Pos_Skew': positive_skew,
    'Neg_Skew': negative_skew
})

print(df.describe())

print("\n--- Pandas Skew and Kurtosis ---")
stats_df = df.agg(['skew', 'kurtosis']).round(4)
print(stats_df)
```

Listing 2: Calculating Skewness and Kurtosis in Python

Explanation

- `scipy.stats.skew()` calculates the skewness of the data distribution.
- `scipy.stats.kurtosis()` measures the tailedness of the distribution.
- When `fisher=True`, the function returns excess kurtosis where a normal distribution has a value of 0.
- The Pandas method `.agg()` allows multiple statistical functions such as `skew` and `kurtosis` to be applied simultaneously to DataFrame columns.

Numerical Example for Covariance, Correlation, Skewness, and Kurtosis

1. Numerical Example for Covariance and Correlation

Given Data

- $X = \{2, 4, 6, 8, 10\}$
- $Y = \{4, 6, 8, 10, 12\}$
- Number of observations: $n = 5$

Step 1: Compute Means

$$\bar{X} = \frac{2+4+6+8+10}{5} = 6$$

$$\bar{Y} = \frac{4+6+8+10+12}{5} = 8$$

Step 2: Compute Deviations

- $X_1 - \bar{X} = 2 - 6 = -4, \quad Y_1 - \bar{Y} = 4 - 8 = -4, \quad \text{product} = 16$
- $X_2 - \bar{X} = 4 - 6 = -2, \quad Y_2 - \bar{Y} = 6 - 8 = -2, \quad \text{product} = 4$
- $X_3 - \bar{X} = 6 - 6 = 0, \quad Y_3 - \bar{Y} = 8 - 8 = 0, \quad \text{product} = 0$
- $X_4 - \bar{X} = 8 - 6 = 2, \quad Y_4 - \bar{Y} = 10 - 8 = 2, \quad \text{product} = 4$
- $X_5 - \bar{X} = 10 - 6 = 4, \quad Y_5 - \bar{Y} = 12 - 8 = 4, \quad \text{product} = 16$

$$\sum (X - \bar{X})(Y - \bar{Y}) = 16 + 4 + 0 + 4 + 16 = 40$$

Step 3: Covariance

$$Cov(X, Y) = \frac{\sum (X - \bar{X})(Y - \bar{Y})}{n - 1}$$

$$Cov(X, Y) = \frac{40}{4} = 10$$

Step 4: Standard Deviations
For X

$$(-4)^2 + (-2)^2 + 0^2 + 2^2 + 4^2$$

$$= 16 + 4 + 0 + 4 + 16 = 40$$

$$s_X = \sqrt{\frac{40}{4}} = \sqrt{10} = 3.162$$

For Y

$$(-4)^2 + (-2)^2 + 0^2 + 2^2 + 4^2 = 40$$

$$s_Y = \sqrt{\frac{40}{4}} = \sqrt{10} = 3.162$$

Step 5: Correlation

$$r = \frac{Cov(X, Y)}{s_X s_Y}$$

$$r = \frac{10}{(3.162)(3.162)}$$

$$r = \frac{10}{10} = 1$$

Result

- Correlation $= 1$
- Interpretation: Perfect positive linear relationship

2. Numerical Example for Skewness

Data

$$X = \{2, 4, 6, 8, 10\}$$

Step 1: Mean

$$\bar{X} = 6$$

Step 2: Deviations

- $2 - 6 = -4, \quad (-4)^3 = -64$
- $4 - 6 = -2, \quad (-2)^3 = -8$
- $6 - 6 = 0, \quad 0^3 = 0$
- $8 - 6 = 2, \quad 2^3 = 8$
- $10 - 6 = 4, \quad 4^3 = 64$

$$\sum (X - \bar{X})^3 = -64 - 8 + 0 + 8 + 64 = 0$$

Step 3: Standard Deviation

$$s = \sqrt{\frac{40}{4}} = 3.162$$

Step 4: Skewness

$$Skewness = \frac{\frac{1}{n}\sum (X - \bar{X})^3}{s^3}$$

$$Skewness = \frac{0/5}{(3.162)^3} = 0$$

Result

- Skewness $= 0$
- Interpretation: Distribution is symmetrical

3. Numerical Example for Kurtosis

Step 1: Fourth Power Deviations

- $(-4)^4 = 256$
- $(-2)^4 = 16$
- $0^4 = 0$
- $2^4 = 16$
- $4^4 = 256$

$$\sum (X - \bar{X})^4 = 256 + 16 + 0 + 16 + 256 = 544$$

Step 2: Kurtosis

$$Kurtosis = \frac{\frac{1}{n}\sum (X - \bar{X})^4}{s^4}$$

$$s = 3.162$$

$$s^4 = 100$$

$$Kurtosis = \frac{544/5}{100}$$

$$Kurtosis = \frac{108.8}{100} = 1.088$$

Step 3: Excess Kurtosis

$$Excess\ Kurtosis = Kurtosis - 3$$

$$= 1.088 - 3 = -1.912$$

Result

- Kurtosis = 1.088
- Excess Kurtosis = −1.912
- Interpretation: Distribution is Platykurtic (flat)

Final Results Summary

- Covariance = 10 (Positive relationship)
- Correlation = 1 (Perfect positive linear relationship)
- Skewness = 0 (Symmetrical distribution)
- Kurtosis = 1.088 (Platykurtic distribution)

4.2 Probability Review, Sampling, and Hypothesis Testing: Theory

This section covers the foundational concepts that allow us to move from describing a sample to making inferences about a larger population.

4.2.1 Probability Review

Probability forms the mathematical foundation for statistics. A probability distribution defines the likelihood of different outcomes for a random variable.

- **Random Variable:** A variable whose possible values are numerical outcomes of a random phenomenon.
- **Probability Density Function (PDF):** For continuous variables (e.g., Normal distribution), the PDF describes the relative likelihood that the variable takes on a given value. The area under the PDF curve over an interval represents the probability of the variable falling in that interval.
- **Cumulative Distribution Function (CDF):** The CDF gives the probability that a random variable takes a value less than or equal to a specific point.

$$F(x) = P(X \leq x)$$

- **Common Distributions:** The `scipy.stats` module provides a wide range of distributions, including:
  - Normal distribution (`norm`)
  - Exponential distribution (`expon`)
  - Binomial distribution (`binom`)

  Each distribution provides methods for calculating PDF, CDF, and generating random samples.

4.2.2 Sampling Concepts

Sampling is the process of selecting a subset of individuals from a population to estimate characteristics of the whole population.

- **Population vs. Sample:**

  A *parameter* is a numerical summary of a population (e.g., population mean μ).

  A *statistic* is a numerical summary calculated from a sample (e.g., sample mean $\bar{x}$).

  Statistics are used to estimate population parameters.

- **Random Sample:**

  A sample drawn such that every member of the population has an equal and independent chance of being selected. This is essential for making valid statistical inferences.

- **Sampling with Replacement:**

  After an item is selected from the population, it is returned to the pool and can be chosen again. This method is commonly used in simulation studies.

- **Sampling without Replacement:**

  An item, once selected, is removed from the pool and cannot be chosen again. This method is commonly used in surveys and real-world data collection.

- **Sampling Distribution:**

  The probability distribution of a given statistic (such as the sample mean) obtained from many random samples drawn from the same population.

  The standard deviation of this distribution is called the *standard error*, which measures the precision of the sample statistic as an estimate of the population parameter.

4.2.3 Hypothesis Testing

Hypothesis testing is a formal statistical framework used to make decisions about population parameters using sample data.

Steps in Hypothesis Testing

1. **State the Hypotheses**

   - Null Hypothesis (H_0): A statement of no effect or no difference.

   $$H_0 : \theta = \theta_0$$

   - Alternative Hypothesis (H_a or H_1): A statement that contradicts the null hypothesis.

   $$H_a : \theta \neq \theta_0$$

2. **Choose a Test Statistic**

   A test statistic is a single numerical value calculated from the sample data used to evaluate the null hypothesis. Examples include the t-statistic and the χ^2 statistic.

3. **Determine the Sampling Distribution**

   This is the probability distribution of the test statistic assuming the null hypothesis is true.

4. **Set the Significance Level**

   The significance level (α) is the probability of rejecting the null hypothesis when it is actually true (Type I error).

   Commonly used value:

   $$\alpha = 0.05$$

5. **Compute the p-value**

   The p-value is the probability of observing a test statistic as extreme as or more extreme than the observed value, assuming the null hypothesis is true.

   - If p-value $\leq \alpha$, reject H_0 (statistically significant).
   - If p-value $> \alpha$, fail to reject H_0.

6. **Critical Region**

   The critical region is the set of values of the test statistic that leads to rejection of the null hypothesis. The boundaries of this region are known as critical values, determined by α and the sampling distribution.

Program 2: Sampling Techniques

This program demonstrates different ways to sample from a dataset using NumPy.

```python
import numpy as np
import pandas as pd

print("--- Sampling Techniques ---")

# Create a sample population dataset
np.random.seed(42)
population = pd.DataFrame({
    'ID': range(1, 101),
    'Value': np.random.normal(100, 15, 100)
})
print(f"Population size: {len(population)}")

# 1. Sampling with replacement (default)
sample_with_replacement = np.random.choice(population['ID'], size=10, replace=True)
print(f"\n1. Sample with replacement (IDs): {sample_with_replacement}")
print(f"   Note: IDs can appear multiple times")

# 2. Sampling without replacement
sample_without_replacement = np.random.choice(population['ID'], size=10, replace=False)
print(f"\n2. Sample without replacement (IDs): {sample_without_replacement}")
print(f"   Note: All IDs are unique")

# 3. Sampling with probability weights
weights = population['Value'] / population['Value'].sum()

weighted_sample = np.random.choice(
    population['ID'],
    size=5,
    replace=False,
    p=weights
)

print(f"\n3. Weighted sample (IDs): {weighted_sample}")
print(f"   Note: IDs with larger 'Value' have higher selection probability")

# 4. Random shuffling
print(f"\n4. Random shuffling:")
original_array = np.arange(10)
print(f"   Original: {original_array}")

shuffled_array = np.random.permutation(original_array)
print(f"   Permuted copy: {shuffled_array}")

np.random.shuffle(original_array)
print(f"   In-place shuffle: {original_array}")
```

Listing 3: Sampling Techniques in Python

Explanation

- `np.random.choice()` is the primary function for random sampling in NumPy.
- The parameter `replace=True` enables sampling with replacement.
- The parameter `replace=False` enables sampling without replacement.
- The parameter `p` allows assigning probability weights to each element.
- `np.random.permutation()` returns a randomly permuted copy of a sequence.
- `np.random.shuffle()` randomly shuffles a sequence in-place.

Program 3: Hypothesis Testing – Binomial Test

This program demonstrates hypothesis testing by recreating the ESP study example, finding critical regions, and performing a binomial test.

```
import numpy as np
import scipy.stats as stats
import matplotlib.pyplot as plt

print("--- Hypothesis Testing: ESP Study Example ---")

# Parameters
n_trials = 100
n_simulations = 10000
alpha = 0.05

# Problem 0: Find critical region using simulation
print(f"\nProblem 0: Finding critical region (alpha={alpha})")

np.random.seed(42)

# Simulate experiments under null hypothesis
simulated_results = np.random.binomial(n=n_trials, p=0.5, size=n_simulations)

# Determine critical values
lower_critical = np.percentile(simulated_results, 100 * alpha/2)
upper_critical = np.percentile(simulated_results, 100 * (1 - alpha/2))

print(f"Critical region: X <= {lower_critical:.0f} or X >= {upper_critical:.0f}")
print(f"(based on {n_simulations} simulations)")

# Problem 1: Effect of sample size
print(f"\nProblem 1: Effect of sample size on critical region")

for N in [100, 200, 500]:
    sim_props = np.random.binomial(n=N, p=0.5, size=n_simulations) / N
    lower_crit_prop = np.percentile(sim_props, 100 * alpha/2)
    upper_crit_prop = np.percentile(sim_props, 100 * (1 - alpha/2))

    print(f"N={N}: Critical proportion <= {lower_crit_prop:.3f} or >= {upper_crit_prop:.3f}")

# Problem 2: Exact binomial test
print(f"\nProblem 2: Exact binomial test")

observed_successes = 62

p_value = stats.binomtest(
    observed_successes,
    n=n_trials,
    p=0.5,
    alternative='two-sided'
)

print(f"Observed: {observed_successes} correct out of {n_trials}")
print(f"P-value: {p_value.pvalue:.4f}")

if p_value.pvalue < alpha:
    print(f"Conclusion: Reject null hypothesis (p < {alpha})")
    print("Evidence suggests people guess better than chance")
else:
    print(f"Conclusion: Fail to reject null hypothesis (p >= {alpha})")
    print("No significant evidence of better-than-chance guessing")
```

Listing 4: Hypothesis Testing using Binomial Test in Python

Explanation

- **Simulation-based critical region:** Thousands of experiments are simulated under the null hypothesis to construct the sampling distribution.
- **Critical values:** The boundaries of the rejection region correspond to the percentiles defined by the significance level α.
- **Effect of sample size:** Increasing sample size reduces the width of the critical region, showing that larger samples produce more precise estimates.
- `stats.binomtest()` performs an exact binomial hypothesis test and returns the corresponding p-value for the observed data.

Program 4: Common Statistical Tests with SciPy and Statsmodels

This program demonstrates various hypothesis tests available in SciPy and statsmodels.

```
import numpy as np
import scipy.stats as stats
import statsmodels.api as sm
from statsmodels.formula.api import ols

print("--- Common Statistical Tests ---")

# Generate sample data
np.random.seed(42)
group1 = np.random.normal(100, 10, 30)
group2 = np.random.normal(105, 10, 30)  # Slightly different mean
group3 = np.random.normal(100, 10, 30)

# 1. Independent t-test
print("\n1. Independent t-test:")
t_stat, p_value = stats.ttest_ind(group1, group2)
print(f"   t-statistic: {t_stat:.4f}")
print(f"   p-value: {p_value:.4f}")
print(f"   Conclusion: {'Significant difference' if p_value < 0.05 else 'No significant difference'}")

# 2. Mann-Whitney U test
print("\n2. Mann-Whitney U test:")
u_stat, p_value = stats.mannwhitneyu(group1, group2, alternative='two-sided')
print(f"   U-statistic: {u_stat:.4f}")
print(f"   p-value: {p_value:.4f}")

# 3. Chi-square goodness-of-fit test
print("\n3. Chi-square goodness-of-fit test:")
observed = [25, 35, 40]
expected = [33.3, 33.3, 33.4]

chi2_stat, p_value = stats.chisquare(observed, expected)
print(f"   Chi-square statistic: {chi2_stat:.4f}")
print(f"   p-value: {p_value:.4f}")

# 4. Correlation tests
print("\n4. Correlation tests:")
x = np.random.normal(0, 1, 50)
y = x * 0.7 + np.random.normal(0, 0.5, 50)

pearson_r, p_pearson = stats.pearsonr(x, y)
spearman_r, p_spearman = stats.spearmanr(x, y)

print(f"   Pearson correlation: r={pearson_r:.4f}, p={p_pearson:.4f}")
print(f"   Spearman correlation: rho={spearman_r:.4f}, p={p_spearman:.4f}")

# 5. One-way ANOVA using statsmodels
print("\n5. One-way ANOVA:")

import pandas as pd

df_anova = pd.DataFrame({
    'value': np.concatenate([group1, group2, group3]),
    'group': ['A']*30 + ['B']*30 + ['C']*30
})

model = ols('value ~ group', data=df_anova).fit()
anova_table = sm.stats.anova_lm(model, typ=2)

print(anova_table)
```

Listing 5: Common Statistical Tests in Python

Explanation

- `stats.ttest_ind()` performs an independent samples t-test to compare the means of two independent groups.
- `stats.mannwhitneyu()` is a non-parametric alternative to the t-test used when the data does not meet normality assumptions.
- `stats.chisquare()` performs a chi-square goodness-of-fit test to compare observed frequencies with expected frequencies.
- `stats.pearsonr()` calculates the Pearson correlation coefficient and its p-value, measuring linear relationships between variables.
- `stats.spearmanr()` computes the Spearman rank correlation coefficient, which measures monotonic relationships and is more robust to outliers.
- `statsmodels` provides advanced statistical modeling tools. The `OLS` function fits a linear model, and `anova_lm()` generates an ANOVA table for testing differences among multiple groups.

4.3 Regression and Trend Analysis using Statsmodels

Based on the reference materials, this section introduces regression and trend analysis along with their theoretical foundations and practical implementation using the `statsmodels` library in Python.

4.3.1 Theoretical Foundations

What is Regression Analysis?

Regression analysis is a statistical method used for modeling the relationship between a *dependent variable* (target) and one or more *independent variables* (predictors). It helps us understand how the dependent variable changes when one of the independent variables changes while the others remain fixed.

Key Concepts

- **Dependent Variable (Y):** The outcome variable that we want to predict or explain.
- **Independent Variables (X):** Predictor variables used to explain variation in the dependent variable.
- **Regression Coefficients (β):** Parameters representing the change in Y for a one-unit change in X.
- **Residuals (ϵ):** The difference between observed and predicted values.

Ordinary Least Squares (OLS) Regression

Ordinary Least Squares (OLS) is the most widely used method for estimating regression parameters. It estimates the regression coefficients by minimizing the sum of squared residuals.

Mathematical Formulation

For **simple linear regression** with one predictor:

$$Y_i = \beta_0 + \beta_1 X_i + \epsilon_i$$

where

- β_0 is the intercept
- β_1 is the slope coefficient
- ϵ_i is the random error term

For **multiple linear regression** with k predictors:

$$Y_i = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + \cdots + \beta_k X_{ik} + \epsilon_i$$

The OLS estimator minimizes the sum of squared residuals:

$$\sum_{i=1}^{n} (Y_i - \hat{Y}_i)^2$$

$$= \sum_{i=1}^{n} \left(Y_i - (\hat{\beta}_0 + \hat{\beta}_1 X_{i1} + \cdots + \hat{\beta}_k X_{ik}) \right)^2$$

Key Regression Statistics

Several statistical measures are used to evaluate the performance and reliability of a regression model.

- **R-squared (R^2)**

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

  Measures the proportion of variance in the dependent variable explained by the regression model. Values range from 0 to 1.

- **Adjusted R-squared**

$$R^2_{adj} = 1 - (1 - R^2)\frac{n-1}{n-k-1}$$

  Adjusted R^2 penalizes the model for including too many predictors and provides a more reliable measure when multiple variables are used.

- **F-statistic**

$$F = \frac{MS_{reg}}{MS_{res}}$$

  Tests whether the regression model explains a significant amount of variance in the dependent variable.

- **t-statistic**

$$t = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)}$$

  Used to test whether an individual regression coefficient is significantly different from zero.

- **Standard Error of Coefficient**

$$SE(\hat{\beta}) = \sqrt{\frac{\hat{\sigma}^2}{\sum (X_i - \bar{X})^2}}$$

  Measures the precision of the estimated regression coefficient.

Model Assumptions

For valid statistical inference, Ordinary Least Squares (OLS) regression relies on several key assumptions:

- **Linearity:** The relationship between the independent variable(s) X and the dependent variable Y is linear.
- **Independence:** Observations in the dataset are independent of each other.
- **Homoscedasticity:** The variance of the residuals remains constant across all levels of the independent variables.
- **Normality:** Residuals are normally distributed. This assumption is particularly important for statistical inference such as hypothesis testing and confidence intervals.

Trend Analysis

Trend analysis examines patterns and long-term movements in data over time. It is commonly used in economics, finance, and time series analysis to identify systematic changes in a variable.

Common Types of Trend Models

- **Linear Trend**

$$Y_t = \beta_0 + \beta_1 t + \varepsilon_t$$

This model assumes a constant absolute change in the dependent variable for each unit increase in time.

- **Quadratic Trend**

$$Y_t = \beta_0 + \beta_1 t + \beta_2 t^2 + \varepsilon_t$$

This model allows for acceleration or deceleration in the trend over time.

Seasonal-Trend Decomposition using LOESS (STL)

Seasonal-Trend Decomposition using LOESS (STL) is a technique used to decompose a time series into three main components.

- **Trend (T_t):** Represents the long-term progression of the time series.
- **Seasonal (S_t):** Represents regular patterns that repeat over fixed time intervals.
- **Residual (R_t):** Represents the irregular or random component remaining after removing trend and seasonal effects.

The STL decomposition model can be represented as:

$$Y_t = T_t + S_t + R_t$$

where:

- T_t represents the trend component
- S_t represents the seasonal component
- R_t represents the residual (remainder) component

4.3.2 Python Implementation with Statsmodels

Installation and Imports

```python
# Installation (if needed)
# pip install statsmodels

# Required imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.tsa.seasonal import STL
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.forecasting.stl import STLForecast

# Set visualization style
plt.style.use('seaborn-v0_8-darkgrid')
```

Listing 6: Installing and Importing Required Libraries

Example 1: Simple Linear Regression

This example demonstrates the relationship between education and income.

```
# Create sample dataset
np.random.seed(42)
n = 100
education = np.random.normal(12, 3, n)  # Years of education
true_intercept = 15000
true_slope = 3500
error = np.random.normal(0, 10000, n)
income = true_intercept + true_slope * education + error

# Create DataFrame
df = pd.DataFrame({
    'education': education,
    'income': income
})

# Method 1: Using formula API (recommended)
model_formula = smf.ols('income ~ education', data=df)
results_formula = model_formula.fit()

print("=== Simple Linear Regression Results (Formula API) ===")
print(results_formula.summary().tables[1])

# Method 2: Using array API
X = sm.add_constant(df['education'])
y = df['income']

model_array = sm.OLS(y, X)
results_array = model_array.fit()

print("\n=== Coefficient Comparison ===")
print(f"Formula API - Intercept: {results_formula.params['Intercept']:.2f}")
print(f"Formula API - Slope: {results_formula.params['education']:.2f}")
print(f"Array API - Intercept: {results_array.params[0]:.2f}")
print(f"Array API - Slope: {results_array.params[1]:.2f}")

# Calculate predicted values
df['predicted'] = results_formula.predict(df)
df['residuals'] = results_formula.resid

# Visualization
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Scatter plot with regression line
axes[0].scatter(df['education'], df['income'], alpha=0.6, label='Observed')
axes[0].plot(df['education'], df['predicted'], 'r-', label='Fitted line', linewidth=2)

axes[0].set_xlabel('Education (years)')
axes[0].set_ylabel('Income ($)')
axes[0].set_title('Simple Linear Regression: Income vs Education')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Residual plot
axes[1].scatter(df['predicted'], df['residuals'], alpha=0.6)
axes[1].axhline(y=0, color='r', linestyle='--', linewidth=1)

axes[1].set_xlabel('Predicted Values')
axes[1].set_ylabel('Residuals')
axes[1].set_title('Residual Plot')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Model diagnostics
print("\n=== Model Diagnostics ===")
print(f"R-squared: {results_formula.rsquared:.4f}")
print(f"Adjusted R-squared: {results_formula.rsquared_adj:.4f}")
print(f"F-statistic: {results_formula.fvalue:.2f}")
print(f"F-statistic p-value: {results_formula.f_pvalue:.4e}")
print(f"Mean of residuals: {results_formula.resid.mean():.4e}")
```

Listing 7: Simple Linear Regression using Statsmodels

Expected Output Interpretation

- The slope coefficient (approximately 3500) indicates that each additional year of education is associated with roughly $3500 higher income.
- The intercept represents the estimated income when education is zero (mainly used for model positioning).
- The R^2 value around 0.50–0.60 suggests that education explains approximately 50–60% of the variation in income.
- The residual plot helps assess assumptions such as homoscedasticity and independence.
- The mean of residuals should be close to zero, confirming that the model provides unbiased estimates.

Example 2: Multiple Regression with Non-linear Terms

This example demonstrates how to model non-linear relationships in multiple regression using quadratic terms.

```python
# Extend the dataset with age
np.random.seed(42)
n = 500
education = np.random.normal(13, 3, n)
age = np.random.uniform(25, 65, n)

# True relationship: income increases with age but at decreasing rate
true_income = (-50000 + 3500 * education +
               2500 * age - 25 * age**2 +
               np.random.normal(0, 15000, n))

df_multi = pd.DataFrame({
    'education': education,
    'age': age,
    'income': true_income
})

# Create quadratic terms
df_multi['age_squared'] = df_multi['age'] ** 2
df_multi['education_squared'] = df_multi['education'] ** 2

# Multiple regression model
model_multi = smf.ols('income ~ education + age + age_squared', data=df_multi)
results_multi = model_multi.fit()

print("=== Multiple Regression with Quadratic Terms ===")
print(results_multi.summary().tables[1])
```

Listing 8: Multiple Regression with Quadratic Terms

Prediction and Visualization

```python
# Predictions for visualization
age_range = pd.DataFrame({
    'age': np.linspace(25, 65, 50),
    'age_squared': np.linspace(25, 65, 50)**2,
    'education': [12] * 50  # Hold education constant at 12 years
})

# Predict income for each age
age_range['predicted_income'] = results_multi.predict(age_range)

# Calculate mean income by age group
df_multi['age_group'] = pd.cut(df_multi['age'], bins=20)
mean_by_age = df_multi.groupby('age_group')['income'].mean()
age_centers = [(interval.left + interval.right)/2 for interval in mean_by_age.index]

# Visualization
plt.figure(figsize=(10, 6))
plt.scatter(df_multi['age'], df_multi['income'], alpha=0.3, label='Observed data')
plt.plot(age_centers, mean_by_age.values, 'bo', markersize=8, label='Mean by age group')
plt.plot(age_range['age'], age_range['predicted_income'], 'r-', linewidth=3,
         label='Quadratic fit (educ=12)')
plt.xlabel('Age (years)')
plt.ylabel('Income ($)')
plt.title('Multiple Regression: Non-linear Age Effect')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Listing 9: Predictions and Visualization

Interpreting the Quadratic Effect

```python
# Interpret quadratic effect
age_coef = results_multi.params['age']
age2_coef = results_multi.params['age_squared']

# Find age where income peaks (first derivative = 0)
peak_age = -age_coef / (2 * age2_coef)
print(f"Income peaks at age: {peak_age:.1f} years")
print(f"Age coefficient: {age_coef:.2f} (positive - initial increase)")
print(f"Age  coefficient: {age2_coef:.2f} (negative - diminishing returns)")

# Residual diagnostics
residuals = results_multi.resid
fitted = results_multi.fittedvalues

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Residuals vs fitted
axes[0].scatter(fitted, residuals, alpha=0.6)
axes[0].axhline(y=0, color='r', linestyle='--')
axes[0].set_xlabel('Fitted Values')
axes[0].set_ylabel('Residuals')
axes[0].set_title('Residuals vs Fitted')
axes[0].grid(True, alpha=0.3)

# Q-Q plot for normality
sm.qqplot(residuals, line='s', ax=axes[1])
axes[1].set_title('Q-Q Plot (Normality Check)')

plt.tight_layout()
plt.show()
```

Listing 10: Quadratic Effect Interpretation and Residual Diagnostics

Interpretation:

- The positive coefficient for `age` indicates income initially increases with age.

- The negative coefficient for `age_squared` shows diminishing returns, leading to a peak in income at a certain age.
- Using the formula peak_age $= -\frac{\beta_{\text{age}}}{2\beta_{\text{age\_squared}}}$, the model estimates the age at which income is highest.
- Residual plots and Q-Q plots are used to check model assumptions: homoscedasticity and normality of residuals.

Example 3: Interaction Effects

Interaction terms capture situations where the effect of one variable depends on the level of another variable.

```
# Create dataset with interaction
np.random.seed(42)
n = 300
education = np.random.normal(13, 3, n)
experience = np.random.uniform(0, 40, n)

# True model: education effect is stronger with more experience
true_income = (25000 + 2000 * education + 500 * experience +
               150 * education * experience +  # Interaction term
               np.random.normal(0, 8000, n))

df_interact = pd.DataFrame({
    'education': education,
    'experience': experience,
    'income': true_income
})

# Model with interaction
model_interact = smf.ols('income ~ education * experience', data=df_interact)
# Equivalent to: income ~ education + experience + education:experience
results_interact = model_interact.fit()

print("=== Regression with Interaction Term ===")
print(results_interact.summary().tables[1])
```

Listing 11: Regression with Interaction Term

Visualization of Interaction Effects

```
education_levels = [10, 13, 16]  # Low, medium, high education
experience_range = np.linspace(0, 40, 50)

plt.figure(figsize=(10, 6))

for edu in education_levels:
    # Create prediction data
    pred_data = pd.DataFrame({
        'education': [edu] * len(experience_range),
        'experience': experience_range
    })
    pred_data['education:experience'] = pred_data['education'] * pred_data['experience']

    # Predict income
    pred_income = (results_interact.params['Intercept'] +
                   results_interact.params['education'] * edu +
                   results_interact.params['experience'] * experience_range +
                   results_interact.params['education:experience'] * edu * experience_range)

    plt.plot(experience_range, pred_income, linewidth=2,
             label=f'Education = {edu} years')

plt.xlabel('Experience (years)')
plt.ylabel('Predicted Income ($)')
plt.title('Interaction Effect: Education Modifies Experience Effect')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Listing 12: Visualizing Interaction Effects

Interpretation of Interaction Effect

```
print("\n=== Interaction Interpretation ===")
print("The effect of experience on income depends on education level:")
for edu in education_levels:
    effect = results_interact.params['experience'] + results_interact.params['education:experience'] *
    edu
    print(f"  At {edu} years education: 1 year experience     ${effect:.0f} income increase")
```

Listing 13: Quantifying Interaction Effect

- The positive coefficient for the interaction term (`education:experience`) indicates that the effect of experience on income grows as education increases.
- For example, at 10 years of education, one additional year of experience increases income by a smaller amount compared to 16 years of education.
- The plot shows how income curves steepen for higher education levels, demonstrating that education amplifies the return to experience.

Example 4: STL Decomposition for Trend Analysis

This example demonstrates **Seasonal-Trend decomposition using LOESS (STL)** to analyze long-term trends and seasonal patterns in time series data.

```python
# Load CO2 dataset (monthly atmospheric CO2 measurements)
from statsmodels.datasets import co2

data = co2.load().data
co2_series = data['co2']
co2_series.index = pd.date_range('1958-03-29', periods=len(co2_series), freq='W-WED')

# Fill missing values
co2_series = co2_series.fillna(co2_series.interpolate())

print("=== CO2 Data Summary ===")
print(f"Date range: {co2_series.index[0]} to {co2_series.index[-1]}")
print(f"Number of observations: {len(co2_series)}")
print(f"Mean CO2: {co2_series.mean():.2f} ppm")
print(f"Std CO2: {co2_series.std():.2f} ppm")

# STL Decomposition
# Seasonal period = 52 for weekly data with annual cycle
stl = STL(co2_series, period=52, robust=True)
result = stl.fit()

# Plot decomposition
fig = result.plot()
fig.set_size_inches(12, 10)
plt.suptitle('STL Decomposition of Atmospheric CO2', fontsize=16)
plt.tight_layout()
plt.show()

# Extract components
trend = result.trend
seasonal = result.seasonal
residual = result.resid

print("\n=== STL Decomposition Results ===")
print(f"Trend component variance: {trend.var():.2f}")
print(f"Seasonal component variance: {seasonal.var():.2f}")
print(f"Residual component variance: {residual.var():.2f}")
print(f"Proportion explained by trend: {trend.var() / co2_series.var():.3f}")
print(f"Proportion explained by seasonality: {seasonal.var() / co2_series.var():.3f}")

# Analyze the trend
print(f"\nTrend start (1958): {trend.iloc[0]:.2f} ppm")
print(f"Trend end (2001): {trend.iloc[-1]:.2f} ppm")
total_increase = trend.iloc[-1] - trend.iloc[0]
years = (co2_series.index[-1] - co2_series.index[0]).days / 365.25
print(f"Total increase over {years:.1f} years: {total_increase:.2f} ppm")
print(f"Average annual increase: {total_increase / years:.2f} ppm/year")

# Check residual patterns
plt.figure(figsize=(12, 4))
plt.plot(residual.index, residual.values, 'b-', alpha=0.7)
plt.axhline(y=0, color='r', linestyle='--')
plt.axhline(y=2*residual.std(), color='gray', linestyle=':', label='  2  ')
plt.axhline(y=-2*residual.std(), color='gray', linestyle=':')
plt.xlabel('Year')
plt.ylabel('Residual (ppm)')
plt.title('STL Residuals - Should Show No Pattern')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Listing 14: STL Decomposition of CO2 Data

Interpretation of STL Decomposition:

- The STL decomposition separates the CO2 time series into **trend**, **seasonal**, and **residual** components: $Y_t = T_t + S_t + R_t$.
- **Trend Component**: Captures the long-term upward progression of atmospheric CO2. In this dataset, CO2 increased from approximately 315 ppm (1958) to 370 ppm (2001), averaging ~ 1.1 ppm/year.
- **Seasonal Component**: Shows regular yearly fluctuations in CO2 concentrations.
- **Residual Component**: Should contain no systematic patterns; it represents random variation after removing trend and seasonality. The residual plot confirms minimal autocorrelation.
- Variance explained: The trend and seasonal components account for the majority of variation in the CO2 series, while residuals are small.

Example 5: Forecasting with STL and ARIMA

This example demonstrates how to combine **STL decomposition** with **ARIMA modeling** for forecasting seasonal time series data, using electrical equipment production as an example.

```python
# Load dataset
from statsmodels.datasets import elec_equip as ds
elec_equip = ds.load().data
elec_equip.index = pd.date_range('1995-01-01', periods=len(elec_equip), freq='M')
elec_equip.columns = ['production']

# STL + ARIMA forecasting
from statsmodels.tsa.seasonal import STL
from statsmodels.tsa.forecasting.stl import STLForecast
from statsmodels.tsa.arima.model import ARIMA

stlf = STLForecast(
    elec_equip['production'],
```

```
    ARIMA,
    model_kwargs=dict(order=(1,1,1), trend='t')
)
stlf_res = stlf.fit()

forecast_steps = 24
forecast = stlf_res.forecast(forecast_steps)

# Visualization
plt.figure(figsize=(14,8))
plt.plot(elec_equip.index[-60:], elec_equip['production'][-60:], 'b-', label='Historical', linewidth=2)
plt.plot(forecast.index, forecast, 'r-', label='Forecast', linewidth=2)
plt.xlabel('Date'); plt.ylabel('Production')
plt.title('STL + ARIMA Forecast of Electrical Equipment Production')
plt.legend(); plt.grid(True, alpha=0.3)
plt.show()
```

Listing 15: STL + ARIMA Forecasting

Key Points:

- STL decomposes the series into trend, seasonal, and residual components.
- ARIMA models the deseasonalized (trend + residual) component.
- The final forecast is the sum of ARIMA predictions and seasonal component.
- Model evaluation: AIC, BIC, and log-likelihood help assess fit.

Example 6: Multiple Seasonal Decomposition (MSTL)

For time series with **multiple seasonal periods**, MSTL can decompose trend, multiple seasonal components, and residuals.

```
# Create synthetic time series with multiple seasonalities
import numpy as np; import pandas as pd
from statsmodels.tsa.seasonal import MSTL

np.random.seed(42)
t = np.arange(1, 2000)
hourly_seasonality = 5 * np.sin(2*np.pi*t/24)
weekly_seasonality = 10 * np.sin(2*np.pi*t/(24*7))
trend = 0.001 * t**1.5
noise = np.random.normal(0,2,len(t))
y = trend + hourly_seasonality + weekly_seasonality + noise

ts_index = pd.date_range(start="2020-01-01", periods=len(t), freq="H")
ts_data = pd.Series(y, index=ts_index, name="value")

# MSTL decomposition with daily and weekly periods
mstl = MSTL(ts_data, periods=[24,168], iterate=3)
mstl_result = mstl.fit()

# Variance decomposition
total_var = ts_data.var()
trend_var = mstl_result.trend.var()
daily_var = mstl_result.seasonal['seasonal_24'].var()
weekly_var = mstl_result.seasonal['seasonal_168'].var()
resid_var = mstl_result.resid.var()

print(f"Trend: {trend_var/total_var:.3f}")
print(f"Daily seasonal: {daily_var/total_var:.3f}")
print(f"Weekly seasonal: {weekly_var/total_var:.3f}")
print(f"Residual: {resid_var/total_var:.3f}")
```

Listing 16: MSTL Decomposition

Interpretation:

- MSTL separates the time series into trend, multiple seasonalities (daily, weekly), and residuals.
- Variance decomposition quantifies how much of the total variation is explained by each component.
- Useful for high-frequency data (hourly, minute-level) with nested seasonal patterns.

Exploratory Data Analysis (EDA)

1. Introduction to EDA

Exploratory Data Analysis (EDA) is an approach to analyzing datasets to summarize their main characteristics, often using visual methods. As John Tukey, the pioneer of EDA, stated:

> "Exploratory data analysis is detective work—numerical detective work—or graphical detective work."

Key Distinction: EDA differs from confirmatory data analysis:

- **Exploratory (EDA):** Detective work—finding clues, generating hypotheses, understanding structure.
- **Confirmatory (Inferential):** Judicial work—evaluating evidence, testing hypotheses.

2. The Two Pillars: Descriptive and Inferential Methods in EDA

Descriptive Methods

Descriptive methods summarize features of a dataset and answer *"what happened?"*.

Purpose:

- Understand data distribution and central tendencies.
- Detect outliers and anomalies.
- Identify patterns and relationships.
- Assess data quality and completeness.

Key Descriptive Measures:

Measure Type	Specific Measures	Purpose
Central Tendency	Mean, Median, Mode	Identify typical values
Dispersion	Range, Variance, Standard Deviation, IQR	Understand spread
Shape	Skewness, Kurtosis	Understand distribution shape
Position	Percentiles, Quartiles	Understand relative standing

Inferential Methods

Inferential methods use sample data to make generalizations about populations. In EDA, inferential thinking helps:

- Assess whether observed patterns are likely to be real.
- Quantify uncertainty in exploratory findings.
- Guide hypothesis generation for confirmatory analysis.

Key Concepts: Sampling, uncertainty, and generalizability.

3. Types of Data Analysis Questions

Analysis Type	Goal	Characteristics
Descriptive	Describe a set of data	First analysis performed; cannot generalize without modeling
Exploratory	Find relationships unknown	Discover connections; correlation $\neq$ causation
Inferential	Use sample to infer population	Estimate quantity + uncertainty; depends on population and sampling

4. The EDA Process

1. **Data Acquisition and Understanding**
   - Load data and examine structure.
   - Identify data types (numerical, categorical, temporal).
2. **Data Quality Assessment**
   - Check missing values.
   - Identify outliers.
   - Ensure completeness and reliability.
3. **Univariate Analysis**
   - Examine each variable individually.
   - Summary statistics: mean, median, mode, standard deviation.
   - Visualizations: histograms, box plots.
4. **Bivariate/Multivariate Analysis**
   - Explore relationships between variables.
   - Correlation analysis, scatter plots, heatmaps.
5. **Generate Hypotheses**
   - Formulate questions for further investigation.
   - Consider sampling and generalizability.

5. Descriptive Statistics Detailed

Measures of Central Tendency:

- **Mode:** Most frequent value; only measure for nominal data.
- **Mean:** Arithmetic average

$$\text{Population mean: } \mu = \frac{\sum X}{N}, \quad \text{Sample mean: } \bar{X} = \frac{\sum x}{n}$$

- **Median:** Middle value; robust to outliers.

Measures of Dispersion:

- Range: Max − Min
- Variance: Average squared deviation from mean
- Standard deviation: $SD = \sqrt{\text{Variance}}$
- Interquartile Range (IQR): $Q_3 - Q_1$

6. Inferential Statistics in EDA Context

Key Concepts:

- **Population vs. Sample:** Entire group vs. subset.
- **Random Variable:** Value determined by chance.

Sampling Strategies:

- Random sampling: Equal chance for each member.
- Stratified sampling: Proportional representation of subgroups.
- Cluster sampling: Random selection of entire clusters.
- Convenience sampling: Easily accessible individuals.

Goal: Make inferences from a sample about a larger population, estimating both value and uncertainty.

7. Importance of EDA Before Modeling

- Detect patterns, outliers, and data quality issues.
- Ensure models are built on reliable foundations.
- Generate hypotheses for further analysis.

8. Summary: Descriptive vs. Inferential in EDA

Aspect	Descriptive Methods	Inferential Thinking
Purpose	Summarize sample data	Consider generalization beyond sample
Application	Univariate/bivariate analysis	Hypothesis generation
Tools	Mean, median, plots, tables	Sampling concepts, uncertainty assessment
Output	Summary statistics, visualizations	Questions for confirmatory analysis

4.5 Automation of EDA Workflows Using Python

1. Introduction to EDA Automation

Exploratory Data Analysis (EDA) is the critical first step in any data analysis project. Automating EDA workflows allows data scientists to quickly understand datasets, detect patterns, and make decisions about further analysis without writing repetitive code for each new dataset.

Core Philosophy: Automation of EDA means creating reusable workflows that systematically explore data, generate summaries, and produce visualizations with minimal manual intervention. This aligns with the "detective work" approach to understanding data before formal modeling.

2. The Python Ecosystem for EDA Automation

Python provides a rich set of libraries for automating EDA:

Library	Purpose in EDA Automation	Reference
pandas	Core data structures (DataFrame, Series); data loading, cleaning, manipulation	McKinney
NumPy	Numerical computations; array operations; mathematical functions	VanderPlas
Matplotlib	Foundation for data visualization; customizable plots	VanderPlas
Seaborn	Statistical visualizations; high-level interface for attractive plots	VanderPlas
IPython/Jupyter	Interactive computing environment; iterative exploration	VanderPlas

3. Key Components of Automated EDA

3.1 Data Loading and Initial Inspection

Automated EDA begins with programmatically loading data from various sources and performing initial inspection. Tasks to automate include:

- Display first/last rows of data
- Check data types of each column
- Identify missing values
- Assess dataset dimensions

3.2 Data Quality Assessment

Evaluate data quality systematically:

- Missing value detection: identify columns with null values and calculate percentages
- Duplicate detection: report duplicate rows
- Data type validation: ensure columns have appropriate types
- Outlier identification: flag potential outliers using IQR or z-score

3.3 Univariate Analysis Automation

Generate descriptive statistics and visualizations for individual variables:

Descriptive statistics:

- Measures of central tendency: mean, median, mode
- Measures of dispersion: standard deviation, variance, range, IQR
- Shape statistics: skewness, kurtosis
- Percentiles and quartiles

Automated visualizations:

- Histograms for distribution
- Box plots for outlier detection
- Bar charts for categorical variables

3.4 Bivariate and Multivariate Analysis Automation

Explore relationships between variables:

- Correlation matrices: compute and visualize correlations
- Pair plots: scatterplot matrices
- Cross-tabulations: for categorical relationships
- Grouped summaries: statistics by categorical groups

4. The EDA Automation Workflow

1. **Data Acquisition:** Read data from CSV, Excel, SQL, APIs; handle file formats and encodings.
2. **Data Cleaning:** Handle missing values, remove duplicates, standardize data types.
3. **Univariate Analysis:** Generate summary statistics; create appropriate visualizations; flag potential issues.
4. **Bivariate Analysis:** Analyze relationships; generate correlation matrices, scatter plots, cross-tabs.
5. **Report Generation:** Compile findings into a structured report with key visualizations and patterns.

5. Benefits of EDA Automation

Benefit	Description
Efficiency	Eliminates repetitive coding for each new dataset
Consistency	Ensures thorough analysis across projects
Reproducibility	Creates documented, repeatable workflows
Scalability	Handles large datasets efficiently
Collaboration	Standardized outputs facilitate teamwork

6. Integration with Other Data Science Libraries

Automated EDA integrates with Python libraries for advanced analysis:

- SciPy: Statistical tests and advanced math functions
- scikit-learn: Machine learning preprocessing and modeling
- statsmodels: Statistical modeling and hypothesis testing

7. Practical Considerations for Automation

- Flexibility: Customization based on data types and goals
- Scalability: Efficient handling of datasets of varying sizes
- Extensibility: Easily add new analysis components
- Interpretability: Outputs should be clear and shareable

8. Summary

Automation of EDA workflows using Python leverages libraries such as pandas, NumPy, Matplotlib, and Seaborn to systematically explore datasets. This approach embodies the “detective work” philosophy of EDA while making the process efficient, reproducible, and scalable for real-world data analysis tasks. listings xcolor

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

class AutomatedEDA:
    def __init__(self, df, target=None):
        self.df = df.copy()
        self.target = target
        self.numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
        self.categorical_cols = df.select_dtypes(include=['object', 'category']).columns.tolist()
        self.datetime_cols = df.select_dtypes(include=['datetime64']).columns.tolist()

    def run_complete_eda(self):
        print("="*70)
        print("AUTOMATED EXPLORATORY DATA ANALYSIS REPORT")
        print("="*70)
        self.data_overview()
        self.data_quality_check()
        self.univariate_analysis()
        self.bivariate_analysis()
        self.generate_summary()

    def data_overview(self):
        print("\n1. DATASET OVERVIEW")
        print("-"*40)
        print(f"Dataset Shape: {self.df.shape[0]} rows    {self.df.shape[1]} columns")
        print(f"\nFirst 5 rows:")
        print(self.df.head())
        print(f"\nLast 5 rows:")
        print(self.df.tail())
        print(f"\nColumn Names:")
        for col in self.df.columns:
            print(f"      {col}")

    def data_quality_check(self):
        print("\n2. DATA QUALITY ASSESSMENT")
        print("-"*40)
        print("\n2.1 Data Types:")
        dtype_counts = self.df.dtypes.value_counts()
        for dtype, count in dtype_counts.items():
            print(f"  {dtype}: {count} columns")

        print("\n2.2 Missing Values:")
        missing = self.df.isnull().sum()
        missing_pct = (missing / len(self.df)) * 100
        missing_df = pd.DataFrame({'Missing_Count': missing, 'Missing_Percent': missing_pct}).sort_values('Missing_Percent',
ascending=False)
        missing_with_data = missing_df[missing_df['Missing_Count'] > 0]
        if len(missing_with_data) > 0:
            print(missing_with_data)
            print(f"\nTotal Missing Values: {self.df.isnull().sum().sum()}")
        else:
            print("      No missing values found")

        print("\n2.3 Duplicate Rows:")
        duplicates = self.df.duplicated().sum()
        if duplicates > 0:
            print(f"      Found {duplicates} duplicate rows ({duplicates/len(self.df)*100:.2f}%)")
        else:
            print("      No duplicate rows found")

        print("\n2.4 Memory Usage:")
        memory_usage = self.df.memory_usage(deep=True).sum() / 1024**2
        print(f"  Total memory: {memory_usage:.2f} MB")

    def univariate_analysis(self):
        print("\n3. UNIVARIATE ANALYSIS")
        print("-"*40)
        if self.numeric_cols:
            print("\n3.1 Numeric Variables Summary:")
            print(self.df[self.numeric_cols].describe().round(3))
            print("\nShape Statistics:")
            for col in self.numeric_cols[:5]:
                data = self.df[col].dropna()
                skewness = stats.skew(data)
                kurt = stats.kurtosis(data)
                print(f"\n  {col}:")
                print(f"    Skewness: {skewness:.3f}")
                print(f"    Kurtosis: {kurt:.3f}")
                print(f"    Range: {data.max()-data.min():.3f}")
                print(f"    IQR: {data.quantile(0.75)-data.quantile(0.25):.3f}")

        if self.categorical_cols:
            print("\n3.2 Categorical Variables Summary:")
            for col in self.categorical_cols[:5]:
                print(f"\n  {col}:")
                print(f"    Unique values: {self.df[col].nunique()}")
                print(f"    Top value: {self.df[col].mode()[0]}")
                print(f"    Top frequency: {self.df[col].value_counts().iloc[0]}")

        self.plot_univariate()

    def plot_univariate(self):
        n_numeric = len(self.numeric_cols)
        n_categorical = len(self.categorical_cols)

        if n_numeric > 0:
            n_cols = min(3, n_numeric)
            n_rows = (min(n_numeric, 6) + n_cols - 1) // n_cols
            fig, axes = plt.subplots(n_rows, n_cols, figsize=(5*n_cols, 4*n_rows))
            axes = axes.flatten() if n_rows > 1 else [axes] if n_cols == 1 else axes
            for idx, col in enumerate(self.numeric_cols[:6]):
                if idx < len(axes):
                    self.df[col].hist(ax=axes[idx], bins=30, alpha=0.7, color='steelblue', edgecolor='black')
                    axes[idx].axvline(self.df[col].mean(), color='red', linestyle='--', label='Mean')
                    axes[idx].axvline(self.df[col].median(), color='green', linestyle='-', label='Median')
                    axes[idx].set_title(f'Distribution of {col}')
```

```
                    axes[idx].legend()
            plt.suptitle('Univariate Analysis - Numeric Variables', fontsize=14)
            plt.tight_layout()
            plt.show()

        if n_categorical > 0:
            n_cols = min(3, n_categorical)
            n_rows = (min(n_categorical, 6) + n_cols - 1) // n_cols
            fig, axes = plt.subplots(n_rows, n_cols, figsize=(5*n_cols, 4*n_rows))
            axes = axes.flatten() if n_rows > 1 or n_categorical > 1 else [axes]
            for idx, col in enumerate(self.categorical_cols[:6]):
                if idx < len(axes):
                    value_counts = self.df[col].value_counts().head(10)
                    value_counts.plot(kind='bar', ax=axes[idx], color='coral', edgecolor='black')
                    axes[idx].set_title(f'Top Categories in {col}')
                    axes[idx].set_xlabel(col)
                    axes[idx].set_ylabel('Count')
                    axes[idx].tick_params(axis='x', rotation=45)
            plt.suptitle('Univariate Analysis - Categorical Variables', fontsize=14)
            plt.tight_layout()
            plt.show()

    def bivariate_analysis(self):
        print("\n4. BIVARIATE ANALYSIS")
        print("-"*40)

        if len(self.numeric_cols) > 1:
            print("\n4.1 Correlation Matrix:")
            corr_matrix = self.df[self.numeric_cols].corr()
            print(corr_matrix.round(3))
            corr_pairs = []
            for i in range(len(self.numeric_cols)):
                for j in range(i+1, len(self.numeric_cols)):
                    corr_pairs.append({'var1': self.numeric_cols[i], 'var2': self.numeric_cols[j], 'correlation': corr_matrix.
    iloc[i, j]})
            corr_pairs = sorted(corr_pairs, key=lambda x: abs(x['correlation']), reverse=True)
            print("\nTop 5 Strongest Correlations:")
            for pair in corr_pairs[:5]:
                print(f"  {pair['var1']} vs {pair['var2']}: {pair['correlation']:.3f}")

            plt.figure(figsize=(10, 8))
            sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0, square=True, linewidths=1, cbar_kws={"shrink":
    0.8})
            plt.title('Correlation Matrix Heatmap')
            plt.tight_layout()
            plt.show()

        if self.target and self.target in self.numeric_cols:
            print(f"\n4.2 Relationships with Target Variable: {self.target}")
            features = [col for col in self.numeric_cols if col != self.target]
            fig, axes = plt.subplots(2, 3, figsize=(15, 10))
            axes = axes.flatten()
            for idx, feature in enumerate(features[:6]):
                if idx < len(axes):
                    axes[idx].scatter(self.df[feature], self.df[self.target], alpha=0.5)
                    z = np.polyfit(self.df[feature], self.df[self.target], 1)
                    p = np.poly1d(z)
                    x_range = np.linspace(self.df[feature].min(), self.df[feature].max(), 100)
                    axes[idx].plot(x_range, p(x_range), 'r-', linewidth=2)
                    corr = self.df[feature].corr(self.df[self.target])
                    axes[idx].set_title(f'{feature} vs {self.target}\nr = {corr:.3f}')
                    axes[idx].set_xlabel(feature)
                    axes[idx].set_ylabel(self.target)
            plt.suptitle(f'Relationships with Target Variable: {self.target}', fontsize=14)
            plt.tight_layout()
            plt.show()

        if len(self.numeric_cols) <= 5:
            print("\n4.3 Pair Plot Analysis:")
            sns.pairplot(self.df[self.numeric_cols], diag_kind='kde')
            plt.suptitle('Pair Plot of Numeric Variables', y=1.02)
            plt.show()

    def generate_summary(self):
        print("\n5. EDA SUMMARY REPORT")
        print("-"*40)
        print("\n5.1 Dataset Characteristics:")
        print(f"      Total observations: {self.df.shape[0]:,}")
        print(f"      Total features: {self.df.shape[1]}")
        print(f"      Numeric features: {len(self.numeric_cols)}")
        print(f"      Categorical features: {len(self.categorical_cols)}")
        print(f"      DateTime features: {len(self.datetime_cols)}")

        missing_total = self.df.isnull().sum().sum()
        missing_pct = (missing_total / (self.df.shape[0]*self.df.shape[1]))*100
        print(f"\n5.2 Data Quality:")
        print(f"      Complete cases: {self.df.dropna().shape[0]:,} ({self.df.dropna().shape[0]/self.df.shape[0]*100:.1f}%)")
        print(f"      Missing values: {missing_total:,} ({missing_pct:.2f}% of all cells)")

        duplicates = self.df.duplicated().sum()
        if duplicates > 0:
            print(f"      Duplicate rows: {duplicates:,} ({duplicates/self.df.shape[0]*100:.1f}%)")

        print(f"\n5.3 Feature Types:")
        for col in self.df.columns:
            dtype = str(self.df[col].dtype)
            missing = self.df[col].isnull().sum()
            unique = self.df[col].nunique()
            print(f"      {col}: {dtype}, {unique} unique values, {missing} missing")

        print("\n" + "="*70)
        print("AUTOMATED EDA COMPLETED SUCCESSFULLY")
        print("="*70)

# --- Example 1: Iris Dataset ---
from sklearn.datasets import load_iris
iris = load_iris()
df_iris = pd.DataFrame(iris.data, columns=iris.feature_names)
df_iris['species'] = pd.Categorical.from_codes(iris.target, iris.target_names)
eda = AutomatedEDA(df_iris, target='sepal length (cm)')
eda.run_complete_eda()

# --- Example 2: Titanic Dataset ---
np.random.seed(42)
n = 500
df_titanic = pd.DataFrame({
    'passenger_id': range(1, n+1),
    'survived': np.random.choice([0,1], n, p=[0.6,0.4]),
    'pclass': np.random.choice([1,2,3], n, p=[0.2,0.3,0.5]),
    'name': [f'Passenger_{i}' for i in range(1, n+1)],
    'sex': np.random.choice(['male','female'], n, p=[0.6,0.4]),
```

```
    'age': np.random.normal(30,15,n).clip(0,80).round(1),
    'sibsp': np.random.choice([0,1,2,3,4,5], n, p=[0.6,0.2,0.1,0.05,0.03,0.02]),
    'parch': np.random.choice([0,1,2,3,4,5], n, p=[0.7,0.15,0.08,0.04,0.02,0.01]),
    'fare': np.random.gamma(5,10,n).round(2),
    'embarked': np.random.choice(['C','Q','S'], n, p=[0.3,0.2,0.5])
})
df_titanic.loc[np.random.choice(n,50),'age'] = np.nan
df_titanic.loc[np.random.choice(n,20),'embarked'] = np.nan
eda2 = AutomatedEDA(df_titanic, target='survived')
eda2.run_complete_eda()

# --- Example 3: Custom Dataset ---
np.random.seed(42)
n = 1000
df_custom = pd.DataFrame({
    'customer_id': range(1000,1000+n),
    'age': np.random.normal(40,15,n).clip(18,90).astype(int),
    'income': np.random.gamma(5,10000,n).round(0),
    'credit_score': np.random.normal(700,50,n).clip(300,850).astype(int),
    'purchase_amount': np.random.exponential(100,n).round(2),
    'frequency': np.random.poisson(5,n),
    'satisfaction': np.random.choice([1,2,3,4,5], n, p=[0.1,0.15,0.25,0.3,0.2]),
    'region': np.random.choice(['North','South','East','West'], n),
    'member_type': np.random.choice(['Bronze','Silver','Gold','Platinum'], n, p=[0.4,0.3,0.2,0.1]),
    'last_purchase': pd.date_range('2023-01-01', periods=n, freq='D')
})
eda3 = AutomatedEDA(df_custom, target='purchase_amount')
eda3.run_complete_eda()
```

Listing 17: Full AutomatedEDA Python Workflow

Chapter-5
Data Visualization and Storytelling

0.1 Principles of Effective Visualization and Dashboard Design

0.1.1 Introduction to Data Visualization

Data visualization is the graphical representation of information and data. By using visual elements like charts, graphs, and maps, data visualization tools provide an accessible way to see and understand trends, outliers, and patterns in data. Effective visualization is not just about making *"pretty pictures"* but about communicating information clearly and efficiently [**?**, **?**].

0.1.2 Why Visualize Data?

Graphics are very effective for communicating numerical information quickly. Many design choices are based on subjective measures such as personal taste or disciplinary conventions rather than objective criteria. Understanding principles of effective visualization helps move from subjective to objective design decisions.

0.1.3 Perceptual Principles in Visualization

Preattentive Features

Preattentive features are visual properties processed very rapidly by the human brain without conscious effort. They can highlight important elements in a visualization.

Key Preattentive Attributes:

- **Color hue**: Different colors processed preattentively.
- **Color intensity**: Lighter vs. darker shades.
- **Orientation**: Horizontal, vertical, diagonal lines.
- **Shape**: Circles, squares, triangles.
- **Size**: Larger vs. smaller objects.
- **Motion**: Moving vs. stationary elements.
- **Spatial position**: Grouping and proximity.

Gestalt Heuristics

Gestalt principles describe how humans naturally perceive visual elements as organized patterns and wholes.

Principle	**Description**	**Application in Visualization**
Proximity	Objects near each other are perceived as a group	Group related data points together
Similarity	Similar objects are perceived as related	Use consistent colors for same categories
Enclosure	Objects with a boundary are seen as a group	Use boxes to group related charts
Closure	Mind fills in missing information	Use partial borders effectively
Continuity	Aligned objects are perceived as continuous	Line charts show trends smoothly
Connection	Connected objects are seen as related	Network diagrams, connected scatter plots

Table 1: Gestalt Principles in Visualization

0.1.4 Principles of Effective Visualization

Show the Data Clearly

Present data accurately and without distortion:

- Choose appropriate chart types
- Avoid misleading scales
- Ensure high data-ink ratio (minimize non-data ink)

Maximize Data-Ink Ratio

Edward Tufte's concept of data-ink ratio:

$$\text{Data-Ink Ratio} = \frac{\text{Data-Ink}}{\text{Total Ink Used}}$$

Remove non-data ink wherever possible, e.g., unnecessary gridlines, borders, or decorations.

Avoid Chartjunk

Chartjunk includes unnecessary visual elements that distract from the data:

- Heavy grid lines
- Unnecessary 3D effects
- Excessive decorations or patterns

Use Appropriate Chart Types

Data Relationship	Recommended Chart
Comparison	Bar charts, column charts
Trends over time	Line charts, area charts
Distribution	Histograms, box plots, density plots
Correlation	Scatter plots, bubble charts
Composition	Pie charts (limited), stacked bars
Geospatial	Maps, choropleth maps

Table 2: Recommended Chart Types for Different Data Relationships

Color Usage Principles

- Use color purposefully
- Consider colorblind-friendly palettes
- Sequential palettes for ordered data
- Diverging palettes for data with meaningful midpoint
- Qualitative palettes for categorical data

0.1.5 Dashboard Design Principles

What is a Dashboard?

A dashboard is a visual display of the most important information needed to achieve objectives, consolidated on a single screen for at-a-glance monitoring.

Key Design Principles

1. **Know Your Audience:** Understand who will use the dashboard and what decisions they need to make.
2. **Establish a Visual Hierarchy:** Most important info should be most prominent; use size, position, and color.
3. **Maintain Consistency:** Consistent colors, fonts, labels, and chart types.
4. **Provide Context:** Show comparisons, benchmarks, and goals; include explanatory titles.
5. **Limit Scope:** One dashboard should answer related questions; use tabs for multiple aspects.
6. **Ensure Accessibility:** Consider screen sizes, colorblind viewers, and readable text.

0.1.6 Common Visualization Mistakes

Mistake	Why Problematic	Better Approach
3D charts	Distorts perception	Use 2D charts
Truncated y-axis	Exaggerates differences	Start axis at zero
Too many colors	Confuses viewer	Limit color palette
Pie charts with many slices	Hard to compare	Use bar charts
Missing labels	Context lost	Always label axes
Inconsistent scales	Misleads comparisons	Keep scales consistent

Table 3: Common Visualization Mistakes and Corrections

0.1.7 Storytelling Aspect

Effective visualization tells a story:

- **Narrative:** Guide the viewer through the data
- **Context:** Explain why the data matters
- **Insight:** Highlight key findings
- **Call to action:** Suggest what to do

0.1.8 Testing Visualization Effectiveness

- Design choices should be tested with users
- What seems clear to the designer may not be clear to viewers
- Iterative refinement based on feedback improves effectiveness

0.1.9 Summary of Key Principles

Principle	Key Takeaway
Preattentive Features	Use color, size, position to guide attention automatically
Gestalt Principles	Leverage natural perception of groups and patterns
Data-Ink Ratio	Maximize information, minimize decoration
Appropriate Charts	Match chart type to data and message
Dashboard Design	Know audience, establish hierarchy, provide context
Storytelling	Guide viewers to insights and action

Table 4: Summary of Visualization and Dashboard Principles

0.2 Visualization with Matplotlib: Line, Bar, Histogram, Scatter, Subplots

0.2.1 Introduction to Matplotlib

Matplotlib is the primary plotting library in Python, designed to create high-quality visualizations. Developed by John Hunter in 2003, it serves as the foundation for most Python visualization libraries. Matplotlib allows intuitive plotting while providing complete control over figure appearance [**?**, **?**].

0.2.2 The Two Interfaces: Pyplot vs. Object-Oriented

Pyplot Interface (MATLAB-style)

- Simple, state-based interface
- Good for quick, interactive plotting
- Uses commands like `plt.plot()`, `plt.xlabel()`, etc.
- Matplotlib tracks the "current figure" and "current axes"

Object-Oriented Interface

- Provides more control and customization
- Explicit creation of `Figure` and `Axes` objects
- Better for complex plots and subplots
- Recommended for production code and reusability

0.2.3 Anatomy of a Matplotlib Figure

Understanding the components of a figure is essential for customization:

- **Figure**: The overall window or page that contains all elements
- **Axes**: The plotting area containing x and y axes
- **Axis**: Number-line objects that set scale and limits
- **Artist**: Everything visible in the figure (lines, text, patches)

```
Figure
  Title
  Axes
      Plot Area
      X-axis Label
      Y-axis Label
```

0.2.4 Basic Plot Types

Line Plot

- **Purpose:** Show trends over time or ordered categories
- **When to use:** Time series data, trends, patterns
- **Key parameters:** `linestyle='-'`, `marker='o'`, `linewidth=2`

Bar Plot

- **Purpose:** Compare quantities across categories
- **Types:** Vertical, horizontal, stacked, grouped
- **When to use:** Frequency counts, categorical comparison, survey responses

Histogram

- **Purpose:** Show distribution of continuous variables
- **When to use:** Understanding data distribution, normality, skewness
- **Key concepts:**
  - Bins: Intervals to group data
  - Frequency: Count of observations
  - Density: Normalized area under histogram

Scatter Plot

- **Purpose:** Show relationship between two continuous variables
- **When to use:** Correlations, clusters, outliers, data density
- **Variations:** Bubble charts (size as third variable), colored scatter (categorical data as color)

Subplots

- **Purpose:** Display multiple plots in one figure
- **Layout options:** Regular grids (`plt.subplots()`), GridSpec, nested subplots

0.2.5 Customization Elements

Element	Purpose	Common Functions
Title	Describe the plot	`plt.title()`, `ax.set_title()`
Axis Labels	Explain axes	`plt.xlabel()`, `plt.ylabel()`
Legend	Identify data series	`plt.legend()`
Grid	Improve readability	`plt.grid()`
Ticks	Control axis markings	`plt.xticks()`, `plt.yticks()`
Colors	Enhance distinction	`color='red'`, `cmap='viridis'`
Annotations	Highlight points	`plt.annotate()`, `plt.text()`

Table 5: Common Plot Customization Elements

0.2.6 Saving Figures

To save a figure in Matplotlib:

```
plt.savefig('plot.png', dpi=300, bbox_inches='tight')
```

Common formats:

- PNG: Raster graphics
- PDF/SVG: Vector graphics
- JPG: Photographs or web images

0.3 Program 1: Line Plots

0.3.1 Introduction

Line plots are primarily used to show trends over time or ordered categories. They are ideal for continuous data, time series, or comparing multiple sequences. The following examples demonstrate different ways to use line plots in Matplotlib.

0.3.2 Python Code Examples

```python
import numpy as np
import matplotlib.pyplot as plt

# Sample data
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)
y3 = np.sin(x) * np.exp(-x/3)  # Damped sine wave

# 1. Basic Line Plot
plt.figure(figsize=(10, 6))
plt.plot(x, y1, 'b-', linewidth=2, label='sin(x)')
plt.plot(x, y2, 'r--', linewidth=2, label='cos(x)')
plt.plot(x, y3, 'g-.', linewidth=2, label='damped sin(x)')
plt.xlabel('X axis (radians)')
plt.ylabel('Y axis (amplitude)')
plt.title('Basic Line Plot - Trigonometric Functions')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Listing 1: Basic Line Plots with Matplotlib

```python
# 2. Line Plot with Markers
plt.figure(figsize=(10, 6))
x_markers = np.arange(0, 11, 1)
y_markers = x_markers ** 2
plt.plot(x_markers, y_markers, 'ro-', markersize=8, markerfacecolor='white',
         markeredgewidth=2, linewidth=2, label='y = x  ')
plt.xlabel('X values')
plt.ylabel('Y values')
plt.title('Line Plot with Markers')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Listing 2: Line Plot with Markers

```python
# 3. Multiple Lines with Different Styles
plt.figure(figsize=(12, 6))
styles = ['-', '--', '-.', ':']
for i, style in enumerate(styles):
    y = (i+1) * x + np.random.normal(0, 0.5, len(x))
    plt.plot(x, y, style, linewidth=2, label=f'Line {i+1} ({style})')
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.title('Different Line Styles')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Listing 3: Multiple Lines with Different Styles

```python
# 4. Fill Between (Area Plot)
plt.figure(figsize=(10, 6))
x = np.linspace(0, 2*np.pi, 100)
y_upper = np.sin(x) + 0.2
y_lower = np.sin(x) - 0.2
plt.plot(x, np.sin(x), 'b-', linewidth=2, label='sin(x)')
plt.fill_between(x, y_lower, y_upper, alpha=0.3, color='blue', label=' 0 .2 range')
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.title('Line Plot with Confidence Interval')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Listing 4: Line Plot with Fill Between (Confidence Interval)

0.3.3 Explanation of Examples

- **Basic Line Plot:** Demonstrates multiple lines with different colors and styles.
- **Line Plot with Markers:** Adds markers to highlight individual data points.
- **Multiple Lines with Styles:** Shows how to plot several lines with varying linestyles and random noise.
- **Fill Between (Area Plot):** Illustrates how to display a confidence interval or range around a line using `fill_between`.

0.4 Program 2: Bar Plots

0.4.1 Introduction

Bar plots are used to compare quantities across different categories. They are ideal for displaying categorical data, showing frequency counts, or comparing multiple series. Both vertical and horizontal orientations are possible, and additional elements such as grouped bars, stacked bars, and error bars can be included for clarity.

0.4.2 Python Code Examples

```python
import numpy as np
import matplotlib.pyplot as plt

# Sample data
categories = ['Product A', 'Product B', 'Product C', 'Product D', 'Product E']
sales_2024 = [52, 68, 45, 85, 61]

# 1. Basic Vertical Bar Plot
plt.figure(figsize=(10, 6))
x_pos = np.arange(len(categories))
plt.bar(x_pos, sales_2024, color='steelblue', edgecolor='black', alpha=0.7)
plt.xlabel('Products')
plt.ylabel('Sales (thousands)')
plt.title('Product Sales 2024')
plt.xticks(x_pos, categories, rotation=45)
plt.grid(True, alpha=0.3, axis='y')
plt.show()
```

Listing 5: Basic Vertical Bar Plot

```python
# 2. Horizontal Bar Plot
plt.figure(figsize=(10, 6))
y_pos = np.arange(len(categories))
plt.barh(y_pos, sales_2024, color='coral', edgecolor='black', alpha=0.7)
plt.ylabel('Products')
plt.xlabel('Sales (thousands)')
```

```
plt.title('Product Sales 2024 - Horizontal')
plt.yticks(y_pos, categories)
plt.grid(True, alpha=0.3, axis='x')
plt.show()
```

Listing 6: Horizontal Bar Plot

```
# 3. Grouped Bar Plot (Comparing Two Years)
sales_2023 = [45, 62, 38, 71, 53]
x = np.arange(len(categories))
width = 0.35
plt.figure(figsize=(12, 6))
plt.bar(x - width/2, sales_2023, width, label='2023', color='lightblue', edgecolor='black')
plt.bar(x + width/2, sales_2024, width, label='2024', color='steelblue', edgecolor='black')
plt.xlabel('Products')
plt.ylabel('Sales (thousands)')
plt.title('Product Sales Comparison: 2023 vs 2024')
plt.xticks(x, categories, rotation=45)
plt.legend()
plt.grid(True, alpha=0.3, axis='y')
plt.show()
```

Listing 7: Grouped Bar Plot Comparing Two Years

```
# 4. Stacked Bar Plot
plt.figure(figsize=(10, 6))
plt.bar(categories, sales_2023, label='2023', color='lightblue', edgecolor='black')
plt.bar(categories, sales_2024, bottom=sales_2023, label='2024', color='steelblue', edgecolor='black')
plt.xlabel('Products')
plt.ylabel('Total Sales (thousands)')
plt.title('Stacked Bar Plot - Cumulative Sales')
plt.legend()
plt.grid(True, alpha=0.3, axis='y')
plt.xticks(rotation=45)
plt.show()
```

Listing 8: Stacked Bar Plot

```
# 5. Bar Plot with Error Bars
errors = [3, 4, 3, 5, 4]
plt.figure(figsize=(10, 6))
plt.bar(categories, sales_2024, yerr=errors, capsize=5,
        color='lightgreen', edgecolor='black', alpha=0.7,
        error_kw={'linewidth': 2, 'ecolor': 'red'})
plt.xlabel('Products')
plt.ylabel('Sales (thousands)')
plt.title('Bar Plot with Error Bars (Standard Deviation)')
plt.grid(True, alpha=0.3, axis='y')
plt.xticks(rotation=45)
plt.show()
```

Listing 9: Bar Plot with Error Bars

0.4.3 Explanation of Examples

- **Basic Vertical Bar Plot:** Shows sales for each product as vertical bars.
- **Horizontal Bar Plot:** Useful when category names are long, easier to read.
- **Grouped Bar Plot:** Compares sales across two years side by side for each product.
- **Stacked Bar Plot:** Displays cumulative totals by stacking 2024 sales on top of 2023.
- **Bar Plot with Error Bars:** Includes variability or uncertainty in the data using error bars.

0.5 Program 3: Histograms

0.5.1 Introduction

Histograms are used to visualize the distribution of continuous data by grouping values into intervals (bins) and plotting the frequency of each bin. They are useful for identifying the shape of the data, such as normality, skewness, modality, and detecting outliers. Variations include cumulative histograms, density plots, and 2D histograms for bivariate data.

0.5.2 Python Code Examples

```
import numpy as np
import matplotlib.pyplot as plt

# Generate normal distribution
np.random.seed(42)
normal_data = np.random.normal(100, 15, 1000)

# 1. Basic Histogram
plt.figure(figsize=(10, 6))
plt.hist(normal_data, bins=30, color='steelblue', edgecolor='black', alpha=0.7)
plt.xlabel('Values')
plt.ylabel('Frequency')
plt.title('Basic Histogram - Normal Distribution')
plt.grid(True, alpha=0.3, axis='y')
plt.show()
```

Listing 10: Basic Histogram

```python
# 2. Histogram with Different Bin Sizes
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
bin_sizes = [5, 10, 30, 50]

for idx, bins in enumerate(bin_sizes):
    row, col = idx // 2, idx % 2
    axes[row, col].hist(normal_data, bins=bins, color='steelblue',
                        edgecolor='black', alpha=0.7)
    axes[row, col].set_title(f'Bins = {bins}')
    axes[row, col].set_xlabel('Values')
    axes[row, col].set_ylabel('Frequency')
    axes[row, col].grid(True, alpha=0.3, axis='y')

plt.suptitle('Effect of Bin Size on Histogram', fontsize=14)
plt.tight_layout()
plt.show()
```

Listing 11: Histogram with Different Bin Sizes

```python
# 3. Multiple Distributions Comparison
exponential_data = np.random.exponential(2, 1000)

plt.figure(figsize=(12, 6))
plt.hist(normal_data, bins=30, alpha=0.5, label='Normal', color='blue', edgecolor='black')
plt.hist(exponential_data, bins=30, alpha=0.5, label='Exponential', color='red', edgecolor='black')
plt.xlabel('Values')
plt.ylabel('Frequency')
plt.title('Comparing Two Distributions')
plt.legend()
plt.grid(True, alpha=0.3, axis='y')
plt.show()
```

Listing 12: Comparing Multiple Distributions

```python
# 4. Histogram with Density (KDE) Overlay
from scipy import stats

plt.figure(figsize=(10, 6))
counts, bins, patches = plt.hist(normal_data, bins=30, density=True,
                                 alpha=0.7, color='steelblue',
                                 edgecolor='black', label='Histogram')

# KDE overlay
kde = stats.gaussian_kde(normal_data)
x_range = np.linspace(normal_data.min(), normal_data.max(), 200)
plt.plot(x_range, kde(x_range), 'r-', linewidth=2, label='KDE')

plt.xlabel('Values')
plt.ylabel('Density')
plt.title('Histogram with KDE Overlay')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Listing 13: Histogram with Density (KDE) Overlay

```python
# 5. Cumulative Histogram
plt.figure(figsize=(10, 6))
plt.hist(normal_data, bins=30, cumulative=True, density=True,
         color='steelblue', edgecolor='black', alpha=0.7,
         label='Cumulative Distribution')
plt.xlabel('Values')
plt.ylabel('Cumulative Probability')
plt.title('Cumulative Histogram (CDF)')
plt.grid(True, alpha=0.3)
plt.show()
```

Listing 14: Cumulative Histogram

```python
# 6. 2D Histogram (Hexbin)
x = np.random.normal(0, 1, 5000)
y = x + np.random.normal(0, 0.5, 5000)

plt.figure(figsize=(10, 8))
plt.hexbin(x, y, gridsize=30, cmap='YlOrRd')
plt.colorbar(label='Count')
plt.xlabel('X values')
plt.ylabel('Y values')
plt.title('2D Histogram (Hexbin Plot)')
plt.show()
```

Listing 15: 2D Histogram (Hexbin Plot)

0.5.3 Explanation of Examples

- **Basic Histogram:** Shows frequency distribution of a single continuous variable.
- **Different Bin Sizes:** Demonstrates how bin width affects histogram appearance and interpretation.
- **Multiple Distributions:** Allows visual comparison of two or more distributions.
- **Histogram with KDE Overlay:** Adds smooth density curve to highlight distribution shape.
- **Cumulative Histogram:** Represents cumulative distribution function (CDF) of the data.
- **2D Histogram (Hexbin):** Visualizes the joint distribution of two continuous variables.

0.6 Program 4: Scatter Plots

0.6.1 Introduction

Scatter plots are used to visualize relationships between two continuous variables. They help identify correlations, clusters, outliers, and patterns. Variations include bubble charts (size as a third variable), color-coded categories, and regression lines to show trends.

0.6.2 Python Code Examples

```
import numpy as np
import matplotlib.pyplot as plt

# Generate positively correlated data
np.random.seed(42)
n = 200
x = np.random.normal(0, 1, n)
y = 0.7 * x + np.random.normal(0, 0.5, n)

# 1. Basic Scatter Plot
plt.figure(figsize=(10, 6))
plt.scatter(x, y, alpha=0.6, color='steelblue', edgecolor='black', s=50)
plt.xlabel('X variable')
plt.ylabel('Y variable')
plt.title('Basic Scatter Plot - Positive Correlation')
plt.grid(True, alpha=0.3)
plt.show()
```

Listing 16: Basic Scatter Plot

```
# 2. Scatter Plot for Positive, Negative, and No Correlation
x2 = np.random.normal(0, 1, n)
y2 = -0.6 * x2 + np.random.normal(0, 0.5, n)  # Negative correlation
x3 = np.random.normal(0, 1, n)
y3 = np.random.normal(0, 1, n)  # No correlation

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].scatter(x, y, alpha=0.6, color='steelblue', edgecolor='black')
axes[0].set_title('Positive Correlation (r     0.7)')
axes[0].set_xlabel('X'); axes[0].set_ylabel('Y')
axes[0].grid(True, alpha=0.3)

axes[1].scatter(x2, y2, alpha=0.6, color='coral', edgecolor='black')
axes[1].set_title('Negative Correlation (r     -0.6)')
axes[1].set_xlabel('X'); axes[1].set_ylabel('Y')
axes[1].grid(True, alpha=0.3)

axes[2].scatter(x3, y3, alpha=0.6, color='green', edgecolor='black')
axes[2].set_title('No Correlation (r     0)')
axes[2].set_xlabel('X'); axes[2].set_ylabel('Y')
axes[2].grid(True, alpha=0.3)

plt.suptitle('Scatter Plots - Different Correlation Types', fontsize=14)
plt.tight_layout()
plt.show()
```

Listing 17: Scatter Plot with Different Correlation Types

```
# 3. Bubble Chart
n = 100
x = np.random.uniform(0, 100, n)
y = np.random.uniform(0, 100, n)
population = np.random.gamma(2, 10, n) * 1000  # Bubble size
colors = np.random.uniform(0, 1, n)  # Color scale

scatter = plt.scatter(x, y, s=population/100, c=colors, alpha=0.6,
                      cmap='viridis', edgecolor='black', linewidth=0.5)
plt.colorbar(scatter, label='Color scale')
plt.xlabel('X coordinate')
plt.ylabel('Y coordinate')
plt.title('Bubble Chart - Size = Population, Color = Category')
plt.grid(True, alpha=0.3)
plt.show()
```

Listing 18: Bubble Chart (Size

```
# 4. Scatter Plot with Regression Line
plt.figure(figsize=(10, 6))
plt.scatter(x, y, alpha=0.6, color='steelblue', edgecolor='black', label='Data')

# Regression line
z = np.polyfit(x, y, 1)
p = np.poly1d(z)
x_line = np.linspace(x.min(), x.max(), 100)
plt.plot(x_line, p(x_line), 'r-', linewidth=2,
         label=f'Regression line (y={z[0]:.2f}x+{z[1]:.2f})')

plt.xlabel('X variable')
plt.ylabel('Y variable')
plt.title('Scatter Plot with Regression Line')
plt.legend()
plt.grid(True, alpha=0.3)
```

```
plt.show()
```

Listing 19: Scatter Plot with Regression Line

```
# 5. Scatter Plot with Color by Category
categories = np.random.choice(['Group A', 'Group B', 'Group C'], n)
colors = {'Group A': 'red', 'Group B': 'blue', 'Group C': 'green'}

plt.figure(figsize=(10, 6))
for category in colors:
    mask = categories == category
    plt.scatter(x[mask], y[mask], c=colors[category], label=category,
                alpha=0.6, edgecolor='black', s=80)
plt.xlabel('X variable')
plt.ylabel('Y variable')
plt.title('Scatter Plot - Colored by Category')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Listing 20: Scatter Plot Colored by Category

0.6.3 Explanation of Examples

- **Basic Scatter Plot:** Shows the relationship between two continuous variables.
- **Different Correlation Types:** Demonstrates positive, negative, and no correlation visually.
- **Bubble Chart:** Introduces a third variable via bubble size and a fourth via color.
- **Regression Line:** Adds a trend line to indicate the linear relationship between variables.
- **Color by Category:** Highlights categorical distinctions in the scatter plot.

0.7 Program 5: Subplots

0.7.1 Introduction

Subplots allow multiple plots to be displayed within a single figure. They are useful for comparing different datasets, visualizing related variables, and creating dashboards. Matplotlib provides flexible methods for creating regular grids, irregular layouts, shared axes, and nested plots using `GridSpec`.

0.7.2 Python Code Examples

```
import numpy as np
import matplotlib.pyplot as plt

# Generate sample data
x = np.linspace(0, 10, 100)
categories = ['A', 'B', 'C', 'D']
values = [23, 45, 56, 78]
data1 = np.random.normal(0, 1, 1000)
data2 = np.random.normal(2, 1.5, 1000)

# 1. Basic Subplot (2x2 grid)
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Line plot
axes[0, 0].plot(x, np.sin(x), 'b-', linewidth=2)
axes[0, 0].set_title('Line Plot - sin(x)')
axes[0, 0].set_xlabel('x')
axes[0, 0].set_ylabel('sin(x)')
axes[0, 0].grid(True, alpha=0.3)

# Bar plot
axes[0, 1].bar(categories, values, color='steelblue', edgecolor='black')
axes[0, 1].set_title('Bar Plot')
axes[0, 1].set_xlabel('Categories')
axes[0, 1].set_ylabel('Values')
axes[0, 1].grid(True, alpha=0.3, axis='y')

# Histogram
axes[1, 0].hist(data1, bins=30, alpha=0.7, color='green', edgecolor='black')
axes[1, 0].set_title('Histogram')
axes[1, 0].set_xlabel('Values')
axes[1, 0].set_ylabel('Frequency')
axes[1, 0].grid(True, alpha=0.3, axis='y')

# Scatter plot
axes[1, 1].scatter(data1, data2, alpha=0.5, color='coral', edgecolor='black')
axes[1, 1].set_title('Scatter Plot')
axes[1, 1].set_xlabel('X values')
axes[1, 1].set_ylabel('Y values')
axes[1, 1].grid(True, alpha=0.3)

plt.suptitle('2 2  Subplot Grid - Different Plot Types', fontsize=16)
plt.tight_layout()
plt.show()
```

Listing 21: Basic 2x2 Subplot Grid

```
# 2. Irregular Subplot Layout
fig = plt.figure(figsize=(15, 10))
ax1 = plt.subplot(2, 2, 1)  # Top-left
ax2 = plt.subplot(2, 2, 2)  # Top-right
ax3 = plt.subplot(2, 1, 2)  # Bottom (spans both columns)

ax1.plot(x, np.sin(x), 'b-', linewidth=2); ax1.set_title('sin(x)'); ax1.grid(True, alpha=0.3)
ax2.plot(x, np.cos(x), 'r-', linewidth=2); ax2.set_title('cos(x)'); ax2.grid(True, alpha=0.3)
ax3.plot(x, np.sin(x)*np.cos(x), 'g-', linewidth=2); ax3.set_title('sin(x)    cos(x)'); ax3.set_xlabel(
    'x'); ax3.grid(True, alpha=0.3)

plt.suptitle('Irregular Subplot Layout', fontsize=16)
plt.tight_layout()
plt.show()
```

Listing 22: Irregular Subplot Layout

```
# 3. Subplots with Shared X-Axis
fig, axes = plt.subplots(3, 1, figsize=(10, 12), sharex=True)
axes[0].plot(x, np.sin(x), 'b-', linewidth=2); axes[0].set_ylabel('sin(x)'); axes[0].set_title('Three
    Plots with Shared X-Axis'); axes[0].grid(True, alpha=0.3)
axes[1].plot(x, np.cos(x), 'r-', linewidth=2); axes[1].set_ylabel('cos(x)'); axes[1].grid(True, alpha
    =0.3)
axes[2].plot(x, np.sin(x)+np.cos(x), 'g-', linewidth=2); axes[2].set_xlabel('X axis (shared)'); axes
    [2].set_ylabel('sin(x)+cos(x)'); axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Listing 23: Subplots with Shared Axes

```
# 4. Nested Subplots (GridSpec)
from matplotlib.gridspec import GridSpec
fig = plt.figure(figsize=(12, 8))
gs = GridSpec(3, 3, figure=fig)

ax1 = fig.add_subplot(gs[0, :])    # Top row, all columns
ax2 = fig.add_subplot(gs[1, :-1])  # Middle row, first two columns
ax3 = fig.add_subplot(gs[1:, -1])  # Right column, last two rows
ax4 = fig.add_subplot(gs[2, 0])    # Bottom-left
ax5 = fig.add_subplot(gs[2, 1])    # Bottom-middle

ax1.plot(x, np.sin(x), 'b-', linewidth=2); ax1.set_title('Wide Plot (Top)'); ax1.grid(True, alpha=0.3)
ax2.plot(x, np.cos(x), 'r-', linewidth=2); ax2.set_title('Medium Plot'); ax2.grid(True, alpha=0.3)
ax3.barh(categories, values, color='steelblue'); ax3.set_title('Tall Plot'); ax3.grid(True, alpha=0.3,
    axis='x')
ax4.hist(data1, bins=30, color='green', edgecolor='black'); ax4.set_title('Small Plot 1'); ax4.grid(
    True, alpha=0.3)
ax5.scatter(data1, data2, alpha=0.5, color='coral'); ax5.set_title('Small Plot 2'); ax5.grid(True,
    alpha=0.3)

plt.suptitle('Complex GridSpec Layout', fontsize=16)
plt.tight_layout()
plt.show()
```

Listing 24: Nested Subplots Using GridSpec

0.7.3 Explanation of Examples

- **Basic 2x2 Grid:** Shows multiple plot types together for comparison.
- **Irregular Layouts:** Combines subplots of different sizes in one figure.
- **Shared Axes:** Synchronizes axes for easier comparison of trends.
- **Nested Subplots/GridSpec:** Provides full control over layout and subplot sizes.

0.8 Program 6: Complete Customization Example

0.8.1 Introduction

This program demonstrates a fully customized Matplotlib plot, incorporating multiple elements such as multiple datasets, regression lines, annotations, text boxes, secondary axes, gridlines, legends, and axis customization. It serves as a reference for creating publication-quality visualizations.

0.8.2 Python Code Example

```
import numpy as np
import matplotlib.pyplot as plt

# Generate data
x = np.linspace(0, 10, 50)
y1 = 2*x + 1 + np.random.normal(0, 1, 50)
y2 = 3*x - 2 + np.random.normal(0, 1.5, 50)

# Create figure and axis
fig, ax = plt.subplots(figsize=(12, 8))

# Scatter plots
ax.scatter(x, y1, color='blue', marker='o', s=80, alpha=0.7, edgecolor='black', linewidth=1, label='
    Dataset A')
```

```
ax.scatter(x, y2, color='red', marker='s', s=80, alpha=0.7, edgecolor='black', linewidth=1, label='
    Dataset B')

# Regression lines
z1 = np.polyfit(x, y1, 1)
z2 = np.polyfit(x, y2, 1)
p1 = np.poly1d(z1)
p2 = np.poly1d(z2)
ax.plot(x, p1(x), 'b--', linewidth=2, label=f'A trend (slope={z1[0]:.2f})')
ax.plot(x, p2(x), 'r--', linewidth=2, label=f'B trend (slope={z2[0]:.2f})')

# Axis customization
ax.set_xlabel('Time (hours)', fontsize=12, fontweight='bold')
ax.set_ylabel('Measurement (units)', fontsize=12, fontweight='bold')
ax.set_title('Complete Customized Plot Example\nwith All Elements', fontsize=16, fontweight='bold', pad
    =20)
ax.set_xlim(-0.5, 10.5)
ax.set_ylim(-5, 35)
ax.set_xticks(np.arange(0, 11, 1))
ax.set_yticks(np.arange(-5, 36, 5))
ax.tick_params(axis='both', which='major', labelsize=10)

# Grid, legend, and annotations
ax.grid(True, alpha=0.3, linestyle='--', linewidth=0.5)
ax.legend(loc='upper left', fontsize=10, framealpha=0.9)
ax.annotate('Important\nobservation', xy=(5, 15), xytext=(6, 25),
            arrowprops=dict(facecolor='black', shrink=0.05, width=1.5),
            fontsize=11, ha='center', bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))
ax.text(0.02, 0.98, 'Correlation: 0.85', transform=ax.transAxes,
        fontsize=11, verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='lightblue', alpha=0.8))

# Horizontal and vertical lines
ax.axhline(y=0, color='gray', linestyle='-', linewidth=0.5, alpha=0.5)
ax.axvline(x=5, color='gray', linestyle='-', linewidth=0.5, alpha=0.5)

# Secondary y-axis
ax2 = ax.twinx()
ax2.set_ylabel('Secondary Scale', fontsize=12, color='purple')
ax2.tick_params(axis='y', labelcolor='purple')
ax2.set_ylim(-10, 70)

plt.tight_layout()
plt.show()
```

Listing 25: Complete Customization with Scatter Plots and Regression Lines

0.8.3 Explanation of Customizations

- **Multiple datasets:** Visualize two related datasets on the same plot with distinct markers and colors.
- **Regression lines:** Add trend lines to highlight patterns.
- **Annotations and text boxes:** Highlight important observations and display contextual information.
- **Secondary y-axis:** Display an additional measurement scale.
- **Grid, limits, ticks, and legend:** Improve readability and interpretability.

0.8.4 Summary Table: When to Use Each Plot Type

Plot Type	Best Use Case	Example Code
Line Plot	Time series, trends	`plt.plot(x, y)`
Bar Plot	Category comparisons	`plt.bar(x, height)`
Histogram	Distribution analysis	`plt.hist(data, bins)`
Scatter Plot	Correlation analysis	`plt.scatter(x, y)`
Subplots	Multiple comparisons	`plt.subplots(rows, cols)`

Table 6: Overview of Matplotlib Plot Types and Best Use Cases

0.9 Program 6: Complete Customization Example

0.9.1 Introduction

This program demonstrates a fully customized Matplotlib plot, incorporating multiple elements such as multiple datasets, regression lines, annotations, text boxes, secondary axes, gridlines, legends, and axis customization. It serves as a reference for creating publication-quality visualizations.

0.9.2 Python Code Example

```
import numpy as np
import matplotlib.pyplot as plt

# Generate data
x = np.linspace(0, 10, 50)
y1 = 2*x + 1 + np.random.normal(0, 1, 50)
y2 = 3*x - 2 + np.random.normal(0, 1.5, 50)
```

```

# Create figure and axis
fig, ax = plt.subplots(figsize=(12, 8))

# Scatter plots
ax.scatter(x, y1, color='blue', marker='o', s=80, alpha=0.7, edgecolor='black', linewidth=1, label='
    Dataset A')
ax.scatter(x, y2, color='red', marker='s', s=80, alpha=0.7, edgecolor='black', linewidth=1, label='
    Dataset B')

# Regression lines
z1 = np.polyfit(x, y1, 1)
z2 = np.polyfit(x, y2, 1)
p1 = np.poly1d(z1)
p2 = np.poly1d(z2)
ax.plot(x, p1(x), 'b--', linewidth=2, label=f'A trend (slope={z1[0]:.2f})')
ax.plot(x, p2(x), 'r--', linewidth=2, label=f'B trend (slope={z2[0]:.2f})')

# Axis customization
ax.set_xlabel('Time (hours)', fontsize=12, fontweight='bold')
ax.set_ylabel('Measurement (units)', fontsize=12, fontweight='bold')
ax.set_title('Complete Customized Plot Example\nwith All Elements', fontsize=16, fontweight='bold', pad
    =20)
ax.set_xlim(-0.5, 10.5)
ax.set_ylim(-5, 35)
ax.set_xticks(np.arange(0, 11, 1))
ax.set_yticks(np.arange(-5, 36, 5))
ax.tick_params(axis='both', which='major', labelsize=10)

# Grid, legend, and annotations
ax.grid(True, alpha=0.3, linestyle='--', linewidth=0.5)
ax.legend(loc='upper left', fontsize=10, framealpha=0.9)
ax.annotate('Important\nobservation', xy=(5, 15), xytext=(6, 25),
            arrowprops=dict(facecolor='black', shrink=0.05, width=1.5),
            fontsize=11, ha='center', bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))
ax.text(0.02, 0.98, 'Correlation: 0.85', transform=ax.transAxes,
        fontsize=11, verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='lightblue', alpha=0.8))

# Horizontal and vertical lines
ax.axhline(y=0, color='gray', linestyle='-', linewidth=0.5, alpha=0.5)
ax.axvline(x=5, color='gray', linestyle='-', linewidth=0.5, alpha=0.5)

# Secondary y-axis
ax2 = ax.twinx()
ax2.set_ylabel('Secondary Scale', fontsize=12, color='purple')
ax2.tick_params(axis='y', labelcolor='purple')
ax2.set_ylim(-10, 70)

plt.tight_layout()
plt.show()
```

Listing 26: Complete Customization with Scatter Plots and Regression Lines

0.9.3 Explanation of Customizations

- **Multiple datasets:** Visualize two related datasets on the same plot with distinct markers and colors.
- **Regression lines:** Add trend lines to highlight patterns.
- **Annotations and text boxes:** Highlight important observations and display contextual information.
- **Secondary y-axis:** Display an additional measurement scale.
- **Grid, limits, ticks, and legend:** Improve readability and interpretability.

0.9.4 Summary Table: When to Use Each Plot Type

Plot Type	Best Use Case	Example Code
Line Plot	Time series, trends	`plt.plot(x, y)`
Bar Plot	Category comparisons	`plt.bar(x, height)`
Histogram	Distribution analysis	`plt.hist(data, bins)`
Scatter Plot	Correlation analysis	`plt.scatter(x, y)`
Subplots	Multiple comparisons	`plt.subplots(rows, cols)`

Table 7: Overview of Matplotlib Plot Types and Best Use Cases

5.3 Seaborn for Statistical Visualization: Box Plot, Pair Plot, Heat Map

1. Introduction to Seaborn

Seaborn is a Python data visualization library based on Matplotlib that provides a high-level interface for drawing statistical graphics. It integrates closely with pandas DataFrames and offers sensible defaults for statistical visualizations.

Key Features:

- Built-in themes for attractive graphics
- Functions for visualizing distributions and relationships

- Tools for visualizing linear regression models
- Support for multi-plot grids
- Integration with pandas DataFrames

2. Statistical Visualizations in Seaborn

Seaborn specializes in statistical visualizations that help understand distributions and relationships between variables, incorporating statistical estimations and aggregations.

3. Box Plot

3.1 What is a Box Plot?

A box plot displays the distribution of data based on a five-number summary: minimum, Q1, median, Q3, and maximum. Useful for identifying outliers and comparing distributions across categories.

3.2 Components of a Box Plot

Component	Formula	Description
Median (Q2)	Middle value	Line inside box (50th percentile)
First Quartile (Q1)	25th percentile	Bottom of box
Third Quartile (Q3)	75th percentile	Top of box
Interquartile Range (IQR)	Q3 - Q1	Height of box (middle 50% of data)
Lower Whisker	Q1 - 1.5 $\times$ IQR	Extends to smallest non-outlier
Upper Whisker	Q3 + 1.5 $\times$ IQR	Extends to largest non-outlier
Outliers	Points beyond whiskers	Individual points shown separately

3.3 Statistical Interpretation

- Median centered: symmetric distribution
- Median near bottom: right-skewed
- Median near top: left-skewed
- Wider box: more variability
- Outliers: unusual values

3.4 When to Use Box Plots

- Comparing distributions across categories
- Identifying outliers
- Understanding spread and symmetry
- Showing median and quartiles
- Detecting skewness

3.5 Seaborn Box Plot Functions

Function	Purpose
sns.boxplot()	Standard box plot
sns.boxenplot()	Letter-value plot for large datasets
sns.violinplot()	Box plot + kernel density estimate
sns.barplot()	Mean with error bars

3.6 Key Parameters

```
sns.boxplot(
    x=None, y=None, hue=None, data=None,
    order=None, palette=None, showfliers=True
)
```

4. Pair Plot

4.1 What is a Pair Plot?

Creates a grid showing pairwise relationships, with scatter plots for numeric pairs and distributions on the diagonal.

4.2 Structure of a Pair Plot

- Diagonal: distribution of each variable
- Off-diagonal: scatter plots of variable pairs
- Hue: color-code categories

4.3 What Pair Plots Reveal

Feature	What to Look For
Correlations	Linear patterns
Clusters	Groups of points
Outliers	Points far from main cloud
Distributions	Histogram shape on diagonal
Skewness	Asymmetric distributions
Modality	Multiple peaks
Non-linear relationships	Curved scatter patterns

4.4 When to Use Pair Plots

- Initial dataset exploration
- Identifying relationships
- Checking multicollinearity
- Discovering patterns and anomalies

4.5 Limitations

- Cluttered with many variables (¿7)
- Computationally intensive
- Only shows pairwise relationships

4.6 Key Parameters

```
sns.pairplot(
    data, hue=None, vars=None,
    kind='scatter', diag_kind='hist',
    palette=None, markers=None,
    height=2.5, aspect=1, corner=False
)
```

5. Heat Map

5.1 What is a Heat Map?

Graphical representation of data where values are shown by color, useful for matrices (correlation, confusion, etc.)

5.2 Structure

- Grid cells: values
- Color mapping: magnitude
- Color bar: legend
- Row/column labels
- Optional annotations

5.3 Common Applications

Application	Purpose
Correlation Matrix	Show variable relationships
Confusion Matrix	Classification performance
Missing Data Pattern	Show missing data
Time Series	Patterns across time
Genomic Data	Gene expression patterns

5.4 Interpreting Correlation Heat Maps

- Red/Blue: positive/negative correlation
- Color intensity: strength
- Dark diagonal: perfect self-correlation
- Patterns: correlated variable groups

5.5 When to Use Heat Maps

- Visualizing correlation matrices
- Confusion matrices
- Missing data patterns
- Identifying clusters
- Comparing multiple variables

5.6 Key Parameters

```
sns.heatmap(
    data, annot=False, fmt='.2g', cmap='coolwarm',
    center=None, vmin=None, vmax=None, cbar=True,
    square=False, linewidths=0, linecolor='white',
    xticklabels='auto', yticklabels='auto',
    mask=None, ax=None
)
```

5.7 Color Maps for Heat Maps

Color Map	Best For
coolwarm	Diverging data
viridis	Sequential data
RdBu_r	Red-Blue diverging
YlOrRd	Yellow-Orange-Red intensity
Blues	Single-color sequential

6. Comparison of Seaborn Statistical Plots

Feature	Box Plot	Pair Plot	Heat Map
Purpose	Distribution comparison	Multi-variable relationships	Matrix visualization
Data Type	Numeric + Categorical	Multiple numeric	2D Matrix
Shows	Quartiles, outliers	Pairwise relationships	Color-coded values
Best For	Group comparisons	Initial exploration	Correlation/confusion
Limitation	Limited detail	Cluttered ¿7 vars	Needs structured data

7. Summary of Key Functions

Plot Type	Function	Basic Syntax
Box Plot	sns.boxplot()	`sns.boxplot(x='category', y='value', data=df)`
Pair Plot	sns.pairplot()	`sns.pairplot(df, hue='category')`
Heat Map	sns.heatmap()	`sns.heatmap(df.corr(), annot=True)`

5.5 Interactive Visualization using Plotly

1. Introduction to Plotly

Plotly is an open-source graphing library for creating interactive, publication-quality visualizations. Unlike static libraries like Matplotlib, Plotly generates web-based visualizations that allow zooming, panning, hovering, and clicking interactions. Built on Plotly.js, Plotly for Python brings interactive web graphics directly into Python.

2. The Plotly Ecosystem

Plotly provides two primary interfaces for visualization:

2.1 Plotly Express

Plotly Express (px) is a high-level API for creating charts with concise syntax and sensible defaults. It is ideal for exploratory data analysis and rapid prototyping.

Key characteristics:

- One-line chart creation
- Direct integration with Pandas DataFrames
- Automatic interactive features
- Pre-styled charts

2.2 Graph Objects

Graph Objects (go) is a low-level interface offering fine-grained control over every aspect of a visualization.

Key characteristics:

- Complete customization
- Multi-layered visualizations
- Modify figures after creation
- Access to advanced chart types

3. Core Concepts in Plotly

3.1 Figure Structure

Every Plotly visualization is built around a Figure object:

Component	Description	Contents
data	Visual representation	List of traces (scatter, bar, etc.)
layout	Chart environment	Titles, axes, legends, annotations

3.2 Traces

A trace is the fundamental building block of a figure. Each trace represents one dataset and its visualization. Multiple traces can be combined in one figure.

Common trace types:

- Scatter: points, lines, or both
- Bar: vertical or horizontal bars
- Box: statistical box plots
- Heatmap: color-coded matrices

3.3 Interactivity Features

Feature	Description	User Benefit
Hover Tooltips	Display values on hover	Immediate access to data
Zoom & Pan	Scale and move across plot	Examine details
Legend Toggle	Show/hide traces via legend	Focus on specific series
Range Sliders	Select ranges dynamically	Filter data interactively
Download Options	Save as PNG or SVG	Share or include in reports

4. The Two APIs: Express vs. Graph Objects

4.1 Plotly Express (px)

```
import plotly.express as px
fig = px.scatter(df, x='column1', y='column2', color='category')
fig.show()
```

Best for:

- Quick exploratory analysis
- Standard chart types
- Beginners and rapid prototyping

4.2 Graph Objects (go)

```
import plotly.graph_objects as go
fig = go.Figure()
fig.add_trace(go.Scatter(x=df['x'], y=df['y'], mode='markers'))
fig.update_layout(title='Custom Title')
fig.show()
```

Best for:

- Complex, multi-trace visualizations
- Fine-tuned customization
- Specialized chart types

5. Chart Types in Plotly

Category	Chart Types
Basic Charts	Scatter, line, bar, pie, histogram
Statistical Charts	Box plot, violin plot, histogram, heatmap
Scientific Charts	Contour, error bars, ternary plots
Financial Charts	Time series, candlestick, OHLC
Maps	Choropleth, scatter geo, line geo
3D Charts	3D scatter, 3D surface, 3D mesh

6. Integration with Data Analysis Workflow

- **With Pandas:** Plotly Express accepts DataFrames directly
- **With Jupyter:** Figures render inline, supporting interactive exploration
- **With Dash:** Plotly provides the visualization foundation for interactive web apps

7. Customization Capabilities

7.1 Visual Styling

- Templates: 'plotly', 'plotly$_{d}ark'$,' $ggplot2'$,' $seaborn'$,' $simple_{w}hite'$
- Color scales: sequential, diverging, qualitative
- Fonts: size, family, color

7.2 Layout Control

- Margins: control spacing
- Annotations: text, shapes, arrows
- Axes: tick marks, grids, ranges

7.3 Hover Information

- Tooltips: control displayed data
- Hover mode: single, closest, compare

8. Export and Sharing

Format	Method	Use Case
HTML	fig.write_html('file.html')	Share interactive charts
Static Images	fig.write_image('file.png')	Include in reports
Plotly Cloud	fig.show(renderer="plotly_mimetype")	Host on Plotly's platform

9. Advantages of Interactive Visualization

- **Deeper Data Exploration:** Probe points, zoom, and filter dynamically
- **Enhanced Communication:** Users explore data at their own pace
- **Efficient EDA Workflow:** Rapid iteration and hypothesis testing

10. Summary: Key Takeaways

Concept	Key Point
Plotly Express	High-level API for simple charts
Graph Objects	Low-level API for full control
Interactivity	Hover, zoom, pan, toggle built-in
Figure Structure	Data (traces) + Layout (environment)
Integration	Works with Pandas and Jupyter
Export	HTML for sharing, images for reports

listings hyperref tikz
margin=1in

5.6 Visualization Driven Insight Generation

1. Introduction to Insight Generation

Data visualization is not merely about creating charts; it is about generating insights that drive decision-making. Visualization-driven insight generation uses graphical representations to discover patterns, trends, and anomalies leading to actionable understanding.

2. The Insight Generation Process

Data → Visualization → Pattern Recognition → Hypothesis → Insight → Action

Stage	Description
Data	Raw information collected from various sources
Visualization	Graphical representation of data
Pattern Recognition	Identifying trends, clusters, outliers
Hypothesis	Formulating explanations for patterns
Insight	Deep understanding with business implications
Action	Decisions based on insights

3. Types of Insights from Visualizations

3.1 Descriptive Insights

Answers "What happened?" based on historical data. **Examples:**

- Sales peaked in December over the last three years
- Customer churn is highest among users in their first 90 days
- Product A consistently outsells Product B across regions

3.2 Diagnostic Insights

Answers "Why did it happen?" by revealing relationships and causes. **Examples:**

- Sales decline correlates with competitor price reductions
- High churn coincides with poor customer service
- Regional variations linked to local economic indicators

3.3 Predictive Insights

Answers "What might happen?" using trends. **Examples:**

- Upward trend in website traffic suggests continued growth
- Seasonal patterns indicate next peak in July
- Leading indicators point to potential downturn

3.4 Prescriptive Insights

Answers "What should we do?" combining patterns with domain knowledge. **Examples:**

- Focus marketing on high-growth segments
- Adjust inventory based on visualized demand
- Target interventions where risk factors cluster

4. Visual Cues That Generate Insights

4.1 Patterns

Pattern	What It Reveals	Example Insight
Trend	Directional movement	"Sales increased 15% annually"
Seasonality	Predictable patterns	"December sees 30% sales spike"
Cyclicality	Long-term waves	"Market correction every 5-7 years"
Correlation	Variables move together	"Satisfaction correlates with retention"
Clustering	Groups of points	"High-value customers cluster in urban areas"

4.2 Anomalies

Anomaly	What It Reveals	Example Insight
Outliers	Points far from norm	"One transaction is 10× larger than average"
Structural Breaks	Sudden pattern changes	"Website traffic dropped after redesign"
Gaps	Missing expected data	"No sales from a key region"
Unexpected Stability	No change where expected	"Prices didn't drop despite competitor sale"

4.3 Comparisons

Comparison	What It Reveals	Example Insight
Benchmarking	Against standards	"Performance 20% below industry avg"
Before/After	Impact of interventions	"Satisfaction improved after training"
Segments	Differences between groups	"Younger users prefer mobile"
Actual vs Forecast	Prediction accuracy	"Overestimate Q1 sales by 15%"

5. Role of Visualization Types in Insight Generation

5.1 Time Series Visualizations

Best for: trend, seasonality, cyclical insights

Chart Type	Insight Generated
Line Chart	Long-term trends, rate of change
Area Chart	Magnitude of change over time
Seasonal Plot	Year-over-year patterns
Time Series Decomposition	Separating trend from seasonality

5.2 Comparison Visualizations

Best for: ranking, benchmarking, segment insights

Chart Type	Insight Generated
Bar Chart	Relative performance across categories
Grouped Bar Chart	Multi-factor comparisons
Diverging Bar Chart	Deviation from baseline
Dot Plot	Precise comparisons

5.3 Distribution Visualizations

Best for: spread, central tendency, outlier insights

Chart Type	Insight Generated
Histogram	Shape of distribution, modality
Box Plot	Quartiles, spread, outliers
Violin Plot	Distribution shape + density
Density Plot	Smooth distribution visualization

5.4 Relationship Visualizations

Best for: correlation, clustering, interaction

Chart Type	Insight Generated
Scatter Plot	Correlation strength and direction
Bubble Chart	Three-variable relationships
Heatmap	Correlation matrix patterns
Pair Plot	Multi-variable relationships

5.5 Composition Visualizations

Best for: part-to-whole, proportion

Chart Type	Insight Generated
Stacked Bar Chart	Category composition across groups
Pie/Donut Chart	Simple proportions
Treemap	Hierarchical proportions
Waterfall Chart	Sequential composition changes

6. Insight Generation Framework

6.1 Ask the Right Questions

Question Type	Examples
Descriptive	What happened? How much? How many?
Temporal	When did it happen? Has it changed?
Comparative	How does this compare to that?
Relational	What is connected to what?
Causal	What causes what?

6.2 Choose the Right Visualization

Question	Visualization
"How has this changed over time?"	Line chart
"How do categories compare?"	Bar chart
"What is the distribution?"	Histogram, box plot
"What is the relationship?"	Scatter plot, heatmap
"What is the composition?"	Stacked bar, treemap

6.3 Look for Patterns Systematically

Scan visualizations with intention:

- Overall pattern: What is the big picture?
- Exceptions: What doesn't fit the pattern?
- Groups: Are there natural clusters?
- Trends: Direction over time?
- Relationships: Do variables move together?

6.4 Generate Hypotheses

Transform observations into testable statements:

- "Sales drop in January → Hypothesis: Post-holiday spending reduction"
- "Churn spikes after 30 days → Hypothesis: Onboarding is insufficient"
- "Regional variations → Hypothesis: Local competitors affect pricing"

6.5 Validate and Refine

- Drill down into specific segments
- Add context variables
- Analyze different time scales
- Compare to external benchmarks

7. Common Insight Patterns

- **Unexpected Finding:** Data contradicts expectation → Shift focus
- **Hidden Relationship:** Two unrelated variables correlate → Identify drivers
- **Tipping Point:** Gradual change accelerates → Prepare for scaling
- **Silent Majority:** Large segment behaves differently → Reallocate resources
- **Leading Indicator:** Metric predicts another → Build forecasting

8. From Insight to Action

Insight	Action
"Sales peak in November"	Increase inventory in October
"Customers churn after poor support"	Improve support training
"Product A outperforms in region X"	Increase marketing in region X
"Price sensitivity varies by segment"	Implement segmented pricing

9. Common Pitfalls in Insight Generation

Pitfall	Description	Avoidance
Confirmation Bias	Seeing only confirming evidence	Seek disconfirming evidence
Overinterpretation	Reading noise as signal	Use statistical significance
Correlation/Causation	Assuming related implies caused	Investigate mechanisms
Missing Context	Insights without business context	Connect to business questions
Analysis Paralysis	Too many insights, no action	Prioritize actionable findings

11. Key Takeaways

Principle	Description
Purpose	Visualization exists to generate insights, not just show data
Process	Systematic: visualize → detect patterns → hypothesize → validate → act
Patterns	Look for trends, outliers, clusters, correlations, comparisons
Questions	Start with clear questions to guide visualization choice
Validation	Verify insights with additional analysis
Action	Insights must lead to decisions

Case Study: End-to-End Visualization and Reporting Project

Project Title: E-commerce Sales Performance Dashboard

This case study simulates a real-world scenario where a data analyst uses Python to build an interactive reporting solution for a company's sales team. The goal is to transform raw transactional data into actionable insights through a structured workflow.

Phase 1: Problem Definition and Data Acquisition

Objective: The sales director requires a centralized dashboard to monitor key performance indicators (KPIs) such as revenue, profit, and customer acquisition across different channels and regions. The aim is to replace manual Excel reports with an automated, interactive solution.

Data Source: Raw transactional data from CSV files or SQL database with columns:

- order_id
- order_date
- customer_id
- product_category
- revenue
- cost
- country
- acquisition_channel

Phase 2: Data Ingestion and Cleaning (using Pandas)

Process: Load the raw data into a Pandas DataFrame. Address missing values, inconsistent types, and duplicate records.

Implementation:

```
import pandas as pd

# Load data
df = pd.read_csv('sales_data.csv')

# Overview
df.info()
df.describe()

# Handle missing values
df['revenue'].fillna(df['revenue'].mean(), inplace=True)
df['cost'].fillna(df['cost'].median(), inplace=True)

# Remove duplicates
df.drop_duplicates(subset='order_id', inplace=True)

# Convert date column
df['order_date'] = pd.to_datetime(df['order_date'])
```

Phase 3: Exploratory Data Analysis (EDA)

Process: Understand data structure, distributions, and relationships to guide dashboard design.

Implementation:

- **Univariate Analysis:** Histograms and box plots for revenue and cost to detect outliers.
- **Bivariate Analysis:** Pair plots and correlation heatmaps using Seaborn.
- **Feature Engineering:**
  - Extract month and year from order_date
  - Calculate profit: revenue - cost

Phase 4: Metrics Definition and Calculation

Process: Define KPIs to display on the dashboard.

Implementation:

```
# Total Revenue
total_revenue = df['revenue'].sum()

# Total Profit
df['profit'] = df['revenue'] - df['cost']
total_profit = df['profit'].sum()

# Average Order Value
aov = df['revenue'].mean()

# Profit Margin
profit_margin = (total_profit / total_revenue) * 100
```

Phase 5: Static and Interactive Visualization

Process: Create plots to answer key business questions.

Implementation:

- **Line Chart (Matplotlib/Seaborn):** Monthly revenue trend with trend line.
- **Bar Chart (Seaborn):** Revenue by product category or acquisition channel.
- **Treemap (Plotly):** Hierarchical revenue contribution by product categories.
- **Geographical Map (Plotly):** Choropleth map of total revenue by country.

Phase 6: Dashboard Development and Automation

Process: Combine visualizations and KPIs into an interactive dashboard.

Implementation (Streamlit):

```
import streamlit as st
import plotly.express as px

st.title("E-commerce Sales Dashboard")

# Sidebar filters
start_date = st.sidebar.date_input("Start Date")
end_date = st.sidebar.date_input("End Date")
category_filter = st.sidebar.multiselect("Category", df['product_category'].unique())

# KPI cards
st.metric("Total Revenue", f"${total_revenue:,.2f}")
st.metric("Total Profit", f"${total_profit:,.2f}")
st.metric("Profit Margin", f"{profit_margin:.2f}%")

# Visualizations
fig_line = px.line(df, x='order_date', y='revenue', title='Monthly Revenue')
st.plotly_chart(fig_line)
```

Phase 7: Deployment and Insights Communication

Deployment: Streamlit app deployed on Streamlit Cloud for team access.

Final Insights:

- Overall revenue reached $9.8M with an average margin of 30.9%.
- Web and Mobile App channels contributed 75% of total sales.
- Electronics category is the dominant revenue driver, followed by Home Goods.

Chapter-6
Data Engineering and Automation

6.1 Overview of Data Engineering in Applied Data Science

1. Introduction to Data Engineering

Data engineering is the foundation upon which data science is built. While data scientists focus on analysis, modeling, and deriving insights, data engineering is concerned with the practical aspects of acquiring, preparing, and maintaining data for these tasks. It involves the systems and processes that enable data to be accessible, reliable, and usable at scale.

2. The Role of Data Engineering in the Data Science Lifecycle

In an applied data science project, data engineering occupies the critical early stages and supports the entire lifecycle. The typical workflow includes:

- **Data Acquisition:** Getting raw data from various sources such as databases, files, APIs, or streams.
- **Data Preparation:** Cleaning, transforming, and structuring data for analysis. This often represents the bulk of data engineering work.
- **Data Modeling & Analysis:** The core work of a data scientist, which relies entirely on the quality of the prepared data.
- **Deployment & Monitoring:** Putting models into production, which requires robust and reliable data pipelines to feed the system.

3. Key Concepts and Components of Data Engineering

The following concepts are central to data engineering in applied data science:

- **Data Pipelines and ETL/ELT:** Moving data from source systems to destinations for analysis.
  - *ETL (Extract, Transform, Load):* Transform data before loading.
  - *ELT (Extract, Load, Transform):* Load first, transform in place, common with modern cloud data warehouses.
- **Data Wrangling:** Hands-on cleaning and transforming of raw data into structured, usable formats. Includes:
  - Handling missing values
  - Standardizing formats
  - Correcting errors
  - Merging datasets
- **Data Automation:** Using technology to execute repetitive tasks without manual intervention.
  - Improves efficiency and reliability
  - Automates quality checks and regular updates
- **Imperative vs. Declarative Approaches in Automation:**
  - *Imperative:* Explicitly define step-by-step process (like a detailed recipe).
  - *Declarative:* Specify desired outcome and let the system determine how to achieve it.

4. The Data Engineer and Data Scientist Relationship

The responsibilities of data engineers and data scientists are distinct but highly complementary:

- **Data Engineer:**
  - *Primary Focus:* Building and maintaining data infrastructure
  - *Typical Tasks:* Design and implement ETL/ELT pipelines, ensure data quality, manage data storage, optimize retrieval
  - *Key Tools:* SQL, Spark, cloud platforms (AWS, GCP, Azure), workflow managers (Airflow)
- **Data Scientist:**
  - *Primary Focus:* Analyzing data to solve business problems
  - *Typical Tasks:* Perform statistical analysis, build machine learning models, create visualizations, communicate insights
  - *Key Tools:* Python (pandas, scikit-learn), R, Jupyter notebooks

The handoff between these roles is a critical point in any project. A well-structured data engineering foundation allows data scientists to focus on core competencies without being bogged down by data acquisition and cleaning.

6.2 Designing and Implementing ETL Pipelines

1. Introduction to ETL Pipelines

An ETL pipeline is a fundamental data engineering pattern that moves data from source systems to a destination data warehouse or data lake. ETL stands for **Extract, Transform, Load**, representing the three core phases.

In applied data science, ETL pipelines are crucial because they ensure data scientists work with clean, reliable, and well-structured data rather than spending most of their time on manual data wrangling.

2. The Three Phases of ETL

2.1 Extract Phase

The Extract phase involves retrieving data from various sources. This forms the foundation for the rest of the pipeline.

Common Data Sources:

- Relational databases: PostgreSQL, MySQL, SQL Server
- Files: CSV, Excel, JSON, Parquet, text files
- APIs: RESTful web services, SaaS platforms
- Data streams: Real-time event data from Kafka or similar platforms

Extraction Strategies:

- Full extraction: Pull all data each time
- Incremental extraction: Pull only changed data
- Batch processing: Process data in scheduled chunks
- Streaming: Continuous processing of incoming data

Key Python Considerations:

- `requests` for API calls
- `pandas` (`read_csv`, `read_json`, `read_excel`) for file ingestion
- `SQLAlchemy` for database connections
- Handle API rate limits and authentication

2.2 Transform Phase

The Transform phase cleans, standardizes, and enriches data for analysis. This is typically the most complex stage.

Common Transformation Operations:

- **Data cleaning:** Remove duplicates, handle missing values (`drop_duplicates()`, `fillna()`, `dropna()`)
- **Data standardization:** Ensure consistent formats (`pd.to_datetime()`, string operations)
- **Data type conversion:** Correct data types (`astype()`, `infer_objects()`)
- **Filtering:** Remove irrelevant records (Boolean indexing, `query()`)
- **Joining/merging:** Combine datasets (`merge()`, `concat()`)
- **Aggregation:** Summarize data (`groupby()`, `agg()`)
- **Feature engineering:** Create derived columns (custom functions, `apply()`)
- **Validation:** Check data quality (custom logic)

Transformation Approaches:

- In-Notebook: Interactive exploration (common in EDA)
- Scripted: Reusable Python functions and classes
- Distributed: Using Spark for large datasets

2.3 Load Phase

The Load phase writes the transformed data to its final destination.

Load Destinations:

- Data warehouses: Snowflake, Redshift, BigQuery
- Data lakes: S3, Azure Data Lake, Google Cloud Storage
- Databases: PostgreSQL, MySQL
- File systems: Parquet, CSV

Load Strategies:

- Full load: Replace entire destination table
- Incremental load: Append new records

- Upsert: Update existing, insert new
- Partitioned load: Write partitioned by date/category

File Formats for Loading:

- Parquet: Columnar, compressed, preserves types
- CSV: Human-readable, widely compatible
- JSON: Flexible for nested data

3. ETL vs. ELT

Traditional ETL: Transform data before loading. **Modern ELT:** Load first, transform inside the warehouse.

Key Differences:

- Transform location: ETL on server, ELT in warehouse
- Performance: ETL limited, ELT leverages warehouse scalability
- Raw data availability: ETL loses raw, ELT preserves raw
- Use case: ETL for limited warehouse power, ELT for cloud warehouses

4. Designing Robust ETL Pipelines

Best Practices:

- **Modular Architecture:** Separate extraction, transformation, and loading components
- **Data Quality Validation:** Schema checks, completeness, uniqueness, range, and count validation
- **Error Handling and Logging:** Comprehensive logs, defensive reading, isolate bad data, alerting
- **Incremental Processing:** Process only new/changed data, track timestamps, ensure idempotency
- **Orchestration:** Use tools like Apache Airflow, Prefect, or cloud-native services for scheduling and dependency management

5. Python Libraries for ETL

- **Pandas:** Core data manipulation and transformation
- **NumPy:** Numerical operations
- **SQLAlchemy:** Database connections and ORM
- **Requests:** API data extraction
- **PySpark:** Distributed processing for big data
- **Apache Airflow:** Workflow orchestration
- **Great Expectations:** Data validation and testing
- **Boto3:** AWS services integration (S3, Redshift)

7. Practical Example: A Simple ETL Pipeline in Python

While reference books do not provide a single complete example, the following pattern combines concepts from multiple sources.

Python ETL Pipeline Structure

```python
import pandas as pd
import logging
from datetime import datetime

# Configure logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def extract_data(file_path):
    """Extract data from source file"""
    logger.info(f"Extracting data from {file_path}")
    df = pd.read_csv(file_path)  # Could also be JSON, Excel, API, etc.
    logger.info(f"Extracted {len(df)} rows")
    return df

def transform_data(df):
    """Clean and transform the data"""
    logger.info("Starting transformation")

    # Remove duplicates
    initial_count = len(df)
    df = df.drop_duplicates()
    logger.info(f"Removed {initial_count - len(df)} duplicates")

    # Handle missing values
    df = df.fillna({
```

```
        'numeric_column': 0,
        'categorical_column': 'Unknown'
    })

    # Standardize date formats
    df['date'] = pd.to_datetime(df['date'], errors='coerce')

    # Filter invalid records
    df = df[df['value'] > 0]

    # Create derived columns
    df['processed_date'] = datetime.now()

    logger.info(f"Transformation complete. {len(df)} rows remain")
    return df

def validate_data(df):
    """Run quality checks"""
    logger.info("Validating data")

    # Schema validation
    expected_columns = ['id', 'date', 'value', 'category']
    assert all(col in df.columns for col in expected_columns), "Missing columns"

    # Completeness check
    null_counts = df.isnull().sum()
    logger.info(f"Null counts: {null_counts}")

    # Uniqueness check
    assert df['id'].is_unique, "Duplicate IDs found"

    logger.info("Validation passed")
    return True

def load_data(df, output_path):
    """Load to destination"""
    logger.info(f"Loading {len(df)} rows to {output_path}")
    df.to_parquet(output_path, index=False)
    logger.info("Load complete")

def run_pipeline(input_file, output_file):
    """Orchestrate the full ETL process"""
    try:
        logger.info("Starting ETL pipeline")

        # Extract
        data = extract_data(input_file)

        # Transform
        transformed = transform_data(data)

        # Validate
        validate_data(transformed)

        # Load
        load_data(transformed, output_file)

        logger.info("Pipeline completed successfully")

    except Exception as e:
        logger.error(f"Pipeline failed: {str(e)}")
        raise

# Execute pipeline
if __name__ == "__main__":
    run_pipeline("data/raw/input.csv", "data/curated/output.parquet")
```

Extract Phase

- Function: `extract_data(file_path)`
- Reads data from a CSV file (can be JSON, Excel, API, etc.)
- Logs the number of rows extracted

Transform Phase

- Function: `transform_data(df)`
- Removes duplicates
- Handles missing values using `fillna()`
- Standardizes date formats (`pd.to_datetime`)
- Filters invalid records (e.g., `value > 0`)
- Creates derived columns (e.g., `processed_date = datetime.now()`)
- Logs row counts before and after transformation

Validation Phase

- Function: `validate_data(df)`
- Schema validation: checks for expected columns
- Completeness check: counts null values
- Uniqueness check: ensures unique IDs
- Logs validation results

Load Phase

- Function: `load_data(df, output_path)`
- Writes transformed data to Parquet file
- Logs completion of loading

Pipeline Orchestration

- Function: `run_pipeline(input_file, output_file)`
- Steps executed in order: Extract → Transform → Validate → Load
- Logs start, success, and errors

8. Scaling Considerations

- **Pandas:** For datasets that fit in memory (up to a few GB)
- **Dask:** For larger-than-memory datasets on a single machine
- **Spark:** Distributed processing across clusters
- **Cloud Services:** Managed ETL (AWS Glue, Google Dataflow)

9. Testing ETL Pipelines

Reliable pipelines require comprehensive testing:

- **Unit tests:** Test individual transformation functions
- **Integration tests:** Test the flow between components
- **Data quality tests:** Validate against known good datasets
- **Regression tests:** Ensure changes don't break existing functionality

10. Summary

- **ETL Definition:** Extract, Transform, Load – the three phases of data movement
- **Extract:** Get data from sources; choose full or incremental strategies
- **Transform:** Clean, standardize, enrich; the most complex phase
- **Load:** Write to destination; choose full or incremental loading
- **Design Principles:** Modularity, validation, error handling, logging
- **Python Tools:** Pandas for transformation, orchestration tools for workflow
- **Scaling:** Move from Pandas to Dask to Spark as data grows

6.3 Automating Workflows with Schedulers (CRON, Schedule)

1. Introduction to Workflow Automation

Workflow automation is the process of automating repetitive tasks and processes to improve efficiency, reduce errors, and ensure consistency. In data engineering, schedulers execute data pipelines, scripts, and maintenance tasks at predetermined times without manual intervention. This allows data professionals to focus on higher-value activities while routine operations run reliably in the background.

2. CRON: The Unix/Linux Scheduler

2.1 What is CRON?

Cron is a time-based job scheduler in Unix-like operating systems. It runs as a daemon (background process) and executes commands or scripts at specified dates and times. Cronjobs are stored in a crontab file, and Cron is commonly used on systems like Ubuntu Linux.

2.2 Installing and Managing Cron

- Update package index: `apt update`
- Install Cron: `apt install cron`
- Ensure service is enabled and running: `systemctl enable cron.service`

Output example:

> Synchronizing state of cron.service with SysV service script with /lib/systemd/systemd-sysv-install.
> Executing: /lib/systemd/systemd-sysv-install enable cron

2.3 Understanding Crontab Structure

Crontab entries follow this order:

- **MINUTE**: 0-59
- **HOUR**: 0-23
- **DAY OF MONTH**: 1-31
- **MONTH**: 1-12 (or JAN-DEC)
- **DAY OF WEEK**: 0-6 (0 = Sunday, or SUN-SAT)
- **COMMAND**: Path to the script/command

Example: Run a backup every night at 4:30 AM:

```
30 04 * * * /var/www/websites/backup.sh
```

2.4 Special Characters in Cron Syntax

- `*` : Wildcard representing "all" (e.g., * * * * * runs every minute)
- `,` : List of values (e.g., 0,30 * * * * runs at minute 0 and 30)
- `-` : Range (e.g., 0-29 * * * * runs every minute for first 30 minutes)
- `*/` : Step value (e.g., */10 * * * * runs every 10 minutes)
- `?` : No specific value (used in Quartz Scheduler)

2.5 Special Time Strings

- `@reboot` : Run once at startup
- `@yearly / @annually` : Run once a year
- `@monthly` : Run once a month
- `@weekly` : Run once a week
- `@daily / @midnight` : Run once a day
- `@hourly` : Run once an hour

2.6 Managing Crontabs

- `crontab -e` : Edit crontab (creates new if none exists)
- `crontab -l` : List current cronjobs
- `crontab -r` : Remove all cronjobs
- `crontab -r -i` : Remove with confirmation

2.7 Redirecting Output

To capture output and errors for logging:

```
0 5 * * * /path/to/script.sh >> /var/log/script.log 2>&1
```

2.8 Practical Cron Examples

- `30 14 15 6 3 /path/to/script.sh` : Run on Wednesday, June 15, at 2:30 PM
- `0 9-17/2 * * 1-5 /path/to/script.sh` : Run every second hour between 9 AM-5 PM, Monday-Friday
- `@reboot /path/to/startup.sh` : Run at system startup

2.9 Cron Daemon Management

- Check status (systemd): `systemctl status cron`
- Check status (older systems): `service cron status`
- Restart cron if needed: `systemctl start cron`

0.1 Schedule Library for Workflow Automation

0.1.1 Overview

The `schedule` library is a Python package that provides an in-process scheduler for periodic jobs. It offers a simple and readable syntax for scheduling functions to run at specific intervals or times.

0.1.2 Installation and Setup

```
pip install schedule
```

0.1.3 Core Concepts

Job

A job is a task (function) scheduled to run at specific times. Each job includes:

- The function that will be executed
- The timing interval for execution
- Optional arguments passed to the function

Scheduler

The scheduler manages all registered jobs and determines when each task should be executed according to its schedule.

Job Runner

The `run_pending()` function checks scheduled jobs and executes those whose execution time has arrived.

0.1.4 Basic Syntax and Structure

Fundamental Pattern

```
import schedule
import time

def my_task():
    print("Task executed")

# Schedule the task
schedule.every(10).minutes.do(my_task)

# Keep the scheduler running
while True:
    schedule.run_pending()
    time.sleep(1)
```

0.1.5 Time Units and Intervals

Seconds

```
schedule.every(10).seconds.do(task)      # Every 10 seconds
schedule.every().second.do(task)         # Every second
```

Minutes

```
schedule.every(5).minutes.do(task)       # Every 5 minutes
schedule.every().minute.do(task)         # Every minute
```

Hours

```
schedule.every(3).hours.do(task)         # Every 3 hours
schedule.every().hour.do(task)           # Every hour
```

Days

```
schedule.every(2).days.do(task)          # Every 2 days
schedule.every().day.do(task)            # Every day
```

Weeks

```
schedule.every().week.do(task)           # Every week
schedule.every(2).weeks.do(task)         # Every 2 weeks
```

0.1.6 Specific Day Scheduling

Weekday Scheduling

```
# Specific weekdays
schedule.every().monday.do(task)
schedule.every().tuesday.do(task)
schedule.every().wednesday.do(task)
schedule.every().thursday.do(task)
schedule.every().friday.do(task)
schedule.every().saturday.do(task)
schedule.every().sunday.do(task)

# Weekdays only (Monday to Friday)
schedule.every().monday.to.friday.do(task)

# Multiple days
schedule.every().monday.and.wednesday.do(task)
```

6.4 Logging, Monitoring, and Error Handling in Data Pipelines

Overview

In data engineering, logging, monitoring, and error handling are essential practices that ensure pipeline reliability, maintainability, and observability. These mechanisms help detect problems early, diagnose failures quickly, and maintain data quality throughout the workflow.

1. Logging Fundamentals

Python's Logging Module

Python provides a built-in `logging` module that offers a flexible framework for generating log messages from applications.

Logging Levels

Python defines five standard logging levels arranged by increasing severity:

- **DEBUG (10)** – Detailed information used for diagnostic purposes.
- **INFO (20)** – Confirmation that operations are functioning as expected.
- **WARNING (30)** – Indicates unexpected situations or potential problems.
- **ERROR (40)** – A serious issue preventing a function from executing properly.
- **CRITICAL (50)** – A severe error indicating a system-level failure requiring immediate attention.

Basic Logging Configuration

```
import logging

# Basic configuration
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    filename='pipeline.log',
    filemode='a'
)

# Create different types of logs
logging.debug("Variable has value: %s", path)
logging.info("Data has been transformed and will now be loaded")
logging.warning("Unexpected number of rows detected: %d", row_count)
logging.error("KeyError arose in execution: %s", str(e))
```

Example output:

```
INFO: Data has been transformed and will now be loaded
WARNING: Unexpected number of rows detected
ERROR: KeyError arose in execution
```

Module-Level Logging Pattern

For production pipelines, it is recommended to use named loggers rather than the root logger.

```
import logging

# Create module-level logger
logger = logging.getLogger(__name__)

def process_data():
    logger.info("Starting data processing")
    try:
        # Processing logic
        logger.debug("Processing row %d", row_num)
    except Exception as e:
        logger.error("Processing failed: %s", str(e))
        raise
```

2. Advanced Logging Configuration

Using Dictionary Configuration

For complex pipelines, centralized logging configuration can be implemented using `dictConfig`.

```
import logging.config

LOGGING_CONFIG = {
    'version': 1,
    'formatters': {
        'detailed': {
            'format': '%(asctime)s - %(name)s - %(levelname)s - %(message)s'
        },
        'simple': {
            'format': '%(levelname)s: %(message)s'
        }
    },
    'handlers': {
        'console': {
            'class': 'logging.StreamHandler',
            'level': 'INFO',
            'formatter': 'simple',
            'stream': 'ext://sys.stdout'
        },
        'file': {
            'class': 'logging.handlers.RotatingFileHandler',
            'level': 'DEBUG',
            'formatter': 'detailed',
            'filename': 'pipeline.log',
            'maxBytes': 10485760,
            'backupCount': 5
        },
        'error_file': {
            'class': 'logging.handlers.RotatingFileHandler',
            'level': 'ERROR',
            'formatter': 'detailed',
            'filename': 'errors.log',
            'maxBytes': 10485760,
            'backupCount': 3
        }
    },
    'loggers': {
        'pipeline': {
            'handlers': ['console', 'file', 'error_file'],
            'level': 'DEBUG',
            'propagate': False
        }
    },
    'root': {
        'handlers': ['console'],
        'level': 'WARNING'
    }
}

logging.config.dictConfig(LOGGING_CONFIG)
logger = logging.getLogger('pipeline')
```

Structured Logging with JSON

For better integration with log aggregation tools and monitoring platforms, log messages can be structured in JSON format. Structured logging allows logs to be parsed, searched, and analyzed more easily by centralized logging systems.

```
import json
import logging
from datetime import datetime

class JSONFormatter(logging.Formatter):
    def format(self, record):
        log_record = {
            'timestamp': datetime.utcnow().isoformat(),
            'level': record.levelname,
            'logger': record.name,
            'message': record.getMessage(),
            'module': record.module,
            'function': record.funcName
        }

        # Add exception info if present
        if record.exc_info:
            log_record['exception'] = self.formatException(record.exc_info)

        # Add custom attributes
        if hasattr(record, 'pipeline_id'):
            log_record['pipeline_id'] = record.pipeline_id

        return json.dumps(log_record)

# Configure JSON logging
handler = logging.StreamHandler()
handler.setFormatter(JSONFormatter())

logger = logging.getLogger('pipeline')
logger.addHandler(handler)
logger.setLevel(logging.INFO)

```

```
# Usage with custom attributes
logger.info("Pipeline started", extra={'pipeline_id': 'daily_etl_001'})
```

3. Error Handling Strategies

Try-Except Blocks

Basic exception handling combined with logging provides detailed context when failures occur during pipeline execution.

```
import logging
import pandas as pd

logger = logging.getLogger(__name__)

def extract_data(source_path):
    """Extract data from source with error handling"""
    try:
        logger.info(f"Extracting data from {source_path}")
        data = pd.read_csv(source_path)
        logger.info(f"Successfully extracted {len(data)} rows")
        return data

    except FileNotFoundError as e:
        logger.error(f"Source file not found: {source_path}")
        raise  # Re-raise after logging

    except pd.errors.EmptyDataError as e:
        logger.error(f"Source file is empty: {source_path}")
        raise

    except Exception as e:
        logger.error(
            f"Unexpected error during extraction: {str(e)}",
            exc_info=True
        )
        raise
```

Handling Specific Exceptions

Different types of exceptions should be handled appropriately to ensure that the pipeline can either recover from errors or fail gracefully with meaningful logs.

```
def transform_data(raw_data):
    """Transform data with specific error handling"""
    try:
        # Try to filter by price_change
        clean_data = raw_data[raw_data['price_change'].notna()]
        logger.info("Successfully filtered DataFrame by 'price_change'")
        return clean_data

    except KeyError as ke:
        # Handle missing column by creating it
        logger.warning(f"{ke}: Cannot filter by 'price_change', creating column")
        raw_data['price_change'] = raw_data['close'] - raw_data['open']
        clean_data = raw_data[raw_data['price_change'].notna()]
        return clean_data

    except Exception as e:
        logger.error(f"Transformation failed: {str(e)}", exc_info=True)
        raise
```

Data Validation and Quality Checks

Data validation ensures that datasets meet expected quality standards before further processing. Performing validation early in the pipeline helps detect data issues quickly and prevents downstream failures.

```
def validate_data(df, expected_columns, min_rows=1):
    """Validate dataframe quality"""
    errors = []

    # Check if dataframe is empty
    if len(df) == 0:
        errors.append("DataFrame is empty")

    # Check minimum rows
    if len(df) < min_rows:
        errors.append(
            f"DataFrame has {len(df)} rows, minimum required: {min_rows}"
        )

    # Check expected columns
    missing_columns = set(expected_columns) - set(df.columns)
    if missing_columns:
        errors.append(f"Missing columns: {missing_columns}")

    # Check for nulls in critical columns
    for col in expected_columns:
        null_count = df[col].isnull().sum()
        if null_count > 0:
            errors.append(
                f"Column '{col}' has {null_count} null values"
            )

    if errors:
```

```
        error_msg = "; ".join(errors)
        logger.error(f"Data validation failed: {error_msg}")
        raise ValueError(f"Data validation failed: {error_msg}")

    logger.info(
        f"Data validation passed: {len(df)} rows, {len(df.columns)} columns"
    )
    return True

# Usage
try:
    validate_data(data,
                  expected_columns=['id', 'date', 'value'],
                  min_rows=100)
except ValueError as e:
    # Handle validation failure
    logger.critical(
        f"Pipeline stopped due to validation error: {e}"
    )
    send_alert(f"Pipeline validation failed: {e}")
    sys.exit(1)
```

4. Retry Mechanisms

Basic Retry Logic

Retry mechanisms are used to handle temporary or transient failures such as network timeouts, API rate limits, or temporary service unavailability. A common approach is to retry the operation multiple times with an increasing delay between attempts, known as exponential backoff.

```
import time
import logging

logger = logging.getLogger(__name__)

def retry_operation(operation, max_retries=3, initial_delay=1):
    """
    Retry an operation with exponential backoff

    Args:
        operation: Callable to retry
        max_retries: Maximum number of retry attempts
        initial_delay: Initial delay in seconds (doubles each retry)
    """
    for attempt in range(max_retries):
        try:
            logger.info(f"Attempt {attempt + 1}/{max_retries}")
            return operation()

        except Exception as e:
            wait_time = initial_delay * (2 ** attempt)

            if attempt < max_retries - 1:
                logger.warning(
                    f"Attempt {attempt + 1} failed: {str(e)}. "
                    f"Retrying in {wait_time} seconds..."
                )
                time.sleep(wait_time)
            else:
                logger.error(f"All {max_retries} attempts failed")
                raise  # Re-raise on final failure


# Usage example
def unstable_operation():
    # Simulate flaky operation
    if random.random() < 0.7:  # 70% failure rate
        raise ConnectionError("Network timeout")
    return "Success"


try:
    result = retry_operation(unstable_operation, max_retries=5)
    logger.info(f"Operation succeeded: {result}")
except Exception as e:
    logger.error(f"Operation failed after retries: {e}")
```

Using Tenacity Library

The `tenacity` library provides advanced retry capabilities with configurable stopping conditions, exponential backoff strategies, and logging integration. It is commonly used for handling transient failures such as API timeouts or network interruptions.

```
from tenacity import (
    retry,
    stop_after_attempt,
    wait_exponential,
    retry_if_exception_type,
    before_log,
    after_log
)
import logging
import requests

logger = logging.getLogger(__name__)

```

```python
@retry(
    stop=stop_after_attempt(5),
    wait=wait_exponential(multiplier=1, min=4, max=30),
    retry=retry_if_exception_type((
        requests.exceptions.Timeout,
        requests.exceptions.ConnectionError
    )),
    before=before_log(logger, logging.INFO),
    after=after_log(logger, logging.INFO)
)
def fetch_data_from_api(url):
    """Fetch data with automatic retry for transient failures"""
    logger.info(f"Fetching data from {url}")
    response = requests.get(url, timeout=5)
    response.raise_for_status()
    return response.json()

# Usage
try:
    data = fetch_data_from_api("https://api.example.com/data")
except Exception as e:
    logger.error(f"API request failed after retries: {e}")
```

5. Monitoring and Observability

Monitoring and observability are essential for understanding how data pipelines behave in production. By tracking performance metrics and execution details, engineers can identify bottlenecks, detect failures quickly, and improve pipeline reliability.

Pipeline Performance Monitoring

Execution time and resource usage can be tracked using decorators that wrap pipeline functions.

```python
import time
import logging
from functools import wraps
from datetime import datetime

logger = logging.getLogger(__name__)

def monitor_pipeline(func):
    """Decorator to monitor pipeline execution"""
    @wraps(func)
    def wrapper(*args, **kwargs):
        start_time = time.time()
        start_datetime = datetime.now()

        logger.info(f"Starting {func.__name__} at {start_datetime}")

        try:
            result = func(*args, **kwargs)

            end_time = time.time()
            duration = end_time - start_time

            logger.info(
                f"Completed {func.__name__} in {duration:.2f} seconds. "
                f"Rows processed: {len(result) if hasattr(result, '__len__') else 'N/A'}"
            )

            # Record metrics for monitoring
            record_metric(f"{func.__name__}.duration", duration)
            record_metric(f"{func.__name__}.success", 1)

            return result

        except Exception as e:
            end_time = time.time()
            duration = end_time - start_time

            logger.error(
                f"Failed {func.__name__} after {duration:.2f} seconds: {str(e)}"
            )
            record_metric(f"{func.__name__}.duration", duration)
            record_metric(f"{func.__name__}.success", 0)
            raise

    return wrapper


@monitor_pipeline
def etl_pipeline():
    # Pipeline logic here
    pass
```

Custom Metrics Collection

Custom metrics collection helps track pipeline execution details such as runtime, processed rows, and errors. These metrics can be stored in files or sent to monitoring systems for later analysis.

```python
import json
import os
from datetime import datetime

class PipelineMetrics:
```

```
    def __init__(self, pipeline_name):
        self.pipeline_name = pipeline_name
        self.metrics = {
            'pipeline': pipeline_name,
            'start_time': None,
            'end_time': None,
            'duration': None,
            'status': None,
            'rows_read': 0,
            'rows_written': 0,
            'errors': []
        }

    def start(self):
        self.metrics['start_time'] = datetime.now().isoformat()

    def stop(self, status='success'):
        self.metrics['end_time'] = datetime.now().isoformat()
        self.metrics['duration'] = (
            datetime.fromisoformat(self.metrics['end_time']) -
            datetime.fromisoformat(self.metrics['start_time'])
        ).total_seconds()
        self.metrics['status'] = status

    def add_error(self, error):
        self.metrics['errors'].append({
            'timestamp': datetime.now().isoformat(),
            'message': str(error)
        })

    def save(self, metrics_dir='metrics'):
        os.makedirs(metrics_dir, exist_ok=True)
        filename = f"{metrics_dir}/{self.pipeline_name}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.json
    "

        with open(filename, 'w') as f:
            json.dump(self.metrics, f, indent=2)

        logger.info(f"Metrics saved to {filename}")


# Usage
metrics = PipelineMetrics('daily_etl')
metrics.start()

try:
    # Pipeline execution
    metrics.metrics['rows_read'] = 15000
    metrics.metrics['rows_written'] = 14980
    metrics.stop('success')

except Exception as e:
    metrics.add_error(e)
    metrics.stop('failure')
    raise

finally:
    metrics.save()
```

6. Alerting Mechanisms

Alerting mechanisms notify engineers when a pipeline fails or encounters unusual conditions. Alerts can be delivered through email, messaging platforms, or monitoring systems.

Simple Alert System

```
import smtplib
import requests
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

class AlertManager:
    def __init__(self, config):
        self.config = config
        self.logger = logging.getLogger(__name__)

    def send_email_alert(self, subject, message, severity='ERROR'):
        """Send email alert"""
        try:
            msg = MIMEMultipart()
            msg['From'] = self.config['email_from']
            msg['To'] = self.config['email_to']
            msg['Subject'] = f"[{severity}] {subject}"

            msg.attach(MIMEText(message, 'plain'))

            server = smtplib.SMTP(
                self.config['smtp_server'],
                self.config['smtp_port']
            )
            server.starttls()
            server.login(
                self.config['smtp_user'],
                self.config['smtp_password']
```

```
            )
            server.send_message(msg)
            server.quit()

            self.logger.info(f"Email alert sent: {subject}")

        except Exception as e:
            self.logger.error(f"Failed to send email alert: {e}")

    def send_slack_alert(self, message, severity='ERROR'):
        """Send Slack alert via webhook"""
        try:
            webhook_url = self.config['slack_webhook']

            color = {
                'INFO': '#36a64f',
                'WARNING': '#ffcc00',
                'ERROR': '#ff0000',
                'CRITICAL': '#8b0000'
            }.get(severity, '#808080')

            payload = {
                'attachments': [{
                    'color': color,
                    'title': f'Pipeline Alert: {severity}',
                    'text': message,
                    'fields': [
                        {
                            'title': 'Pipeline',
                            'value': self.config.get('pipeline_name','unknown'),
                            'short': True
                        },
                        {
                            'title': 'Time',
                            'value': datetime.now().strftime(
                                '%Y-%m-%d %H:%M:%S'
                            ),
                            'short': True
                        }
                    ]
                }]
            }

            response = requests.post(webhook_url, json=payload)
            response.raise_for_status()

            self.logger.info(
                f"Slack alert sent: {message[:50]}..."
            )

        except Exception as e:
            self.logger.error(
                f"Failed to send Slack alert: {e}"
            )


# Usage
alert_config = {
    'email_from': 'pipeline@example.com',
    'email_to': 'team@example.com',
    'smtp_server': 'smtp.gmail.com',
    'smtp_port': 587,
    'smtp_user': 'user@gmail.com',
    'smtp_password': 'password',
    'slack_webhook': 'https://hooks.slack.com/services/xxx/yyy/zzz',
    'pipeline_name': 'daily_etl'
}

alerts = AlertManager(alert_config)

try:
    # Pipeline execution
    pass

except Exception as e:
    alerts.send_slack_alert(
        f"Pipeline failed: {str(e)}",
        severity='ERROR'
    )
    alerts.send_email_alert(
        "Pipeline Failure Alert",
        f"The daily ETL pipeline failed with error:\n\n{str(e)}"
    )
```

7. Comprehensive Pipeline Example

This section presents a complete ETL pipeline that integrates logging, monitoring, retry mechanisms, and error handling. The example demonstrates how production pipelines can be structured to ensure reliability, observability, and maintainability.

Complete ETL Pipeline with Logging, Monitoring, and Error Handling

```
import logging
import logging.config
import time
```

```
import json
from datetime import datetime
from tenacity import retry, stop_after_attempt, wait_exponential
import pandas as pd
import requests

# Configure logging
LOGGING_CONFIG = {
    'version': 1,
    'formatters': {
        'detailed': {
            'format': '%(asctime)s - %(name)s - %(levelname)s - %(message)s'
        },
        'json': {
            '()': 'pipeline_utils.JSONFormatter'
        }
    },
    'handlers': {
        'console': {
            'class': 'logging.StreamHandler',
            'level': 'INFO',
            'formatter': 'detailed'
        },
        'file': {
            'class': 'logging.handlers.RotatingFileHandler',
            'level': 'DEBUG',
            'formatter': 'json',
            'filename': 'pipeline.log',
            'maxBytes': 10485760,
            'backupCount': 5
        },
        'error_file': {
            'class': 'logging.handlers.RotatingFileHandler',
            'level': 'ERROR',
            'formatter': 'json',
            'filename': 'errors.log',
            'maxBytes': 10485760,
            'backupCount': 3
        }
    },
    'loggers': {
        'etl_pipeline': {
            'handlers': ['console', 'file', 'error_file'],
            'level': 'DEBUG',
            'propagate': False
        }
    }
}

logging.config.dictConfig(LOGGING_CONFIG)
logger = logging.getLogger('etl_pipeline')

class PipelineMetrics:
    """Track pipeline execution metrics"""

    def __init__(self, pipeline_name):
        self.pipeline_name = pipeline_name
        self.metrics = {
            'pipeline': pipeline_name,
            'run_id': datetime.now().strftime('%Y%m%d_%H%M%S'),
            'start_time': None,
            'end_time': None,
            'duration': None,
            'status': 'running',
            'stages': {},
            'errors': []
        }
        self.logger = logging.getLogger('etl_pipeline.metrics')

    def start_stage(self, stage_name):
        self.metrics['stages'][stage_name] = {
            'start_time': time.time(),
            'status': 'running'
        }
        self.logger.info(f"Starting stage: {stage_name}")

    def end_stage(self, stage_name, status='success', rows_processed=None):
        stage = self.metrics['stages'].get(stage_name, {})
        if stage:
            stage['end_time'] = time.time()
            stage['duration'] = stage['end_time'] - stage.get('start_time',
                                                              stage['end_time'])
            stage['status'] = status
            if rows_processed:
                stage['rows_processed'] = rows_processed

            self.logger.info(
                f"Completed stage: {stage_name} - "
                f"Duration: {stage['duration']:.2f}s, Status: {status}"
            )

    def add_error(self, stage, error):
        error_record = {
            'stage': stage,
            'timestamp': time.time(),
            'error_type': type(error).__name__,
```

```
            'error_message': str(error)
        }
        self.metrics['errors'].append(error_record)
        self.logger.error(f"Error in {stage}: {error}", exc_info=True)

    def complete(self, status='success'):
        self.metrics['status'] = status
        self.metrics['end_time'] = time.time()
        self.metrics['duration'] = self.metrics['end_time'] - \
                                   self.metrics.get('start_time', 0)

        self._save_metrics()

    def _save_metrics(self):
        import os

        metrics_dir = 'metrics'
        os.makedirs(metrics_dir, exist_ok=True)

        filename = f"{metrics_dir}/{self.pipeline_name}_" \
                   f"{self.metrics['run_id']}.json"

        with open(filename, 'w') as f:
            json.dump(self.metrics, f, indent=2)

        self.logger.info(f"Metrics saved to {filename}")


class ETLPipeline:
    """Production ETL Pipeline with monitoring"""

    def __init__(self, config):
        self.config = config
        self.logger = logging.getLogger('etl_pipeline')
        self.metrics = None

    @retry(stop=stop_after_attempt(3),
           wait=wait_exponential(multiplier=1, min=2, max=10))
    def extract(self, source):
        self.logger.info(f"Extracting data from {source}")

        if source.startswith('http'):
            response = requests.get(source, timeout=30)
            response.raise_for_status()
            data = response.json()
        else:
            data = pd.read_csv(source)

        rows = len(data) if hasattr(data, '__len__') else 1
        self.logger.info(f"Extracted {rows} records")
        return data

    def validate(self, data):
        self.logger.info("Validating data quality")

        if data is None or (hasattr(data, '__len__') and len(data) == 0):
            raise ValueError("Empty dataset")

        if isinstance(data, pd.DataFrame):
            required_cols = self.config.get('required_columns', [])
            missing = [c for c in required_cols if c not in data.columns]

            if missing:
                raise ValueError(f"Missing columns: {missing}")

        self.logger.info("Data validation passed")
        return True

    def transform(self, data):
        self.logger.info("Starting transformation")

        if isinstance(data, pd.DataFrame):
            original_rows = len(data)
            data = data.drop_duplicates()

            data = data.fillna({
                'numeric_col': 0,
                'string_col': 'unknown'
            })

            rows_after = len(data)
            self.logger.info(
                f"Transformation complete: {original_rows} -> {rows_after}"
            )

        return data

    def load(self, data, destination):
        self.logger.info(f"Loading data to {destination}")

        if destination.endswith('.csv') and isinstance(data, pd.DataFrame):
            data.to_csv(destination, index=False)

        rows = len(data) if hasattr(data, '__len__') else 1
        self.logger.info(f"Loaded {rows} records")

```

```
        return True

    def run(self):
        self.metrics = PipelineMetrics(self.config.get('name','etl_pipeline'))
        self.metrics.metrics['start_time'] = time.time()

        try:
            self.metrics.start_stage('extract')
            data = self.extract(self.config['source'])
            self.metrics.end_stage('extract')

            self.metrics.start_stage('validate')
            self.validate(data)
            self.metrics.end_stage('validate')

            self.metrics.start_stage('transform')
            data = self.transform(data)
            self.metrics.end_stage('transform')

            self.metrics.start_stage('load')
            self.load(data, self.config['destination'])
            self.metrics.end_stage('load')

            self.metrics.complete('success')
            return data

        except Exception as e:
            self.metrics.add_error('pipeline', e)
            self.metrics.complete('failed')
            raise


if __name__ == "__main__":

    config = {
        'name': 'daily_sales_etl',
        'source': 'sales_data.csv',
        'destination': 'processed_sales.csv',
        'required_columns': ['order_id','customer_id','amount','date'],
        'critical_columns': ['order_id','amount'],
        'max_null_pct': 10,
        'alert_on_failure': True
    }

    pipeline = ETLPipeline(config)

    try:
        result = pipeline.run()
        print("Pipeline executed successfully")
    except Exception as e:
        print(f"Pipeline failed: {e}")
```

6.5 Data Storage and Retrieval Strategies for ETL Pipelines

In ETL (Extract, Transform, Load) and ELT (Extract, Load, Transform) pipelines, data storage is not merely the final destination of processed data. The strategies used for storing and retrieving data significantly influence pipeline performance, scalability, reliability, and overall architecture. Selecting appropriate storage systems allows pipelines to efficiently process large volumes of data while controlling costs and ensuring long-term accessibility.

1. Storage Paradigms for Pipelines

Modern data pipelines typically interact with multiple storage systems, each designed for a specific role in the data lifecycle.

a) Data Warehouses

A data warehouse is a centralized repository designed for analytical processing (OLAP). It stores structured and processed data optimized for reporting, dashboards, and business intelligence.

In traditional ETL pipelines, the data warehouse acts as the final destination. Data is extracted from operational systems, transformed in staging environments, and then loaded into the warehouse using structured schemas such as star or snowflake schemas. In ELT pipelines, raw data is first loaded into the warehouse and transformations are performed using SQL within the warehouse itself.

Key Characteristics

- **Schema-on-Write**: Data structure is enforced during the loading stage.
- **Optimized for Analytical Queries**: Uses techniques such as indexing, partitioning, and columnar storage formats.
- **ACID Compliance**: Ensures data consistency and reliability.

Common Platforms
Amazon Redshift, Google BigQuery, Snowflake, and Azure Synapse Analytics.

b) Data Lakes

A data lake is a large-scale storage repository designed to store raw data in its native format. Unlike warehouses, data lakes support structured, semi-structured, and unstructured data.

In ELT architectures, data lakes often function as the initial landing zone where data from multiple sources is stored with minimal transformation. This approach allows analysts and data scientists to process and analyze raw data later.

Key Characteristics

- **Schema-on-Read**: Structure is applied only when data is analyzed.
- **Cost-Effective Storage**: Built on scalable object storage systems.
- **Flexible Data Formats**: Supports formats such as JSON, XML, images, and log files.

Common Platforms
AWS S3, Azure Data Lake Storage (ADLS), and Google Cloud Storage (GCS).

c) Data Lakehouses

A data lakehouse is a hybrid architecture that combines the flexibility of data lakes with the data management features of data warehouses. This is achieved by adding a transactional metadata layer on top of data lake storage.

In pipeline architectures, the lakehouse can serve both as a landing zone for raw data and as an analytical storage system. This reduces system complexity by eliminating the need for separate data lakes and warehouses.

Key Characteristics

- **ACID Transactions**: Supports reliable concurrent reads and writes.
- **Schema Evolution**: Allows schema changes without breaking existing datasets.
- **Time Travel**: Enables querying previous versions of data.
- **Open File Formats**: Data is typically stored in open formats such as Parquet.

Common Technologies
Delta Lake, Apache Iceberg, and Apache Hudi.

2. Key Data Storage Strategies

When designing the storage layer of a data pipeline, several strategies help improve performance, reliability, and cost efficiency.

Data Partitioning

Data partitioning divides large datasets into smaller logical segments based on column values such as date, region, or category. This enables query engines to scan only relevant partitions instead of the entire dataset.

- Without partitioning: Queries must scan the full dataset.
- With partitioning: Queries access only the required partitions.

Data Replication
Data replication involves maintaining multiple copies of data across systems or geographic locations.
Common uses include:

- Disaster recovery and fault tolerance
- Improved query performance through geographically distributed copies
- Separation between operational databases (OLTP) and analytical systems (OLAP)

Data Caching
Caching stores intermediate or frequently accessed data in high-speed storage systems to reduce latency.
Typical use cases include:

- Storing intermediate ETL results
- Reusing computation results during pipeline reruns
- Accelerating dashboards and applications through cached aggregates

Hot vs. Cold Data Storage

Data tiering strategies classify data based on access frequency.

- **Hot Storage**: Frequently accessed data stored on high-performance storage.
- **Warm Storage**: Moderately accessed data stored on medium-cost storage.
- **Cold/Archive Storage**: Rarely accessed data stored on low-cost archival storage.

This strategy helps organizations manage storage costs while maintaining data availability.

3. Example: Partitioning Data for Efficient Retrieval (PySpark)

The following example demonstrates how large datasets can be written to storage in a partitioned format using PySpark. Partitioning improves query performance by enabling engines to scan only relevant data segments.

```
from pyspark.sql import SparkSession

# Create Spark session
spark = SparkSession.builder \
    .appName("PartitionExample") \
    .getOrCreate()

# Load dataset
df = spark.read.csv("sales_data.csv", header=True, inferSchema=True)

# Convert date column to proper format
from pyspark.sql.functions import year, month

df = df.withColumn("year", year("date")) \
       .withColumn("month", month("date"))
```

```

# Write data partitioned by year and month
df.write \
  .partitionBy("year", "month") \
  .mode("overwrite") \
  .parquet("data_lake/sales_partitioned")

print("Data successfully written in partitioned format")
```

Example 1: Partitioning Data for Efficient Retrieval (PySpark)

Partitioning data in a data lake enables efficient retrieval. Spark can automatically read only relevant partitions based on filter conditions, which reduces I/O and improves query performance.

```
from pyspark.sql import SparkSession

# Initialize Spark Session
spark = SparkSession.builder.appName("PartitioningExample").getOrCreate()

# Assume 'sales_df' is a Spark DataFrame loaded earlier in the pipeline
# sales_df = spark.read.parquet("landing_zone/sales_data/")

# Write the data to a data lake partitioned by 'year' and 'month'
output_path = "data_lake/sales/"

(sales_df.write
 .partitionBy("year", "month")      # Organizes data into year=YYYY/month=MM/ folders
 .mode("overwrite")                  # Overwrite existing data
 .format("parquet")                  # Efficient columnar storage format
 .save(output_path))

print(f"Data successfully written to {output_path} with year/month partitions.")

# Retrieval: Read only sales for a specific month
jan_sales_df = spark.read.parquet(output_path) \
    .where("year = '2024' AND month = '01'")  # Only scans relevant partitions
```

Example 2: Config-Driven ETL Pipeline with Multiple Storage Destinations

A common pattern in modern pipelines is to support multiple storage destinations, such as a raw data lake (ELT) and a curated warehouse (ETL). The following Python class demonstrates a simple configurable ETL pipeline.

```
import pandas as pd
import logging

logger = logging.getLogger(__name__)

class ETLPipeline:
    """A simple ETL pipeline with configurable storage destinations."""

    def __init__(self, config: dict):
        self.config = config
        self.data = None

    def extract(self) -> pd.DataFrame:
        logger.info(f"Extracting data from {self.config['source_path']}")
        self.data = pd.read_csv(self.config['source_path'])
        logger.info(f"Extracted {len(self.data)} rows.")
        return self.data

    def transform(self) -> pd.DataFrame:
        logger.info("Starting transformation...")
        if self.data is not None:
            if 'date' in self.data.columns:
                self.data['date'] = pd.to_datetime(self.data['date'])
            logger.info("Transformation complete.")
        return self.data

    def load(self):
        dest_type = self.config['destination_type']
        dest_path = self.config['destination_path']
        logger.info(f"Loading data to {dest_type} at {dest_path}")

        if dest_type == "data_lake":
            self.data.to_parquet(f"{dest_path}/raw_data.parquet", index=False)
            logger.info("Data written to data lake in Parquet format.")
        elif dest_type == "data_warehouse":
            logger.info(f"Loading transformed data into warehouse table: {dest_path}")
            # Example: self.data.to_sql('sales_table', connection, if_exists='append')
        else:
            raise ValueError(f"Unknown destination type: {dest_type}")

        logger.info("Load phase completed.")

# --- Usage ---
elt_config = {
    'source_path': 'sales_data_raw.csv',
    'destination_type': 'data_lake',
    'destination_path': 's3://my-data-lake-bronze/sales/'
}

etl_config = {
    'source_path': 'sales_data_raw.csv',
    'destination_type': 'data_warehouse',
    'destination_path': 'analytics.sales_fact'
```

```
}

pipeline = ETLPipeline(etl_config)  # or elt_config
pipeline.extract()
pipeline.transform()
pipeline.load()
```

6.6 Automated Report Generation (Excel, HTML, PDF)

Introduction to Automated Reporting

Automated report generation is the process of programmatically creating structured documents that present data analysis results without manual intervention. This ensures consistency, saves time, and enables regular distribution of insights.

Definition: Automated reporting involves pulling data from sources, processing it, formatting results into human-readable formats (Excel, HTML, PDF), and distributing them automatically on a schedule or trigger.

Core Benefits:

Benefit	Description
Time Efficiency	Eliminates repetitive manual reporting work
Accuracy	Reduces human error in data compilation and calculations
Consistency	Same format and quality every time reports are generated
Timeliness	Can be scheduled to run at optimal times or triggered by events
Scalability	Handle growing amounts of data and increasing report frequency easily

Common Use Cases:

Use Case	Application
Sales Reporting	Pull daily sales data, generate revenue trends, email PDF to managers
Website Analytics	Fetch traffic data from APIs, show visitor trends, publish weekly reports
Finance/Accounting	Collect expense/revenue data, create balance sheets, distribute automatically
IT Monitoring	Track server uptime, CPU usage, generate dashboards, notify on critical events

Key Python Libraries for Report Generation

Library	Purpose	Use Case
pandas	Data handling and manipulation	Core data processing for all reports
matplotlib / seaborn	Static visualizations	Charts and graphs for PDF/HTML reports
plotly	Interactive visualizations	Web-based dashboards and HTML reports
fpdf / ReportLab	PDF generation	Printable reports, formal documentation
openpyxl / xlsxwriter	Excel file creation	Spreadsheet reports with formulas and formatting
Jinja2	HTML templating	Dynamic web-based reports

Automated Report Generation Workflow

Extract → **Transform** → **Visualize** → **Export** → **Deliver**

(Gather data) (Process/analyze) (Charts/tables) (PDF/Excel/HTML) (Email/upload)

1. Excel Report Generation

Excel reports are ideal for stakeholders who need to perform additional analysis or work with raw numbers.

Key Concepts:

- DataFrames as foundation: pandas DataFrames provide the structure for tabular data.
- Multiple sheets: Organize related information across worksheet tabs.
- Formatting: Apply styles, colors, and fonts for readability.
- Formulas: Embed Excel formulas for dynamic calculations.

```
import pandas as pd
from datetime import datetime

def generate_excel_report(data_path, output_path):
    """
    Generate formatted Excel report with multiple sheets
    """
    # Extract data
    data = pd.read_csv(data_path)

    # Transform - create summary statistics
    summary = pd.DataFrame({
        'Metric': ['Total', 'Average', 'Maximum', 'Minimum'],
        'Value': [
```

```
            data['amount'].sum(),
            data['amount'].mean(),
            data['amount'].max(),
            data['amount'].min()
        ]
    })

    # Export to Excel with multiple sheets
    with pd.ExcelWriter(output_path, engine='openpyxl') as writer:
        data.to_excel(writer, sheet_name='Raw Data', index=False)
        summary.to_excel(writer, sheet_name='Summary', index=False)

        # Add metadata
        workbook = writer.book
        summary_sheet = writer.sheets['Summary']
        summary_sheet['A1'] = f"Report Generated: {datetime.now()}"
```

2. HTML Report Generation

HTML reports enable web-based viewing with interactive elements and are easily shareable via email or web servers.

Key Concepts:

- **Templating:** Use Jinja2 to separate content from presentation.
- **Interactivity:** Embed JavaScript libraries like Plotly for interactive charts.
- **Responsive design:** Create reports that work on different devices.
- **Linking:** Connect related sections with hyperlinks.

HTML Template Example (template.html):

```
<!DOCTYPE html>
<html>
<head>
    <title>{{ report_title }}</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 40px; }
        h1 { color: #333; }
        table { border-collapse: collapse; width: 100%; }
        th, td { border: 1px solid #ddd; padding: 8px; text-align: left; }
        th { background-color: #4CAF50; color: white; }
    </style>
</head>
<body>
    <h1>{{ report_title }}</h1>
    <p>Generated: {{ generation_date }}</p>

    <h2>Summary Statistics</h2>
    <table>
        <tr>
            <th>Metric</th>
            <th>Value</th>
        </tr>
        {% for item in summary_data %}
        <tr>
            <td>{{ item.Metric }}</td>
            <td>{{ item.Value }}</td>
        </tr>
        {% endfor %}
    </table>
</body>
</html>
```

6.6 Automated Report Generation (Excel, HTML, PDF)

3. PDF Report Generation

PDF reports are suitable for formal documentation, printing, and sharing with external stakeholders.

Key Concepts for PDF Generation:

- Page layout: Control margins, page breaks, and orientation
- Font management: Use different fonts for headings and body text
- Image embedding: Include charts and graphs as images
- Headers and footers: Add consistent page numbering and document information

```
from fpdf import FPDF

pdf = FPDF()
pdf.add_page()

# Title
pdf.set_font('Arial', 'B', 16)
pdf.cell(0, 10, "Daily Sales Report", ln=True, align='C')
pdf.cell(0, 10, f"Date: {datetime.now().strftime('%Y-%m-%d')}", ln=True, align='C')
pdf.ln(10)

# KPIs
pdf.set_font('Arial', 'B', 12)
pdf.cell(0, 10, "Key Performance Indicators", ln=True)
```

```
pdf.set_font('Arial', '', 10)

for kpi, value in kpis.items():
    pdf.cell(0, 8, f"{kpi}: {value:,.2f}", ln=True)

pdf.output("daily_sales_report.pdf")
```

Best Practices for Automated Reporting

- Parameterize Everything: Make report elements configurable (dates, thresholds, formats)
- Include Metadata: Always add generation timestamp, data sources, and version information
- Handle Missing Data Gracefully: Implement checks for empty datasets
- Log Generation Process: Track when reports are generated and any errors encountered
- Validate Outputs: Verify file sizes and content before distribution
- Archive Previous Versions: Maintain history for audit purposes

6.7 Case Study: End-to-End Automated Analytics Pipeline

Overview

This case study synthesizes concepts from Chapter 6 into a complete, real-world example of an automated analytics pipeline. It demonstrates how data engineering components work together in production.

Business Scenario:

- Company: E-commerce Retailer "ShopSmart"
- Problem: Manual reporting on daily sales, inventory, and customer metrics takes 3 hours each morning
- Goal: Fully automate data collection, processing, analysis, and reporting with zero manual intervention

Pipeline Architecture

The pipeline follows a modern ELT (Extract, Load, Transform) pattern with multiple storage layers:

$$[\text{Source Systems}] \rightarrow [\text{Landing Zone}] \rightarrow [\text{Processing}] \rightarrow [\text{Serving Layer}] \rightarrow [\text{Reports}]$$

- Source Systems: Database, API, CSV
- Landing Zone: Data Lake (raw storage)
- Processing: Spark or Python ETL jobs
- Serving Layer: Data Warehouse, Excel, PDF

Pipeline Components

Component 1: Data Extraction (Scheduled) Automatically pull data from multiple source systems.

```
class DataExtractor:
    def extract_sales_data(self):
        df = pd.read_csv(self.config['sales_source'])
        return df

    def extract_inventory_data(self):
        response = requests.get(self.config['inventory_api'])
        data = response.json()
        return pd.DataFrame(data)

    def extract_customer_data(self):
        return pd.read_csv(self.config['customer_source'])
```

Component 2: Data Lake Storage (Landing Zone) Store raw data in its original format for auditability and reprocessing.

```
class DataLake:
    def store_raw_data(self, data, source_name):
        date_str = datetime.now().strftime("%Y-%m-%d")
        partition_path = f"{self.base_path}/{source_name}/date={date_str}"
        os.makedirs(partition_path, exist_ok=True)
        data.to_parquet(f"{partition_path}/raw_data.parquet")
        return f"{partition_path}/raw_data.parquet"
```

Component 3: Data Transformation (Processing Layer) Clean, validate, and transform raw data into analytical structures.

```
class DataTransformer:
    def clean_sales_data(self, raw_sales):
        cleaned = raw_sales.drop_duplicates(subset=['order_id'])
        cleaned['customer_id'] = cleaned['customer_id'].fillna('UNKNOWN')
        cleaned['amount'] = cleaned['amount'].fillna(0).astype(float)
        cleaned['order_date'] = pd.to_datetime(cleaned['order_date'])
        return cleaned

    def calculate_kpis(self, sales_df, inventory_df, customer_df):
        kpis = {}
        kpis['total_sales'] = sales_df['amount'].sum()
        kpis['avg_order_value'] = sales_df['amount'].mean()
```

```
        kpis['order_count'] = len(sales_df)
        kpis['unique_customers'] = sales_df['customer_id'].nunique()
        kpis['low_stock_items'] = len(inventory_df[inventory_df['quantity'] < 10])
        return kpis
```

```
class ReportGenerator:
    def create_excel_dashboard(self, sales_df, kpis):
        with pd.ExcelWriter(f"reports/daily_dashboard.xlsx") as writer:
            pd.DataFrame(list(kpis.items()), columns=['KPI', 'Value']).to_excel(writer, sheet_name='
   Dashboard')
            sales_df.to_excel(writer, sheet_name='Sales Details')

    def create_pdf_report(self, sales_df, kpis):
        from fpdf import FPDF
        pdf = FPDF()
        pdf.add_page()
        pdf.set_font('Arial', 'B', 16)
        pdf.cell(0, 10, "Daily Sales Report", ln=True, align='C')
        for kpi, value in kpis.items():
            pdf.cell(0, 8, f"{kpi}: {value:,.2f}", ln=True)
        pdf.output("reports/daily_report.pdf")
```

Component 5: Orchestration (Pipeline Controller) Coordinate extraction, storage, transformation, and report generation.

```
class AnalyticsPipeline:
    def run(self):
        raw_sales = self.extractor.extract_sales_data()
        raw_inventory = self.extractor.extract_inventory_data()
        raw_customers = self.extractor.extract_customer_data()
        self.data_lake.store_raw_data(raw_sales, 'sales')
        cleaned_sales = self.transformer.clean_sales_data(raw_sales)
        kpis = self.transformer.calculate_kpis(cleaned_sales, raw_inventory, raw_customers)
        excel_report = self.reporter.create_excel_dashboard(cleaned_sales, kpis)
        pdf_report = self.reporter.create_pdf_report(cleaned_sales, kpis)
```

Pipeline Monitoring and Alerting

Include metrics, logs, and error tracking to ensure reliability.

```
class PipelineMonitor:
    def generate_health_report(self):
        report = {
            'total_executions': len(self.metrics['execution_time']),
            'avg_execution_time': np.mean([m['duration'] for m in self.metrics['execution_time']]),
            'error_rate': self.metrics['error_count'] / max(len(self.metrics['execution_time']), 1)
        }
        return report
```

Key Lessons and Best Practices

- Separation of Concerns: Each component has a single responsibility
- Idempotency: Pipeline can be safely re-run
- Observability: Logging and metrics at every step
- Error Handling: Graceful failure with detailed logs
- Configuration-Driven: Externalize all settings
- Data Lineage: Preserve raw data for audit trail
- Automated Validation: Prevent bad data from propagating

Performance Optimization Techniques

Technique	**Application**
Vectorized Operations	Use pandas/numpy instead of loops
Partitioning	Organize data by date for faster queries
Caching	Store intermediate results for frequent reports
Parallel Processing	Extract from multiple sources concurrently
Incremental Loading	Process only new/changed data